Nagy hatékonyságú számítások . . .
1. előadás (Bevezetés, alapok)

Nagy Ádám
nagyadam888@gmail.com

Eötvös Loránd Tudományegyetem
Informatikai Kar

2021. szeptember 10.

Miről lesz szó

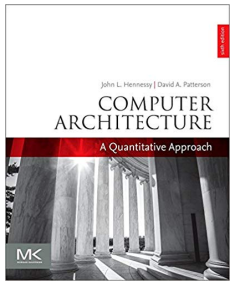
Néhány kérdés melyre választ adunk.

- Hogyan hajtódik végre egy program?
- Miért lehet gyorsabb egy hasonló paraméterekkel rendelkező processzor ugyanannál a programnál.
- Miért lehet két hasonló program futási ideje nagyban különböző?
- Hogyan lehet hogy az utóbbi 10-15 évben a órajel nem nőtt jelentősen, de a számítási kapacitás igen (egy processzor esetében is)?
- Mi is történik pontosan az adatokkal (memórikezelés)?
- Hogyan válasszak "jó vasat,, a programomhoz?
- Hogyan optimalizáljak egy programot (ha ismerem az architektúrát)?

Forrás

Irodalom:

Előadás: *Computer Architekture, 6th Edition - A Quntitive Approach*,J. L. Hennessy, D. A. Patterson

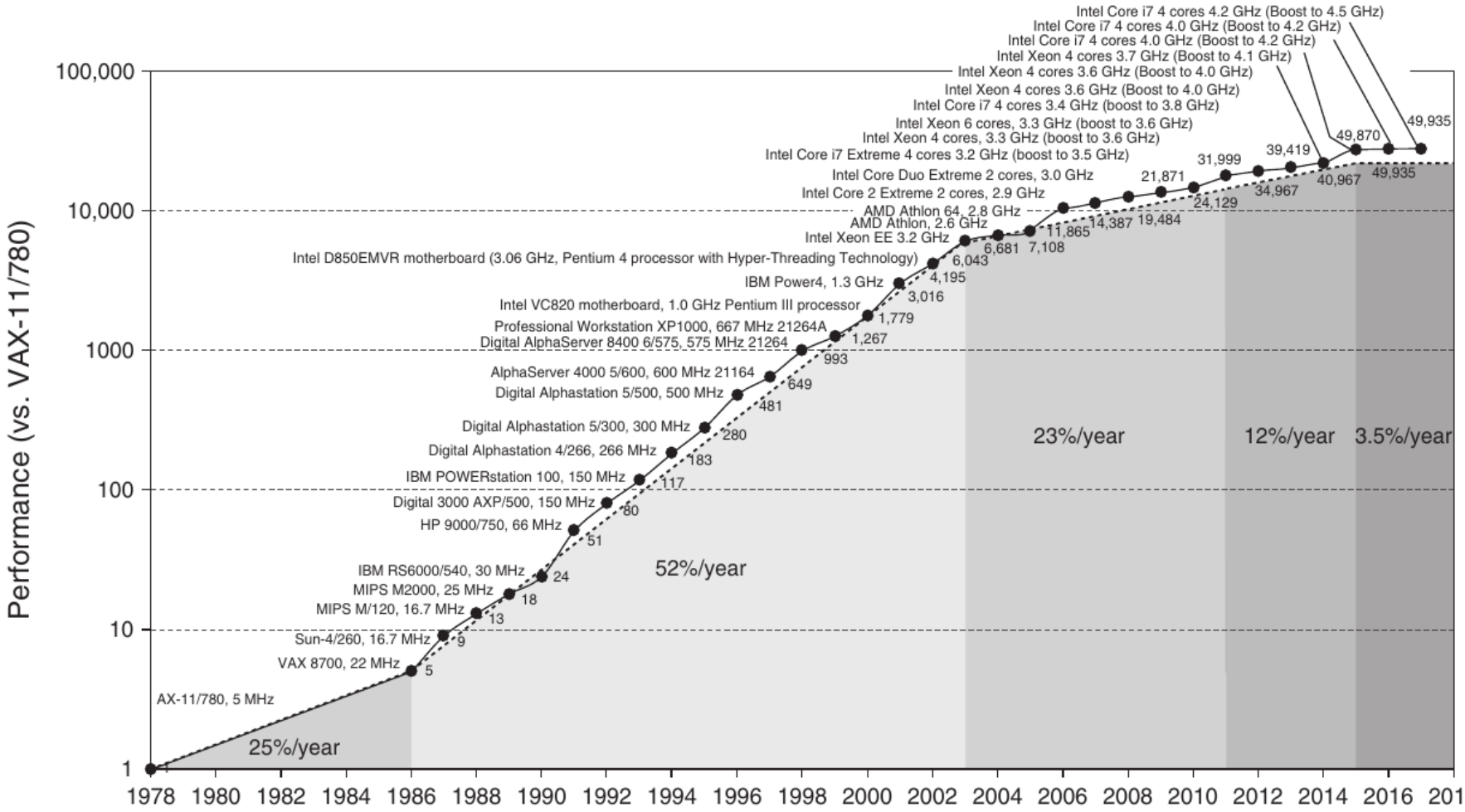


Gyakorlat: MMIX, Donald E. Knuth
`http://www-cs-faculty.stanford.edu/~knuth/mmix.html`

Alapfogalmak:

- Párhuzamosítás szintjei:
  - Utasításszint (ILP)
  - Adatszint (DLP)
  - Szál-szint (TLP)
  - Request-Level Parallelism
- CPU idő: utasítások száma $\times$ órajle utasításonként $\times$ órajel idő
- Amdahl törvénye: 1 Mb memória 1 Mbit/s I/O sávszélesség kell egy kiegyensúlyozott architektúrához
- Lokalitás elve: 90/10 szabály
- Sávszélesség szabály: sávszélesség négyzetesen nő a késleltetéshez képest
- RISC, CISC
- SISD, SIMD, MISD, MIMD

Hatékonyság növekedése



Architektúrák osztályai

Feature	Personal mobile device (PMD)	Desktop	Server	Clusters/warehouse-scale computer	Internet of things/ embedded
Price of system	$100–$1000	$300–$2500	$5000–$10,000,000	$100,000–$200,000,000	$10–$100,000
Price of microprocessor	$10–$100	$50–$500	$200–$2000	$50–$250	$0.01–$100
Critical system design issues	Cost, energy, media performance, responsiveness	Price-performance, energy, graphics performance	Throughput, availability, scalability, energy	Price-performance, throughput, energy proportionality	Price, energy, application-specific performance

Figure 1.2 A summary of the five mainstream computing classes and their system characteristics. Sales in 2015 included about 1.6 billion PMDs (90% cell phones), 275 million desktop PCs, and 15 million servers. The total number of embedded processors sold was nearly 19 billion. In total, 14.8 billion ARM-technology-based chips were shipped in 2015. Note the wide range in system price for servers and embedded systems, which go from USB keys to network routers. For servers, this range arises from the need for very large-scale multiprocessor systems for high-end transaction processing.

Mi is kerül anyiba egy szerveren?

Application	Cost of downtime per hour	Annual losses with downtime of		
		1% (87.6 h/year)	0.5% (43.8 h/year)	0.1% (8.8 h/year)
Brokerage service	$4,000,000	$350,400,000	$175,200,000	$35,000,000
Energy	$1,750,000	$153,300,000	$76,700,000	$15,300,000
Telecom	$1,250,000	$109,500,000	$54,800,000	$11,000,000
Manufacturing	$1,000,000	$87,600,000	$43,800,000	$8,800,000
Retail	$650,000	$56,900,000	$28,500,000	$5,700,000
Health care	$400,000	$35,000,000	$17,500,000	$3,500,000
Media	$50,000	$4,400,000	$2,200,000	$400,000

Figure 1.3 Costs rounded to nearest $100,000 of an unavailable system are shown by analyzing the cost of downtime (in terms of immediately lost revenue), assuming three different levels of availability, and that downtime is distributed uniformly. These data are from Landstrom (2014) and were collected and analyzed by Contingency Planning Research.

Mi határoz meg egy architektúrát?

- **Utasításkészlet**
  - Utasítások típusai (reg-reg, reg-mem, . . . , sp, dp, . . .)
  - Utasítások formátuma (fix, változó), bonyolultsága
  - Memória elérés, címzési mód(ok)
  - Vezérlő utasítások (if, jump, call/return) szerkezete, kezelésük
- Szerkezet (cache/memória, pipeline, mutlicore és kezelése, órajel)
- Hardware

Legfontosabb szempontok: (funkcionalitás), ár, hatékonyság, energiafelhasználás, rendelkezésre állás

Funkcionális követelmények

Functional requirements	Typical features required or supported
Application area	*Target of computer*
Personal mobile device	Real-time performance for a range of tasks, including interactive performance for graphics, video, and audio; energy efficiency (Chapters 2–5 and 7; Appendix A)
General-purpose desktop	Balanced performance for a range of tasks, including interactive performance for graphics, video, and audio (Chapters 2–5; Appendix A)
Servers	Support for databases and transaction processing; enhancements for reliability and availability; support for scalability (Chapters 2, 5, and 7; Appendices A, D, and F)
Clusters/warehouse-scale computers	Throughput performance for many independent tasks; error correction for memory; energy proportionality (Chapters 2, 6, and 7; Appendix F)
Internet of things/embedded computing	Often requires special support for graphics or video (or other application-specific extension); power limitations and power control may be required; real-time constraints (Chapters 2, 3, 5, and 7; Appendices A and E)
Level of software compatibility	*Determines amount of existing software for computer*
At programming language	Most flexible for designer; need new compiler (Chapters 3, 5, and 7; Appendix A)
Object code or binary compatible	Instruction set architecture is completely defined—little flexibility—but no investment needed in software or porting programs (Appendix A)
Operating system requirements	*Necessary features to support chosen OS* (Chapter 2; Appendix B)
Size of address space	Very important feature (Chapter 2); may limit applications
Memory management	Required for modern OS; may be paged or segmented (Chapter 2)
Protection	Different OS and application needs: page versus segment; virtual machines (Chapter 2)
Standards	*Certain standards may be required by marketplace*
Floating point	Format and arithmetic: IEEE 754 standard (Appendix J), special arithmetic for graphics or signal processing
I/O interfaces	For I/O devices: Serial ATA, Serial Attached SCSI, PCI Express (Appendices D and F)
Operating systems	UNIX, Windows, Linux, CISCO IOS
Networks	Support required for different networks: Ethernet, Infiniband (Appendix F)
Programming languages	Languages (ANSI C, C++, Java, Fortran) affect instruction set (Appendix A)

Figure 1.8 Summary of some of the most important functional requirements an architect faces. The left-hand column describes the class of requirement, while the right-hand column gives specific examples. The right-hand column also contains references to chapters and appendices that deal with the specific issues.

Trendek, órajel

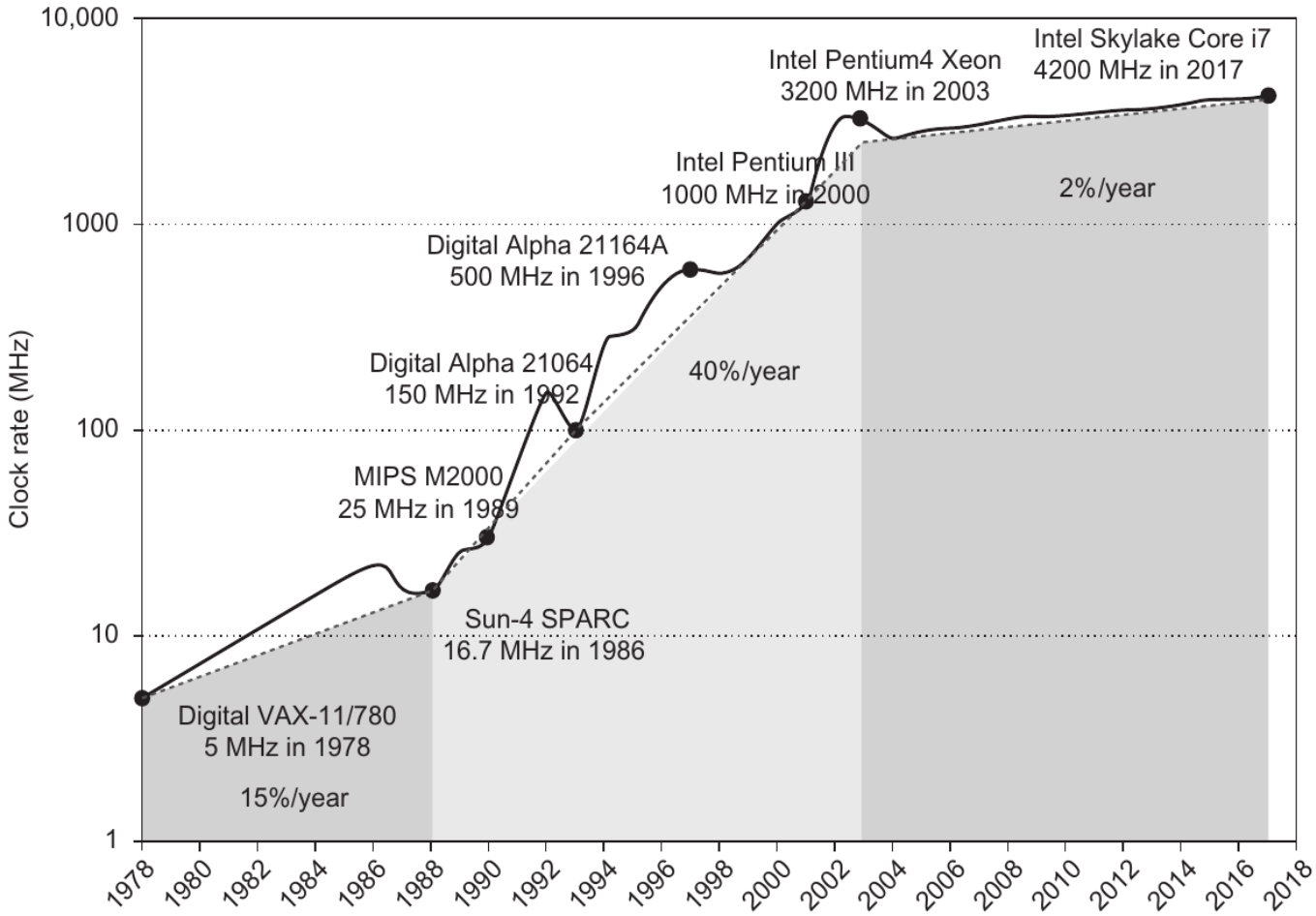


Figure 1.11 Growth in clock rate of microprocessors in Figure 1.1. Between 1978 and 1986, the clock rate improved less than 15% per year while performance improved by 22% per year. During the "renaissance period" of 52% performance improvement per year between 1986 and 2003, clock rates shot up almost 40% per year. Since then, the clock rate has been nearly flat, growing at less than 2% per year, while single processor performance improved recently at just 3.5% per year.

Trendek, RAM

CA:AQA Edition	Year	DRAM growth rate	Characterization of impact on DRAM capacity
1	1990	60%/year	Quadrupling every 3 years
2	1996	60%/year	Quadrupling every 3 years
3	2003	40%–60%/year	Quadrupling every 3 to 4 years
4	2007	40%/year	Doubling every 2 years
5	2011	25%–40%/year	Doubling every 2 to 3 years

Figure 1.8 Change in rate of improvement in DRAM capacity over time. The first two editions even called this rate the DRAM Growth Rule of Thumb, since it had been so dependable since 1977 with the 16-kilobit DRAM through 1996 with the 64-megabit DRAM. Today, some question whether DRAM capacity can improve at all in 5 to 7 years, due to difficulties in manufacturing an increasingly three-dimensional DRAM cell [Kim 2005].

Trendek, Sávszélesség > késleltetés (latency), response t.

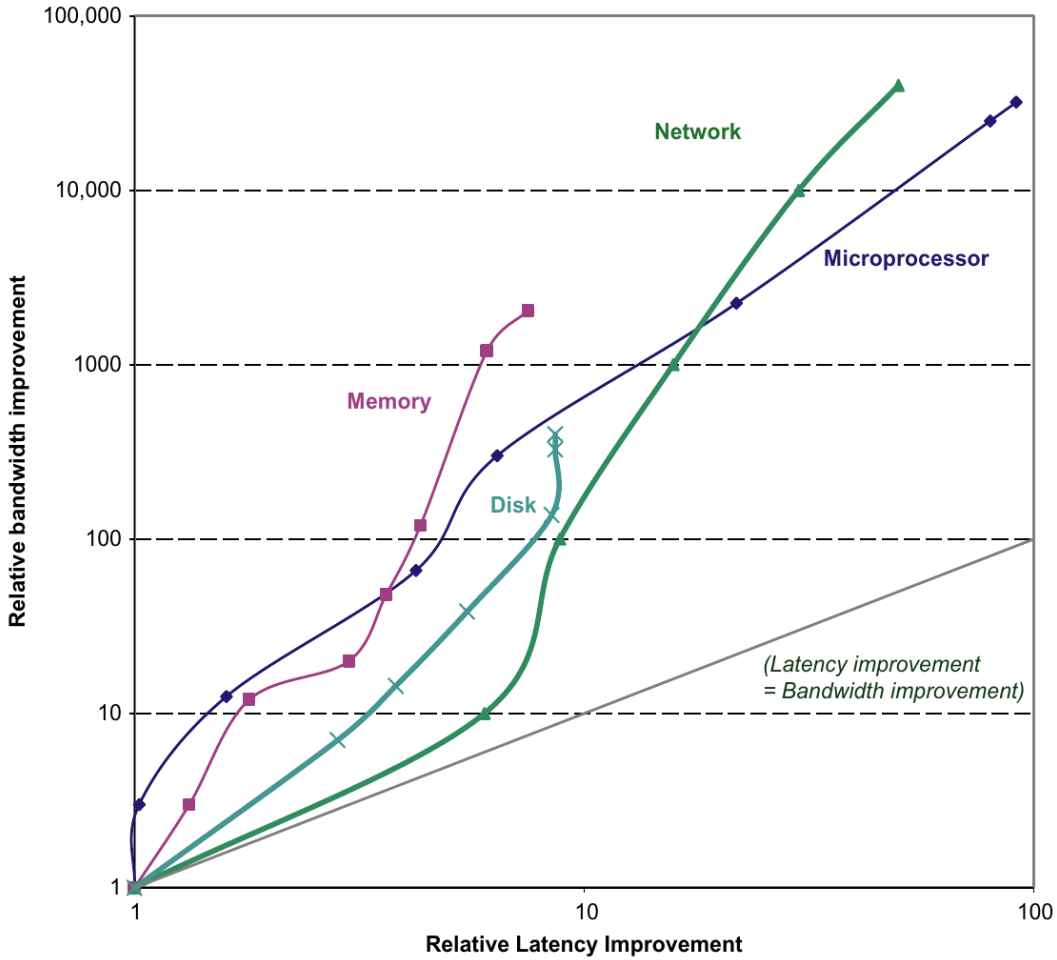


Figure 1.9 Log-log plot of bandwidth and latency milestones in Figure 1.10 relative to the first milestone. Note that latency improved 8–91 ×, while bandwidth improved about 400–32,000 ×. Except for networking, we note that there were modest improvements in latency and bandwidth in the other three technologies in the six years since the last edition: 0%–23% in latency and 23%–70% in bandwidth. Updated from Patterson, D., 2004. Latency lags bandwidth. Commun. ACM 47 (10), 71–75.

Trendek, példa

Microprocessor	16-Bit address/ bus, microcoded	32-Bit address/ bus, microcoded	5-Stage pipeline, on-chip I & D caches, FPU	2-Way superscalar, 64-bit bus	Out-of-order 3-way superscalar	Out-of-order superpipelined, on-chip L2 cache	Multicore OOO 4-way on chip L3 cache, Turbo
Product	Intel 80286	Intel 80386	Intel 80486	Intel Pentium	Intel Pentium Pro	Intel Pentium 4	Intel Core i7
Year	1982	1985	1989	1993	1997	2001	2015
Die size (mm^2)	47	43	81	90	308	217	122
Transistors	134,000	275,000	1,200,000	3,100,000	5,500,000	42,000,000	1,750,000,000
Processors/chip	1	1	1	1	1	1	4
Pins	68	132	168	273	387	423	1400
Latency (clocks)	6	5	5	5	10	22	14
Bus width (bits)	16	32	32	64	64	64	196
Clock rate (MHz)	12.5	16	25	66	200	1500	4000
Bandwidth (MIPS)	2	6	25	132	600	4500	64,000
Latency (ns)	320	313	200	76	50	15	4
Memory module	DRAM	Page mode DRAM	Fast page mode DRAM	Fast page mode DRAM	Synchronous DRAM	Double data rate SDRAM	DDR4 SDRAM
Module width (bits)	16	16	32	64	64	64	64
Year	1980	1983	1986	1993	1997	2000	2016
Mbits/DRAM chip	0.06	0.25	1	16	64	256	4096
Die size (mm^2)	35	45	70	130	170	204	50
Pins/DRAM chip	16	16	18	20	54	66	134
Bandwidth (MBytes/s)	13	40	160	267	640	1600	27,000
Latency (ns)	225	170	125	75	62	52	30
Local area network	Ethernet	Fast Ethernet	Gigabit Ethernet	10 Gigabit Ethernet	100 Gigabit Ethernet	400 Gigabit Ethernet	
IEEE standard	802.3	803.3u	802.3ab	802.3ac	802.3ba	802.3bs	
Year	1978	1995	1999	2003	2010	2017	
Bandwidth (Mbits/seconds)	10	100	1000	10,000	100,000	400,000	
Latency (μs)	3000	500	340	190	100	60	
Hard disk	3600 RPM	5400 RPM	7200 RPM	10,000 RPM	15,000 RPM	15,000 RPM	
Product	CDC WrenI 94145-36	Seagate ST41600	Seagate ST15150	Seagate ST39102	Seagate ST373453	Seagate ST600MX0062	
Year	1983	1990	1994	1998	2003	2016	
Capacity (GB)	0.03	1.4	4.3	9.1	73.4	600	
Disk form factor	5.25 in.	5.25 in.	3.5 in.	3.5 in.	3.5 in.	3.5 in.	
Media diameter	5.25 in.	5.25 in.	3.5 in.	3.0 in.	2.5 in.	2.5 in.	
Interface	ST-412	SCSI	SCSI	SCSI	SCSI	SAS	
Bandwidth (MBytes/s)	0.6	4	9	24	86	250	
Latency (ms)	48.3	17.1	12.7	8.8	5.7	3.6	

Figure 1.10 Performance milestones over 25–40 years for microprocessors, memory, networks, and disks. The microprocessor milestones are several generations of IA-32 processors, going from a 16-bit bus, microcoded 80286 to a 64-bit bus, multicore, out-of-order execution, superpipelined Core i7. Memory module milestones go from 16-bit-wide, plain DRAM to 64-bit-wide double data rate version 3 synchronous DRAM. Ethernet advanced from 10 Mbits/s to 400 Gbits/s. Disk milestones are based on rotation speed, improving from 3600 to 15,000 RPM. Each case is best-case bandwidth, and latency is the time for a simple operation assuming no contention. Updated from Patterson, D., 2004. Latency lags bandwidth. Commun. ACM 47 (10), 71–75.

Energiagazdálkodás

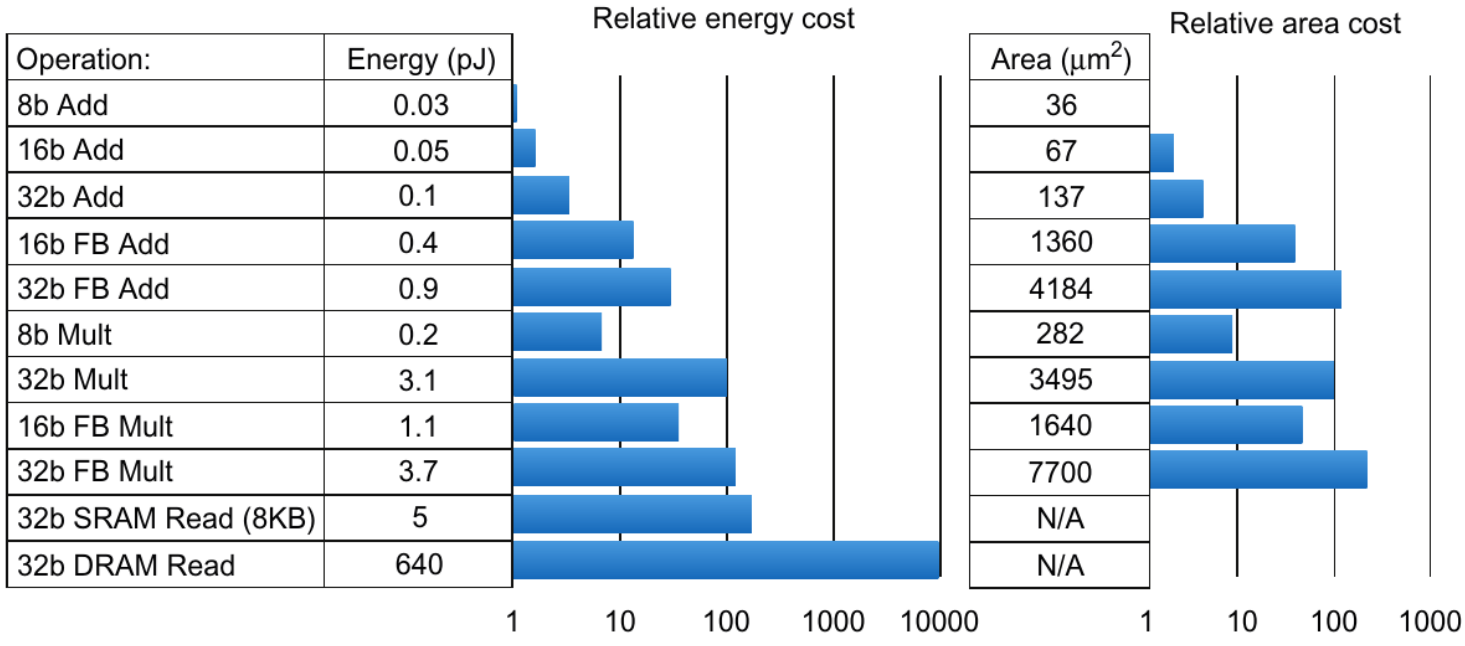
Egyre fontosabb szempont

A kurzus szempontjából nem elsődleges,

DE vannak olyan tényezők/technikák melyek nagyban befolyásolhatják a hatékonságot:

- funkcionális egységek lekapcsolása
- dinamikus frekvenicakezelés
- tipikus felhasználásra tervezve (pl. hűtés nem elég folyamatos teljes kihasználtságra)
- Turbo (overclocking)

Energy/area vs operands



Energy numbers are from Mark Horowitz *Computing's Energy problem (and what we can do about it)*. ISSCC 2014
Area numbers are from synthesized result using Design compiler under TSMC 45nm tech node. FP units used DesignWare Library.

Figure 1.13 Comparison of the energy and die area of arithmetic operations and energy cost of accesses to SRAM and DRAM. [Azizi][Dally]. Area is for TSMC 45 nm technology node.

Specifikáción túl

Mit jelent hogy egy architektúra hatékonyabb egy másiknál?

- MIPS (végrehajtási idő) egy feladatra?
- hány feladatot képes elvégezni adott idő alatt?
- milyen gyorsan válaszol?
- ...

Csak a konkrét program végrehajtási ideje alapján tudunk csak tényleg jó összehasonlítást kapni (ami ritkán áll rendelkezésre).

De a végrehajtási időt is lehet többféleképpen definiálni.

Benchmarking

Valódi programokkal tesztelni az architektúrákat.

Nem igazán jó: program mag, toy programs, szintetikus benchmark (erre a célra készült programok, sajnos sok ilyen elfogadott)

A programoknak nem szabad nagymértékben függeni a fordítási paraméterektől különböző architektúrákon.

Forráskód módosíthatósága szempontjából:

- nem megengedett,
- engedett, de gyakorlatilag lehetetlen (óriási benchmark)
- engedett, ha az eredmény nem sokban változik

Jó gyakorlat: benchmark suit (valódi programok gyüjteménye)

Benchmarking: eredmények kiértékelése

Megismételhetőség

Mérték kiválasztása
Cél általában: high performance, cost-performance

Összesítés:

- aritmetikai közép (hosszabb program lényegesebb lesz)
- súlyok használata (mi alapján válasszuk a súlyokat?)
- referencia használata (eredmények osztása a referenciaértékkel) $\rightarrow$ arányok
- geometriai közép (arányok közepe azonos a közepek arányával $\rightarrow$ a referenciagéptől független eredmények

Benchmarking: példa

Benchmarks	Sun Ultra Enterprise 2 time (seconds)	AMD A10-6800K time (seconds)	SPEC 2006Cint ratio	Intel Xeon E5-2690 time (seconds)	SPEC 2006Cint ratio	AMD/Intel times (seconds)	Intel/AMD SPEC ratios
perlbench	9770	401	24.36	261	37.43	1.54	1.54
bzip2	9650	505	19.11	422	22.87	1.20	1.20
gcc	8050	490	16.43	227	35.46	2.16	2.16
mcf	9120	249	36.63	153	59.61	1.63	1.63
gobmk	10,490	418	25.10	382	27.46	1.09	1.09
hmmer	9330	182	51.26	120	77.75	1.52	1.52
sjeng	12,100	517	23.40	383	31.59	1.35	1.35
libquantum	20,720	84	246.08	3	7295.77	29.65	29.65
h264ref	22,130	611	36.22	425	52.07	1.44	1.44
omnetpp	6250	313	19.97	153	40.85	2.05	2.05
astar	7020	303	23.17	209	33.59	1.45	1.45
xalancbmk	6900	215	32.09	98	70.41	2.19	2.19
Geometric mean			31.91		63.72	2.00	2.00

Figure 1.19 SPEC2006Cint execution times (in seconds) for the Sun Ultra 5—the reference computer of SPEC2006—and execution times and SPECRatios for the AMD A10 and Intel Xeon E5-2690. The final two columns show the ratios of execution times and SPEC ratios. This figure demonstrates the irrelevance of the reference computer in relative performance. The ratio of the execution times is identical to the ratio of the SPEC ratios, and the ratio of the geometric means (63.7231.91/20.86 = 2.00) is identical to the geometric mean of the ratios (2.00). Section 1.11 discusses libquantum, whose performance is orders of magnitude higher than the other SPEC benchmarks.

Összefoglalás: Hogyan legyünk hatékonyak?

- Párhuzamosság használata (ILP, DLP, TLP, RLP).
- Lokalitás elvének használata. (Temporal locality, spatial locality)
- Gyakran használt funkciók hatékonyabbak.
- Amdahl törvénye a gyorsulás becslésére.

Csapdák és tévhitek

- T Több processzor megold mindent.
- Cs Optimalizálni mérés nélkül. (Keveset futó rész optimlizálása majdnem fölösleges.)
- T Benchmark nem évül el.
- T Legjobb eredmény használata.

Csúcs és átlagos aránya

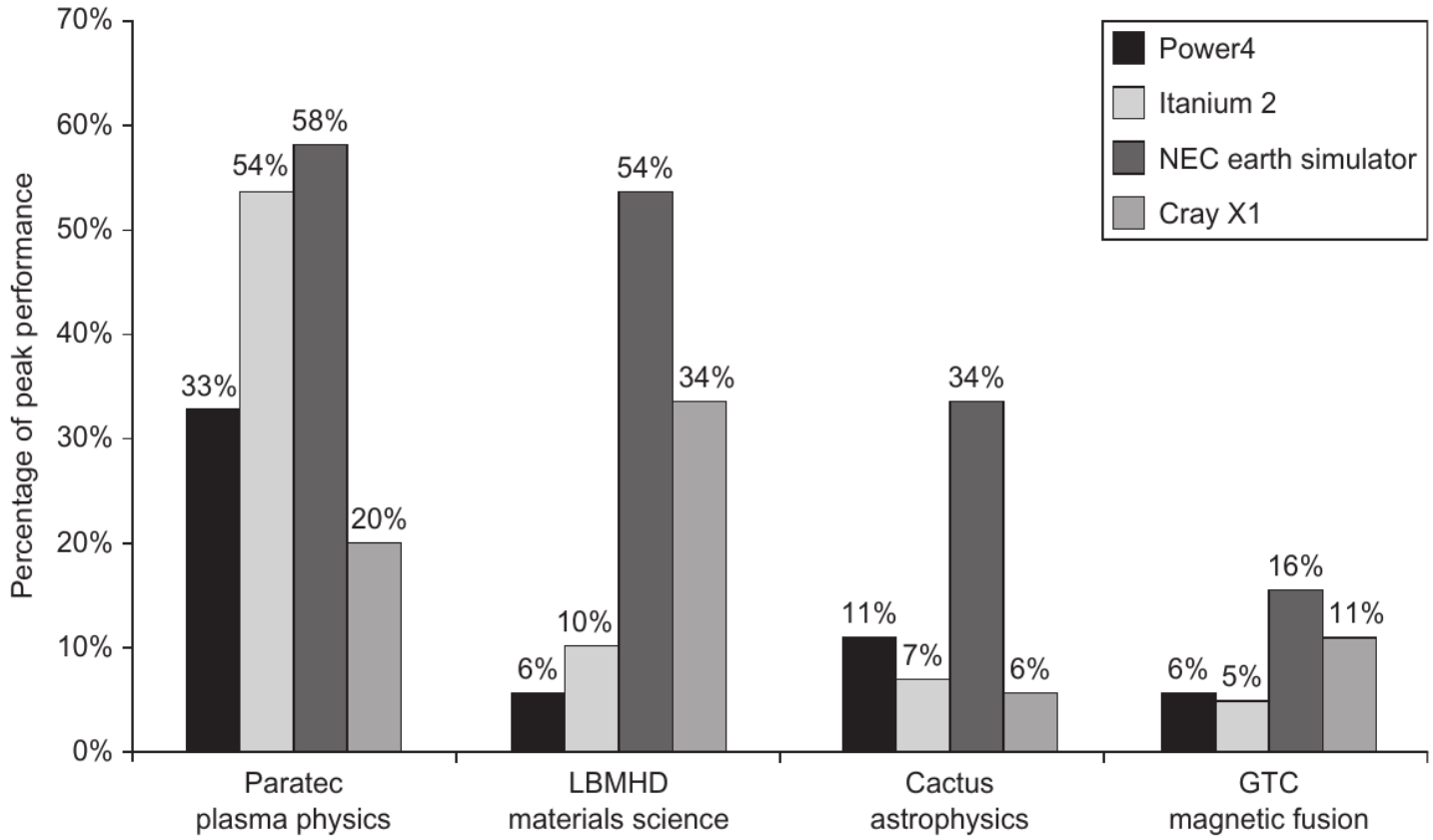


Figure 1.24 Percentage of peak performance for four programs on four multiprocessors scaled to 64 processors. The Earth Simulator and X1 are vector processors (see Chapter 4 and Appendix G). Not only did they deliver a higher fraction of peak performance, but they also had the highest peak performance and the lowest clock rates. Except for the Paratec program, the Power 4 and Itanium 2 systems delivered between 5% and 10% of their peak. From Oliker, L., Canning, A., Carter, J., Shalf, J., Ethier, S., 2004. Scientific computations on modern parallel vector systems. In: Proc. ACM/IEEE Conf. on Supercomputing, November 6–12, 2004, Pittsburgh, Penn., p. 10.

Nagy hatékonyságú számítások . . .
2. előadás (Memória)

Nagy Ádám
nagyadam888@gmail.com

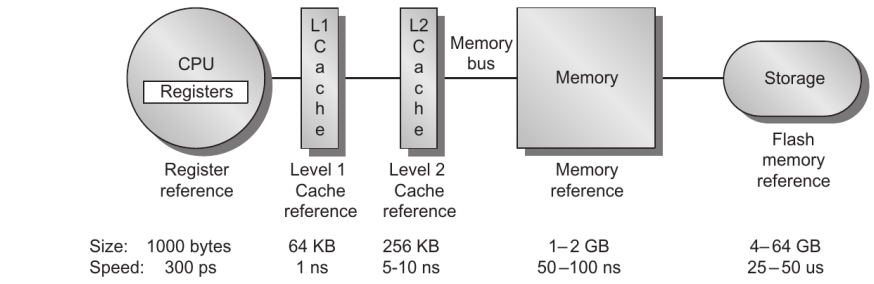
Eötvös Loránd Tudományegyetem
Informatikai Kar

2022. szeptember 21.

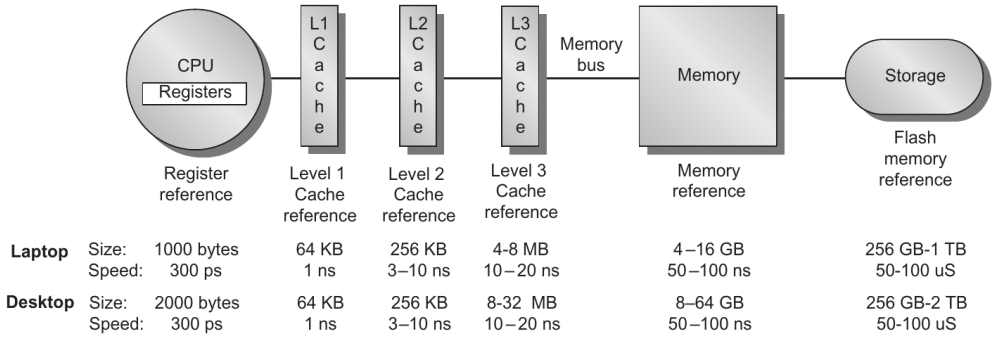
Ideally one would desire an indefinitely large memory capacity such that any particular… word would be immediately available… We are… forced to recognize the possibility of constructing a hierarchy of memories each of which has greater capacity than the preceding but which is less quickly accessible.

A. W. Burks, H. H. Goldstine, and J. von Neumann,
Preliminary Discussion of the Logical Design of an Electronic Computing Instrument (1946).

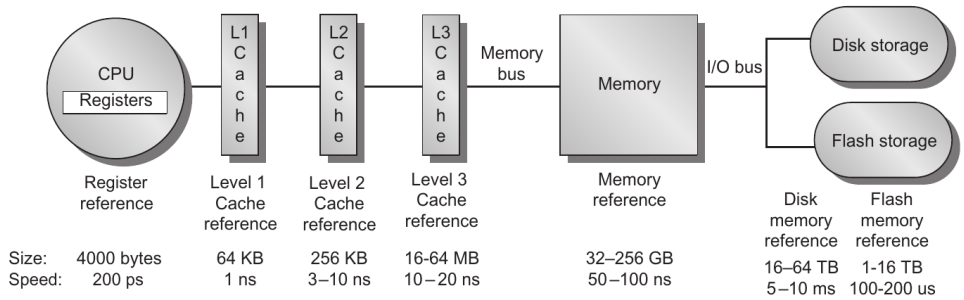
Memória hierarchia



(A) Memory hierarchy for a personal mobile device

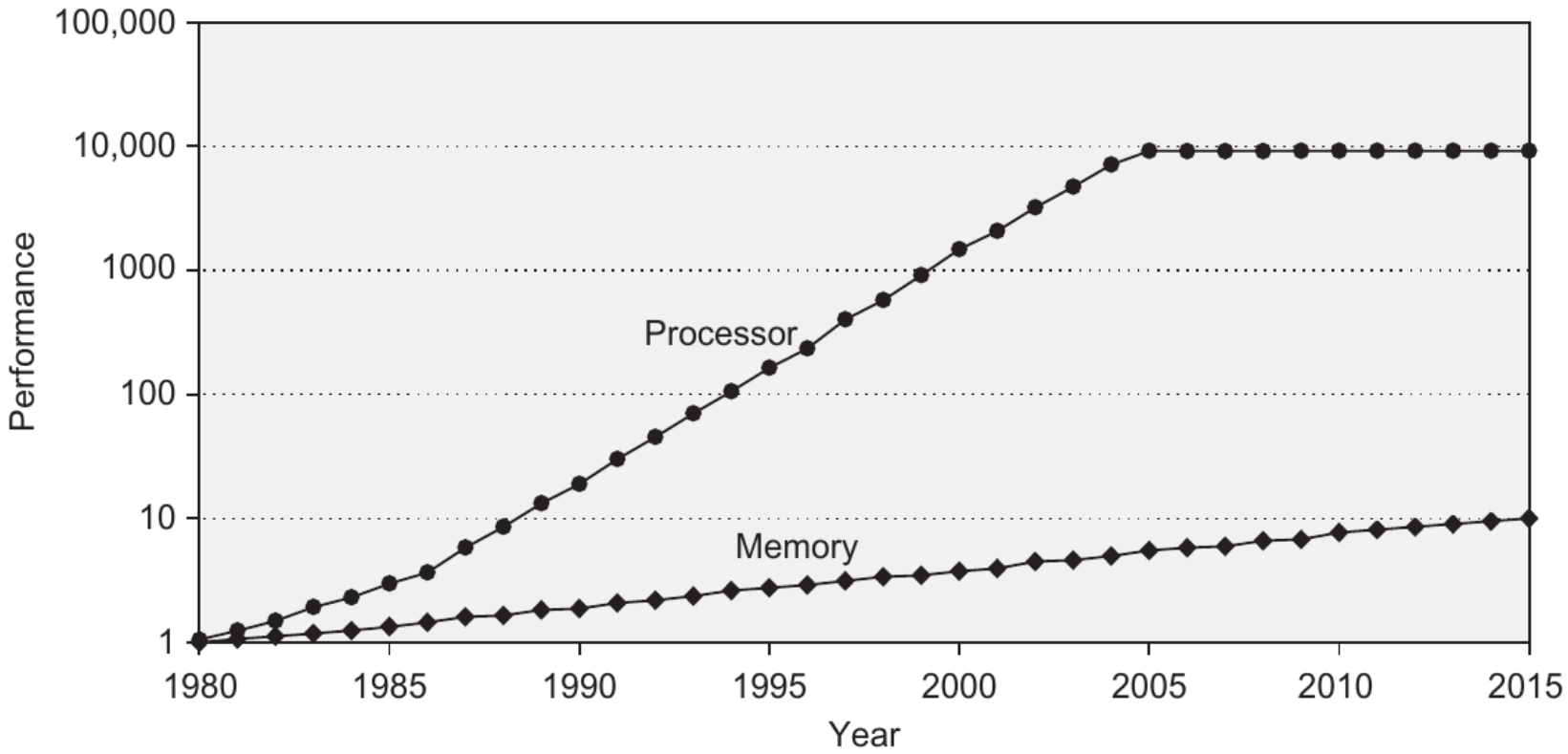


(B) Memory hierarchy for a laptop or a desktop



(C) Memory hierarchy for server

Memória (RAM) és CPU (1 db) sebessége



Alapfogalmak, problémák

Valójában az igény még ennél is nagyobb: több processzor, pipeline, többszörös kibocsájtás, spekuláció. (i7 (4mag) 12.8 bill. 128 bit referencia = 409.6 Gb/s)

De a nagy (kihasználatlan) cache sok erőforrást igényel.

Metrikák: **access time, miss rate, miss penalty**

Leképzés a memóraszintek között és **asszociativitás** (direct $\rightarrow$ full)

Leképezés időben: **write-through, write back, . . .**

Cache - Hová kerül postosan egy blokk?

- **Direkt** Minden blokknak megvan, hogy hova kerülhet. Általában blokk címe/blokkok száma a cache-ben.
- **Teljesen asszociatív** Bárhová kerülhet minden blokk a cache-be.
- **Set associative/*n*-way set associative** Egy blokk egy *n* elemű halmaz bármely helyére kerülhet.

Figure B.3 The three portions of an address in a set associative or direct-mapped cache. The tag is used to check all the blocks in the set, and the index is used to select the set. The block offset is the address of the desired data within the block. Fully associative caches have no index field.

Cache - Cache miss esetén melyik legyen eldobva?

Direkt leképezés esetén nincs döntési kényszer.

Általában 3 (lap/blokk)kezelési stratégia közül az egyik:

- **Random** egyszerű hardware
- **LRU** bonyolult már kis méretű halmazok esetén is $\rightarrow$ **pseudo-LRU**
- **FIFO**

	Associativity								
	Two-way			Four-way			Eight-way		
Size	LRU	Random	FIFO	LRU	Random	FIFO	LRU	Random	FIFO
16 KiB	114.1	117.3	115.5	111.7	115.1	113.3	109.0	111.8	110.4
64 KiB	103.4	104.3	103.9	102.4	102.3	103.1	99.7	100.5	100.3
256 KiB	92.2	92.1	92.5	92.1	92.1	92.5	92.1	92.1	92.5

Figure B.4 Data cache misses per 1000 instructions comparing least recently used, random, and first in, first out replacement for several sizes and associativities. There is little difference between LRU and random for the largest size cache, with LRU outperforming the others for smaller caches. FIFO generally outperforms random in the smaller cache sizes. These data were collected for a block size of 64 bytes for the Alpha architecture using 10 SPEC2000 benchmarks. Five are from SPECint2000 (gap, gcc, gzip, mcf, and perl) and five are from SPECfp2000 (applu, art, equake, lucas, and swim). We will use this computer and these benchmarks in most figures in this appendix.

Cache - Mi történik írás esetén?

Igazából az írás csak negyede-harmada a memóriaelérésnek.

Írás lassabb mint az olvasás találat esetén is és blokkon belül is el kell dönteni mi volt írva.

Írási stratégiák:

- **Write-through** egyszerre kerül be a cache-be és az alacsonabb szintre. Egyszerűbb, koherens memóriát eredményez, terheli az adatsineket akkor is ha ugyanazt az adatot többször írjuk (nem mindig a legalsó szintig történik a propagáció). *Write stall* ha várakozni kell emiatt.
- **Write-back** Csak akkor kerül vissza a memóriába ha kell (dirty bit).

Write buffer: write stall ,,elsimítására".
Write miss esetén nincs szükség az elöző adatra így több lehetőség van:

- *Write allocate*: ugyanugy viselkedik mint read miss esetén
- *No-write allocate*: egyből az alsóbb szinteken történik az írás.

Miss rate

Három kategória:

- Compulsory (nem javítható)
- Capacity (eldobott majd újra bekért)
- Conflict (Ha nem teljesen asszociatív.)

Nem túl beszédes mérték, össz kell vetni az utasítások számával és a memóriaelérések arányával.

Cache optimalizációi I.

Átlagos memóriaelérési idő = hit time + miss rate × miss penalty

Miss rate csökkentése:

- *Nagyobb blokkméret*: lokalitás kihasználása, 1C csökken, 2-3C nőhet (kis cache), miss penalty növekszik
- *Nagyobb cache*: hit time nőhet, több erőforrás
- *Nagyobb asszociativitás*: 3C csökken, hit time növekszik, több erőforrás

Miss penalty csökkentése:

- *Több szint*: . . .
- *Olvasás prioritást élvez (írással szemben)*: write bufferben (majdnem minden proc)

Hit time csökkentése: címfordítás elkerülése/egyszerűsítése

Miss rate vs blokk méret

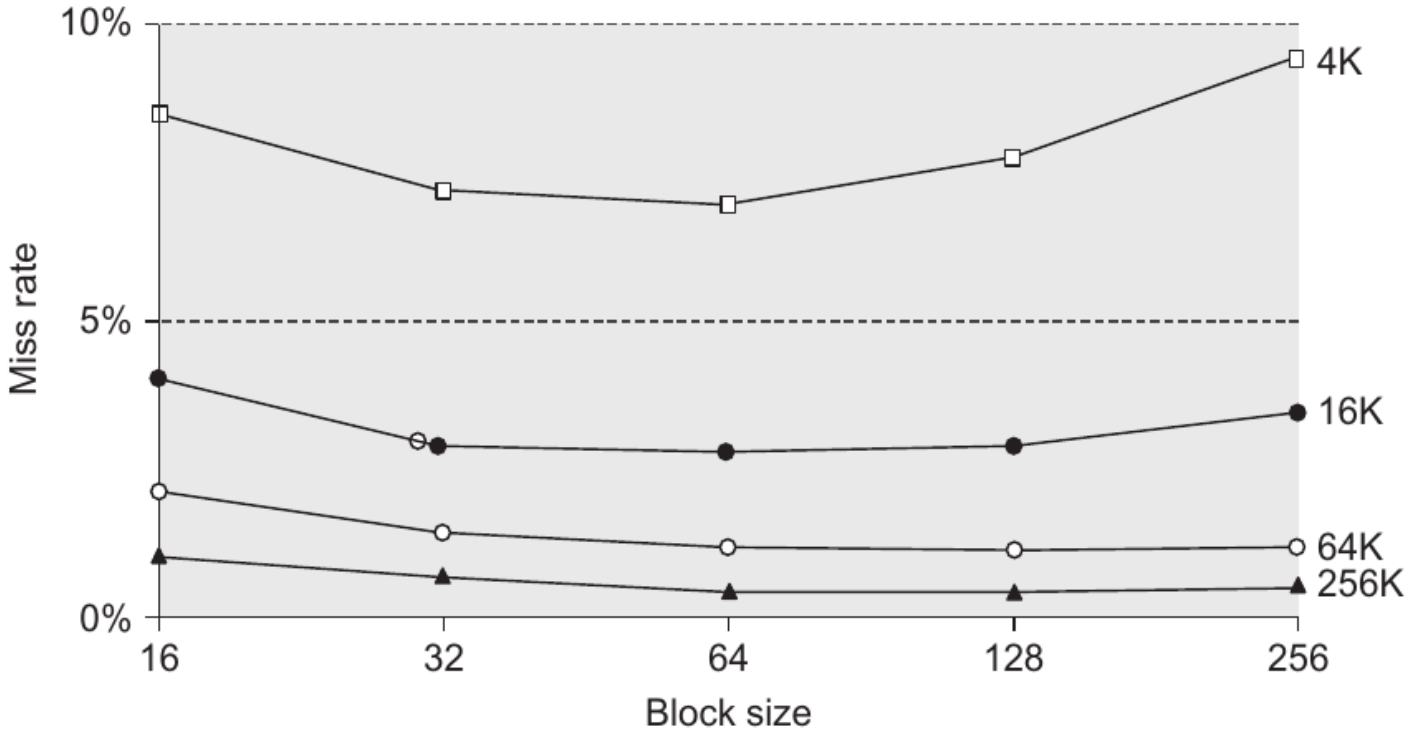


Figure B.10 Miss rate versus block size for five different-sized caches. Note that miss rate actually goes up if the block size is too large relative to the cache size. Each line represents a cache of different size. Figure B.11 shows the data used to plot these lines. Unfortunately, SPEC2000 traces would take too long if block size were included, so these data are based on SPEC92 on a DECstation 5000 (Gee et al. 1993).

Access time vs cache size

		Cache size			
Block size	Miss penalty	4K	16K	64K	256K
16	82	8.027	4.231	2.673	1.894
32	84	**7.082**	3.411	2.134	1.588
64	88	7.160	**3.323**	**1.933**	**1.449**
128	96	8.469	3.659	1.979	1.470
256	112	11.651	4.685	2.288	1.549

Figure B.12 Average memory access time versus block size for five different-sized caches in Figure B.10. Block sizes of 32 and 64 bytes dominate. The smallest average time per cache size is boldfaced.

Access time vs asszociativitás

	Associativity			
Cache size (KiB)	1-way	2-way	4-way	8-way
4	3.44	3.25	3.22	**3.28**
8	2.69	2.58	2.55	**2.62**
16	2.23	**2.40**	**2.46**	**2.53**
32	2.06	**2.30**	**2.37**	**2.45**
64	1.92	**2.14**	**2.18**	**2.25**
128	1.52	**1.84**	**1.92**	**2.00**
256	1.32	**1.66**	**1.74**	**1.82**
512	1.20	**1.55**	**1.59**	**1.66**

Figure B.13 Average memory access time using miss rates in Figure B.8 for parameters in the example. *Boldface* type means that this time is higher than the number to the left, that is, higher associativity *increases* average memory access time.

Miss rate vs cache méret (multilevel)

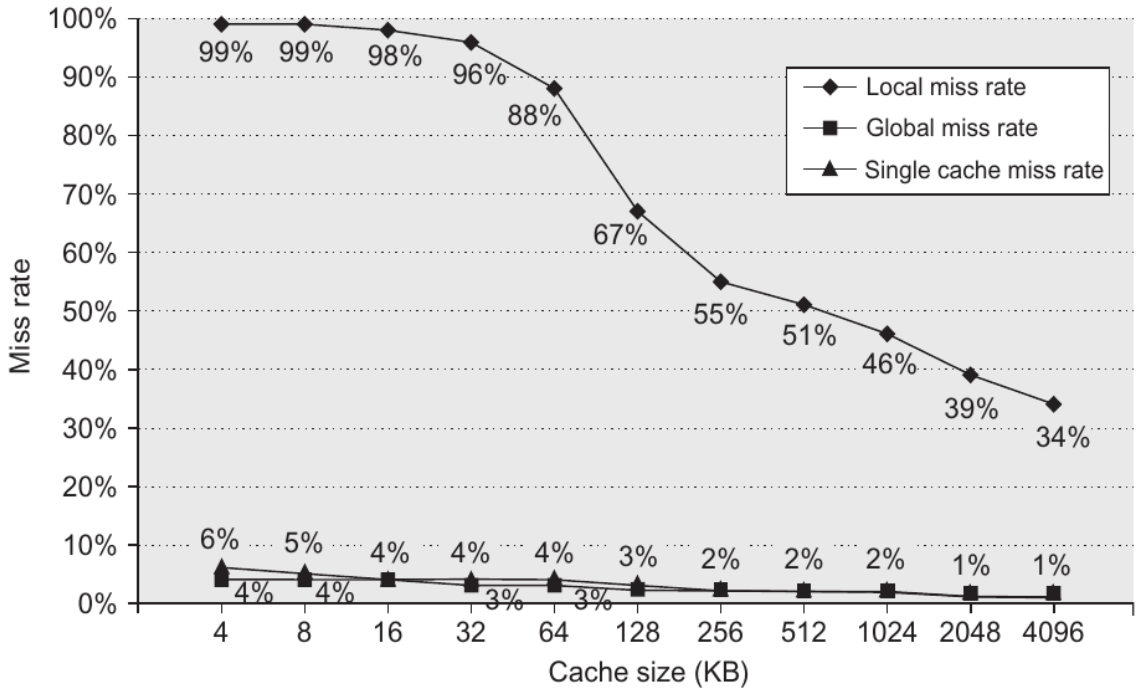


Figure B.14 Miss rates versus cache size for multilevel caches. Second-level caches *smaller* than the sum of the two 64 KiB first-level caches make little sense, as reflected in the high miss rates. After 256 KiB the single cache is within 10% of the global miss rates. The miss rate of a single-level cache versus size is plotted against the local miss rate and global miss rate of a second-level cache using a 32 KiB first-level cache. The L2 caches (unified) were two-way set associative with replacement. Each had split L1 instruction and data caches that were 64 KiB two-way set associative with LRU replacement. The block size for both L1 and L2 caches was 64 bytes. Data were collected as in Figure B.4.

Elérési idő

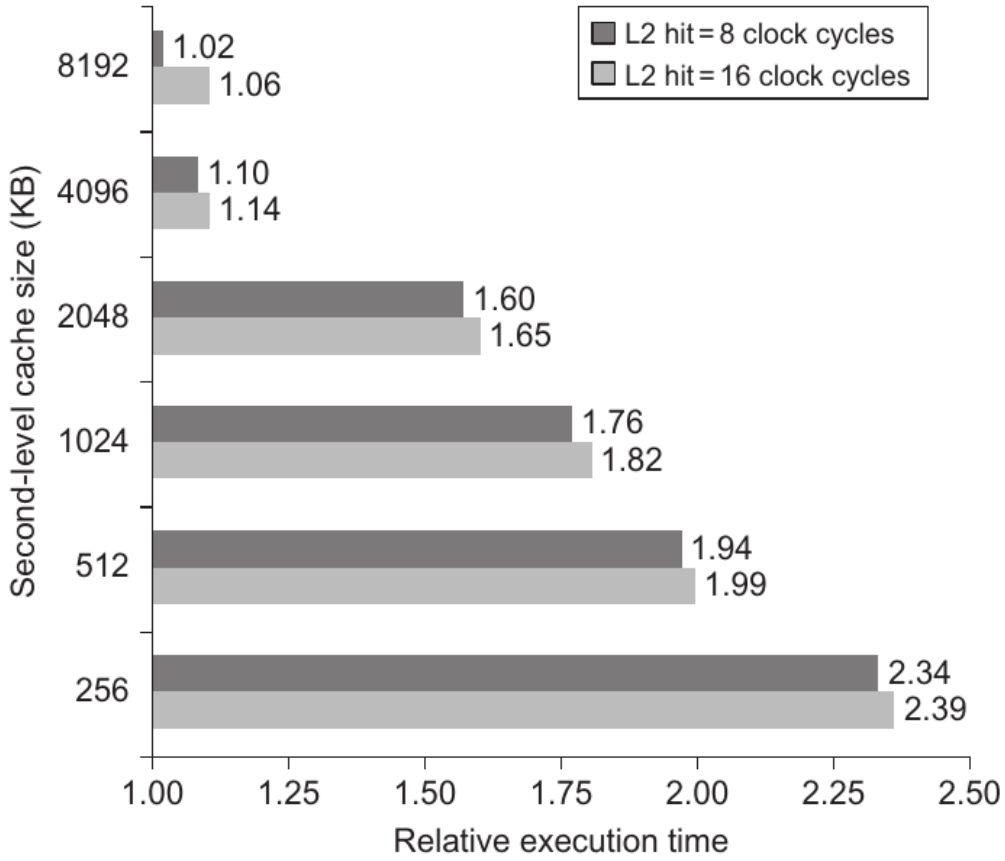


Figure B.15 Relative execution time by second-level cache size. The two bars are for different clock cycles for an L2 cache hit. The reference execution time of 1.00 is for an 8192 KiB second-level cache with a 1-clock-cycle latency on a second-level hit. These data were collected the same way as in Figure B.14, using a simulator to imitate the Alpha 21264.

Gyakorlatban

Technique	Hit time	Miss penalty	Miss rate	Hardware complexity	Comment
Larger block size		–	+	0	Trivial; Pentium 4L2 uses 128 bytes
Larger cache size	–		+	1	Widely used, especially for L2 caches
Higher associativity	–		+	1	Widely used
Multilevel caches		+		2	Costly hardware; harder if L1 block size $\neq$ L2 block size; widely used
Read priority over writes		+		1	Widely used
Avoiding address translation during cache indexing	+			1	Widely used

Figure B.18 Summary of basic cache optimizations showing impact on cache performance and complexity for the techniques in this appendix. Generally a technique helps only one factor. + means that the technique improves the factor, – means it hurts that factor, and blank means it has no impact. The complexity measure is subjective, with 0 being the easiest and 3 being a challenge.

Cache optimalizáció II. (advanced)

1. Egyszerű és kicsi L1 $\rightarrow$ hit time és power consumption csökken
2. Way prediction $\rightarrow$ hit time csökken (extra bits, ICache 9/10, DCache 8/10 pontosság)
3. Pipelined Cache elérés $\rightarrow$ sávszélesség nő (hit time csökkenhet ha nem jó a joslás), jó a asszociativitásnak
4. Nonblocking caches $\rightarrow$ sávszélesség nő (az L1 cache miss elrejthető, de az alacsonyabb szinteken korlátozott)
5. Multibanked caches $\rightarrow$ sávszélesség növekszik

Cache optimalizáció II. (advanced)

6. Critical Word firs, Early restart $\rightarrow$ miss penalty csökken (nem várjuk meg amíg a teljes blokk megérkezik)
7. Merging write buffer $\rightarrow$ miss penalty csökken
8. Compiler optimalizáció (loop interchange, blocking, . . .)
9. Hardware prefetch (ha már a sávszélesség jó nagy)
10. Compiler/user prefetch (register/cache prefetch, faulting/nonfaulting)

Non-blocking cache

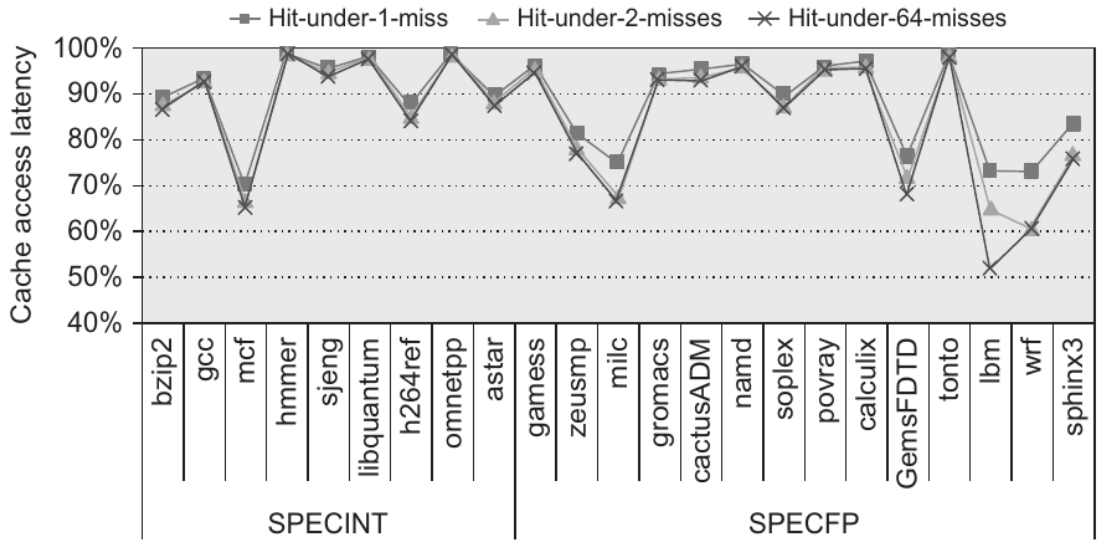


Figure 2.11 The effectiveness of a nonblocking cache is evaluated by allowing 1, 2, or 64 hits under a cache miss with 9 SPECINT (on the left) and 9 SPECFP (on the right) benchmarks. The data memory system modeled after the Intel i7 consists of a 32 KiB L1 cache with a four-cycle access latency. The L2 cache (shared with instructions) is 256 KiB with a 10-clock cycle access latency. The L3 is 2 MiB and a 36-cycle access latency. All the caches are eight-way set associative and have a 64-byte block size. Allowing one hit under miss reduces the miss penalty by 9% for the integer benchmarks and 12.5% for the floating point. Allowing a second hit improves these results to 10% and 16%, and allowing 64 results in little additional improvement.

Hardware prefetch

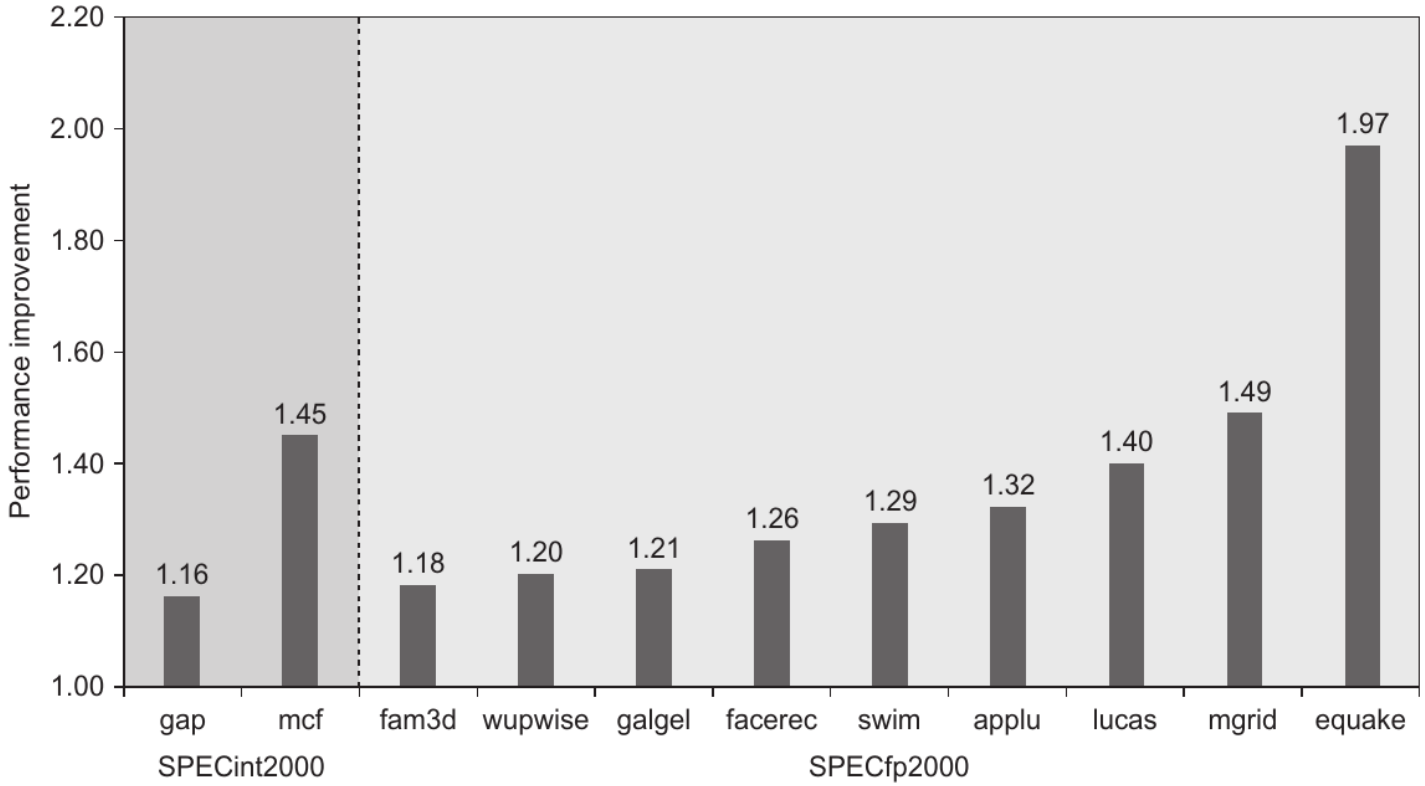


Figure 2.15 Speedup because of hardware prefetching on Intel Pentium 4 with hardware prefetching turned on for 2 of 12 SPECint2000 benchmarks and 9 of 14 SPECfp2000 benchmarks. Only the programs that benefit the most from prefetching are shown; prefetching speeds up the missing 15 SPECCPU benchmarks by less than 15% (Boggs et al., 2004).

Összefoglalás

Technique	Hit time	Band-width	Miss penalty	Miss rate	Power consumption	Hardware cost/ complexity	Comment
Small and simple caches	+			–	+	0	Trivial; widely used
Way-predicting caches	+				+	1	Used in Pentium 4
Pipelined & banked caches	–	+				1	Widely used
Nonblocking caches		+	+			3	Widely used
Critical word first and early restart			+			2	Widely used
Merging write buffer			+			1	Widely used with write through
Compiler techniques to reduce cache misses				+		0	Software is a challenge, but many compilers handle common linear algebra calculations
Hardware prefetching of instructions and data			+	+	–	2 instr., 3 data	Most provide prefetch instructions; modern high-end processors also automatically prefetch in hardware
Compiler-controlled prefetching			+	+		3	Needs nonblocking cache; possible instruction overhead; in many CPUs
HBM as additional level of cache		+/–	–	+	+	3	Depends on new packaging technology. Effects depend heavily on hit rate improvements

Figure 2.18 Summary of 10 advanced cache optimizations showing impact on cache performance, power consumption, and complexity. Although generally a technique helps only one factor, prefetching can reduce misses if done sufficiently early; if not, it can reduce miss penalty. + means that the technique improves the factor, – means it hurts that factor, and blank means it has no impact. The complexity measure is subjective, with 0 being the easiest and 3 being a challenge.

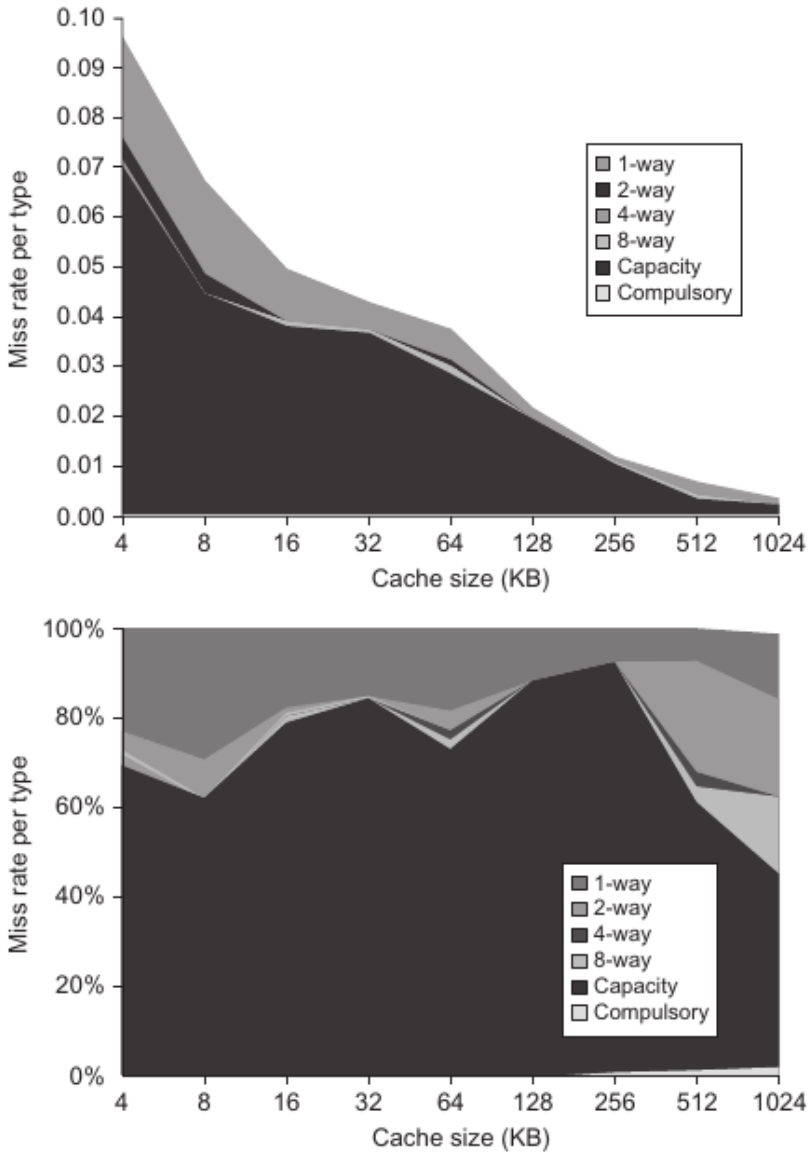
Nagy hatékonyságú számítások . . .
3. előadás (Memória II)

Nagy Ádám
nagyadam888@gmail.com

Eötvös Loránd Tudományegyetem
Informatikai Kar

2022. október 4.

Miss rate vs Cache size



Emlékeztető

cache
virtual memory
memory stall cycles
direct mapped
valid bit
block address
write through
instruction cache
average memory access time
cache hit
page
miss penalty
fully associative
dirty bit
block offset
write back
data cache
hit time
cache miss
page fault
miss rate
n-way set associative
least recently used
tag field
write allocate
unified cache
misses per instruction
block
locality
address trace
set
random replacement
index field
no-write allocate
write buffer
write stall

Virtuális címzés

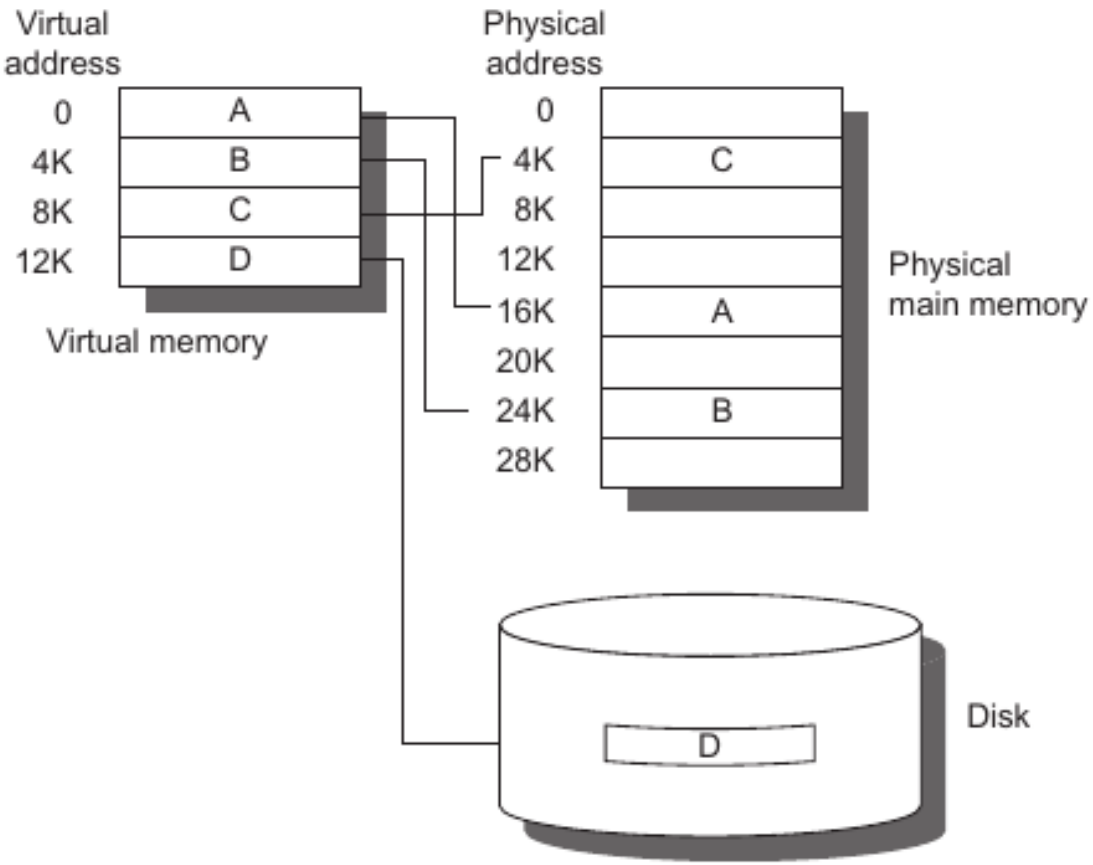


Figure B.19 The logical program in its contiguous virtual address space is shown on the left. It consists of four pages, A, B, C, and D. The actual location of three of the blocks is in physical main memory and the other is located on the disk.

Virtuális címzés

Mire jó?

- Nagy programok is elférhetnek kis memóriában.
- Védelem programok/eljárások között.
- A programozó számára egy program memóriája egységes.
- Egyszerűbb porgramhívás (nem kell a teljes programmemória és relocation regiszter sem kell)

A processzor virtuális címekkel számol $\Rightarrow$ címfordítóra van szükség (hardware + software).

Manapság két szint: Cache (processzor) $\leftrightarrow$ DRAM, memória $\leftrightarrow$ merevlemez.

Virtualizáció

Parameter	First-level cache	Virtual memory
Block (page) size	16–128 bytes	4096–65,536 bytes
Hit time	1–3 clock cycles	100–200 clock cycles
Miss penalty	8–200 clock cycles	1,000,000–10,000,000 clock cycles
(access time)	(6–160 clock cycles)	(800,000–8,000,000 clock cycles)
(transfer time)	(2–40 clock cycles)	(200,000–2,000,000 clock cycles)
Miss rate	0.1%–10%	0.00001%–0.001%
Address mapping	25–45-bit physical address to 14–20-bit cache address	32–64-bit virtual address to 25–45-bit physical address

Figure B.20 Typical ranges of parameters for caches and virtual memory. Virtual memory parameters represent increases of 10–1,000,000 times over cache parameters. Usually, first-level caches contain at most 1 MiB of data, whereas physical memory contains 256 MiB to 1 TB.

Egyiket OS, másikat hardware vezérli, így a kivalásztás bonyolultabb de pontosabb az OS esetében (tévedés drága).

Segments and pages

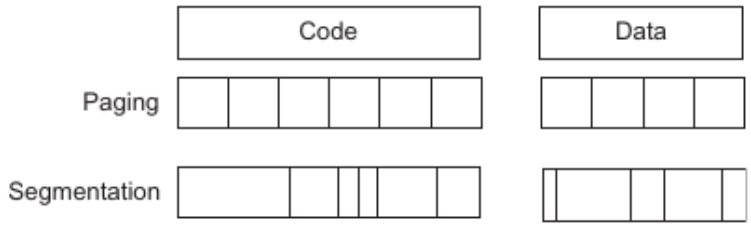


Figure B.21 Example of how paging and segmentation divide a program.

	Page	Segment
Words per address	One	Two (segment and offset)
Programmer visible?	Invisible to application programmer	May be visible to application programmer
Replacing a block	Trivial (all blocks are the same size)	Difficult (must find contiguous, variable-size, unused portion of main memory)
Memory use inefficiency	Internal fragmentation (unused portion of page)	External fragmentation (unused pieces of main memory)
Efficient disk traffic	Yes (adjust page size to balance access time and transfer time)	Not always (small segments may transfer just a few bytes)

Figure B.22 Paging versus segmentation. Both can waste memory, depending on the block size and how well the segments fit together in main memory. Programming languages with unrestricted pointers require both the segment and the address to be passed. A hybrid approach, called *paged segments*, shoots for the best of both worlds: segments are composed of pages, so replacing a block is easy, yet a segment may be treated as a logical unit.

Page table

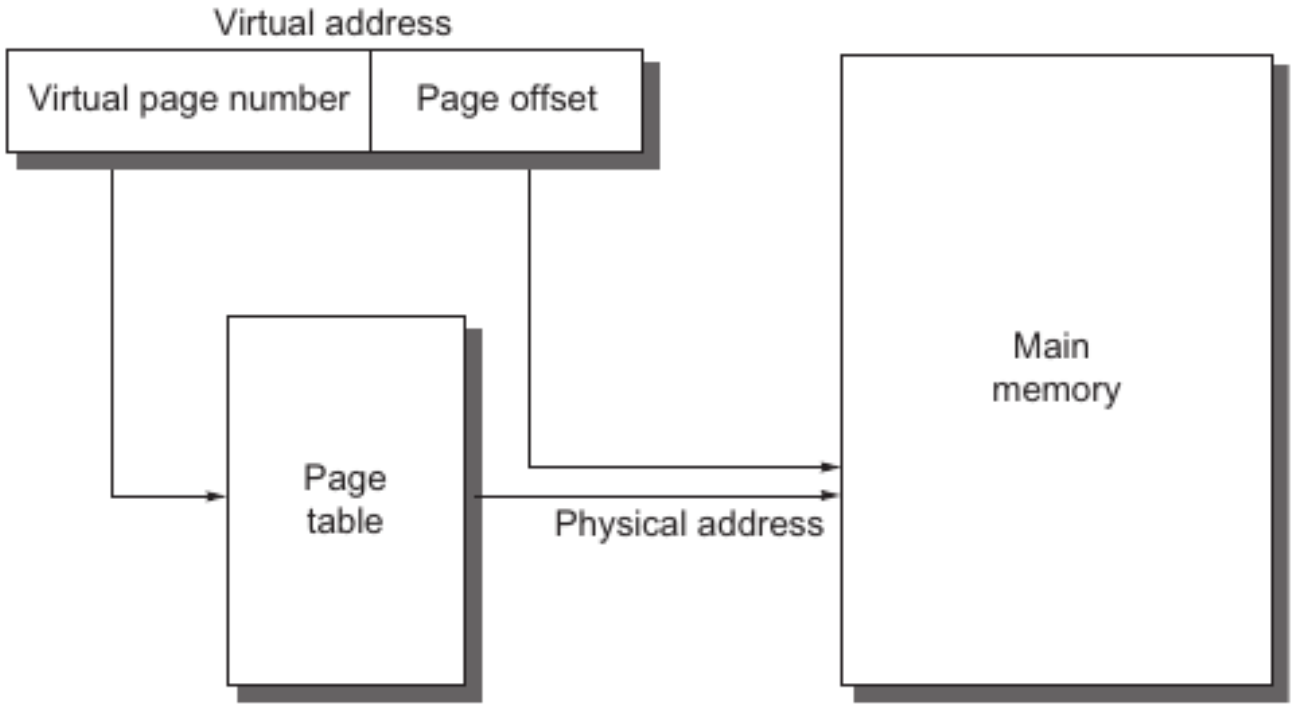


Figure B.23 The mapping of a virtual address to a physical address via a page table.

Más technika: TLB (cache), inverted hash table (kisebb tábla)

Translation lookaside buffer – TLB

A page table nagyon nagy, sokszor maga is lapozott (2xtime).

Lokalitás kihasználásával megéri a már jelen lévő címeket eltárolni egy cache-ben (TLB).

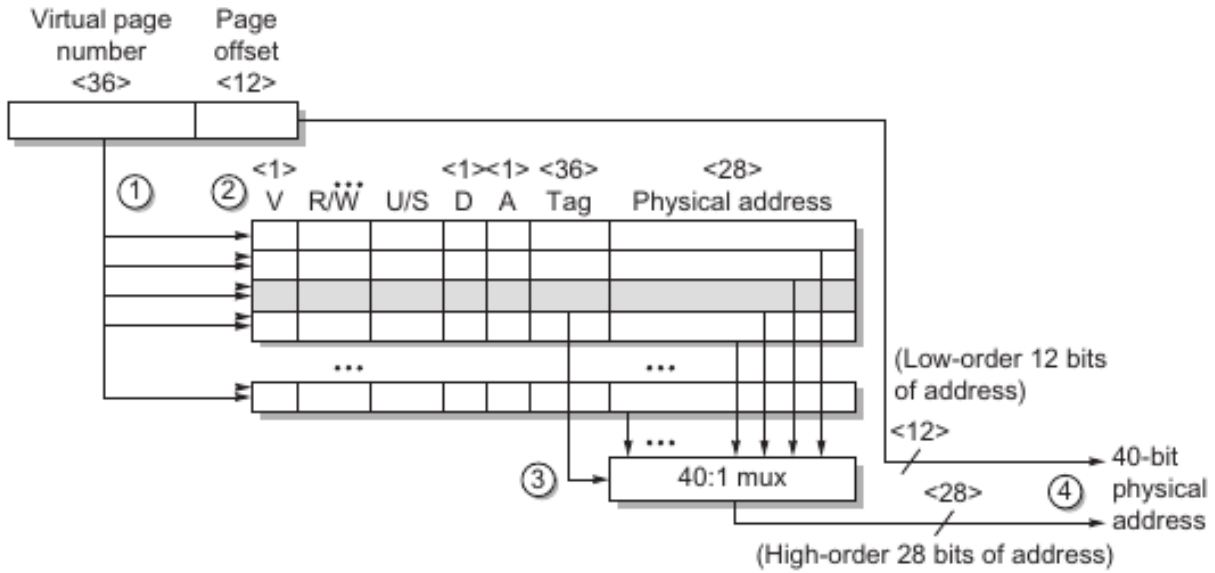


Figure B.24 Operation of the Opteron data TLB during address translation. The four steps of a TLB hit are shown as *circled numbers*. This TLB has 40 entries. Section B.5 describes the various protection and access fields of an Opteron page table entry.

Lapméret

1. Nagyobb lapméret $\Rightarrow$ kisebb laptábla (egyszerűbb leképezés és kezelés)
2. Nagyobb cachnél kis lapméret esetén a hit time nőhet.
3. Nagyobb lapméretet $\Rightarrow$ hatékonyabb transzfer
4. Nagyobb lapméret esetén a TLB miss csökken (kevesebbet kell használni).
5. Nagyobb lapméret pazarló lehet, különösen ha az utlsó lap csak alig kitöltött.

Példa

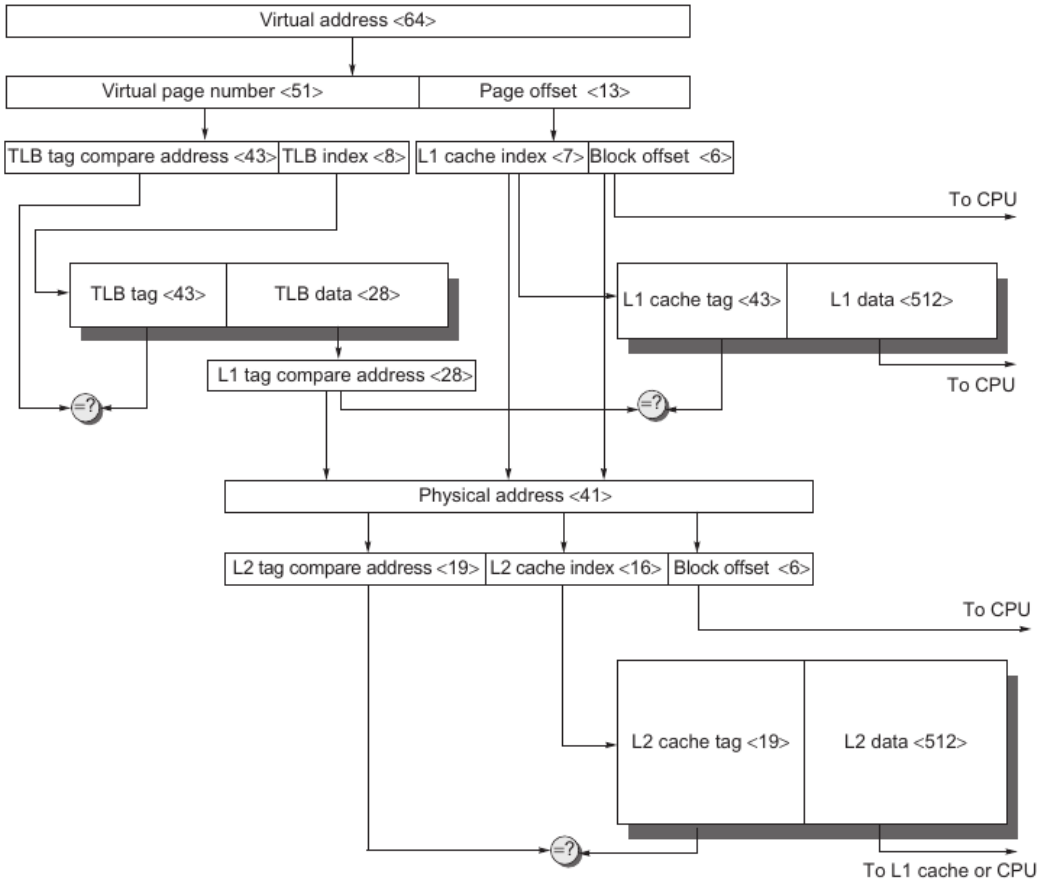


Figure B.25 The overall picture of a hypothetical memory hierarchy going from virtual address to L2 cache access. The page size is 8 KiB. The TLB is direct mapped with 256 entries. The L1 cache is a direct-mapped 8 KiB, and the L2 cache is a direct-mapped 4 MiB. Both use 64-byte blocks. The virtual address is 64 bits and the physical address is 41 bits. The primary difference between this simple figure and a real cache is replication of pieces of this figure.

Main memory

Dimenziók

- **latency** (cach miss penalty)
- **bandwidth** (multiprocesszor, I/O, nagy blokkméretű L3)

Latency csökkentése nehéz, a sávszélesség növelése könnyű.
Feladat a sávszélesség kihasználása.

$$\Downarrow$$

Új technikákra/technológiákra van szükség;
és új mértékre (a régiek mellett):

- access time (az adat megérkezéséig eltelt idő read után)
- cycle time (idő két független memóriaelérés között)

Memóriatípusok

1. SRAM (Static RAM)
   - 6 tranzisztor/byte, power efficient
   - Nincs szükség újraírásra (access time $\approx$ cycle time)
   - Max. 10-20 Mb, 3-5x gyorsabb DRAM-nál
   - Cache készül ilyen típusból
2. DRAM (Dynamic RAM)
   - 1 tranzisztor/byte
   - Olvasáskor elveszik a tartalom (újra kell írni)
   - Időnként újra ki kell olvasni és vissza kell írni (hardware, stalling, cycle time nő)
   - Az adatok mátrix szerűen helyezkednek el
   - Bankok használata

DRAM felépítése

Négyzetes szerkezet. $\Rightarrow$ A címek két részre vannak ,,multiplexálva".

- row access strobe (RAS) - latency
- column acces strobe (CAS) - bandwith

Szerencsére a frissítés is soronként történik. $\Rightarrow$ Ideje a méret négyzetgyökével arányos és általában nem több mint a használat 5%-a.

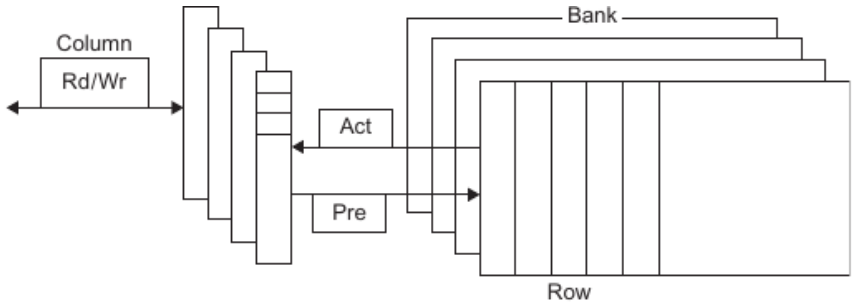


Figure 2.3 Internal organization of a DRAM. Modern DRAMs are organized in banks, up to 16 for DDR4. Each bank consists of a series of rows. Sending an ACT (Activate) command opens a bank and a row and loads the row into a row buffer. When the row is in the buffer, it can be transferred by successive column addresses at whatever the width of the DRAM is (typically 4, 8, or 16 bits in DDR4) or by specifying a block transfer and the starting address. The Precharge commend (PRE) closes the bank and row and readies it for a new access. Each command, as well as block transfers, are synchronized with a clock. See the next section discussing SDRAM. The row and column signals are sometimes called RAS and CAS, based on the original names of the signals.

DIMM (dual inline memory modules) - 4-16 DRAM, általában 8 byte széles

DRAM optimalizáció

Általában a bandwidth (néha a latency kárára is).
A sorok egy Row buffer segítségével érhetőek el.

1. Time signal több oszlop kiolvasására egy soron belül.
   Mivel az olvasás a sorbufferből történik, RAS nem kell.
2. **SDRAM** (Synchronized DRAM)
   - Clock signal hozzáadása DRAM interface-hez.
   - Programozható regiszter burst adatforgalomhoz a clock-hoz ütemezve. (critical word first)
3. Szélesebb adatforgalom (nagyobb sűrűség miatt).
4. **DDR DRAM** (Double Data Rate DRAM)
   Le- és a felmenő (DRAM) órajel esetén is van adatforgalom.
   (2xbandwidth)
5. Memóriabankok használata. $\Rightarrow$ Memóriaszerkezet téglatest.
   Új bank megnyitása némi késleltetést jelenthet.

Standard	I/O clock rate	M transfers/s	DRAM name	MiB/s/DIMM	DIMM name
DDR1	133	266	DDR266	2128	PC2100
DDR1	150	300	DDR300	2400	PC2400
DDR1	200	400	DDR400	3200	PC3200
DDR2	266	533	DDR2-533	4264	PC4300
DDR2	333	667	DDR2-667	5336	PC5300
DDR2	400	800	DDR2-800	6400	PC6400
DDR3	533	1066	DDR3-1066	8528	PC8500
DDR3	666	1333	DDR3-1333	10,664	PC10700
DDR3	800	1600	DDR3-1600	12,800	PC12800
DDR4	1333	2666	DDR4-2666	21,300	PC21300

Figure 2.5 Clock rates, bandwidth, and names of DDR DRAMS and DIMMs in 2016. Note the numerical relationship between the columns. The third column is twice the second, and the fourth uses the number from the third column in the name of the DRAM chip. The fifth column is eight times the third column, and a rounded version of this number is used in the name of the DIMM. DDR4 saw significant first use in 2016.

Graphics Data RAM

Speciális DRAM modul még nagyobb sávszélességgeli (2-10x).

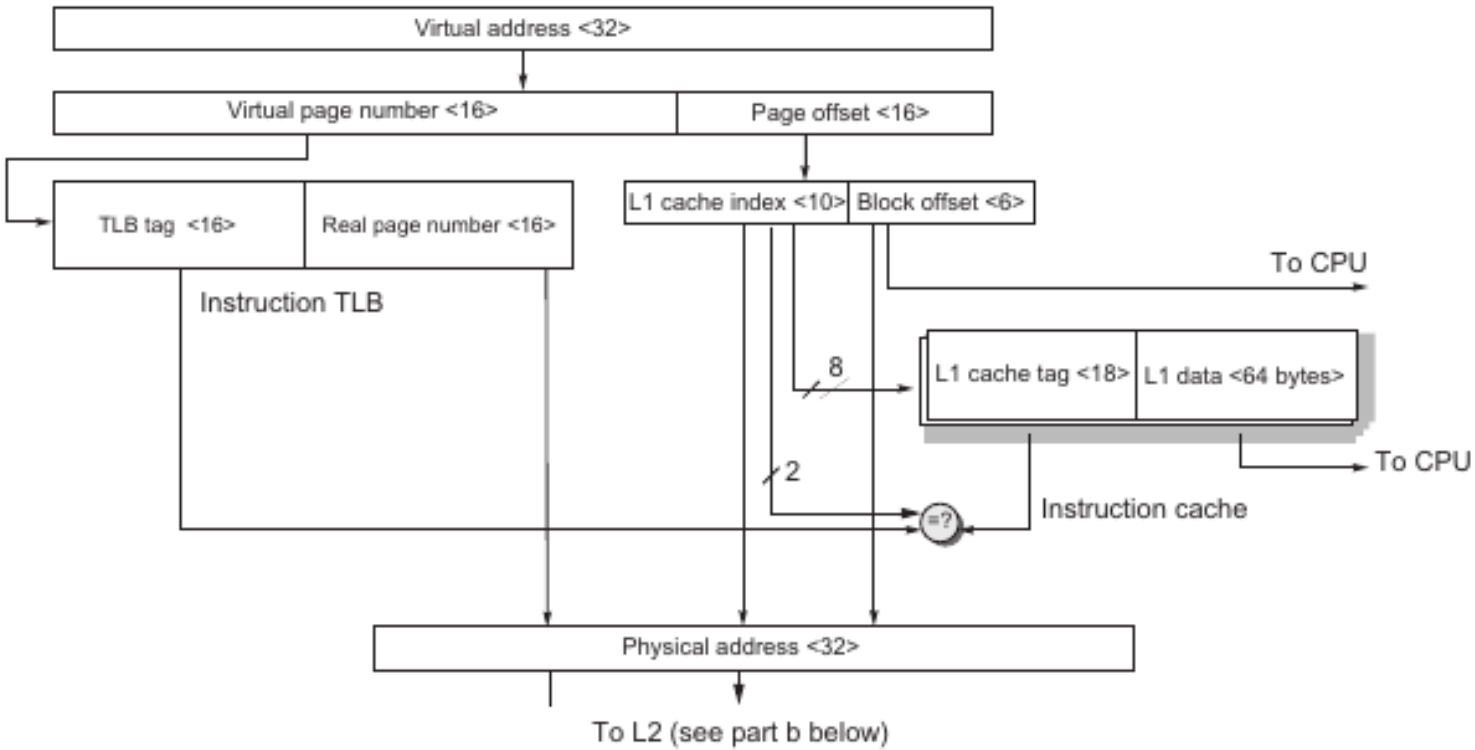
Különbségek:

- Szélesebb interface $\geq$ 32 bit vs 4,8,16 bit
- Magasabb órajel és közvetlen kapcsolat.

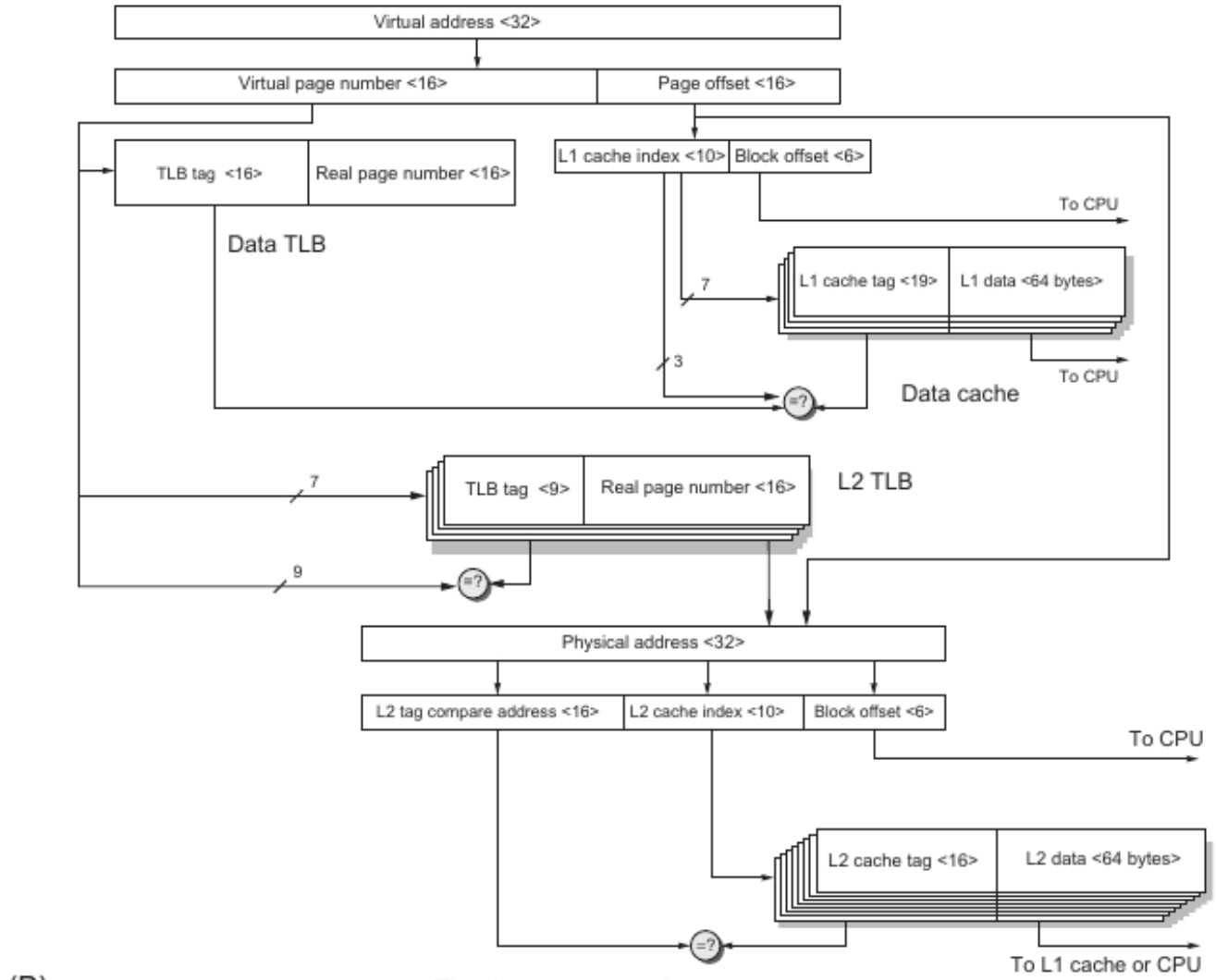
Példa I. – ARM Cortex-A53

Structure	Size	Organization	Typical miss penalty (clock cycles)
Instruction MicroTLB	10 entries	Fully associative	2
Data MicroTLB	10 entries	Fully associative	2
L2 Unified TLB	512 entries	4-way set associative	20
L1 Instruction cache	8–64 KiB	2-way set associative; 64-byte block	13
L1 Data cache	8–64 KiB	2-way set associative; 64-byte block	13
L2 Unified cache	128 KiB to 2 MiB	16-way set associative; LRU	124

Figure 2.19 The memory hierarchy of the Cortex A53 includes multilevel TLBs and caches. A page map cache keeps track of the location of a physical page for a set of virtual pages; it reduces the L2 TLB miss penalty. The L1 caches are virtually indexed and physically tagged; both the L1 D cache and L2 use a write-back policy defaulting to allocate on write. Replacement policy is LRU approximation in all the caches. Miss penalties to L2 are higher if both a MicroTLB and L1 miss occur. The L2 to main memory bus is 64–128 bits wide, and the miss penalty is larger for the narrow bus.



(A) **The instruction access path**



(B) **The data access path**

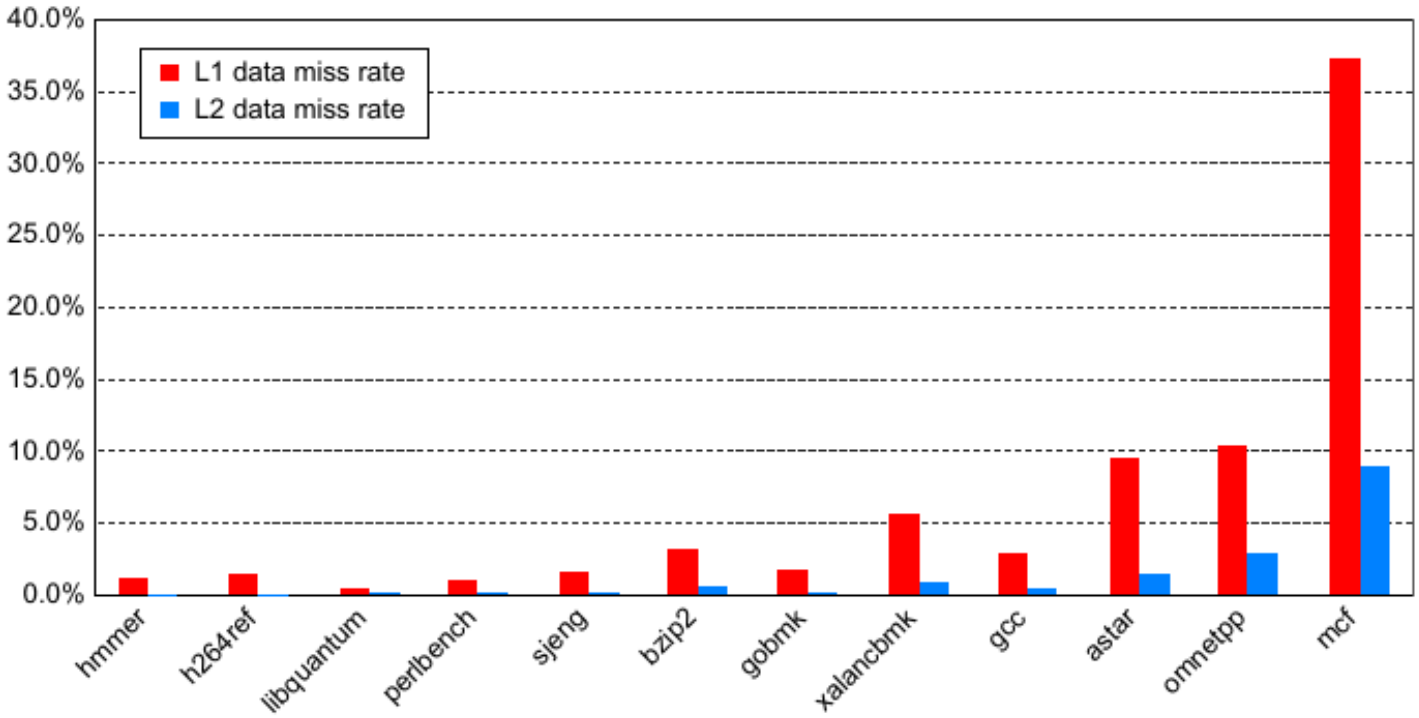


Figure 2.21 The data miss rate for ARM with a 32 KiB L1 and the global data miss rate for a 1 MiB L2 using the SPECInt2006 benchmarks are significantly affected by the applications. Applications with larger memory footprints tend to have higher miss rates in both L1 and L2. Note that the L2 rate is the global miss rate that is counting all references, including those that hit in L1. MCF is known as a cache buster.

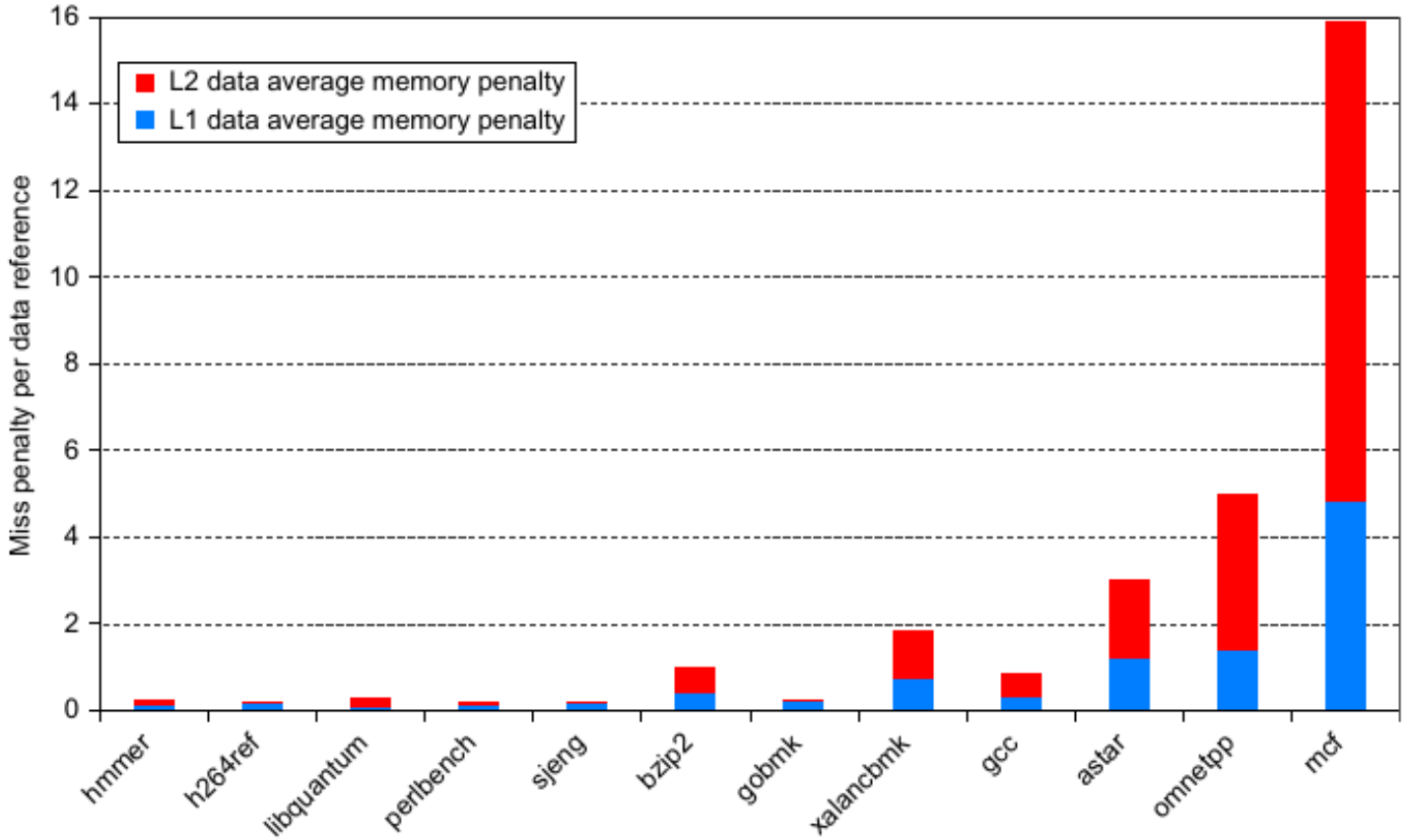


Figure 2.22 The average memory access penalty per data memory reference coming from L1 and L2 is shown for the A53 processor when running SPECInt2006. Although the miss rates for L1 are significantly higher, the L2 miss penalty, which is more than five times higher, means that the L2 misses can contribute significantly.

Intel Core i7, 1 mag

- 1 utasítás/órajel, 3.3 GHz
- multiple issue, dynamically scheduled, 16-stage pipeline, 2 threads
- 3 memória csatorna $\Rightarrow$ peak bandwidth 25Gb/sec
- 48 bit virtuális cím, 36bit fizikális cím
- L1: virtuális indexelés, L2,L3: fizikális indexelés
- L1, L2, L3: fizikális címkézés

Intel Core i7 TLBs

Characteristic	Instruction TLB	Data DLB	Second-level TLB
Entries	128	64	1536
Associativity	8-way	4-way	12-way
Replacement	Pseudo-LRU	Pseudo-LRU	Pseudo-LRU
Access latency	1 cycle	1 cycle	8 cycles
Miss	9 cycles	9 cycles	Hundreds of cycles to access page table

Figure 2.23 Characteristics of the i7's TLB structure, which has separate first-level instruction and data TLBs, both backed by a joint second-level TLB. The first-level TLBs support the standard 4 KiB page size, as well as having a limited number of entries of large 2–4 MiB pages; only 4 KiB pages are supported in the second-level TLB. The i7 has the ability to handle two L2 TLB misses in parallel. See Section L.3 of online Appendix L for more discussion of multilevel TLBs and support for multiple page sizes.

Intel Core i7 Cache

Characteristic	L1	L2	L3
Size	32 KiB I/32 KiB D	256 KiB	2 MiB per core
Associativity	both 8-way	4-way	16-way
Access latency	4 cycles, pipelined	12 cycles	44 cycles
Replacement scheme	Pseudo-LRU	Pseudo-LRU	Pseudo-LRU but with an ordered selection algorithm

Figure 2.24 Characteristics of the three-level cache hierarchy in the i7. All three caches use write back and a block size of 64 bytes. The L1 and L2 caches are separate for each core, whereas the L3 cache is shared among the cores on a chip and is a total of 2 MiB per core. All three caches are nonblocking and allow multiple outstanding writes. A merging write buffer is used for the L1 cache, which holds data in the event that the line is not present in L1 when it is written. (That is, an L1 write miss does not cause the line to be allocated.) L3 is inclusive of L1 and L2; we explore this property in further detail when we explain multiprocessor caches. Replacement is by a variant on pseudo-LRU; in the case of L3, the block replaced is always the lowest numbered way whose access bit is off. This is not quite random but is easy to compute.

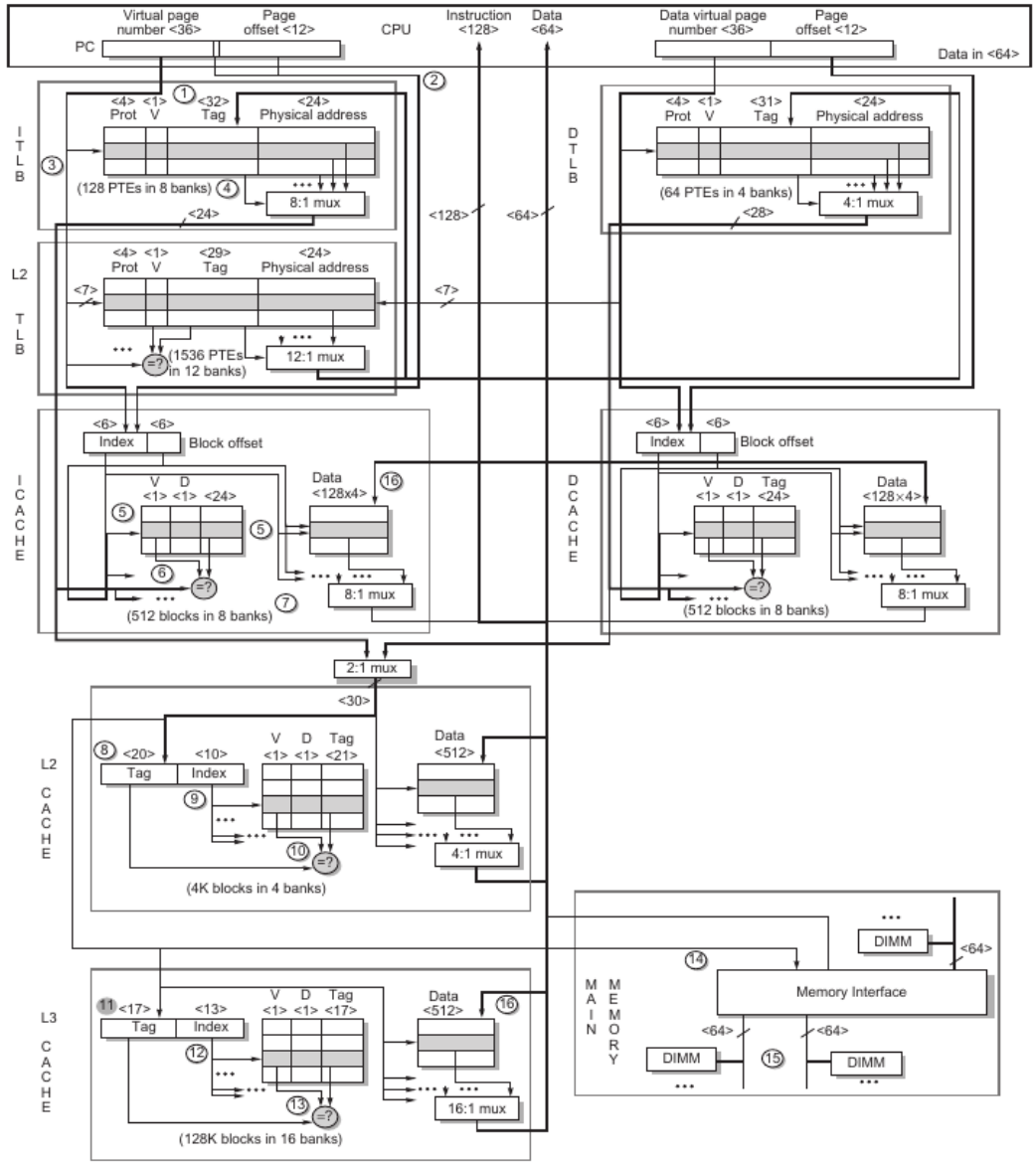


Figure 2.25 The Intel i7 memory hierarchy and the steps in both instruction and data access. We show only reads. Writes are similar, except that misses are handled by simply placing the data in a write buffer, because the L1 cache is not write-allocated.

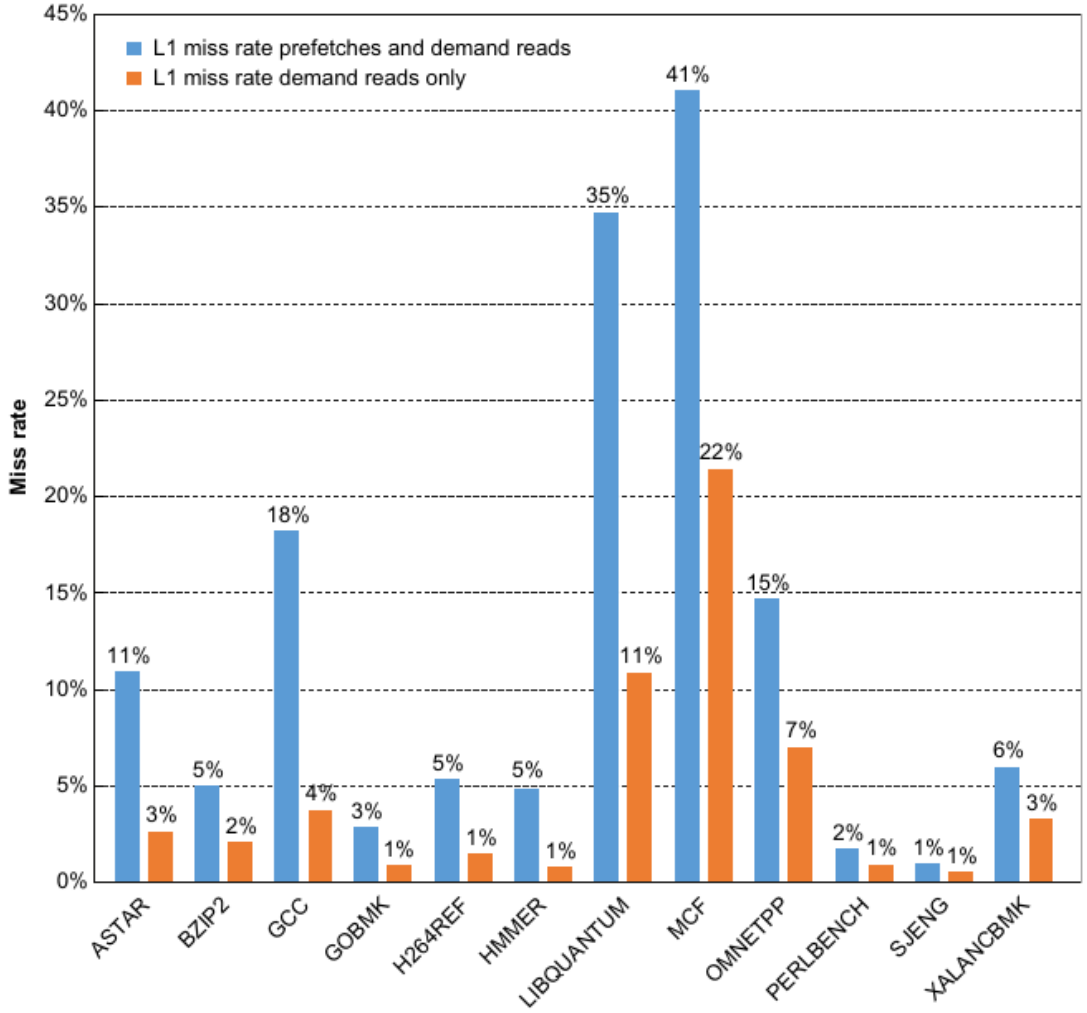


Figure 2.26 The L1 data cache miss rate for the SPECint2006 benchmarks is shown in two ways relative to the demand L1 reads: one including both demand and prefetch accesses and one including only demand accesses. The i7 separates out L1 misses for a block not present in the cache and L1 misses for a block already outstanding that is being prefetched from L2; we treat the latter group as hits because they would hit in a blocking cache. These data, like the rest in this section, were collected by Professor Lu Peng and PhD student Qun Liu, both of Louisiana State University, based on earlier studies of the Intel Core Duo and other processors (see Peng et al., 2008).

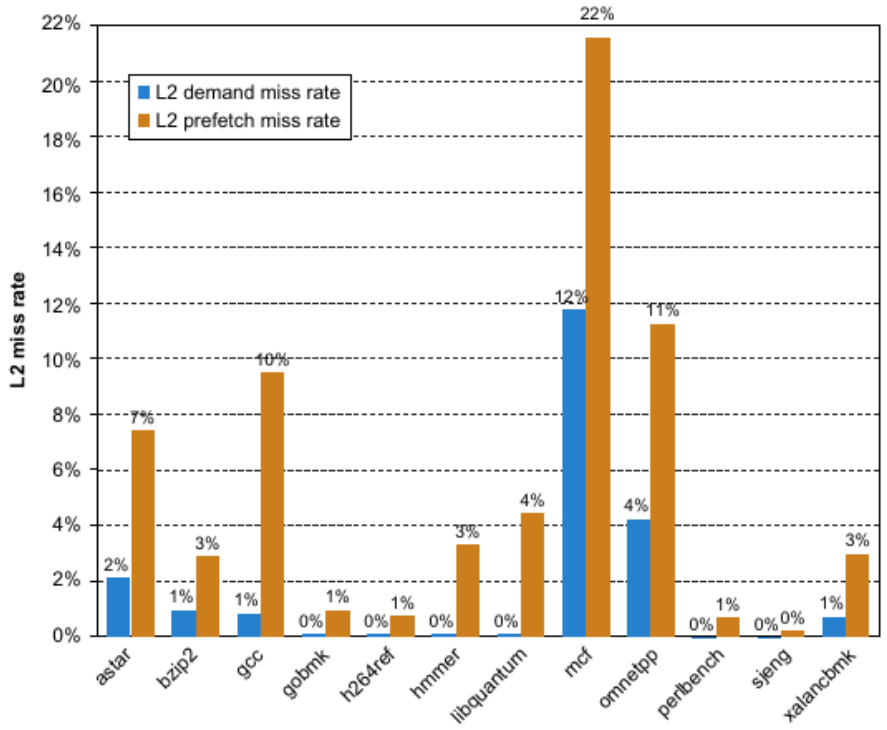
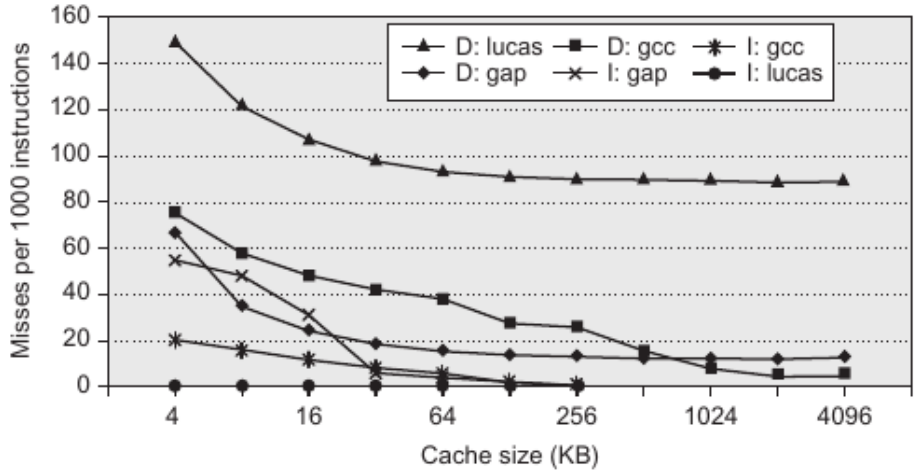


Figure 2.28 **The L2 demand miss rate and prefetch miss rate, both shown relative to all the references to L1, which also includes prefetches, speculative loads that do not complete, and program-generated loads and stores (demand references).** These data, like the rest in this section, were collected by Professor Lu Peng and PhD student Qun Liu, both of Louisiana State University.

Tévhitek és csapdák

F Cache preformancia egy másik program alapján.



P Keves információból ítélni meg a memótiahierarchia tulajdonságait.

1. Nagy cache tesztelése kis programmal/kevés adattal.
2. A tesztprogram lokalitás szempontjából nem konstans.
3. A program nagyban függ az input tulajdonságaitól.

P A töltések nem használják ki a sávszélességet.

Nagy hatékonyságú . . .
4. előadás (ILP)

Nagy Ádám
nagyadam888@gmail.com

Eötvös Loránd Tudományegyetem
Informatikai Kar

2022. október 4.

ILP - Instruction-Level Parallelism

Mi is az ILP?
Utasítások párhuzamosan történő végrehajtásának lehetősége, alapvetően egyszerű utasításblokkban (egyelőre nincs elágazás, ciklus, stb.)

Hogyan használjuk ki?

1. HW-re támaszkodva dinamikus ütemezéssel.
2. Fordító vagy programozó által statikus ütemezéssel.

Pipelineing

Gyakorlatban a legfontosabb (HW) technika ILP kihasználására.

A kiszemelt utasításokat részekre (pipe stage vagy pipe segment) osztjuk. Ezek a részek általában ugyanannyi ideig tartanak (ált. 1 órajel), ui. a leghosszabb ideig tartó határozza meg az teljes időt (maxtime $\times$ állomások száma).

Szinkronizáció a munkafázisok között: órajel.

Ideális körülmények között:

$$\text{Pipelined utasítás végrahajtási ideje} =$$

$$= \frac{\text{Unpipelined utasítás végrehajtási ideje}}{\text{pipe stage-ek száma}}$$

Pipelineing - Utasítások

RISC utasítások megfelenek a következőeknek (és így segitik a pipeline-t):

- Minen utasítás regisztereken történik és általában a regiszter egészén.
- Csak a load és store utasítások mozgatnak adatot a memóriából/ba.
- Kevés típusú utasítás a formátum (és ált. méret) tekintetében.

Az utasítások általában 3 csoportba sorolhatóak:

1. ALU utasítások.
2. Load és Store utasítások.
3. Branch (contitional Jump) és Jump.

Pipelineing - Utasítások "darabolása"

Egy egyszerűsített model esetén az utasítások 5 lépésből állnak:

1. Instruction Fetch (IF) – set PC, fetch instruction, update PC
2. Instruction Decode/Register fetch cycle (ID)
   fixed-field decoding, branch
3. Execution/effective address cycle (EX)
4. Memory access (MEM)
5. Write-back cycle (WB) - register-register or load

Megj.: Branch: 2 ój., store: 4 ój., minden más: 5 ój.

Pipelineing – Hogyan néz ki

	Clock number								
Instruction number	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
Instruction i	IF	ID	EX	MEM	WB				
Instruction $i+1$		IF	ID	EX	MEM	WB			
Instruction $i+2$			IF	ID	EX	MEM	WB		
Instruction $i+3$				IF	ID	EX	MEM	WB	
Instruction $i+4$					IF	ID	EX	MEM	WB

Figure C.1 Simple RISC pipeline. On each clock cycle, another instruction is fetched and begins its five-cycle execution. If an instruction is started every clock cycle, the performance will be up to five times that of a processor that is not pipelined. The names for the stages in the pipeline are the same as those used for the cycles in the unpipelined implementation: IF = instruction fetch, ID = instruction decode, EX = execution, MEM = memory access, and WB = write-back.

Sajnos az élet nem ilyen egyszerű : (

- Külön instruction és data cache, 5x sávszélesség,
- PC írás ID alatt (branch) ütközhet IF-el
- Órajel max. a leglassabb rész ideje.

Pipelineing - Regiszterek használata

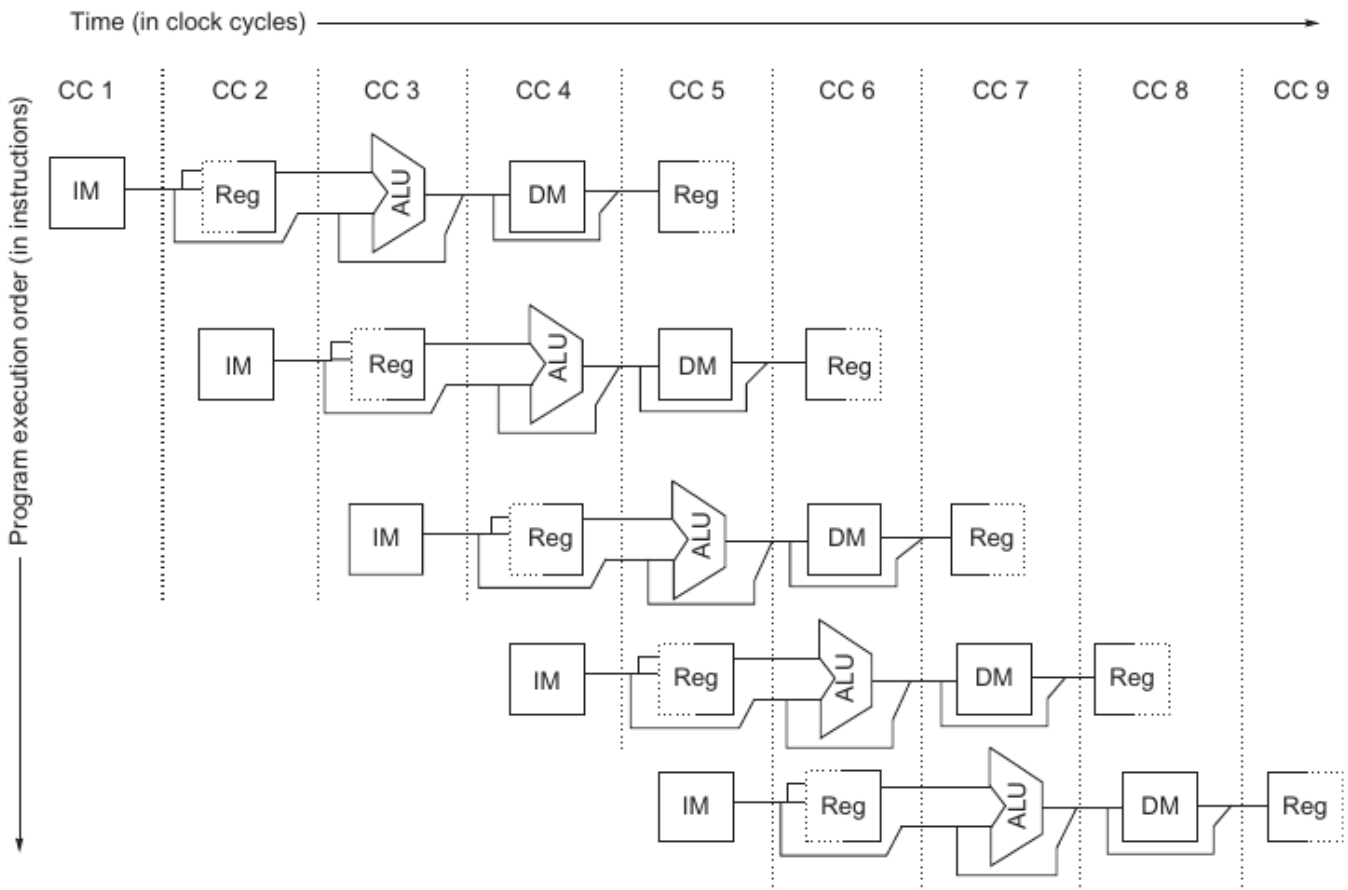


Figure C.2 The pipeline can be thought of as a series of data paths shifted in time. This figure shows the overlap among the parts of the data path, with clock cycle 5 (CC 5) showing the steady-state situation. Because the register file is used as a source in the ID stage and as a destination in the WB stage, it appears twice. We show that it is read in one part of the stage and written in another by using a solid line, on the right or left, respectively, and a dashed line on the other side. The abbreviation IM is used for instruction memory, DM for data memory, and CC for clock cycle.

Pipelineing - Pipline regiszterek használata

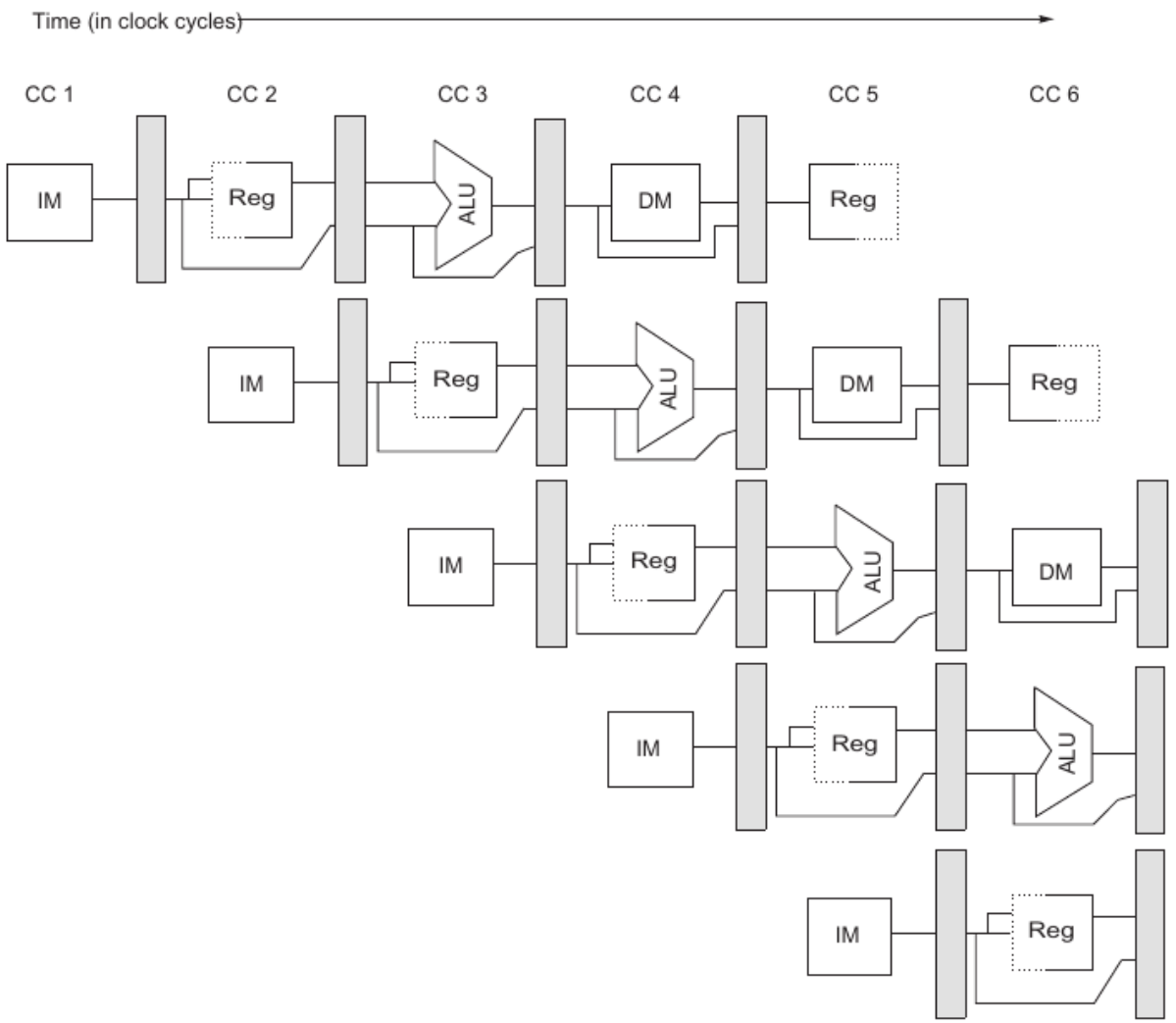


Figure C.3 A pipeline showing the pipeline registers between successive pipeline stages. Notice that the registers prevent interference between two different instructions in adjacent stages in the pipeline. The registers also play the critical role of carrying data for a given instruction from one stage to the other. The edge-triggered property of registers—that is, that the values change instantaneously on a clock edge—is critical. Otherwise, the data from one instruction could interfere with the execution of another!

Pipelining - Kockázatok

Utasítások, melyek akadályozzák a következő utasítások végrehajtását.
Ilyen kozkázatok lehetnek:

- **szerkezeti** hw nem tud kiszolgálni két párhuzamos állomást
- **adat** adatfüggöség
- **vezérlési** PC-t módosító utasítás

Megoldás: stall (ha nem lehet mashogy)

Szerkezeti kockázatok

- Nem pipeline-olt utasítások (nem azonos órajel)
- Közös erőforrás használata
- Nem pipline-olt egység használata (nagyobb órajel mint a többi)
- Kevés funkcionális egység a párhuzmos végrehajtáshoz (pl. 1 reg file).

Legtöbbször: nem pipline-olt funkcionális egység használata.

Egyetlen memóriakezelő egység

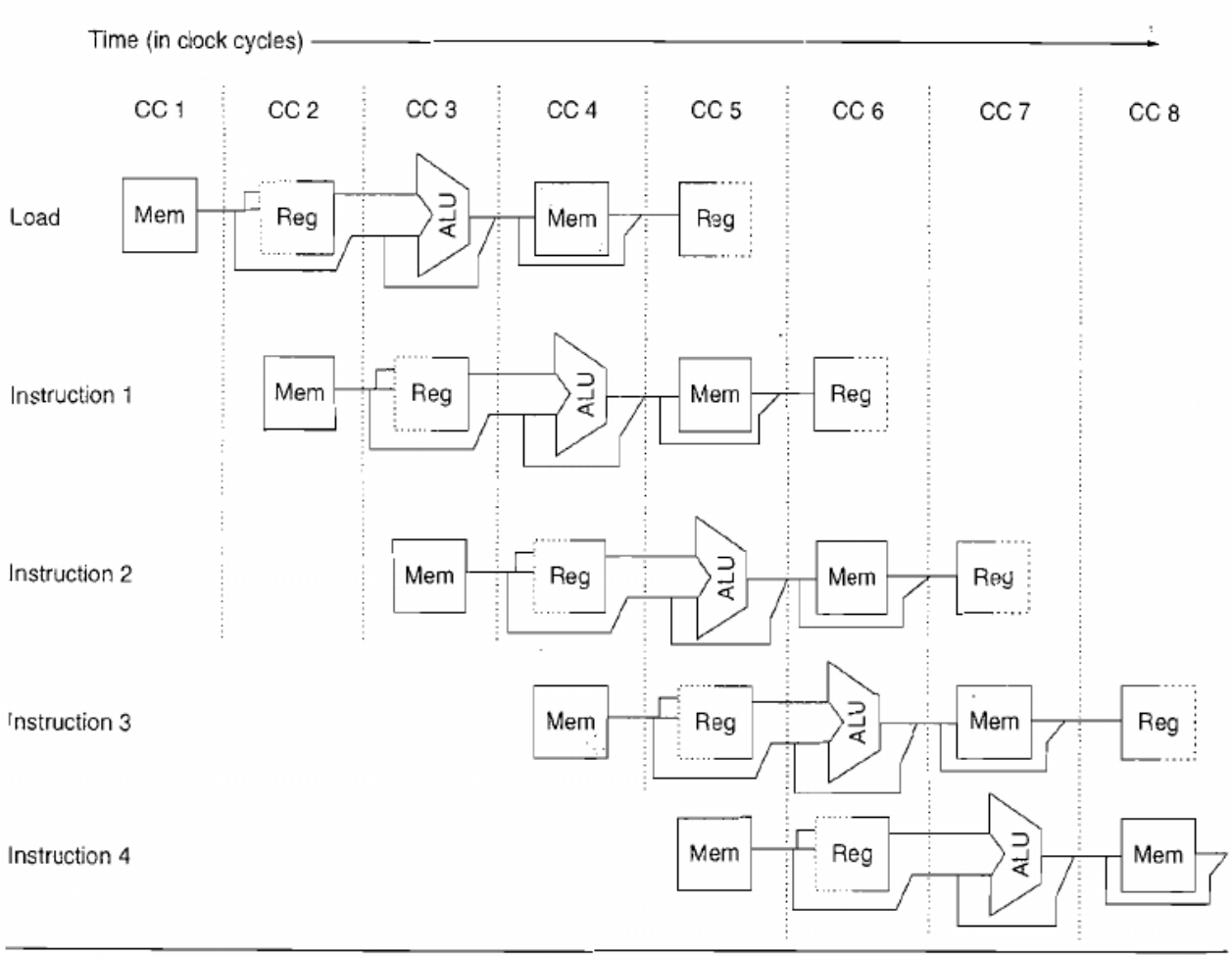


Figure C.4 A processor with only one memory port will generate a conflict whenever a memory reference occurs. In this example the load instruction uses the memory for a data access at the same time instruction 3 wants to fetch an instruction from memory.

Stall (bubble)

	Clock cycle number									
Instruction	1	2	3	4	5	6	7	8	9	10
Load instruction	IF	ID	EX	MEM	WB					
Instruction $i + 1$		IF	ID	EX	MEM	WB				
Instruction $i + 2$			IF	ID	EX	MEM	WB			
Instruction $i + 3$				Stall	IF	ID	EX	MEM	WB	
Instruction $i + 4$						IF	ID	EX	MEM	WB
Instruction $i + 5$							IF	ID	EX	MEM
Instruction $i + 6$								IF	ID	EX

Figure C.5 A pipeline stalled for a structural hazard—a load with one memory port. As shown here, the load instruction effectively steals an instruction-fetch cycle, causing the pipeline to stall—no instruction is initiated on clock cycle 4 (which normally would initiate instruction $i + 3$). Because the instruction being fetched is stalled, all other instructions in the pipeline before the stalled instruction can proceed normally. The stall cycle will continue to pass through the pipeline, so that no instruction completes on clock cycle 8. Sometimes these pipeline diagrams are drawn with the stall occupying an entire horizontal row and instruction 3 being moved to the next row; in either case, the effect is the same, since instruction $i + 3$ does not begin execution until cycle 5. We use the form above, since it takes less space in the figure. Note that this figure assumes that instructions $i + 1$ and $i + 2$ are not memory references.

A szerkezeti kockázatok kiküszöbölthetőek lennénenek, de

1. a végrehajtási egységek duplikálása költséges (cost, power);
2. memóriaelérés I és D esetén egyszerre $\Rightarrow$ dupla sávszélesség;
3. floating point pipleline költséges $\Rightarrow$ ritka műveletek pipeline-olása nem célszerű

Adat kockázat

A pipeline miatt megváltozhat a memória írásának és olvasásának sorrendje.

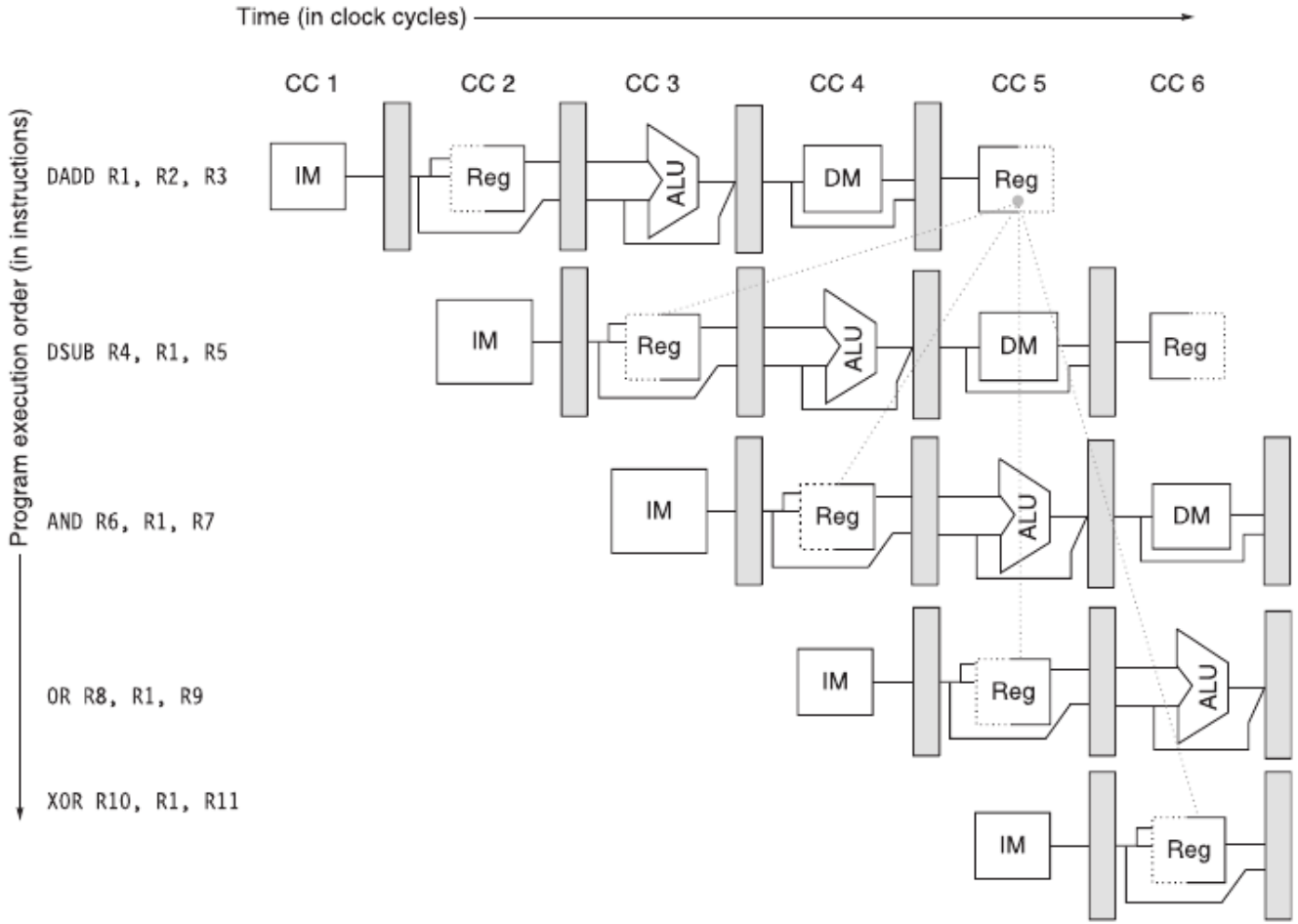


Figure C.4 The use of the result of the `add` instruction in the next three instructions causes a hazard, because the register is not written until after those instructions read it.

Megoldás: forwarding (bypassing, short-circuiting)

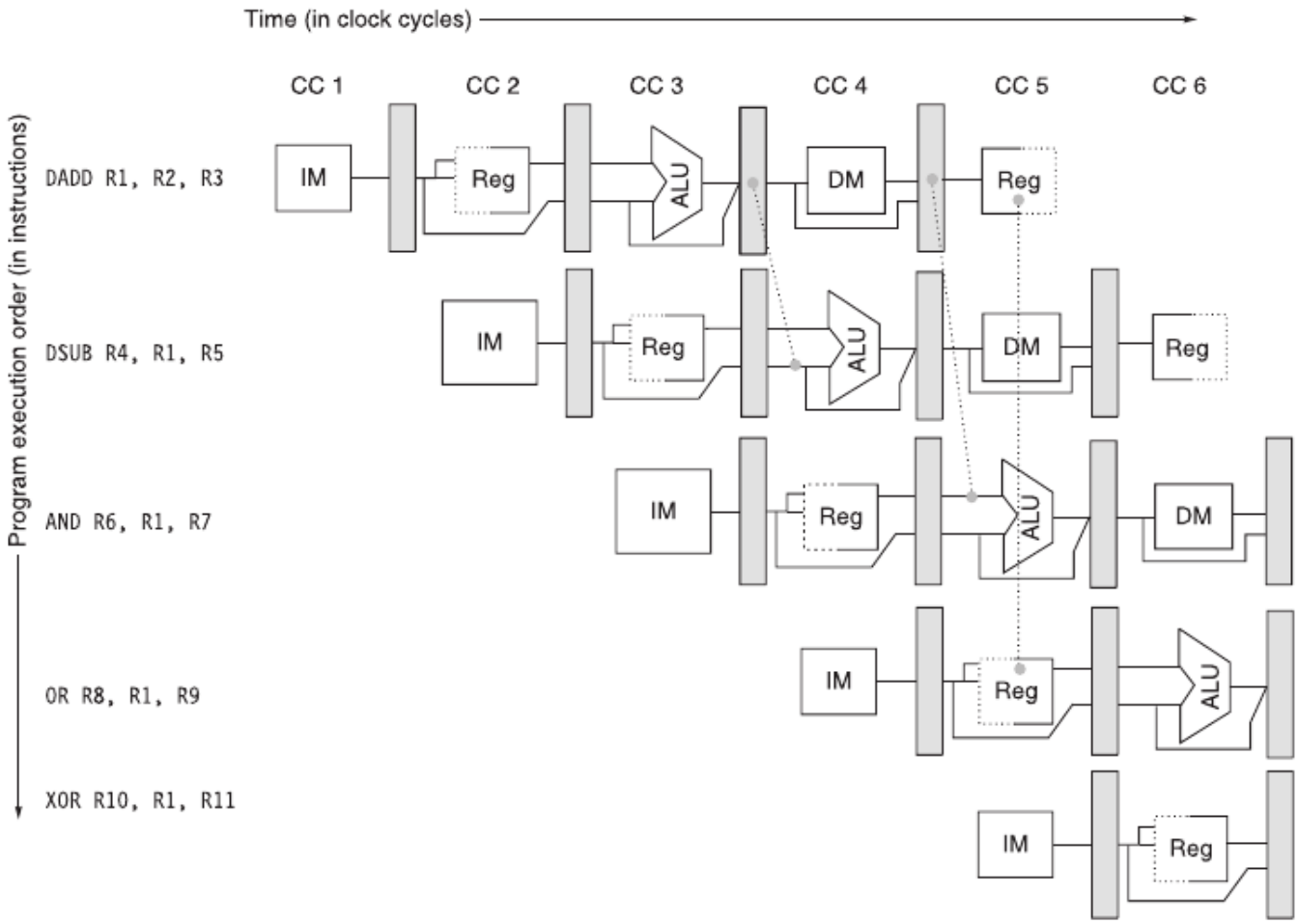


Figure C.5 A set of instructions that depends on the `add` result uses forwarding paths to avoid the data hazard. The inputs for the `sub` and `and` instructions forward from the pipeline registers to the first ALU input. The `or` receives its result by forwarding through the register file, which is easily accomplished by reading the registers in the second half of the cycle and writing in the first half, as the dashed lines on the registers indicate. Notice that the forwarded result can go to either ALU input; in fact, both ALU inputs could use forwarded inputs from either the same pipeline register or from different pipeline registers. This would occur, for example, if the `and` instruction was `and x6,x1,x4`.

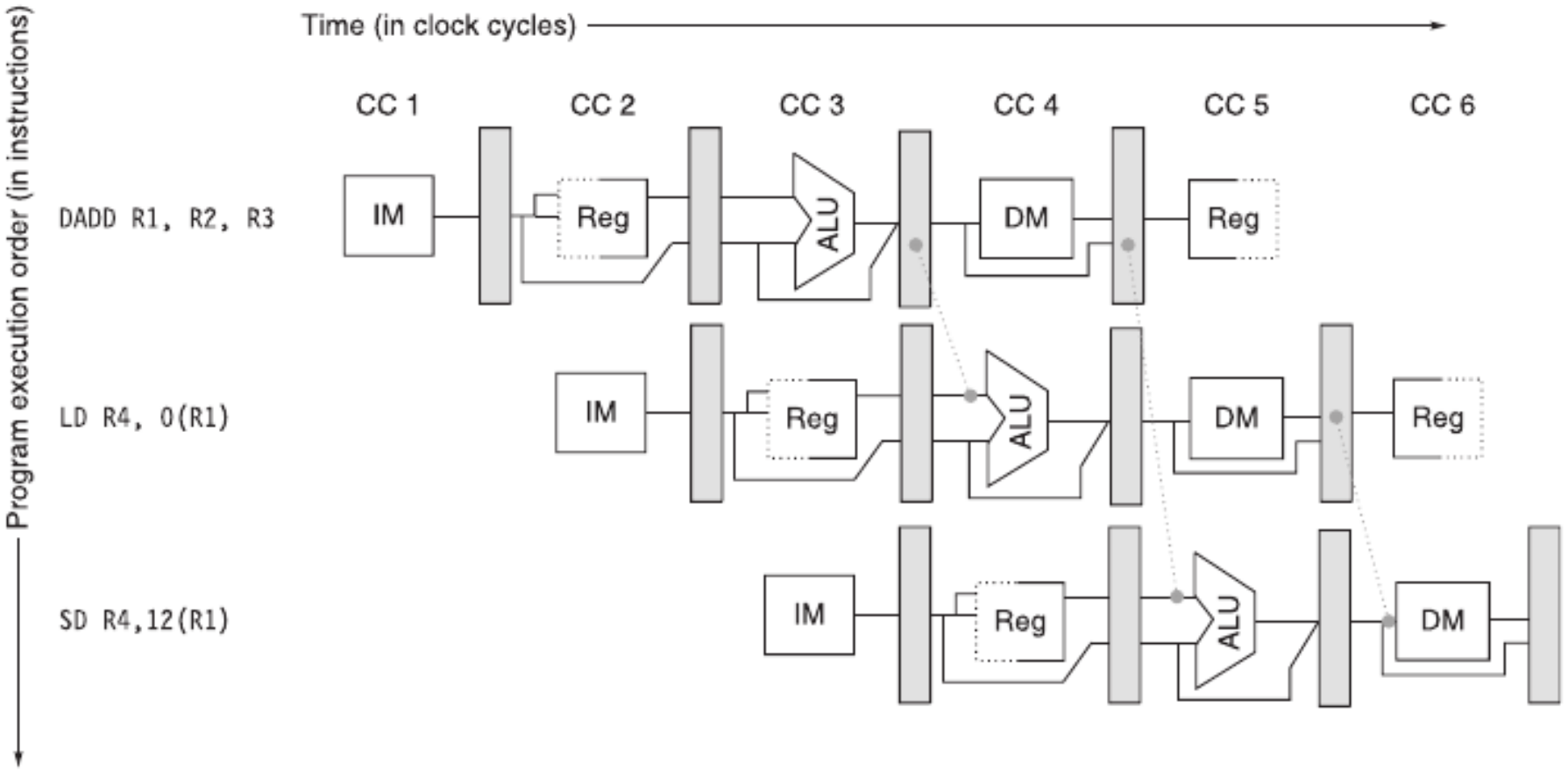


Figure C.6 Forwarding of operand required by stores during MEM. The result of the load is forwarded from the memory output to the memory input to be stored. In addition, the ALU output is forwarded to the ALU input for the address calculation of both the load and the store (this is no different than forwarding to another ALU operation). If the store depended on an immediately preceding ALU operation (not shown herein), the result would need to be forwarded to prevent a stall.

Nem mindenhol alkalmazható forwarding

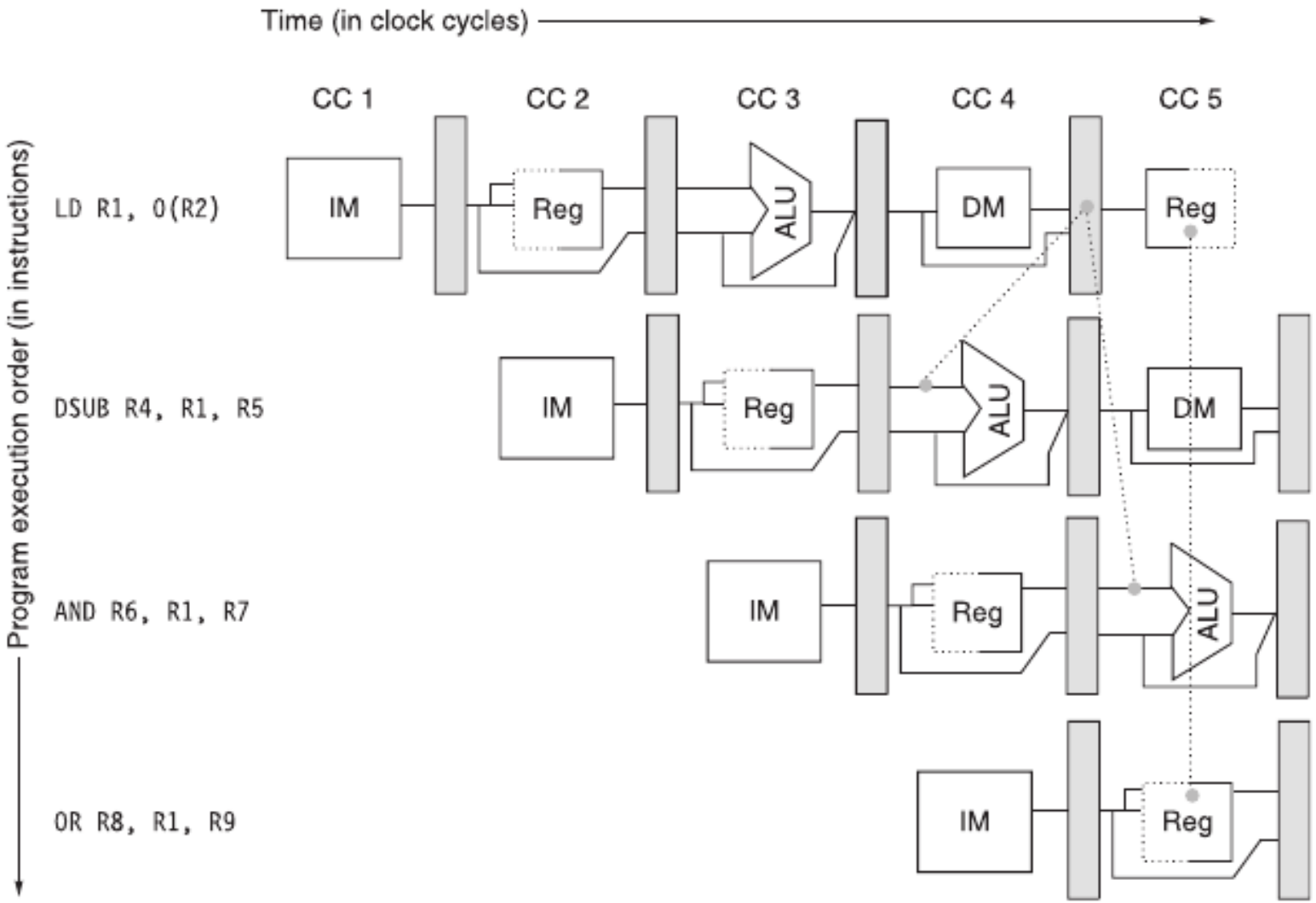


Figure C.7 The load instruction can bypass its results to the `and` and `or` instructions, but not to the `sub`, because that would mean forwarding the result in "negative time."

Stall alkalmazása

`ld x1,0(x2)`	IF	ID	EX	MEM	WB				
`sub x4,x1,x5`		IF	ID	EX	MEM	WB			
`and x6,x1,x7`			IF	ID	EX	MEM	WB		
`or x8,x1,x9`				IF	ID	EX	MEM	WB	
`ld x1,0(x2)`	IF	ID	EX	MEM	WB				
`sub x4,x1,x5`		IF	ID	Stall	EX	MEM	WB		
`and x6,x1,x7`			IF	Stall	ID	EX	MEM	WB	
`or x8,x1,x9`				Stall	IF	ID	EX	MEM	WB

Figure C.8 In the top half, we can see why a stall is needed: the MEM cycle of the load produces a value that is needed in the EX cycle of the `sub`, which occurs at the same time. This problem is solved by inserting a stall, as shown in the bottom half.

Vezérlési (branch) kockázatok

A PC változtatának lehetősége ID fázisban.

Változik (else ág) $\rightarrow$ taken
Nem változik (igaz ág) $\rightarrow$ untaken

Stall elkerülése: IF újra ha PC változik (branch taken)

Branch instruction	IF	ID	EX	MEM	WB		
Branch successor		IF	IF	ID	EX	MEM	WB
Branch successor + 1				IF	ID	EX	MEM
Branch successor + 2					IF	ID	EX

Figure C.9 A branch causes a one-cycle stall in the five-stage pipeline. The instruction after the branch is fetched, but the instruction is ignored, and the fetch is restarted once the branch target is known. It is probably obvious that if the branch is not taken, the second IF for branch successor is redundant. This will be addressed shortly.

Branch penalty csökkentése compilerrel

4 technika a stall elkerülésére (később még lesz több):

1. pipeline freezing or pipeline flushing
   egyszerű (sw-hw), de szoftveresen nem csökkenti a branch penalty-t
2. minden branch továbbengedése (untaken), majd ha kierült, hogy taken "black out"
   kicsit bonyolultabb hw (black out lekezelése), de sokkal hatékonyabb

Untaken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i+1$		IF	ID	EX	MEM	WB			
Instruction $i+2$			IF	ID	EX	MEM	WB		
Instruction $i+3$				IF	ID	EX	MEM	WB	
Instruction $i+4$					IF	ID	EX	MEM	WB

Taken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i+1$		IF	**idle**	**idle**	**idle**	**idle**			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

Figure C.10 The predicted-not-taken scheme and the pipeline sequence when the branch is untaken (top) and taken (bottom). When the branch is untaken, determined during ID, we fetch the fall-through and just continue. If the branch is taken during ID, we restart the fetch at the branch target. This causes all instructions following the branch to stall 1 clock cycle.

3. Mindkét ágról végrehajtani
   1. Amíg a taken cím nem ismert fetch az untaken ágról
   2. Ha a taken ág címe ismert fetch a taken ágról
   3. A brach eredménye után a jót folytatjuk

   Több fázisból álló (5-nél) pipeline kell, ahol a taken címét hamarabb tudjuk mint az eredményét.

4. Delayed branch, melynél a branch után egy ún. branch delay instruction (slot) kerül
   Egyszerű és gyakran (már az elő RISC processoroknál is) használt módszer, sajnos nem alkalmazható ha
   - nincs olyan utasítás, ami a delay slotba-kerülhetne
   - és ekkor a hatékonyság függ attól, hogy mennyire tudjuk megjósolni a branch irányát
   - különben black out (canceling or nullifying branch)

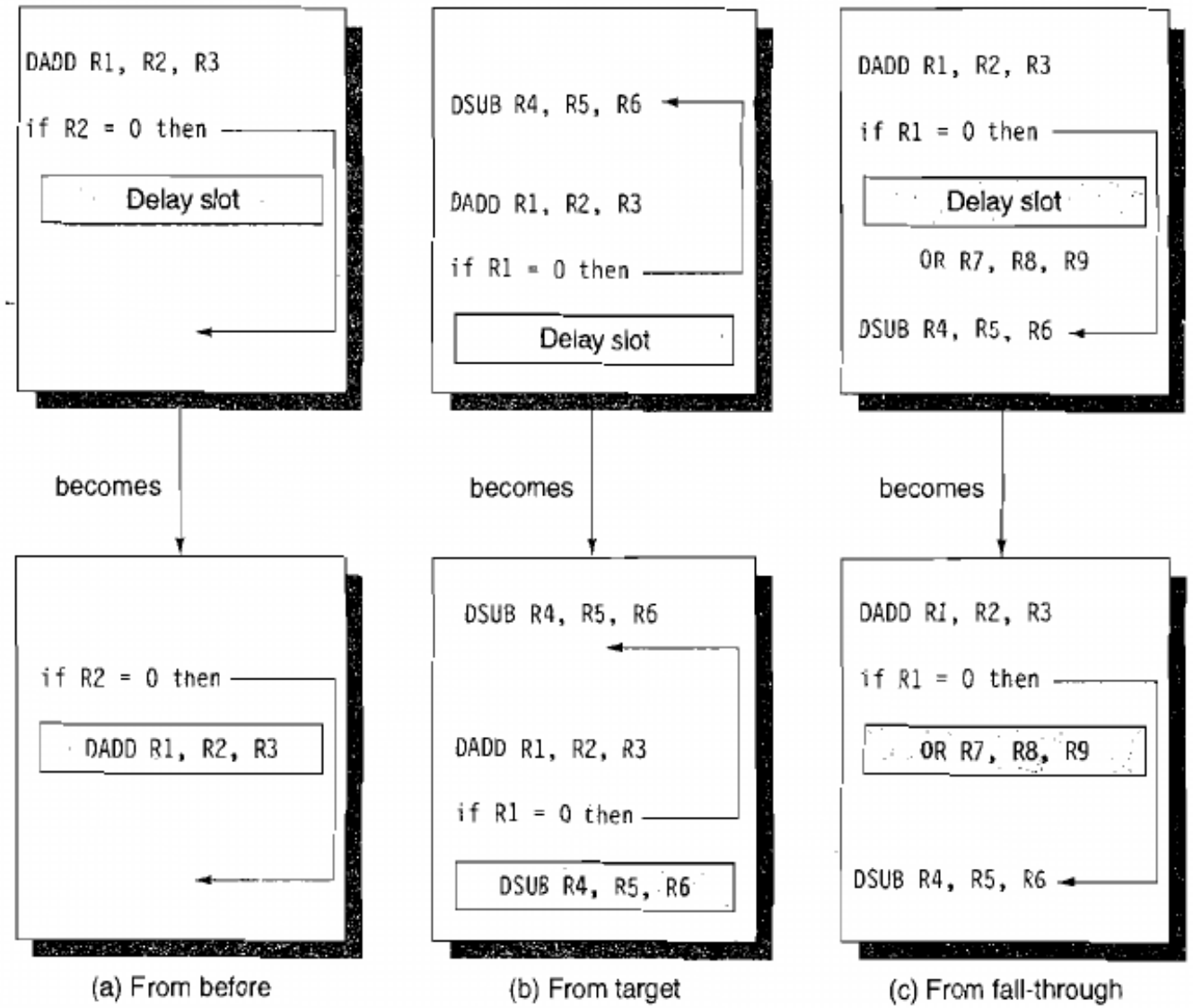
Delayed branch végrehajtása

Untaken branch instruction	IF	ID	EX	MEM	WB				
Branch delay instruction ($i+1$)		IF	ID	EX	MEM	WB			
Instruction $i+2$			IF	ID	EX	MEM	WB		
Instruction $i+3$				IF	ID	EX	MEM	WB	
Instruction $i+4$					IF	ID	EX	MEM	WB

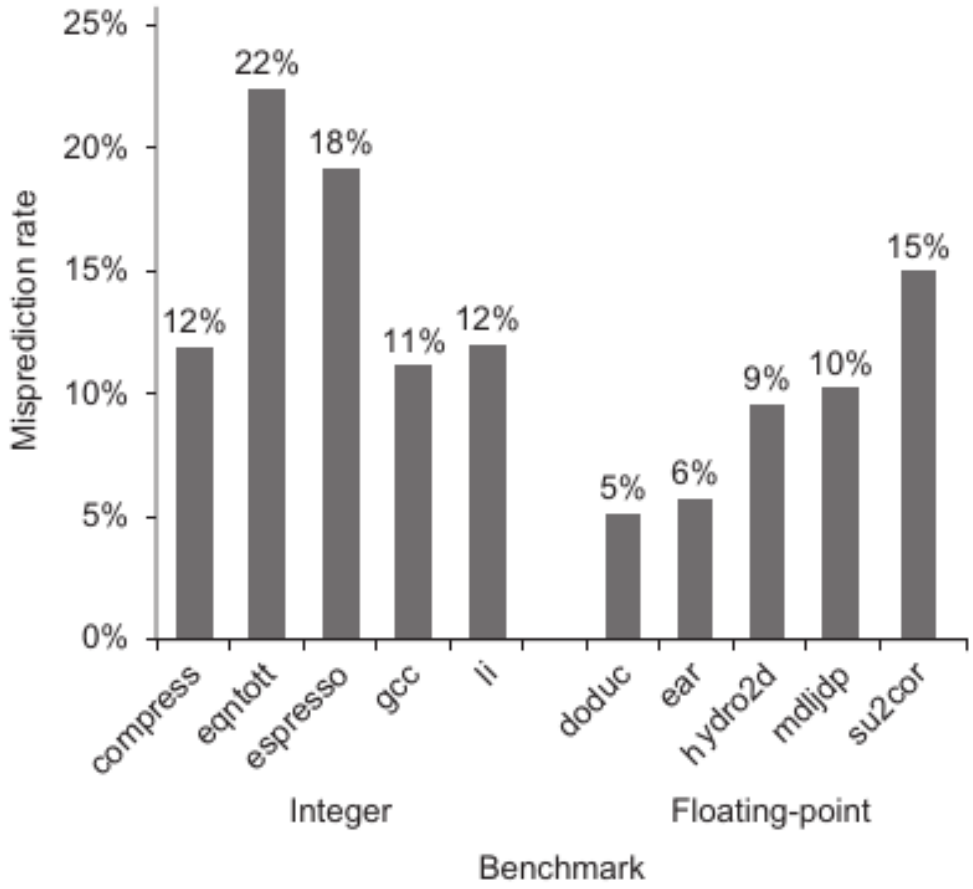
Taken branch instruction	IF	ID	EX	MEM	WB				
Branch delay instruction ($i+1$)		IF	ID	EX	MEM	WB			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

Figure C.11 The behavior of a delayed branch is the same whether or not the branch is taken. The instructions in the delay slot (there was only one delay slot for most RISC architectures that incorporated them) are executed. If the branch is untaken, execution continues with the instruction after the branch delay instruction; if the branch is taken, execution continues at the branch target. When the instruction in the branch delay slot is also a branch, the meaning is unclear: if the branch is not taken, what should happen to the branch in the branch delay slot? Because of this confusion, architectures with delay branches often disallow putting a branch in the delay slot.

Delay slot ütemezése



Branch prediction statikus javítása



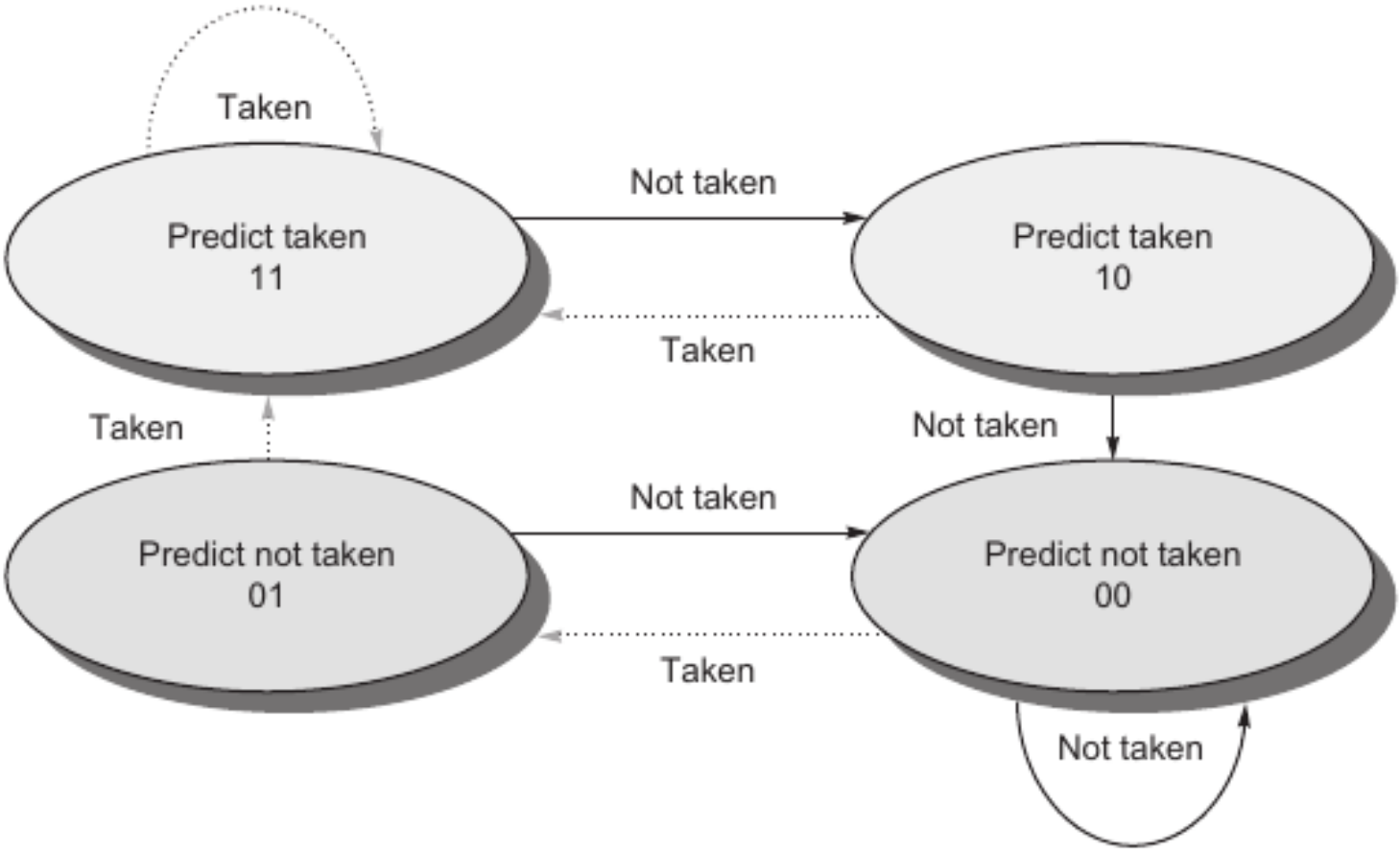
Branch prediction dinamikus javítása

Egy buffer vagy táblában tároljuk az elágazás előző irányait

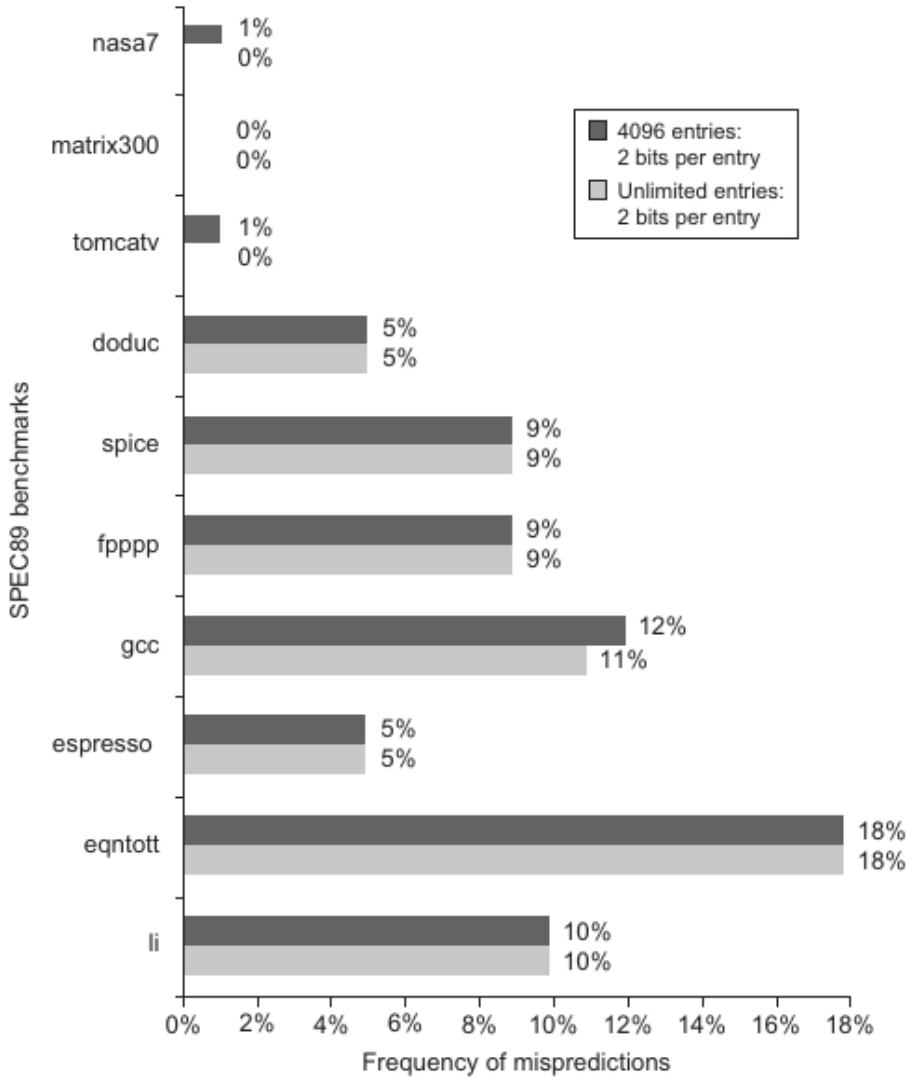
Eredmény függ:

- Hány előző eredményt tárolunki (legegyszerübb 1bit).
- Hány elágazás eredményét tudjuk használni.

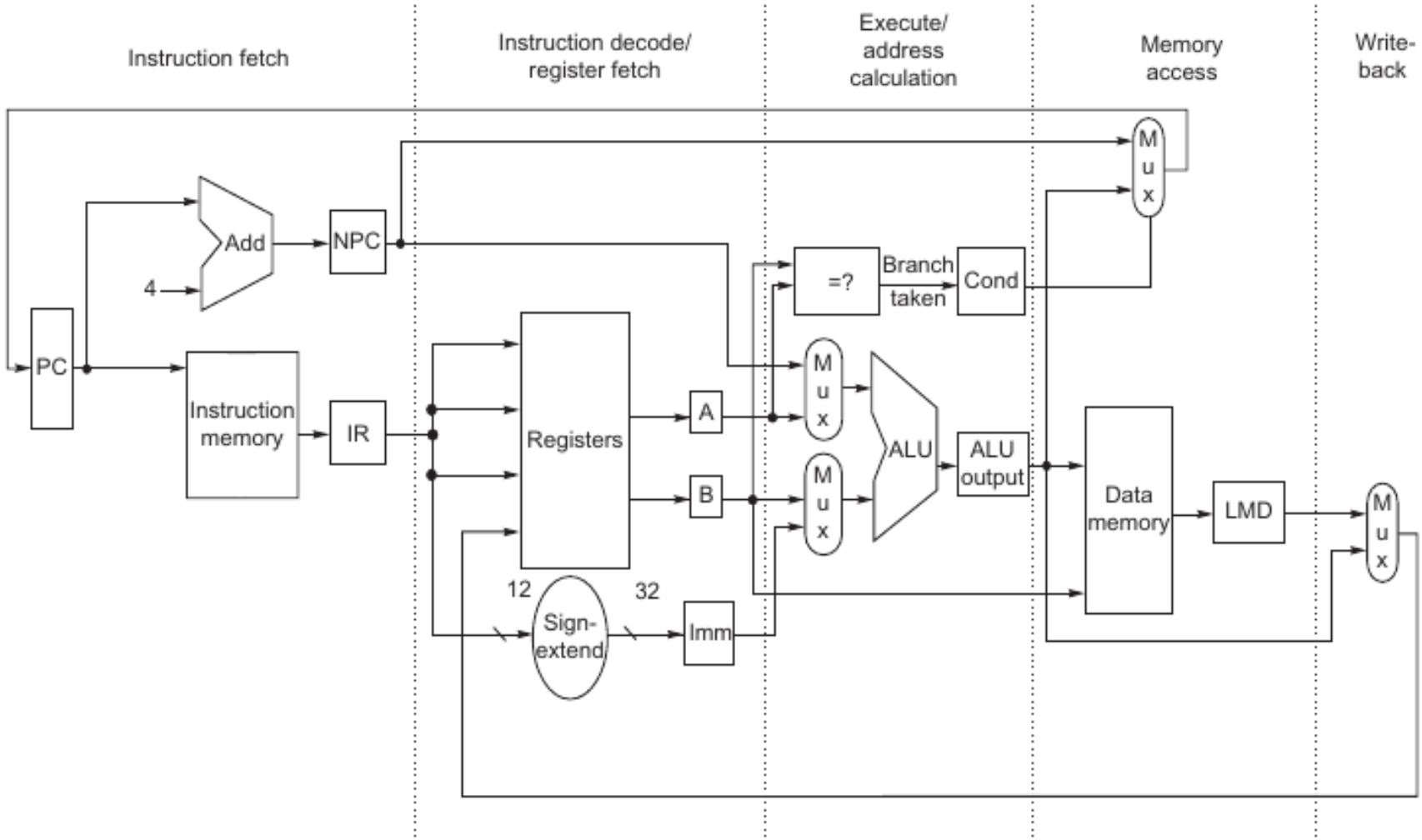
Dynamic branch prediction with 2 bit buffer



Branch prediction buffer size



Példa: 5 stage pipeline



Példa: with pipline registers

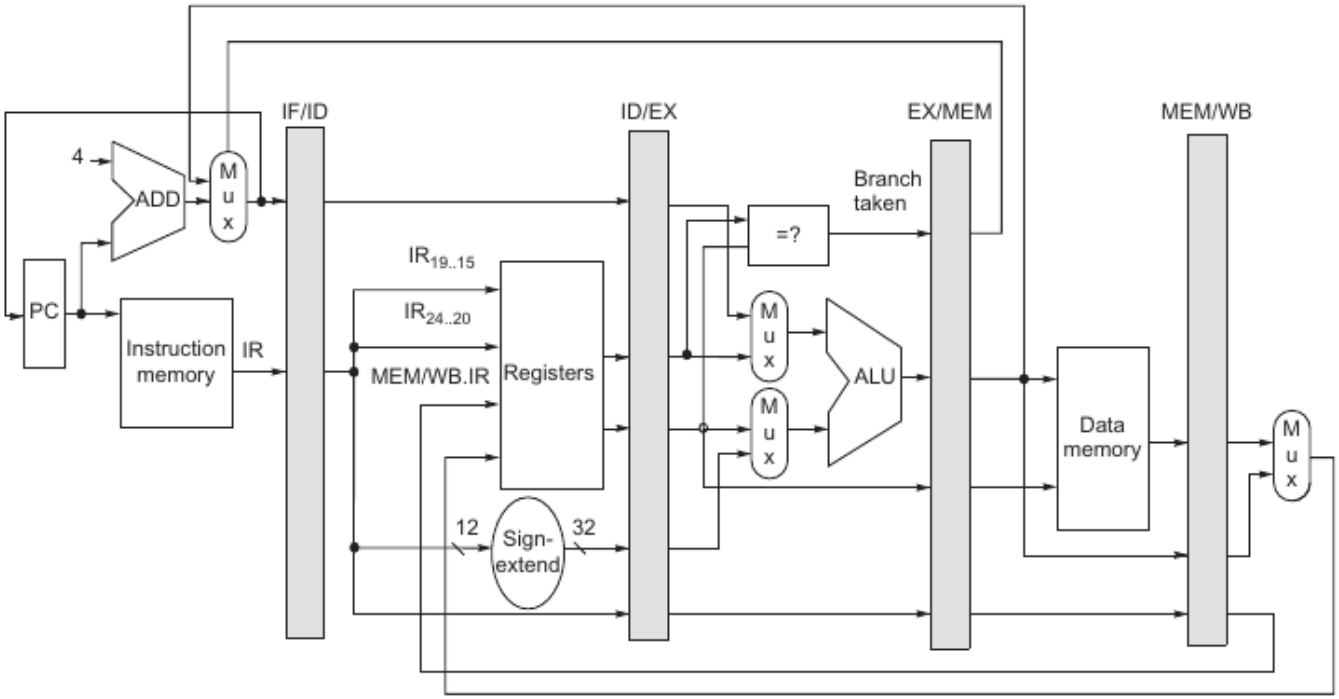


Figure C.19 The data path is pipelined by adding a set of registers, one between each pair of pipe stages. The registers serve to convey values and control information from one stage to the next. We can also think of the PC as a pipeline register, which sits before the IF stage of the pipeline, leading to one pipeline register for each pipe stage. Recall that the PC is an edge-triggered register written at the end of the clock cycle; hence, there is no race condition in writing the PC. The selection multiplexer for the PC has been moved so that the PC is written in exactly one stage (IF). If we didn't move it, there would be a conflict when a branch occurred, because two instructions would try to write different values into the PC. Most of the data paths flow from left to right, which is from earlier in time to later. The paths flowing from right to left (which carry the register write-back information and PC information on a branch) introduce complications into our pipeline.

Példa: what happens

Stage	Any instruction		
IF	IF/ID.IR←Mem[PC] IF/ID.NPC,PC←(if ((EX/MEM.opcode == branch) & EX/MEM.cond){EX/MEM. ALUOutput} else {PC+4});		
ID	ID/EX.A←Regs[IF/ID.IR[rs1]]; ID/EX.B←Regs[IF/ID.IR[rs2]]; ID/EX.NPC←IF/ID.NPC; ID/EX.IR←IF/ID.IR; ID/EX.Imm←sign-extend(IF/ID.IR[immediate field]);		
	ALU instruction	**Load instruction**	**Branch instruction**
EX	EX/MEM.IR←ID/EX.IR; EX/MEM.ALUOutput← ID/EX.A func ID/EX.B; or EX/MEM.ALUOutput← ID/EX.A *op* ID/EX.Imm;	EX/MEM.IR to ID/EX.IR EX/MEM.ALUOutput← ID/EX.A+ID/EX.Imm; EX/MEM.B←ID/EX.B;	EX/MEM.ALUOutput← ID/EX.NPC + (ID/EX.Imm<<2); EX/MEM.cond← (ID/EX.A == ID/EX.B);
MEM	MEM/WB.IR←EX/MEM.IR; MEM/WB.ALUOutput← EX/MEM.ALUOutput;	MEM/WB.IR←EX/MEM.IR; MEM/WB.LMD← Mem[EX/MEM.ALUOutput]; or Mem[EX/MEM.ALUOutput]← EX/MEM.B;	
WB	Regs[MEM/WB.IR[rd]]← MEM/WB.ALUOutput;	For load only: Regs[MEM/WB.IR[rd]]← MEM/WB.LMD;	

Figure C.20 Events on every pipe stage of the RISC V pipeline. Let's review the actions in the stages that are specific

Branch hazard elimination when zero test

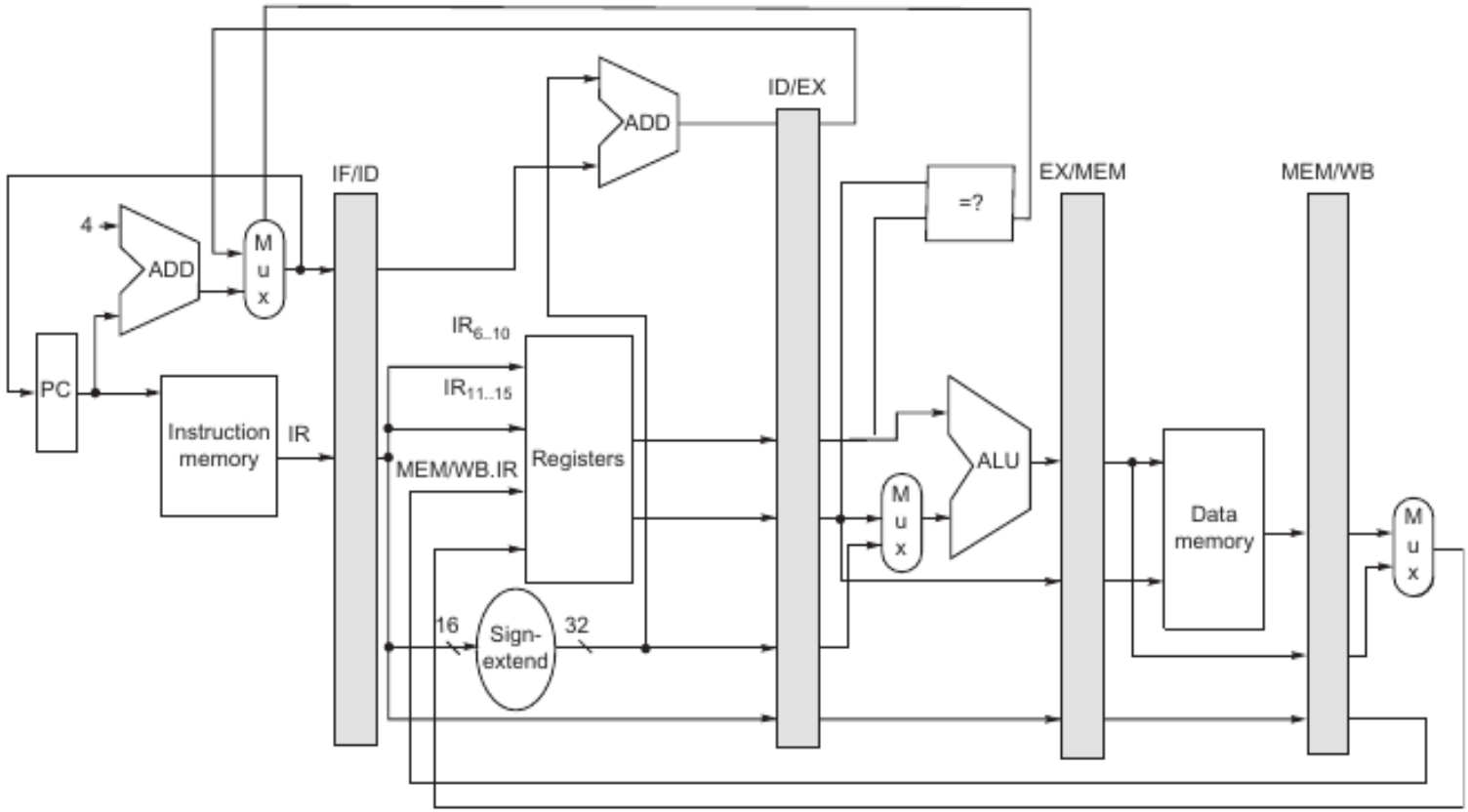


Figure C.25 To minimize the impact of deciding whether a conditional branch is taken, we compute the branch target address in ID while doing the conditional test and final selection of next PC in EX. As mentioned in Figure C.19, the PC can be thought of as a pipeline register (e.g., as part of ID/IF), which is written with the address of the next instruction at the end of each IF cycle.

Pipelining - Problems

Mi történik váratlan helyzetekben?

- Utasítások nem egy órajel alatt kerülnek kiszámolásra.
- Utasítások megszakítása és folyatatás.
- Utasítások végrehajtása alatt kivételek keletkezhetnek. (exception, fault, interrupt)

Exception type	Synchronous vs. asynchronous	User request vs. coerced	User maskable vs. nonmaskable	Within vs. between instructions	Resume vs. terminate
I/O device request	Asynchronous	Coerced	Nonmaskable	Between	Resume
Invoke operating system	Synchronous	User request	Nonmaskable	Between	Resume
Tracing instruction execution	Synchronous	User request	User maskable	Between	Resume
Breakpoint	Synchronous	User request	User maskable	Between	Resume
Integer arithmetic overflow	Synchronous	Coerced	User maskable	Within	Resume
Floating-point arithmetic overflow or underflow	Synchronous	Coerced	User maskable	Within	Resume
Page fault	Synchronous	Coerced	Nonmaskable	Within	Resume
Misaligned memory accesses	Synchronous	Coerced	User maskable	Within	Resume
Memory protection violations	Synchronous	Coerced	Nonmaskable	Within	Resume
Using undefined instructions	Synchronous	Coerced	Nonmaskable	Within	Terminate
Hardware malfunctions	Asynchronous	Coerced	Nonmaskable	Within	Terminate
Power failure	Asynchronous	Coerced	Nonmaskable	Within	Terminate

Figure C.26 Five categories are used to define what actions are needed for the different exception types. Exceptions that must allow resumption are marked as resume, although the software may often choose to terminate the program. Synchronous, coerced exceptions occurring within instructions that can be resumed are the most difficult to implement. We might expect that memory protection access violations would always result in termination; however, modern operating systems use memory protection to detect events such as the first attempt to use a page or the first write to a page. Thus, processors should be able to resume after such exceptions.

Utasítások megállítása és folytatása

A legproblémásabb, ha egy kivétel egy utasítás közben (EX, MEM) történik és tudni kell folytatni.

Saving the pipeline:

1. TRAP írása a következő IF-be
2. A TRAP előtti utasítások fagyasztása.
3. TRAP lementi a PC értékét
   vagy PC értékeket, ha branch és delay slot

Vannak nem "precise instructions" (pl. FP) amik már esetleg az utasítás vége előtt (az exception előtt) is kiírják az eredményt. Ekkor vissza kell állítani a forrást akkor is ha pont az lett írva. (De hatékonyság miatt ezt el lehet kerülni – processzor mód átállítása).

További problémák:

- Lehetnek olyan utasítások, amelyek hamarabb kerülnek commited állapotba minthogy azok (vagy az azt megelőzőek) véget értek volna.
- Lehet olyan utasítás, ami többször írja a regisztereket.
- Memory state kezelése.
- Állapotbitek tárolása és visszaállítása.
- Condition code (branch)
- Különböző mennyiségű állomásból álló utasítasok (microinstuction).

Nagy hatékonyságú . . .
5. előadás (ILP II)

Nagy Ádám
nagyadam888@gmail.com

Eötvös Loránd Tudományegyetem
Informatikai Kar

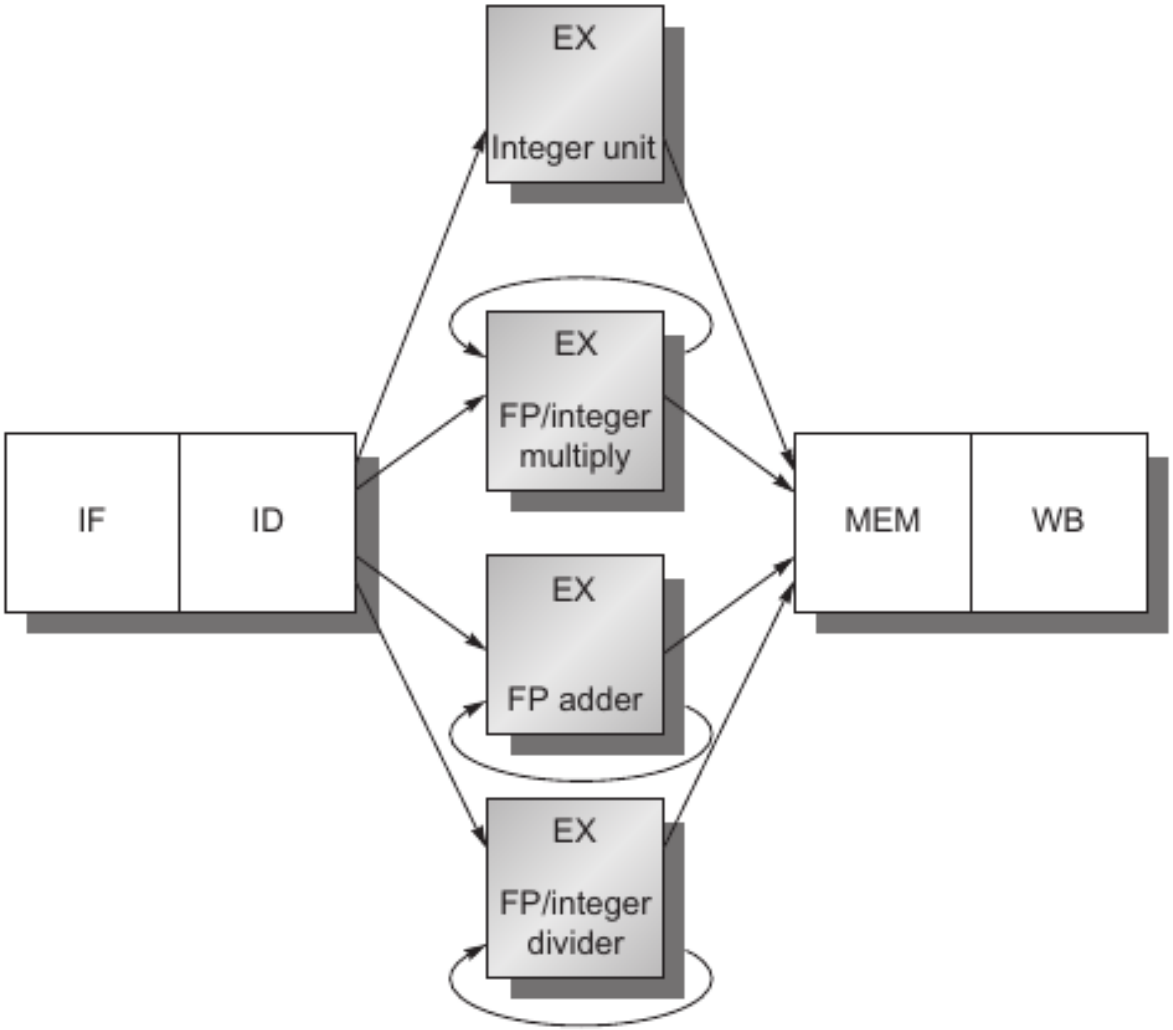
2022. október 10.

Pipeline bonyolultabb utasítások esetén

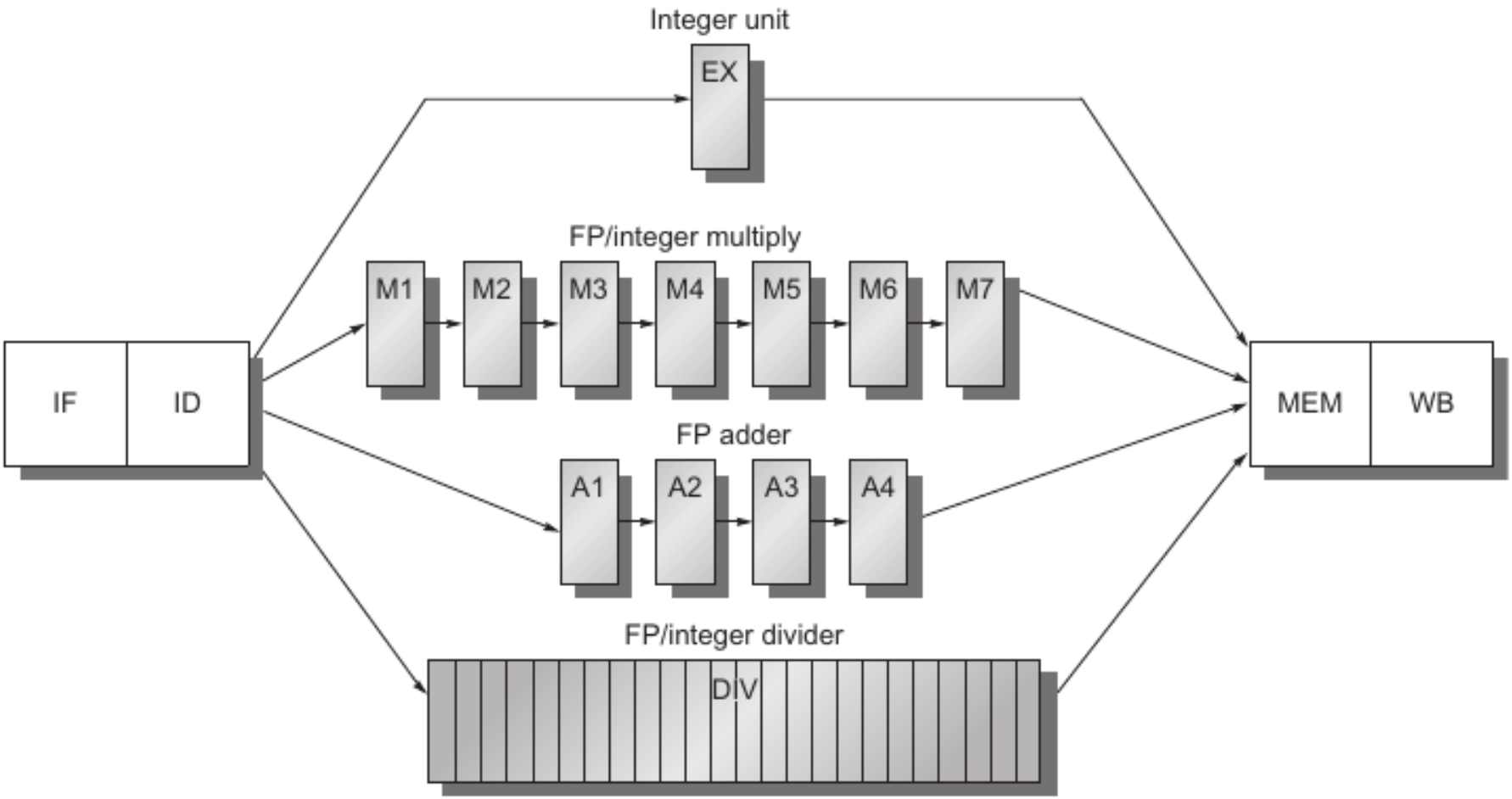
Vannak utasítások, melyek végrehajtásához több órajel kell, ui.

- bonyolultak (FP, int div, stb),
- többféle végrehajtó egység dolgozhat párhuzamosan rajtuk és (ALU, FP add, FP/int mul, FP/int div)
- az egység, ami 1 órajel alatt végre tudná hajtani nagyon költséges lenne.

Megoldás: az EX többször



A hasszabb utasítások több részre oszthatók, így egyszerre több is indítható.



Késleltetések!!!

Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Figure C.29 Latencies and initiation intervals for functional units.

Problémák

1. Structural hazard (pl. int div egy egység)
2. Lehet, hogy többször kell írni a regiszter file-t egy órajel alatt.
3. WAW lehetősége (WB nem eredeti sorrendben)
4. Kivételek kezelése
5. RAW gyakrabban

`fmul.d`	IF	ID	*M1*	M2	M3	M4	M5	M6	**M7**	MEM	WB
`fadd.d`		IF	ID	*A1*	A2	A3	**A4**	**MEM**	WB		
`fadd.d`			IF	ID	*EX*	**MEM**	WB				
`fsd`				IF	ID	*EX*	**MEM**	WB			

Figure C.31 The pipeline timing of a set of independent FP operations. The stages in italics show where data are needed, while the stages in bold show where a result is available. FP loads and stores use a 64-bit path to memory so that the pipelining timing is just like an integer load or store.

	Clock cycle number																
Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
`fld f4,0(x2)`	IF	ID	EX	MEM	WB												
`fmul.d f0,f4,f6`		IF	ID	Stall	M1	M2	M3	M4	M5	M6	M7	MEM	WB				
`fadd.d f2,f0,f8`			IF	Stall	ID	Stall	Stall	Stall	Stall	Stall	Stall	A1	A2	A3	A4	MEM	WB
`fsd f2,0(x2)`					IF	Stall	Stall	Stall	Stall	Stall	Stall	ID	EX	Stall	Stall	Stall	MEM

Figure C.32 A typical FP code sequence showing the stalls arising from RAW hazards. The longer pipeline substantially raises the frequency of stalls versus the shallower integer pipeline. Each instruction in this sequence is dependent on the previous and proceeds as soon as data are available, which assumes the pipeline has full bypassing and forwarding. The `fsd` must be stalled an extra cycle so that its MEM does not conflict with the `fadd.d`. Extra hardware could easily handle this case.

	Clock cycle number										
Instruction	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**
`fmul.d f0,f4,f6`	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
`fadd.d f2,f4,f6`				IF	ID	A1	A2	A3	A4	MEM	WB
...					IF	ID	EX	MEM	WB		
...						IF	ID	EX	MEM	WB	
`fld f2,0(x2)`							IF	ID	EX	MEM	WB

Figure C.33 Three instructions want to perform a write-back to the FP register file simultaneously, as shown in clock cycle 11. This is *not* the worst case, because an earlier divide in the FP unit could also finish on the same clock. Note that although the `fmul.d`, `fadd.d`, and `fld` are in the MEM stage in clock cycle 10, only the `fld` actually uses the memory, so no structural hazard exists for MEM.

- több write port (de általában csak 1 kell)
- ID-nél észrevenni és stall (shift/reservation register)
- ha egyszerre több kérés van akkor ott stall
  (egyszerűnek tűnik, de melyik utasítás, exception?, black out?)
- WAW esetén delay vagy a detecting és nem korábbi esetén nem írni (MEM)
- (Szerencsére) több regiszter típus/file is lehet melyek párhuzamos írása nem okoz gondot (viszont a mozgatás közöttük igen)

Precise utasítások

Out-of-order completion

```
fdiv.d      f0,f2,f4
fadd.d      f10,f10,f8
fsub.d      f12,f12,f14
```

Exception esetén (SUB) valahogy garantálni kellene a korábbi utasítások eredményét (DIV), ami lehetséges lenne pipeline drain-nel, de mi van ha itt is történik egy kivétel.

Megoldások:

- Overlap tiltása.
- Buffer használata (bonyolult)
- history file
- future file
- Imprecise exception (PC tárolása exceptionnél, majd szofveres visszaállítás)
- EX elején megállapítani, hogy történik-e kivétel

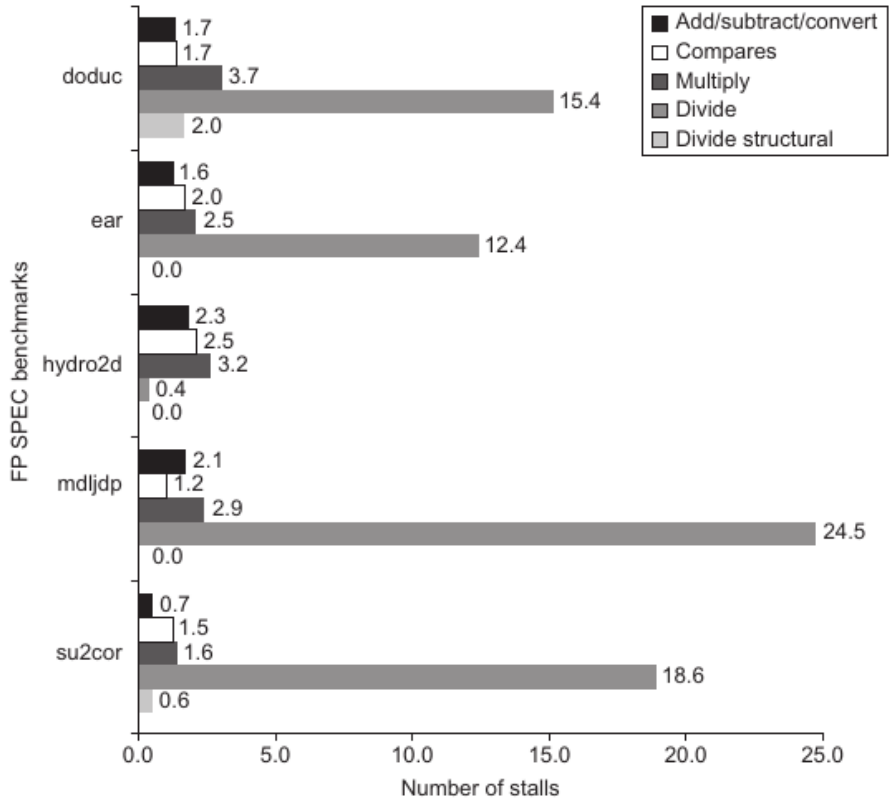


Figure C.34 Stalls per FP operation for each major type of FP operation for the SPEC89 FP benchmarks. Except for the divide structural hazards, these data do not depend on the frequency of an operation, only on its latency and the number of cycles before the result is used. The number of stalls from RAW hazards roughly tracks the latency of the FP unit. For example, the average number of stalls per FP add, subtract, or convert is 1.7 cycles, or 56% of the latency (three cycles). Likewise, the average number of stalls for multiplies and divides are 2.8 and 14.2, respectively, or 46% and 59% of the corresponding latency. Structural hazards for divides are rare, because the divide frequency is low.

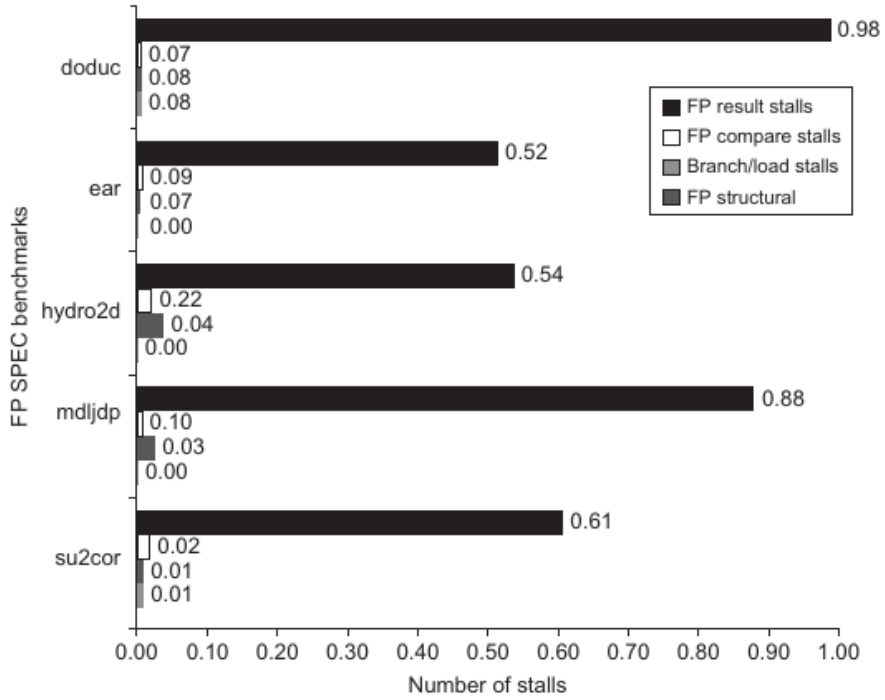


Figure C.35 The stalls occurring for the a simple RISC V FP pipeline for five of the SPEC89 FP benchmarks. The total number of stalls per instruction ranges from 0.65 for su2cor to 1.21 for doduc, with an average of 0.87. FP result stalls dominate in all cases, with an average of 0.71 stalls per instruction, or 82% of the stalled cycles. Compares generate an average of 0.1 stalls per instruction and are the second largest source. The divide structural hazard is only significant for doduc. Branch stalls are not accounted for, but would be small.

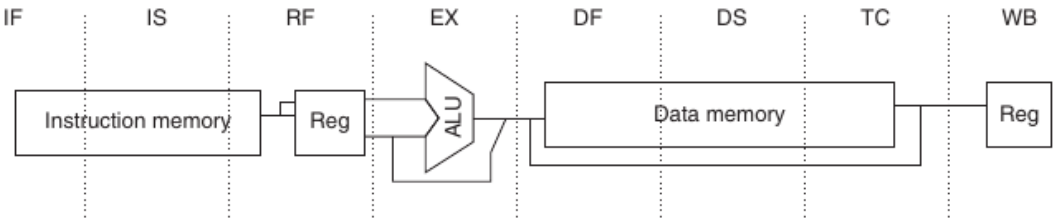


Figure C.36 The eight-stage pipeline structure of the R4000 uses pipelined instruction and data caches. The pipe stages are labeled and their detailed function is described in the text. The vertical dashed lines represent the stage boundaries as well as the location of pipeline latches. The instruction is actually available at the end of IS, but the tag check is done in RF, while the registers are fetched. Thus, we show the instruction memory as operating through RF. The TC stage is needed for data memory access, because we cannot write the data into the register until we know whether the cache access was a hit or not.

Stage	Functional unit	Description
A	FP adder	Mantissa add stage
D	FP divider	Divide pipeline stage
E	FP multiplier	Exception test stage
M	FP multiplier	First stage of multiplier
N	FP multiplier	Second stage of multiplier
R	FP adder	Rounding stage
S	FP adder	Operand shift stage
U		Unpack FP numbers

Figure C.41 The eight stages used in the R4000 floating-point pipelines.

FP instruction	Latency	Initiation interval	Pipe stages
Add, subtract	4	3	U, S+A, A+R, R+S
Multiply	8	4	U, E+M, M, M, M, N, N+A, R
Divide	36	35	U, A, R, D^{28}, D+A, D+R, D+A, D+R, A, R
Square root	112	111	U, E, $(A+R)^{108}$, A, R
Negate	2	1	U, S
Absolute value	2	1	U, S
FP compare	3	2	U, A, R

Figure C.42 The latencies and initiation intervals for the FP operations initiation intervals for the FP operations both depend on the FP unit stages that a given operation must use. The latency values assume that the destination instruction is an FP operation; the latencies are one cycle less when the destination is a store. The pipe stages are shown in the order in which they are used for any operation. The notation S+A indicates a clock cycle in which both the S and A stages are used. The notation D^{28} indicates that the D stage is used 28 times in a row.

		Clock cycle												
Operation	**Issue/stall**	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
Multiply	Issue	U	E+M	M	M	M	N	**N+A**	**R**					
Add	Issue		U	S+A	A+R	R+S								
	Issue			U	S+A	A+R	R+S							
	Issue				U	S+A	A+R	R+S						
	Stall					U	S+A	**A+R**	**R+S**					
	Stall						U	**S+A**	**A+R**	R+S				
	Issue							U	S+A	A+R	R+S			
	Issue								U	S+A	A+R	R+S		

Figure C.43 An FP multiply issued at clock 0 is followed by a single FP add issued between clocks 1 and 7. The second column indicates whether an instruction of the specified type stalls when it is issued *n* cycles later, where *n* is the clock cycle number in which the U stage of the second instruction occurs. The stage or stages that cause a stall are in bold. Note that this table deals with only the interaction between the multiply and *one* add issued between clocks 1 and 7. In this case, the add will stall if it is issued four or five cycles after the multiply; otherwise, it issues without stalling. Notice that the add will be stalled for two cycles if it issues in cycle 4 because on the next clock cycle it will still conflict with the multiply; if, however, the add issues in cycle 5, it will stall for only 1 clock cycle, because that will eliminate the conflicts.

		Clock cycle												
Operation	**Issue/stall**	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
Add	Issue	U	S+A	A+R	R+S									
Multiply	Issue		U	E+M	M	M	M	N	N+A	R				
	Issue			U	M	M	M	M	N	N+A	R			

Figure C.44 A multiply issuing after an add can always proceed without stalling, because the shorter instruction clears the shared pipeline stages before the longer instruction reaches them.

		Clock cycle												
Operation	**Issue/stall**	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
Add	Issue	U	S+A	**A+R**	**R+S**									
Divide	Stall		U	**A**	**R**	D	D	D	D	D	D	D	D	D
	Issue			U	A	R	D	D	D	D	D	D	D	D
	Issue				U	A	R	D	D	D	D	D	D	D

Figure C.46 **A double-precision add is followed by a double-precision divide.** If the divide starts one cycle after the add, the divide stalls, but after that there is no conflict.

		Clock cycle											
Operation	**Issue/stall**	**25**	**26**	**27**	**28**	**29**	**30**	**31**	**32**	**33**	**34**	**35**	**36**
Divide	Issued in cycle 0…	D	D	D	D	D	**D+A**	**D+R**	**D+A**	**D+R**	**A**	**R**	
Add	Issue		U	S+A	A+R	R+S							
	Issue			U	S+A	A+R	R+S						
	Stall				U	S+A	**A+R**	**R+S**					
	Stall					U	**S+A**	**A+R**	R+S				
	Stall						U	S+A	**A+R**	**R+S**			
	Stall							U	**S+A**	**A+R**	R+S		
	Stall								U	S+A	**A+R**	**R+S**	
	Stall									U	**S+A**	**A+R**	R+S
	Issue										U	S+A	A+R
	Issue											U	S+A
	Issue												U

Figure C.45 An FP divide can cause a stall for an add that starts near the end of the divide. The divide starts at cycle 0 and completes at cycle 35; the last 10 cycles of the divide are shown. Because the divide makes heavy use of the rounding hardware needed by the add, it stalls an add that starts in any of cycles 28–33. Notice that the add starting in cycle 28 will be stalled until cycle 36. If the add started right after the divide, it would not conflict, because the add could complete before the divide needed the shared stages, just as we saw in Figure C.44 for a multiply and add. As in the earlier figure, this example assumes *exactly* one add that reaches the U stage between clock cycles 26 and 35.

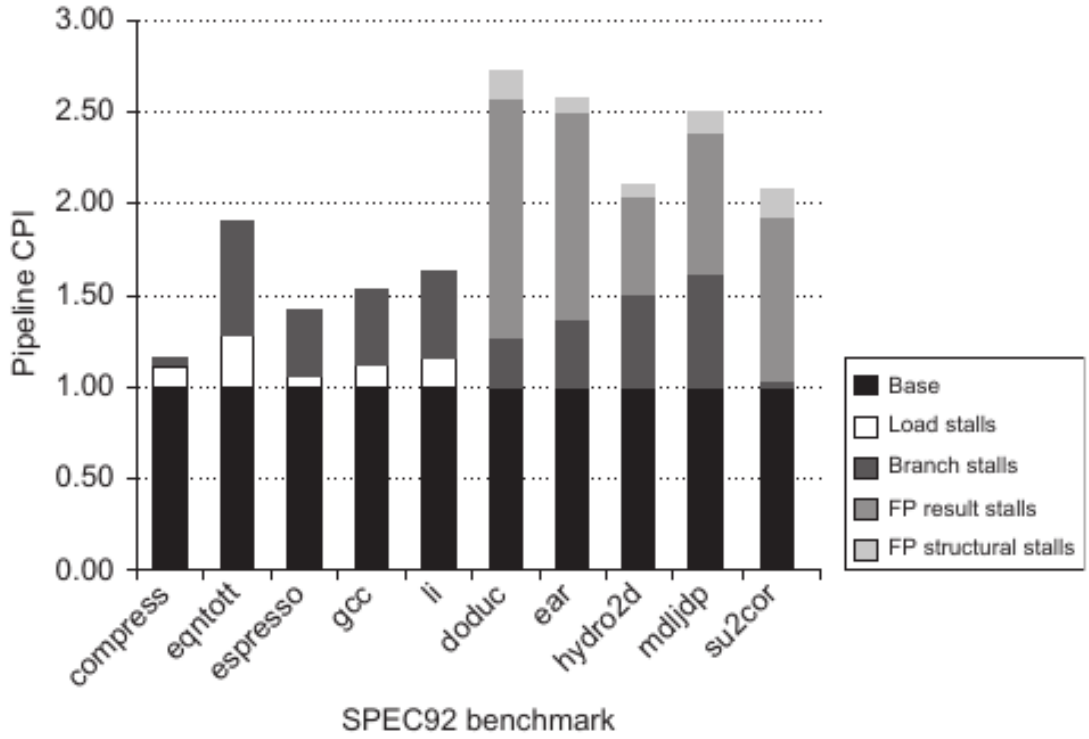


Figure C.47 The pipeline CPI for 10 of the SPEC92 benchmarks, assuming a perfect cache. The pipeline CPI varies from 1.2 to 2.8. The left-most five programs are integer programs, and branch delays are the major CPI contributor for these. The right-most five programs are FP, and FP result stalls are the major contributor for these. Figure C.48 shows the numbers used to construct this plot.

Benchmark	Pipeline CPI	Load stalls	Branch stalls	FP result stalls	FP structural stalls
Compress	1.20	0.14	0.06	0.00	0.00
Eqntott	1.88	0.27	0.61	0.00	0.00
Espresso	1.42	0.07	0.35	0.00	0.00
Gcc	1.56	0.13	0.43	0.00	0.00
Li	1.64	0.18	0.46	0.00	0.00
Integer average	1.54	0.16	0.38	0.00	0.00
Doduc	2.84	0.01	0.22	1.39	0.22
Mdljdp2	2.66	0.01	0.31	1.20	0.15
Ear	2.17	0.00	0.46	0.59	0.12
Hydro2d	2.53	0.00	0.62	0.75	0.17
Su2cor	2.18	0.02	0.07	0.84	0.26
FP average	2.48	0.01	0.33	0.95	0.18
Overall average	2.00	0.10	0.36	0.46	0.09

Figure C.48 The total pipeline CPI and the contributions of the four major sources of stalls are shown. The major contributors are FP result stalls (both for branches and for FP inputs) and branch stalls, with loads and FP structural stalls adding less.

Utasítások ütemezése

Statikus compiler végzi
Dinamikus in-order helyett out-of-ordert szeretnénk
ID két lépésben:

1. Issue (decode and check for hazards) – in-order
2. Read operands (ha nincs hazard, különben várakozás) – out-of-order

WAR és WAW is előfordulhat

```
fdiv.d      f0,f2,f4
fadd.d      f10,f0,f8
fsub.d      f8,f8,f14
```

Scoreboard használata

Az utasítások négy részre osztása ID, EX, WB helyett:

1. Issue
2. Read data (feltételek)
3. Execution
4. Write results

Sajnos forwarding nem lehetséges. Nagy számú funkcionális egységek használata párhuzamosan.

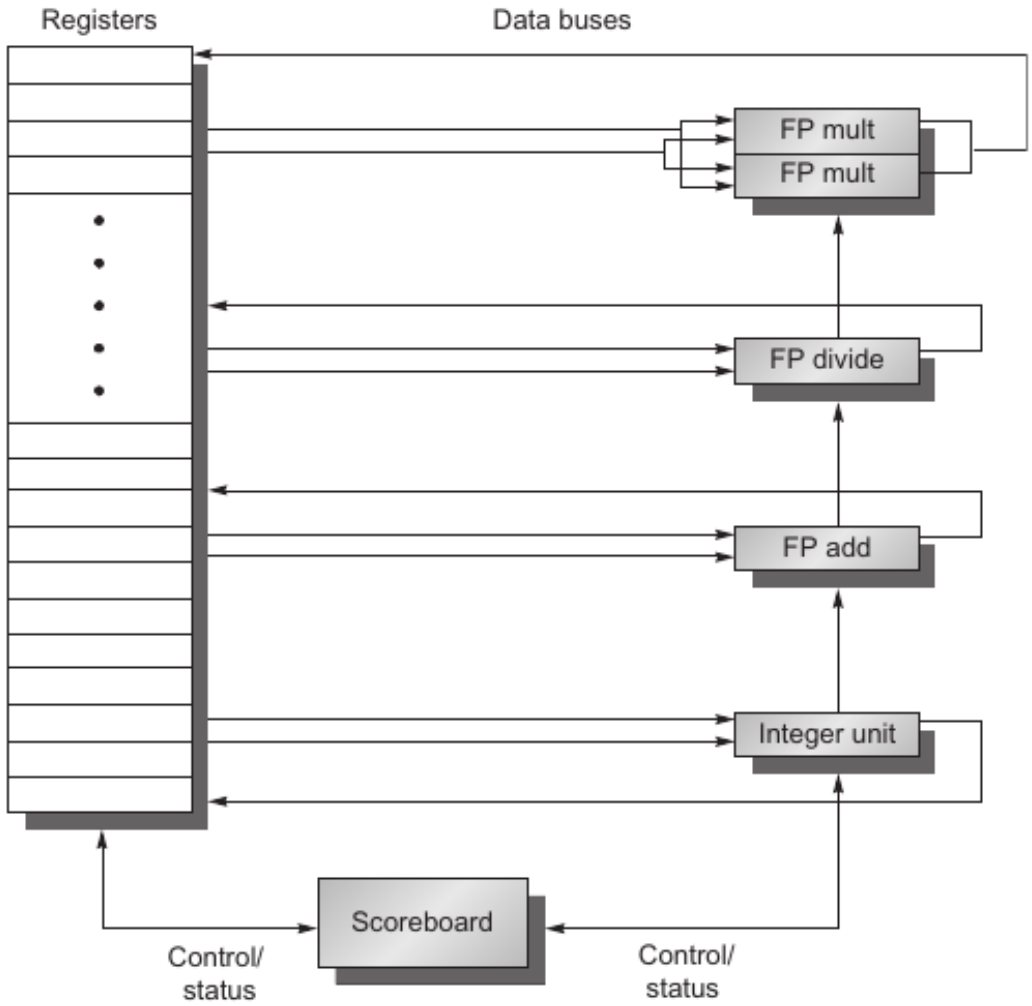


Figure C.49 The basic structure of a RISC V processor with a scoreboard. The score-

		Instruction status			
Instruction		Issue	Read operands	Execution complete	Write result
L.D	F6,34(R2)	√	√	√	√
L.D	F2,45(R3)	√	√	√	
MUL.D	F0,F2,F4	√			
SUB.D	F8,F6,F2	√			
DIV.D	F10,F0,F6	√			
ADD.D	F6,F8,F2				

	Functional unit status								
Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	Yes	Load	F2	R3				No	
Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2		Integer	Yes	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

	Register result status								
	F0	F2	F4	F6	F8	F10	F12	...	F30
FU	Mult1	Integer			Add	Divide			

Hatékonság függ:

1. maximálisan kihasználható ILP
2. scoreboard bejegyzések maximális száma (window)
3. funkcionális egységek száma
4. Függőségek száma (WAR, WAW)

Hogyan használunk ki egy nagy ablakot (branch)?

Ma már van jobb eszközünk is.

Buktatók (és tévhitek)

- P WAW nincs

```
                BNEZ    R1,foo
                DIV.D   F0,F2,F4; moved into delay slot
                        ;from fall through
                .....
                .....
foo:            L.D     F0,qrs
```

- P Nem optimalizált kód alapján megítélni a (statikus/dinamikus) ütemezést
- P Bonyolultság-sebesség-ILP.

Nagy hatékonyságú . . .
6. előadás (ILP)

Nagy Ádám
nagyadam888@gmail.com

Eötvös Loránd Tudományegyetem
Informatikai Kar

2022. október 18.

ILP kihasználása =

"mapping between program structure and hardware structure"

Technique	Reduces	Section
Forwarding and bypassing	Potential data hazard stalls	C.2
Simple branch scheduling and prediction	Control hazard stalls	C.2
Basic compiler pipeline scheduling	Data hazard stalls	C.2, 3.2
Basic dynamic scheduling (scoreboarding)	Data hazard stalls from true dependences	C.7
Loop unrolling	Control hazard stalls	3.2
Advanced branch prediction	Control stalls	3.3
Dynamic scheduling with renaming	Stalls from data hazards, output dependences, and antidependences	3.4
Hardware speculation	Data hazard and control hazard stalls	3.6
Dynamic memory disambiguation	Data hazard stalls with memory	3.6
Issuing multiple instructions per cycle	Ideal CPI	3.7, 3.8
Compiler dependence analysis, software pipelining, trace scheduling	Ideal CPI, data hazard stalls	H.2, H.3
Hardware support for compiler speculation	Ideal CPI, data hazard stalls, branch hazard stalls	H.4, H.5

Figure 3.1 The major techniques examined in Appendix C, Chapter 3, and Appendix H are shown together with the component of the CPI equation that the technique affects.

ILP csak elágazás nélküli programblokkokban

Probléma: az utasítások 15-25%-a branch $\Rightarrow$ túl kicsik a blokkok

Megoldás: egyszerű elágazások és leginkább a ciklusok kezelése

Loop level paralellism
(alapvetően nem lenne lehetőség ILP kihasználására)

```
for (i=0; i<=999; i=i+1)
    x[i] = x[i] + y[i]
```

Dinamikus/statikus unrolling, SIMD

ILP kihasználáshoz: uasítások egymástól való függőségeinek vizsgálata

Data dependencies

RAW, WAW, WAR
Lehet:

1. (true) data dependencies
   Az i utasítás adatfügg j utasítástól, ha
   - j utasítás használja i utasítás eredményét
   - $\exists k$ utasítás, amely adatfügg i-től és j adatfügg tőle
2. name dependencies
   névfüggőség van az i és j utasítások között, ha i megelőzi j-t és
   - (antidependence) i olvassa azt a regisztert, amit j ír
   - (output dependence) i és j is ugyanazt a regisztert írja

   Megoldás lehet: register renaming
3. control dependencies
   Az utasítások végrehajtásának sorrendje függ valamely megelőző utasítástól.

Data dependencies

A program egy tualjdonsága, amely

- lehet, hogy nem okoz többlet futási időt,
- ütemezéssel megoldható (az adatfüggőség megmarad),
- programtranszformációval kiküszöbölhető vagy
- blokkolja a pipeline-t valamilyen formában.

Valódi adatfüggőség $\Rightarrow$

- hazard lehetősége
- az utasítások végrehajtásának sorrendje
- mennyire alkalmazható ILP

Valódi adatfüggőség felderítése és kezelése memóriában jóval problémásabb (pl. $2A + 20 =? 3B - 5 =? 2A' + 20$), statikus elemzés szerepe nagyon fontos.

Control dependencies

Az elágazások meghatároznak egy hatáskört, melyből és melybe elméletileg nem lehet utasítást (át)ütemezni. Egy elégazás feltételének számolása is ilyen.

Hatékonysági okokból ez a függőség sérül. Helyette megőrzik:

- *exception behavior*
  Általában: az utasítások áthelyezés miatt nem lehet kivétel

```
        add     x2,x3,x4
        beqz    x2,x0,L1
        ld      x1,0(x2)
L1:     ...
```

- *data flow* ($>$ data dependence)

```
        add     x1,x2,x3
        beqz    x4,x0,L
        sub     x1,x5,x6
L:      ...
        or      x7,x1,x8
```

  dead utasítások mozgatása (szoftver spekuláció)

Pipeline ütemezés és Unrolling

Feltételek:

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Figure 3.2 Latencies of FP operations used in this chapter. The last column is the number of intervening clock cycles needed to avoid a stall. These numbers are similar to the average latencies we would see on an FP unit. The latency of a floating-point load to a store is 0 because the result of the load can be bypassed without stalling the store. We will continue to assume an integer load latency of 1 and an integer ALU operation latency of 0 (which includes ALU operation to branch).

Feladat:

```
for (i=999l i>=0; i=i-1)
    x[i] = x[i] + s;
```

Unrolling

```
Loop:  fld     f0,0(x1)     //f0=array element
       fadd.d  f4,f0,f2     //add scalar in f2
       fsd     f4,0(x1)     //store result
       addi    x1,x1,-8     //decrement pointer
                            //8 bytes (per DW)
       bne     x1,x2,Loop   //branch x1≠x2
```

			Clock cycle issued
Loop:	fld	f0,0(x1)	1
	stall		2
	fadd.d	f4,f0,f2	3
	stall		4
	stall		5
	fsd	f4,0(x1)	6
	addi	x1,x1,-8	7
	bne	x1,x2,Loop	8

```
Loop:  fld     f0,0(x1)
       addi    x1,x1,-8
       fadd.d  f4,f0,f2
       stall
       stall
       fsd     f4,8(x1)
       bne     x1,x2,Loop
```

Loop unrolling:

```
Loop:  fld     f0,0(x1)
       fadd.d  f4,f0,f2
       fsd     f4,0(x1)      //drop addi & bne
       fld     f6,-8(x1)
       fadd.d  f8,f6,f2
       fsd     f8,-8(x1)     //drop addi & bne
       fld     f0,-16(x1)
       fadd.d  f12,f0,f2
       fsd     f12,-16(x1)   //drop addi & bne
       fld     f14,-24(x1)
       fadd.d  f16,f14,f2
       fsd     f16,-24(x1)
       addi    x1,x1,-32
       bne     x1,x2,Loop
```

Egyszerű, de az új ciklusváltozó meghatározása bonyolult is lehet és a ciklus mérete nem biztosan osztható a kibontott iterációk számával.

Ütemezés a ciklus kibontása után a késleltetéseknek megfelelően:

```
Loop:   fld     f0,0(x1)
        fld     f6,-8(x1)
        fld     f0,-16(x1)
        fld     f14,-24(x1)
        fadd.d  f4,f0,f2
        fadd.d  f8,f6,f2
        fadd.d  f12,f0,f2
        fadd.d  f16,f14,f2
        fsd     f4,0(x1)
        fsd     f8,-8(x1)
        fsd     f12,16(x1)
        fsd     f16,8(x1)
        addi    x1,x1,-32
        bne     x1,x2,Loop
```

Az ütemezés jelentősége és hatása jóval nagyobb a kibontott cikluson.

Ütemezési és transzformációs lépések

- Unroll hasznosságának és a iterációk egymástól való függésének feltárása.
- Különböző regiszterek használata az unroll-nál.
- A fölösleges kódrészek eltávolítása és a feltétel módosítása.
- Memóriaelérések vizsgálata (függetlenek-e).
- Ütemezés úgy, hogy az eredmény megegyezzen az eredeti kóddal.

Problémák:

- Kis ciklusok esetén nem éri meg.
- Megnő a kód mérete $\Rightarrow$ több I cachemiss.
- Sok regiszter lehet szükséges (register pressure).
- Bonyolultabb compiler.

Advanced branch prediction – Correlating branches

Az egymás utáni ,,független" elágazások között sok esetben összefüggés van, amelyet ki lehet használni az elágazás eredményének jóslására.

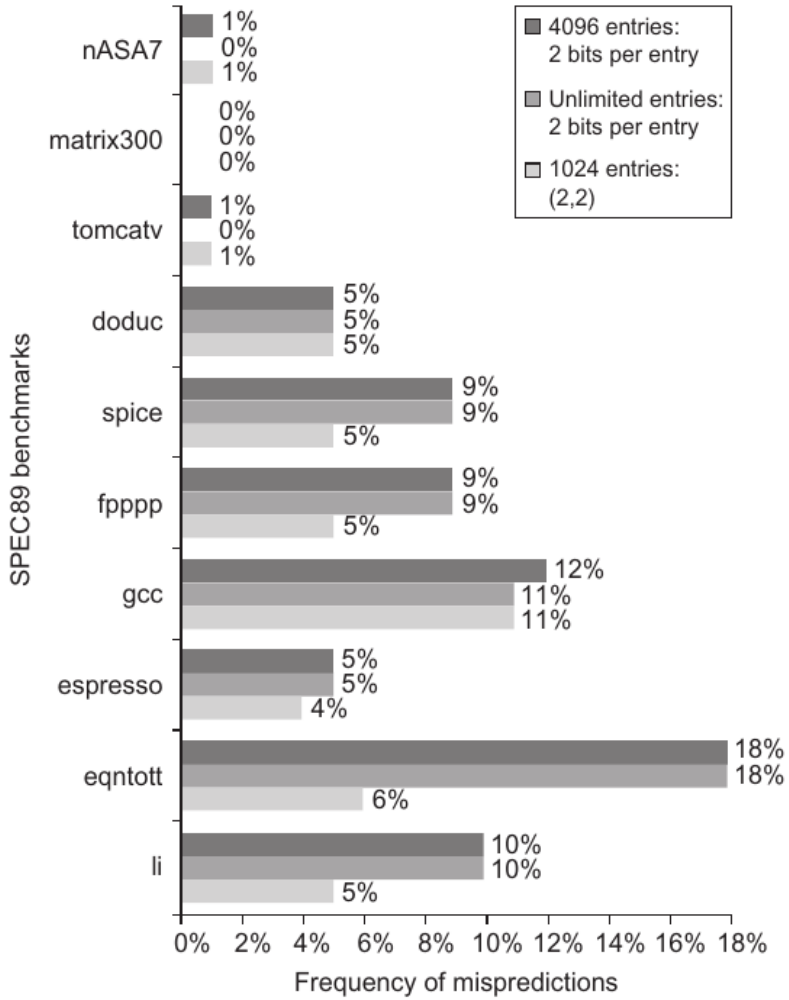
```
if (aa == 2)
    aa = 0;
if (bb == 2)
    bb = 0;
if (aa != bb)
    ...
```

Shiftregiszterben tárolható a korábbi ciklusok jóslásához használt bitek.

(n, m) *predictor*: elágazások jóslása m bittel és az előző n elágazás figyelembevételével.

A hardver nem válik sokkal bonyolultabbá.

(0,2) 4K bejegyzéssel ugyanannyi bitet használ mint (2,2) 1K bejegyzéssel.


4096 entries:
2 bits per entry
Unlimited entries:
2 bits per entry
1024 entries:
(2,2)
nASA7
matrix300
tomcatv
doduc
spice
fpppp
gcc
espresso
eqntott
li
1%
0%
1%
0%
0%
0%
1%
0%
1%
5%
5%
5%
9%
9%
5%
9%
9%
5%
12%
11%
11%
5%
5%
4%
18%
18%
6%
10%
10%
5%
SPEC89 benchmarks
0% 2% 4% 6% 8% 10% 12% 14% 16% 18%
Frequency of mispredictions

Advanced branch prediction – Tournament predictors

Lokális és globális információ használata a jóslásnál.

Több jósló használata (ált. 1 globális és 1 lokális) szelektorral az alapján, hogy melyik volt jobb a legutóbbi jóslásoknál.

Lokális tábla lehet 2 szintű pl. így:

- Felső szint 1024 db 10 bites bejegyzés, ami tárolja a 10 legutóbbi jóslás eredményeit az adott bejegyzéshez.
- A bejegyzések indexelnek egy (0,3) paraméterű jósló táblát, ami használható abban az esetben, ha a bejegyzés nincs a fenti táblában.

További előny: nagy jósló táblák hatékony használata.

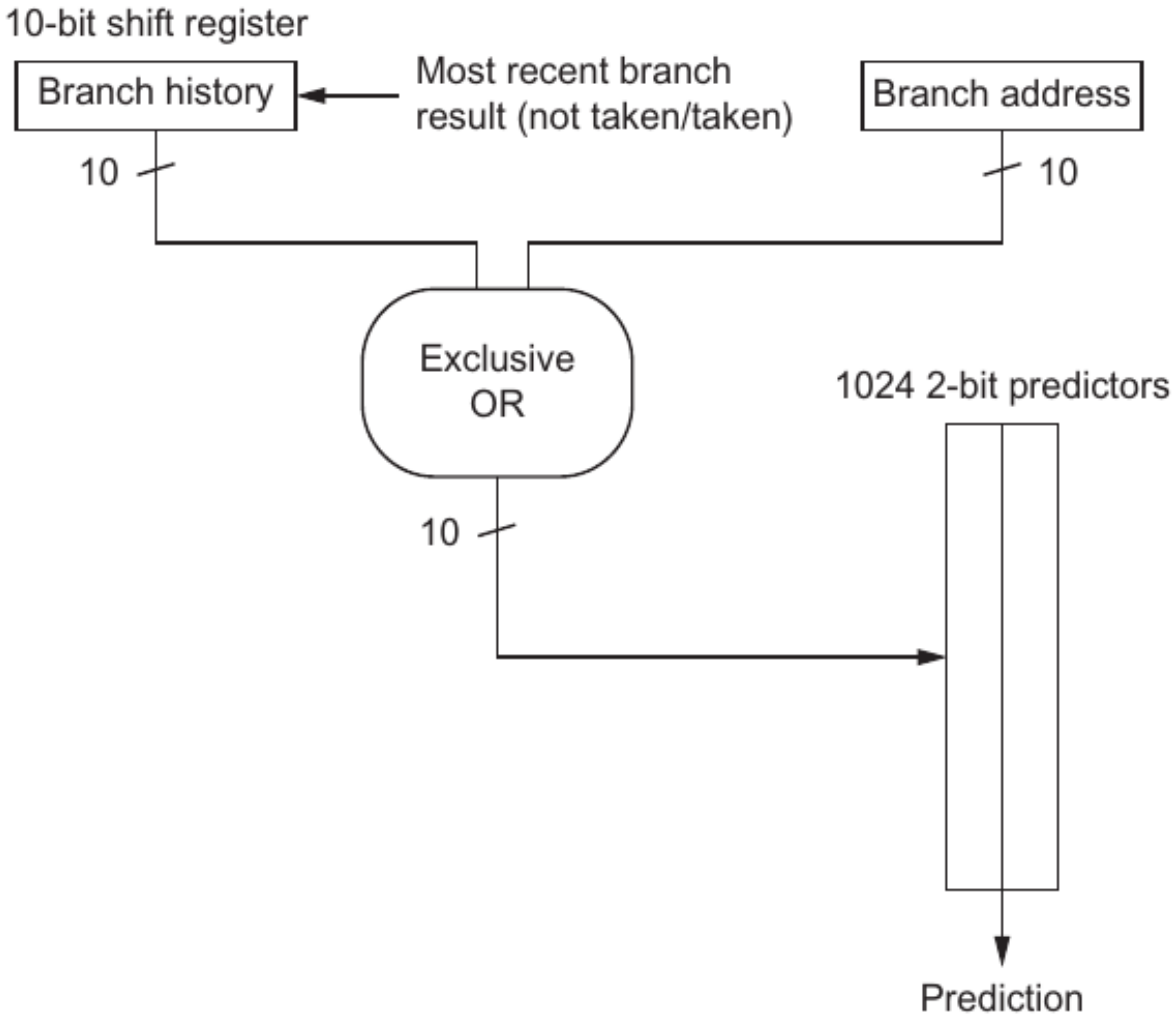


Figure 3.4 A gshare predictor with 1024 entries, each being a standard 2-bit predictor.

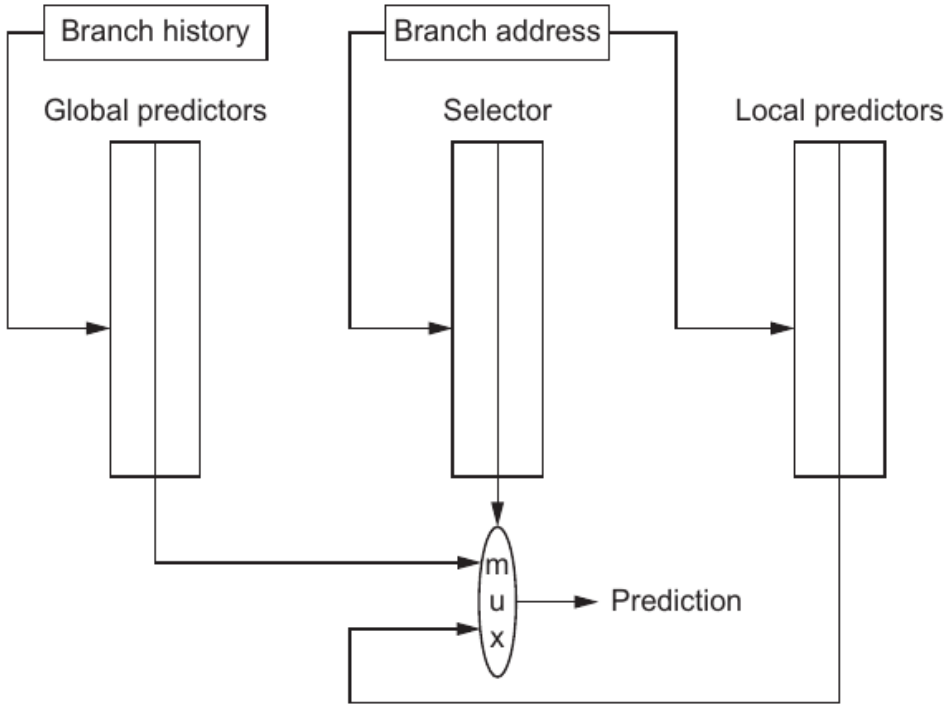


Figure 3.5 A tournament predictor using the branch address to index a set of 2-bit selection counters, which choose between a local and a global predictor. In this case, the index to the selector table is the current branch address. The two tables are also 2-bit predictors that are indexed by the global history and branch address, respectively. The selector acts like a 2-bit predictor, changing the preferred predictor for a branch address when two mispredicts occur in a row. The number of bits of the branch address used to index the selector table and the local predictor table is equal to the length of the global branch history used to index the global prediction table. Note that misprediction is a bit tricky because we need to change both the selector table and either the global or local predictor.

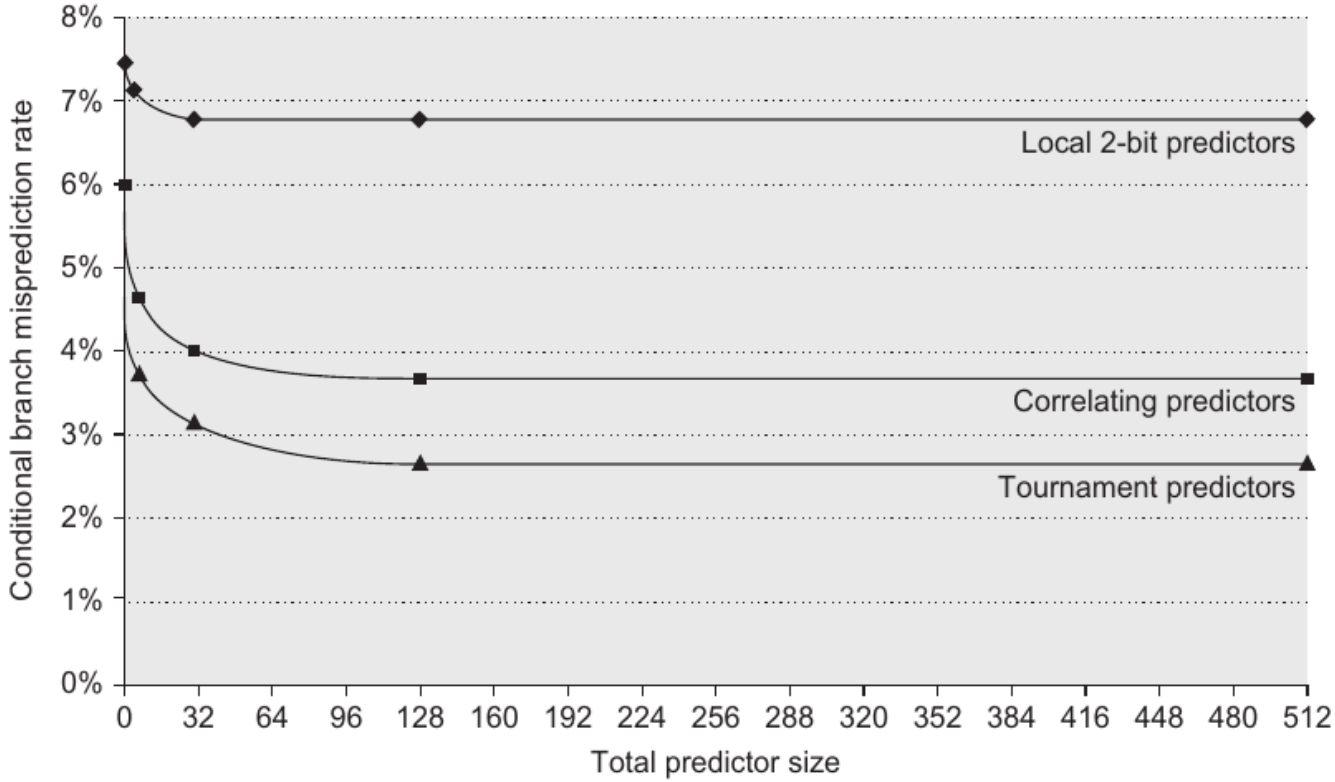


Figure 3.6 The misprediction rate for three different predictors on SPEC89 versus the size of the predictor in kilobits. The predictors are a local 2-bit predictor, a correlating predictor that is optimally structured in its use of global and local information at each point in the graph, and a tournament predictor. Although these data are for an older version of SPEC, data for more recent SPEC benchmarks show similar behavior, perhaps converging to the asymptotic limit at slightly larger predictor sizes.

Tagged Hybrid Predictors – TAgged GEometric

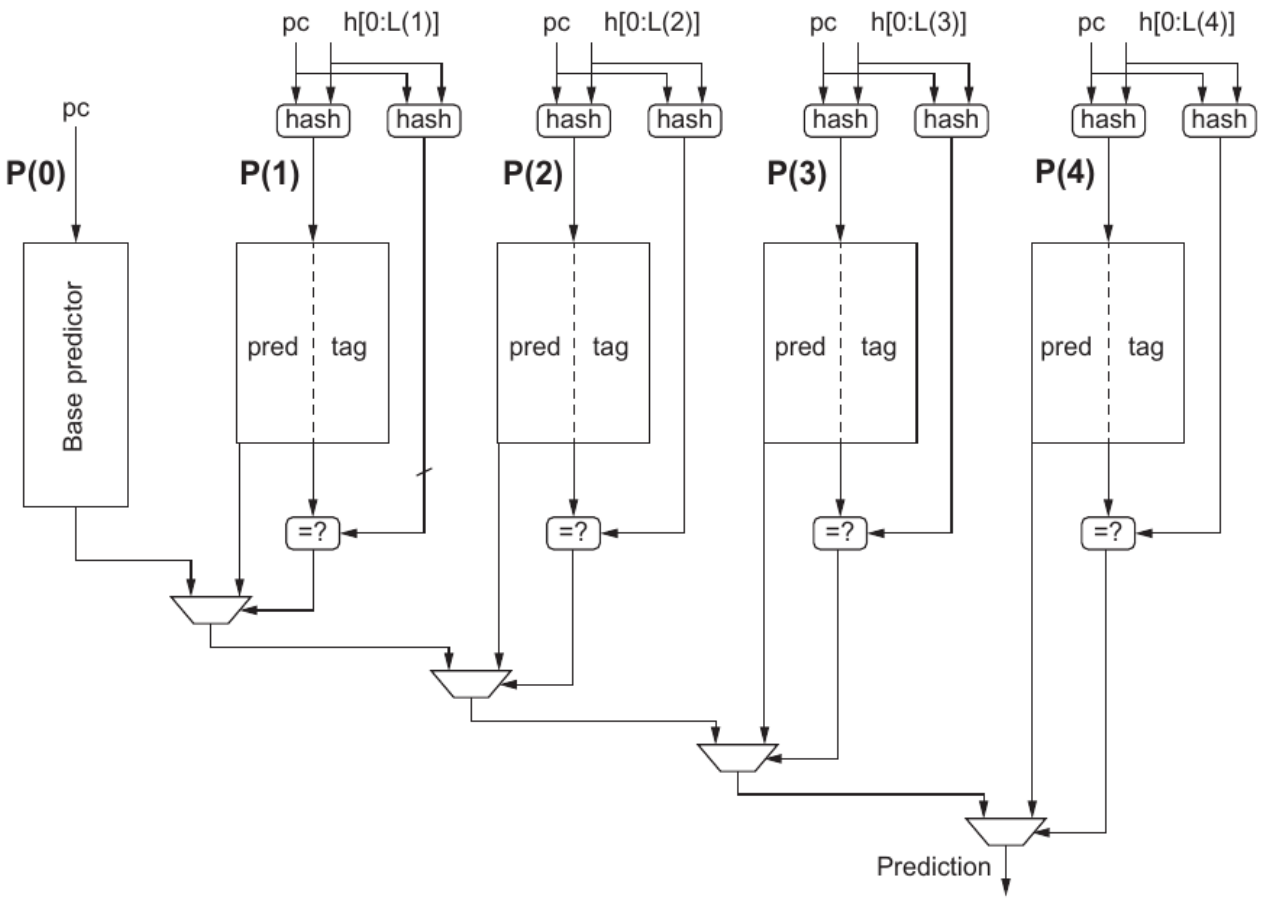


Figure 3.7 A five-component tagged hybrid predictor has five separate prediction tables, indexed by a hash of the branch address and a segment of recent branch history of length 0–4 labeled "h" in this figure. The hash can be as simple as an exclusive-OR, as in gshare. Each predictor is a 2-bit (or possibly 3-bit) predictor. The tags are typically 4–8 bits. The chosen prediction is the one with the longest history where the tags also match.

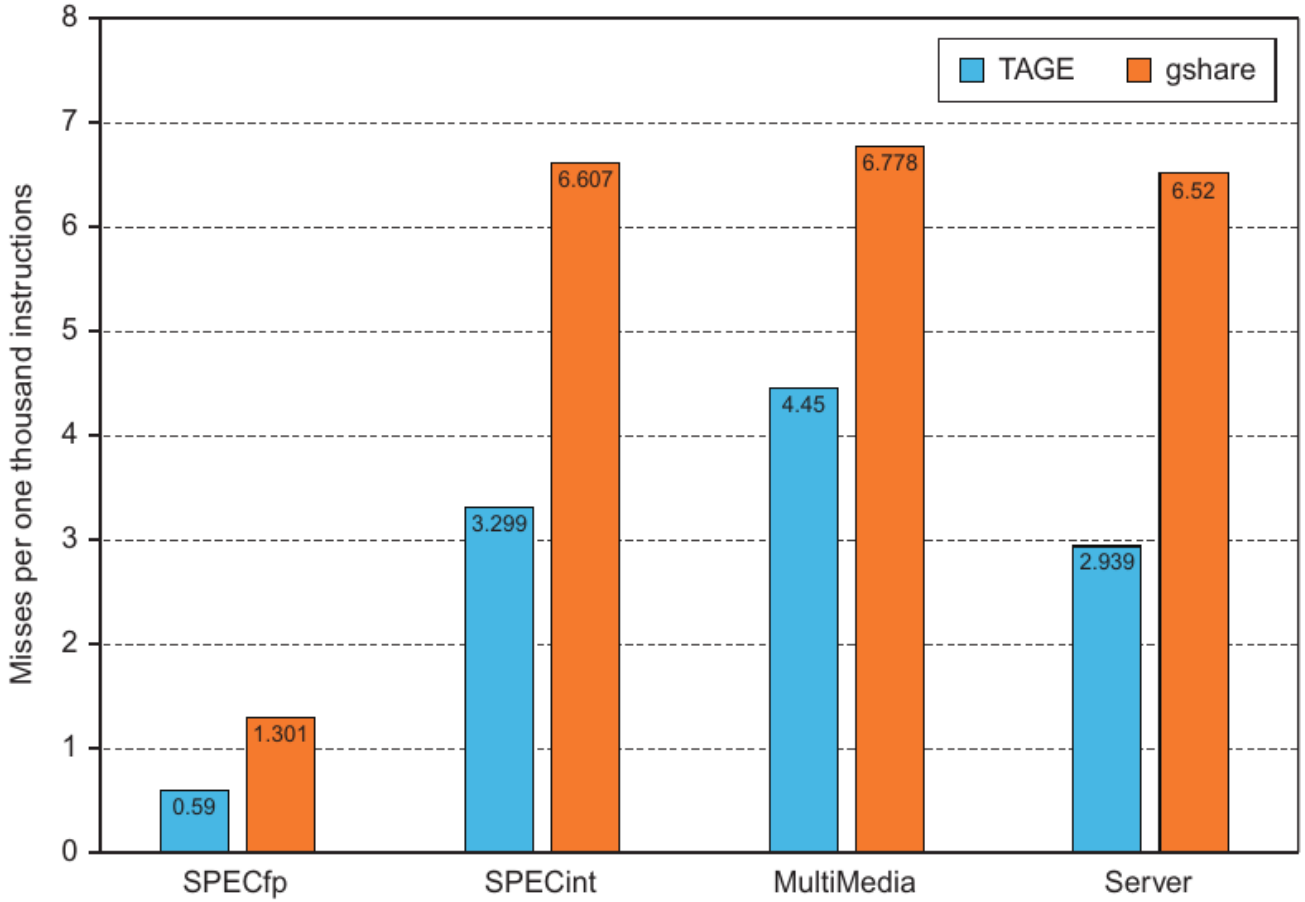


Figure 3.8 A comparison of the misprediction rate (measured as mispredicts per 1000 instructions executed) for tagged hybrid versus gshare. Both predictors use the same total number of bits, although tagged hybrid uses some of that storage for tags, while gshare contains no tags. The benchmarks consist of traces from SPECfp and SPECint, a series of multimedia and server benchmarks. The latter two behave more like SPECint.

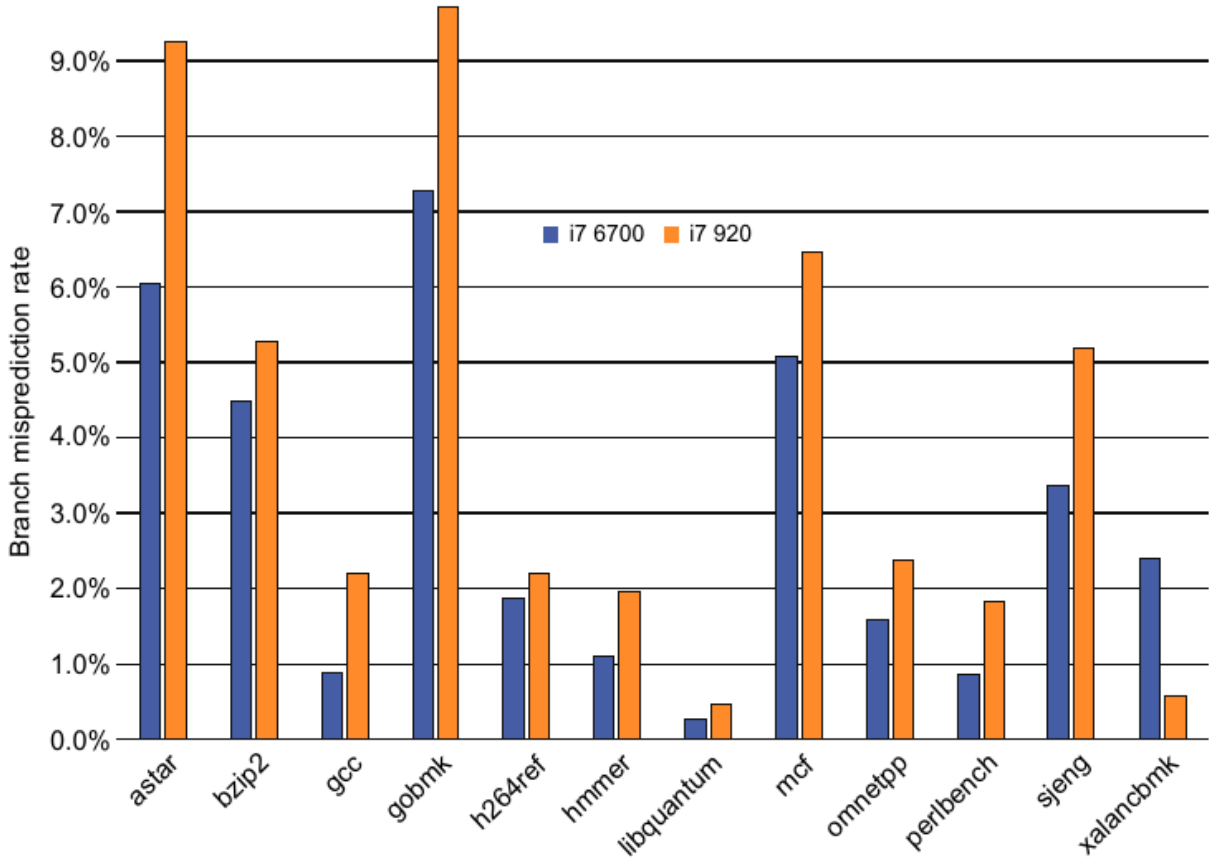


Figure 3.9 The misprediction rate for the integer SPECCPU2006 benchmarks on the Intel Core i7 920 and 6700. The misprediction rate is computed as the ratio of completed branches that are mispredicted versus all completed branches. This could understate the misprediction rate somewhat because if a branch is mispredicted and led to another mispredicted branch (which should not have been executed), it will be counted as only one misprediction. On average, the i7 920 mispredicts branches 1.3 times as often as the i7 6700.

Dinamikus ütemezés

Eml.: Ütemezés scoreboard segítségével.

Dinamikus ütemezés előnyei:

- Fordított kód (más pipeline használatával) hatékony futtatása különböző hardveren.
- Használható olyan esetekben is, amikor fordítási időben nem volt meghatározható a program egésze (dynamic links, dispatching).
- El tudja rejteni a nem várt késleltetéseket (cache, I/O, stb.)

Out-of-order execution/competition $\Rightarrow$ WAW, WAR $\Rightarrow$ register renaming

Dinamikus ütemezés és kivételek

Dinamikus ütemezés esetén nehéz precíz kivételeket garantálni, ui.

- A pipeline lehet, hogy már befejezett olyan utasításokat, amelyek a kivételt kiváltó utasítás után vannak.
- A pipeline lehet, hogy még nem fejezett be olyan utasításokat, amelyek a kivételt kiváltó utasítás előt vannak.

Részleges megoldás lehet a kivételek késleltetése.

Megoldás: spekulatív végrehajtás (később) ötvözése dinamikus ütemezéssel. $\Rightarrow$ Sajnos a scoreboard nem (jól) használható.

Tomasulo algoritmus

Robert Tomasulo IBM 360 FP számítások hatékony ütemezésére, WAR és WAW problémák kiküszöbölésére

Tomasulo algoritmus:

1. dinamikus ütemezés (tudja azt amit a scoreboard)
2. megoldás WAR és WAW problémákra (renaming)
3. kompatibilis spekulatív végrehajtással
4. magasabb kibocsájtású processzorok (> 4) esetén is jól használható
5. ciklus végrehajtása overlapped iterációkkal

Register renaming

WAR és WAW elkerüléséhez használhatunk további regisztereket (T,S)

```
fdiv.d  f0,f2,f4          fdiv.d  f0,f2,f4
fadd.d  f6,f0,f8          fadd.d  S,f0,f8
fsd     f6,0(x1)    ⇒     fsd     S,0(x1)
fsub.d  f8,f10,f14        fsub.d  T,f10,f14
fmul.d  f6,f10,f8         fmul.d  f6,f10,T
```

Statikus esetben a compiler is végezheti, de ekkor elleőrizni kell az elágazásokban is a régi/új regiszterek használatát (bonyolult).

A Tomasulo algoritmus egy *reservation station* elnevezésű buffert használ. A beolvasott adatok és a köztes eredmények ide kerülnek és csak akkor kerülnek a regiszterekbe, ha már nem használjuk őket. A reservation buffer mérete általában nagyobb mint a rendelkezésre álló regiszterek összesen.

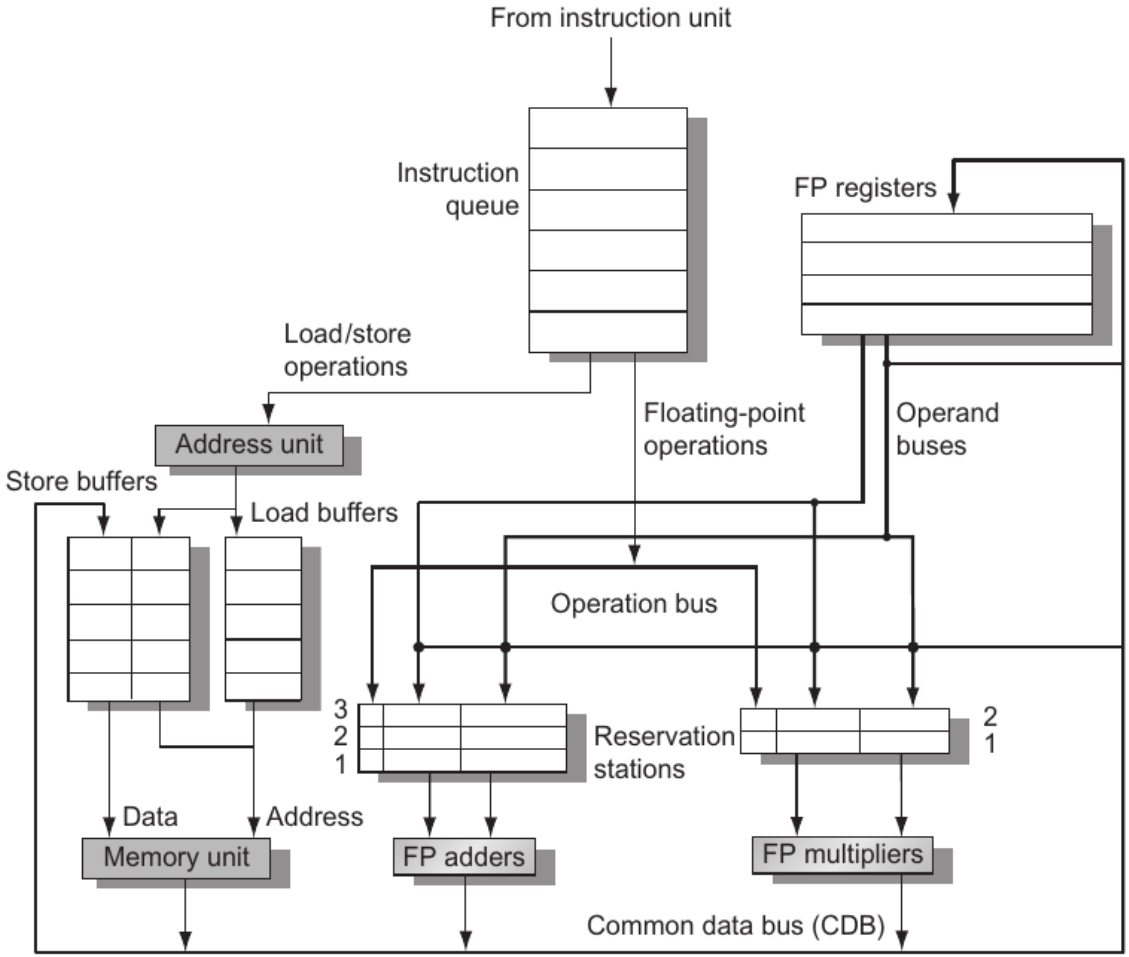


Figure 3.10 The basic structure of a RISC-V floating-point unit using Tomasulo's algorithm. Instructions are sent from the instruction unit into the instruction queue from which they are issued in first-in, first-out (FIFO) order. The reservation stations include the operation and the actual operands, as well as information used for detecting and resolving hazards. Load buffers have three functions: (1) hold the components of the effective address until it is computed, (2) track outstanding loads that are waiting on the memory, and (3) hold the results of completed loads that are waiting for the CDB. Similarly, store buffers have three functions: (1) hold the components of the effective address until it is computed, (2) hold the destination memory addresses of outstanding stores that are waiting for the data value to store, and (3) hold the address and value to store until the memory unit is available. All results from either the FP units or the load unit are put on the CDB, which goes to the FP register file as well as to the reservation stations and store buffers. The FP adders implement addition and subtraction, and the FP multipliers do multiplication and division.

Tomasulo algoritmushoz:

- *Reservation Stations*
- *Common Data Bus – CDB*
- *Load/Store Buffers*

Lépések:

1. *Issue*
   - Utasítás az instruction queue-be (FIFO – data flow)
   - Utasítás egy reservation station-be, ha van üres; különben stall
   - Operandusok beolvasása, vagy várakozás rá (a megfelelő funkcionális egységnél – dispatch/renaming)
2. *Execution*
   - Figyeljük a CDB-t, amíg minden operandus rendelkezésre nem áll.
   - A rendelkezésre álló utasítások közül egy végrehajtása (tetszőleges vagy akár több is, kivéve load/store)
3. *Write results* Ha az írandó érték megjelenik a CDB-n és van funkcionális egység az íráshoz, akkor írunk.

Minden reservation station a CDB monitorozza és a következő mezőket használja:

- *Op* – művelet (az *S*1 és *S*2-n)
- *Qj*, *Qk* – Honnan fog érkezni a megfelelő operandus.
- *Vj*, *Vk* – Az operandus értéke, ha már rendelkezésre áll.
- *A* – Immediate mező értéke (load/store eetén információ a címszámításhoz)
- *Busy*

Minden regiszterhez tartozik egy *Qi* mező, mely megmodja, hogy honnan érkezik az adott regiszter értéke.

	Instruction status		
Instruction	Issue	Execute	Write result
`fld f6,32(x2)`	√	√	√
`fld f2,44(x3)`	√	√	
`fmul.d f0,f2,f4`	√		
`fsub.d f8,f2,f6`	√		
`fdiv.d f0,f0,f6`	√		
`fadd.d f6,f8,f2`	√		

	Reservation stations						
Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	No						
Load2	Yes	Load					`44 + Regs[x3]`
Add1	Yes	SUB		`Mem[32 + Regs[x2]]`	Load2		
Add2	Yes	ADD			Add1	Load2	
Add3	No						
Mult1	Yes	MUL		`Regs[f4]`	Load2		
Mult2	Yes	DIV		`Mem[32 + Regs[x2]]`	Mult1		

	Register status								
Field	f0	f2	f4	f6	f8	f10	f12	…	f30
Qi	Mult1	Load2		Add2	Add1	Mult2			

Az algoritmus használatával nem okoz gondot ciklus több iterációjának egyszerre történő kibocsájtása sem.

A load ill. store műveletek kibocsájtása is történhet párhuzamosan/nem eredeti sorrendben, ha

- load esetén nincs ilyen cím a store bufferben
- store esetén nincs ilyen cím sem a load sem a store bufferben

Hátrányok: bonyolult hardware

- reservation station és a hozzájuk tartozó bufferek
- CDB ,,mindennel" össze van kötve
- hatékony tag keresés

Viszont manapság minden processzor része, ui.

- késleltetések elrejtése (cache, I/O)
- többszörös kibocsájtás, több funkcionális egység
- máshol fordított bináris esetén is jól működik

Nagy hatékonyságú számítások . . .
7. előadás – ILP, Spekuláció

Nagy Ádám
nagyadam888@gmail.com

Eötvös Loránd Tudományegyetem
Informatikai Kar

2022. október 26.

Motiváció

Tomasulo-algoritmussal sikerült:

- dinamikus ütemezés
- több funkcionális egység használata
- többszörös kibocsájtás támogatása
- késleltetések elfedése
- WAW és RAW kiküszöbölése

DE a probléma továbbra is az elágazások kezelése
Legfőképpen a többszörös kibocsájtású processzorok esetén

Eddig: jóslás, majd még fejletebb jóslás

Hardware-based speculation

3 részből:

- dynamic branch prediction
- spekuláció (utasítások végrehajtása mielőt az elágazás eredménye kiderül)
- dinamikus ütemezés, ami engedi a legkisebb programblokkok határainak elmosását

Data flow execution (akkor kerül végrehajtásra egy utasítás, ha oz operandusai készen vannak)

Az utasítások számolhatóak, de nem kell elfogadni az eredményt (instruction commit), amíg az utasítás spekulatív

Ordered issue – out-of-order execution – ordered commit

Reorder Buffer (ROB)

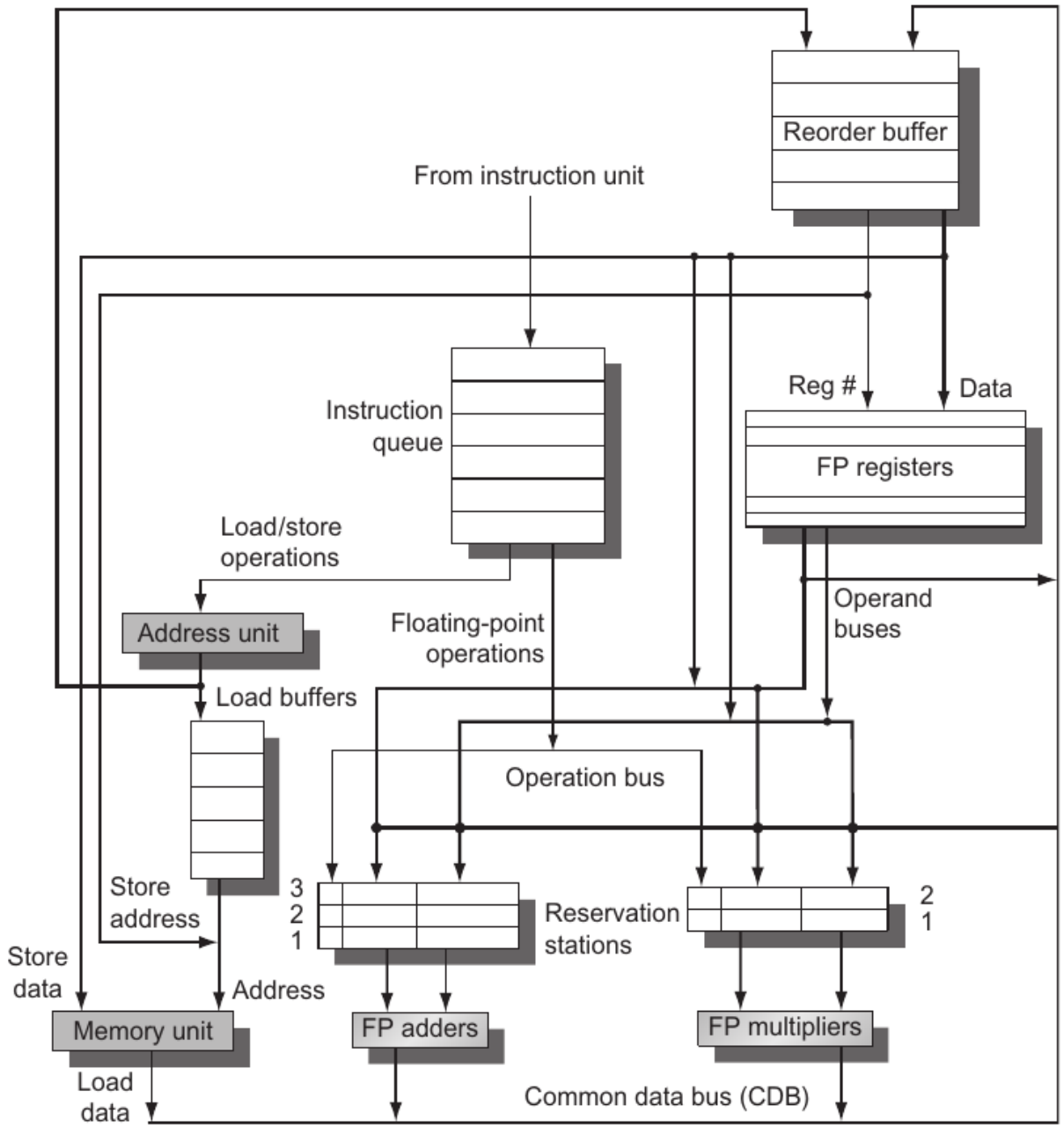
A számolás vége $\neq$ commit $\Rightarrow$ kivételek és állapot frissitése $\Rightarrow$ hw (buffer)

A spekulatív utasítások eredményét tárolni kell $\Rightarrow$ hw (buffer)

ROB buffer hasonló szerkezetű a Store bufferhez (és helyettesítheti azt), 4 mezőt tárol:

- instruction type (branch, store, load, ALU,...)
- destination (reg num, address)
- value (spekulatív utasítások eredménye)
- ready field

Reservation station átnevezési funkciója átkerül ROB-be, de arra továbbra is szükség van, mivel a kibocsájtás és a végrehajtás között késleltetés lehet. A RS tag helyett ROB tag használata.


Reorder buffer
From instruction unit
Instruction queue
Reg #
Data
FP registers
Load/store operations
Address unit
Floating-point operations
Operand buses
Load buffers
Operation bus
Store address
3
2
1
Reservation stations
2
1
Store data
Address
Memory unit
FP adders
FP multipliers
Load data
Common data bus (CDB)

Utasítás végrehajtásának menete

1. **Kibocsájtás (issue)**
   1. utasítás betöltése az instruction queue-be
   2. kibocsájtás, ha van üres hely a RS-ben és a ROB-ban is
   3. Operandusok RS-be regiszterből vagy ROB-ból
   4. ROB elküldi a tag-et a RS-nek (ezzel lesz azonosítva az eredmény)
2. **Végrehajtás (execution)**
   1. Várakozás az utasítás két operandusának értékére (CDB – Reg, Mem, ROB)
   2. Ha mindkét operandus rendelkezésre áll és van szabad végrehajtási egység akkor az utasítás végrehajtása
3. **Eredmény írása (write result)**
   1. Eredmény írása CDB-ba ROB taggel
   2. ROB-ba bekerül (value field), valamint RS-be is, ha kell
   3. Store esetén tovább kell figyelni a CDB-t

Utasítás végrehajtásának menete

4. **Befejezés (commit)** 3 eset az utasítás típusának megfelelően:
   - Store: Bekerül a memóriába a megfelelő érték a ROB-ből és törlésre kerül a ROB bejegyzés
   - Branch rossz jóslással: A spkulált utasítások eredményei törlésre kerülnek (RS-ből nem kell mivel a ROB nem fogja levenni a már érvényes taggal rendelkező eredményeket)
   - Egyéb: Az eredmény bekerül a megfelelő regiszterbe és a ROB bejegyzés törlődik

Kivétel esetén egyszerűen csak meg kell várni, amíg az kivételt megelőző utasítás (eredménye) kikerül a ROB-ből.

	Reorder buffer					
Entry	Busy	Instruction		State	Destination	Value
1	No	`fld`	`f6,32(x2)`	Commit	`f6`	`Mem[32 + Regs[x2]]`
2	No	`fld`	`f2,44(x3)`	Commit	`f2`	`Mem[44 + Regs[x3]]`
3	Yes	`fmul.d`	`f0,f2,f4`	Write result	`f0`	#2 × `Regs[f4]`
4	Yes	`fsub.d`	`f8,f2,f6`	Write result	`f8`	#2 − #1
5	Yes	`fdiv.d`	`f0,f0,f6`	Execute	`f0`	
6	Yes	`fadd.d`	`f6,f8,f2`	Write result	`f6`	#4 + #2

	Reservation stations							
Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	A
Load1	No							
Load2	No							
Add1	No							
Add2	No							
Add3	No							
Mult1	No	`fmul.d`	`Mem[44 + Regs[x3]]`	`Regs[f4]`			#3	
Mult2	Yes	`fdiv.d`		`Mem[32 + Regs[x2]]`	#3		#5	

	FP register status									
Field	f0	f1	f2	f3	f4	f5	f6	f7	f8	f10
Reorder #	3						6		4	5
Busy	Yes	No	No	No	No	No	Yes	…	Yes	Yes

Ciklus (2 iteráció)

	Reorder buffer				
Entry	Busy	Instruction	State	Destination	Value
1	No	`fld f0,0(x1)`	Commit	`f0`	`Mem[0 + Regs[x1]]`
2	No	`fmul.d f4,f0,f2`	Commit	`f4`	#1 × `Regs[f2]`
3	Yes	`fsd f4,0(x1)`	Write result	`0 + Regs[x1]`	#2
4	Yes	`addi x1,x1,−8`	Write result	`x1`	`Regs[x1] − 8`
5	Yes	`bne x1,x2,Loop`	Write result		
6	Yes	`fld f0,0(x1)`	Write result	`f0`	`Mem[#4]`
7	Yes	`fmul.d f4,f0,f2`	Write result	`f4`	#6 × `Regs[f2]`
8	Yes	`fsd f4,0(x1)`	Write result	0 + #4	#7
9	Yes	`addi x1,x1,−8`	Write result	`x1`	#4 − 8
10	Yes	`bne x1,x2,Loop`	Write result		

	FP register status								
Field	f0	f1	f2	f3	f4	F5	f6	F7	f8
Reorder #	6								
Busy	Yes	No	No	No	Yes	No	No	…	No

Figure 3.17 Only the `fld` and `fmul.d` instructions have committed, although all the others have completed execution. Thus no reservation stations are busy and none are shown. The remaining instructions will be committed as quickly as possible. The first two reorder buffers are empty, but are shown for completeness.

Status	Wait until	Action or bookkeeping
Issue all instructions	Reservation station (r) and ROB (b) both available	`if (RegisterStat[rs].Busy)/*in-flight instr. writes rs*/ {h ← RegisterStat[rs].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vj ← ROB[h].Value; RS[r].Qj ← 0;} else {RS[r].Qj ← h;} /* wait for instruction */ } else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0;}; RS[r].Busy ← yes; RS[r].Dest ← b; ROB[b].Instruction ← opcode; ROB[b].Dest ← rd; ROB[b].Ready ← no;`
FP operations and stores		`if (RegisterStat[rt].Busy) /*in-flight instr writes rt*/ {h ← RegisterStat[rt].Reorder; if (ROB[h].Ready)/* Instr completed already */ {RS[r].Vk ← ROB[h].Value; RS[r].Qk ← 0;} else {RS[r].Qk ← h;} /* wait for instruction */ } else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0;};`
FP operations		`RegisterStat[rd].Reorder ← b; RegisterStat[rd].Busy ← yes; ROB[b].Dest ← rd;`
Loads		`RS[r].A ← imm; RegisterStat[rt].Reorder ← b; RegisterStat[rt].Busy ← yes; ROB[b].Dest ← rt;`
Stores		`RS[r].A ← imm;`
Execute FP op	(RS[r].Qj == 0) and (RS[r].Qk == 0)	Compute results—operands are in Vj and Vk
Load step 1	(RS[r].Qj == 0) and there are no stores earlier in the queue	`RS[r].A ← RS[r].Vj + RS[r].A;`
Load step 2	Load step 1 done and all stores earlier in ROB have different address	Read from `Mem[RS[r].A]`
Store	(RS[r].Qj == 0) and store at queue head	`ROB[h].Address ← RS[r].Vj + RS[r].A;`
Write result all but store	Execution done at *r* and CDB available	`b ← RS[r].Dest; RS[r].Busy ← no; ∀x(if (RS[x].Qj==b) {RS[x].Vj ← result; RS[x].Qj ← 0}); ∀x(if (RS[x].Qk==b) {RS[x].Vk ← result; RS[x].Qk ← 0}); ROB[b].Value ← result; ROB[b].Ready ← yes;`
Store	Execution done at *r* and (RS[r].Qk == 0)	`ROB[h].Value ← RS[r].Vk;`
Commit	Instruction is at the head of the ROB (entry h) and ROB[h].ready == yes	`d ← ROB[h].Dest; /* register dest, if exists */ if (ROB[h].Instruction==Branch) {if (branch is mispredicted) {clear ROB[h], RegisterStat; fetch branch dest;};} else if (ROB[h].Instruction==Store) {Mem[ROB[h].Destination] ← ROB[h].Value;} else /* put the result in the register destination */ {Regs[d] ← ROB[h].Value;}; ROB[h].Busy ← no; /* free up ROB entry */ /* free up dest register if no one else writing it */ if (RegisterStat[d].Reorder==h) {RegisterStat[d].Busy ← no;};`

Figure 3.18 Steps in the algorithm and what is required for each step. For the issuing instruction, `rd` is the destination, `rs` and `rt` are the sources, `r` is the reservation station allocated, `b` is the assigned ROB entry, and `h` is the head entry of the ROB. `RS` is the reservation station data structure. The value returned by a reservation station is called the `result`. `Register-Stat` is the register data structure, `Regs` represents the actual registers, and `ROB` is the reorder buffer data structure.

Többszörös kibocsájtású architektúrák

A CPI további növelése érdekében több utasítást bocsájtanak ki a processzorok egy órajel alatt. Erre tipikusan a következő architektúrák képesek:

- Statikusan ütemezett szuperskalár (in-order)
- VLIW (vagy EPIC)
  - Very Long Instruction Word
  - egy bonyolult/hosszú utasítás vagy
  - több utasítás egyben (pl 1 branch, 2 fp, 2 int, 1 mem)
- Dinamikusan ütemezett szuperskalár (out-of-order)

Többszörös kibocsájtású processzorok

Common name	Issue structure	Hazard detection	Scheduling	Distinguishing characteristic	Examples
Superscalar (static)	Dynamic	Hardware	Static	In-order execution	Mostly in the embedded space: MIPS and ARM, including the Cortex-A53
Superscalar (dynamic)	Dynamic	Hardware	Dynamic	Some out-of-order execution, but no speculation	None at the present
Superscalar (speculative)	Dynamic	Hardware	Dynamic with speculation	Out-of-order execution with speculation	Intel Core i3, i5, i7; AMD Phenom; IBM Power 7
VLIW/LIW	Static	Primarily software	Static	All hazards determined and indicated by compiler (often implicitly)	Most examples are in signal processing, such as the TI C6x
EPIC	Primarily static	Primarily software	Mostly static	All hazards determined and indicated explicitly by the compiler	Itanium

Ciklus (statikus) ütemezése VLIW processzoron

Memory reference 1	Memory reference 2	FP operation 1	FP operation 2	Integer operation/branch
`fld f0,0(x1)`	`fld f6,-8(x1)`			
`fld f10,-16(x1)`	`fld f14,-24(x1)`			
`fld f18,-32(x1)`	`fld f22,-40(x1)`	`fadd.d f4,f0,f2`	`fadd.d f8,f6,f2`	
`fld f26,-48(x1)`		`fadd.d f12,f0,f2`	`fadd.d f16,f14,f2`	
		`fadd.d f20,f18,f2`	`fadd.d f24,f22,f2`	
`fsd f4,0(x1)`	`fsd f8,-8(x1)`	`fadd.d f28,f26,f24`		
`fsd f12,-16(x1)`	`fsd f16,-24(x1)`			`addi x1,x1,-56`
`fsd f20,24(x1)`	`fsd f24,16(x1)`			
`fsd f28,8(x1)`				`bne x1,x2,Loop`

Figure 3.20 VLIW instructions that occupy the inner loop and replace the unrolled sequence. This code takes 9 cycles assuming correct branch prediction. The issue rate is 23 operations in 9 clock cycles, or 2.5 operations per cycle. The efficiency, the percentage of available slots that contained an operation, is about 60%. To achieve this issue rate requires a larger number of registers than RISC-V would normally use in this loop. The preceding VLIW code sequence requires at least eight FP registers, whereas the same code sequence for the base RISC-V processor can use as few as two FP registers or as many as five when unrolled and scheduled.

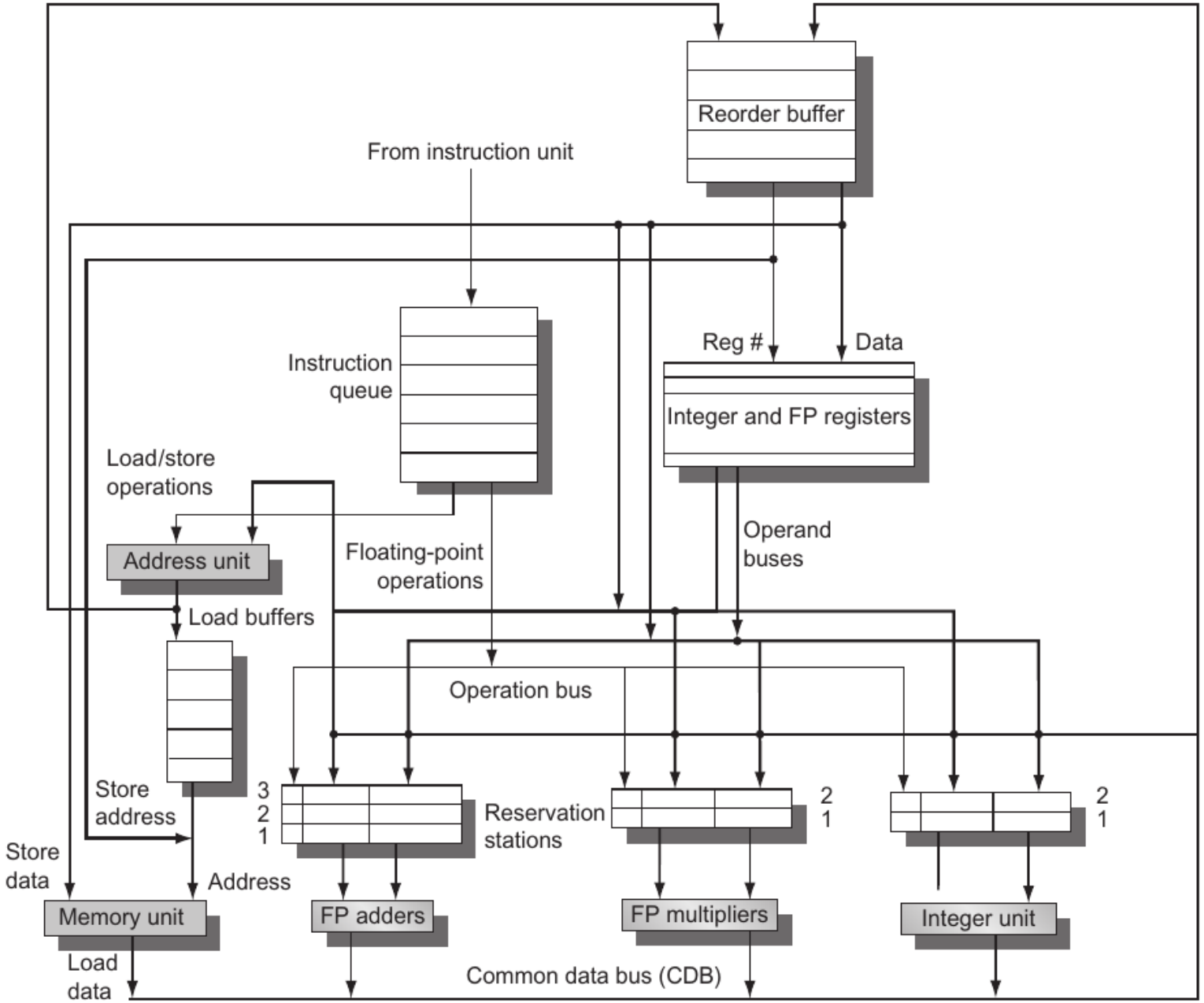
Többszörös kibocsájtás és (spekulatív) Tomasulo-alg.

VLIW esetén korlátozott a dinamikus ütemezés

Probléma a többszörös kibocsájtás és a Tomasulo-algoritmus együtt alkalmazása esetén, hogy egy órjel alatt több utasításnak kell bekerülni az instriction queue-ba majd a megfelelő RS-be úgy hogy a sorrend és az esetleges függőség ne sérüljön.

Megoldás:

1. Az fetch felgyorsítása (egy órajel alatt többet). Kétszeres kibocsájtásnál még alkalmazható, de 3 fölött már nehéz.
2. Áramkör készítése az össze lehetséges függőség figyelembevételével.


Reorder buffer
From instruction unit
Instruction queue
Reg #
Data
Integer and FP registers
Load/store operations
Address unit
Floating-point operations
Operand buses
Load buffers
Operation bus
Store address
Store data
Address
3
2
1
Reservation stations
2
1
2
1
Memory unit
FP adders
FP multipliers
Integer unit
Load data
Common data bus (CDB)

n utasítás kibocsájtása (1 órajel alatt)

1. Minden utasításhoz mező rendelése a ROB-ből akár az utasítások dekódolása előtt.
2. Függőségek feltérképezése.
3. Ha van függőség akkor használjuk a lefoglalt ROB információt a RS frissítésére. Ha nincs akkor az RS-be bekrülhetnek a megfelelő utasítások.

A kibocsájtás mellett probléma lehet a commit fázis is, ahol több utasítást kell elfogadnunk egyetlen órajel alatt. De ez egyszerűbben megoldható a ROB sávszélességének növelésével.

Action or bookkeeping	Comments

`if (RegisterStat[rs1].Busy)/*in-flight instr. writes rs*/`
`  {h ← RegisterStat[rs1].Reorder;`
`  if (ROB[h].Ready)/* Instr completed already */`
`  {RS[x1].Vj ← ROB[h].Value; RS[x1].Qj ← 0;}`
`  else {RS[x1].Qj ← h;} /* wait for instruction */`
`} else {RS[x1].Vj ← Regs[rs]; RS[x1].Qj ← 0;};`
`RS[x1].Busy ← yes; RS[x1].Dest ← b1;`
`ROB[b1].Instruction ← Load; ROB[b1].Dest ← rd1;`
`ROB[b1].Ready ← no;`
`RS[r].A ← imm1; RegisterStat[rt1].Reorder ← b1;`
`RegisterStat[rt1].Busy ← yes; ROB[b1].Dest ← rt1;`
	Updating the reservation tables for the load instruction, which has a single source operand. Because this is the first instruction in this issue bundle, it looks no different than what would normally happen for a load.

`RS[x2].Qj ← b1;} /* wait for load instruction */`	Because we know that the first operand of the FP operation is from the load, this step simply updates the reservation station to point to the load. Notice that the dependence must be analyzed on the fly and the ROB entries must be allocated during this issue step so that the reservation tables can be correctly updated.

`if (RegisterStat[rt2].Busy) /*in-flight instr writes rt*/`
`  {h ← RegisterStat[rt2].Reorder;`
`  if (ROB[h].Ready)/* Instr completed already */`
`  {RS[x2].Vk ← ROB[h].Value; RS[x2].Qk ← 0;}`
`  else {RS[x2].Qk ← h;} /* wait for instruction */`
`} else {RS[x2].Vk ← Regs[rt2]; RS[x2].Qk ← 0;};`
`RegisterStat[rd2].Reorder ← b2;`
`RegisterStat[rd2].Busy ← yes;`
`ROB[b2].Dest ← rd2;`
	Because we assumed that the second operand of the FP instruction was from a prior issue bundle, this step looks like it would in the single-issue case. Of course, if this instruction were dependent on something in the same issue bundle, the tables would need to be updated using the assigned reservation buffer.

`RS[x2].Busy ← yes; RS[x2].Dest ← b2;`
`ROB[b2].Instruction ← FP operation; ROB[b2].Dest ← rd2;`
`ROB[b2].Ready ← no;`
	This section simply updates the tables for the FP operation and is independent of the load. Of course, if further instructions in this issue bundle depended on the FP operation (as could happen with a four-issue superscalar), the updates to the reservation tables for those instructions would be effected by this instruction.

Figure 3.22 The issue steps for a pair of dependent instructions (called 1 and 2), where instruction 1 is FP load and instruction 2 is an FP operation whose first operand is the result of the load instruction; `x1` and `x2` are the assigned reservation stations for the instructions; and `b1` and `b2` are the assigned reorder buffer entries. For the issuing instructions, `rd1` and `rd2` are the destinations; `rs1`, `rs2`, and `rt2` are the sources (the load has only one source); `x1` and `x2` are the reservation stations allocated; and `b1` and `b2` are the assigned ROB entries. `RS` is the reservation station data structure. `RegisterStat` is the register data structure, `Regs` represents the actual registers, and `ROB` is the reorder buffer data structure. Notice that we need to have assigned reorder buffer entries for this logic to operate properly, and recall that all these updates happen in a single clock cycle in parallel, not sequentially.

Spekulatív vs Nem spekulatív 2-szeres kibocsájtású proc.

Iteration number	Instructions	Issues at clock cycle number	Executes at clock cycle number	Memory access at clock cycle number	Write CDB at clock cycle number	Comment
1	`ld x2,0(x1)`	1	2	3	4	First issue
1	`addi x2,x2,1`	1	5		6	Wait for `ld`
1	`sd x2,0(x1)`	2	3	7		Wait for `addi`
1	`addi x1,x1,8`	2	3		4	Execute directly
1	`bne x2,x3,Loop`	3	7			Wait for `addi`
2	`ld x2,0(x1)`	4	8	9	10	Wait for `bne`
2	`addi x2,x2,1`	4	11		12	Wait for `ld`
2	`sd x2,0(x1)`	5	9	13		Wait for `addi`
2	`addi x1,x1,8`	5	8		9	Wait for `bne`
2	`bne x2,x3,Loop`	6	13			Wait for `addi`
3	`ld x2,0(x1)`	7	14	15	16	Wait for `bne`
3	`addi x2,x2,1`	7	17		18	Wait for `ld`
3	`sd x2,0(x1)`	8	15	19		Wait for `addi`
3	`addi x1,x1,8`	8	14		15	Wait for `bne`
3	`bne x2,x3,Loop`	9	19			Wait for `addi`

Figure 3.23 The time of issue, execution, and writing result for a dual-issue version of our pipeline *without* speculation. Note that the `ld` following the `bne` cannot start execution earlier because it must wait until the branch outcome is determined. This type of program, with data-dependent branches that cannot be resolved earlier, shows the strength of speculation. Separate functional units for address calculation, ALU operations, and branch-condition evaluation allow multiple instructions to execute in the same cycle. Figure 3.24 shows this example with speculation.

Spekulatív vs Nem spekulatív 2-szeres kibocsájtású proc.

Iteration number	Instructions	Issues at clock number	Executes at clock number	Read access at clock number	Write CDB at clock number	Commits at clock number	Comment
1	`ld x2,0(x1)`	1	2	3	4	5	First issue
1	`addi x2,x2,1`	1	5		6	7	Wait for `ld`
1	`sd x2,0(x1)`	2	3			7	Wait for `addi`
1	`addi x1,x1,8`	2	3		4	8	Commit in order
1	`bne x2,x3,Loop`	3	7			8	Wait for `addi`
2	`ld x2,0(x1)`	4	5	6	7	9	No execute delay
2	`addi x2,x2,1`	4	8		9	10	Wait for `ld`
2	`sd x2,0(x1)`	5	6			10	Wait for `addi`
2	`addi x1,x1,8`	5	6		7	11	Commit in order
2	`bne x2,x3,Loop`	6	10			11	Wait for `addi`
3	`ld x2,0(x1)`	7	8	9	10	12	Earliest possible
3	`addi x2,x2,1`	7	11		12	13	Wait for `ld`
3	`sd x2,0(x1)`	8	9			13	Wait for `addi`
3	`addi x1,x1,8`	8	9		10	14	Executes earlier
3	`bne x2,x3,Loop`	9	13			14	Wait for `addi`

Figure 3.24 The time of issue, execution, and writing result for a dual-issue version of our pipeline *with* speculation. Note that the `ld` following the `bne` can start execution early because it is speculative.

Instruction fetch sávszélességének növelése

A nagyobb (4–8) utsaítás kibocsájtására képes processzorok esetén fejlettebb technikákra lehet szükség, hogy az utasítás sávszélességet végig tudjuk garantálni.

Ehhez természetesen szükség van megfelelő instruction cache-re, de a

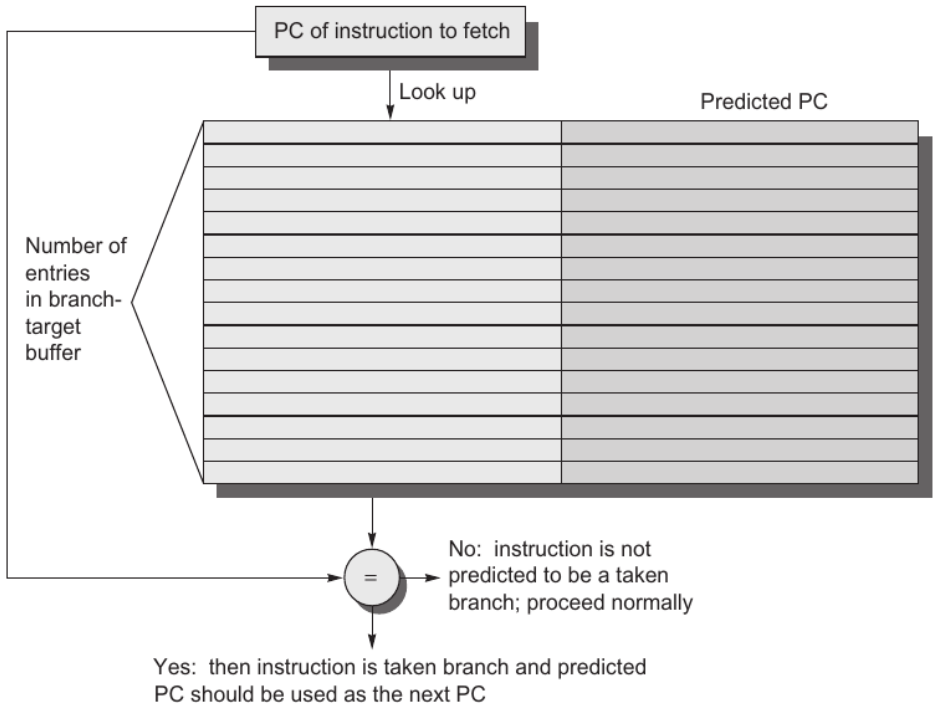
legnagyobb problémát továbbra is az elágazások megfelelő kezelése jelenti, melynek megoldására 2 technikát fogunk megismerni:

1. Branch-Target Buffer
2. Return Address Predictor

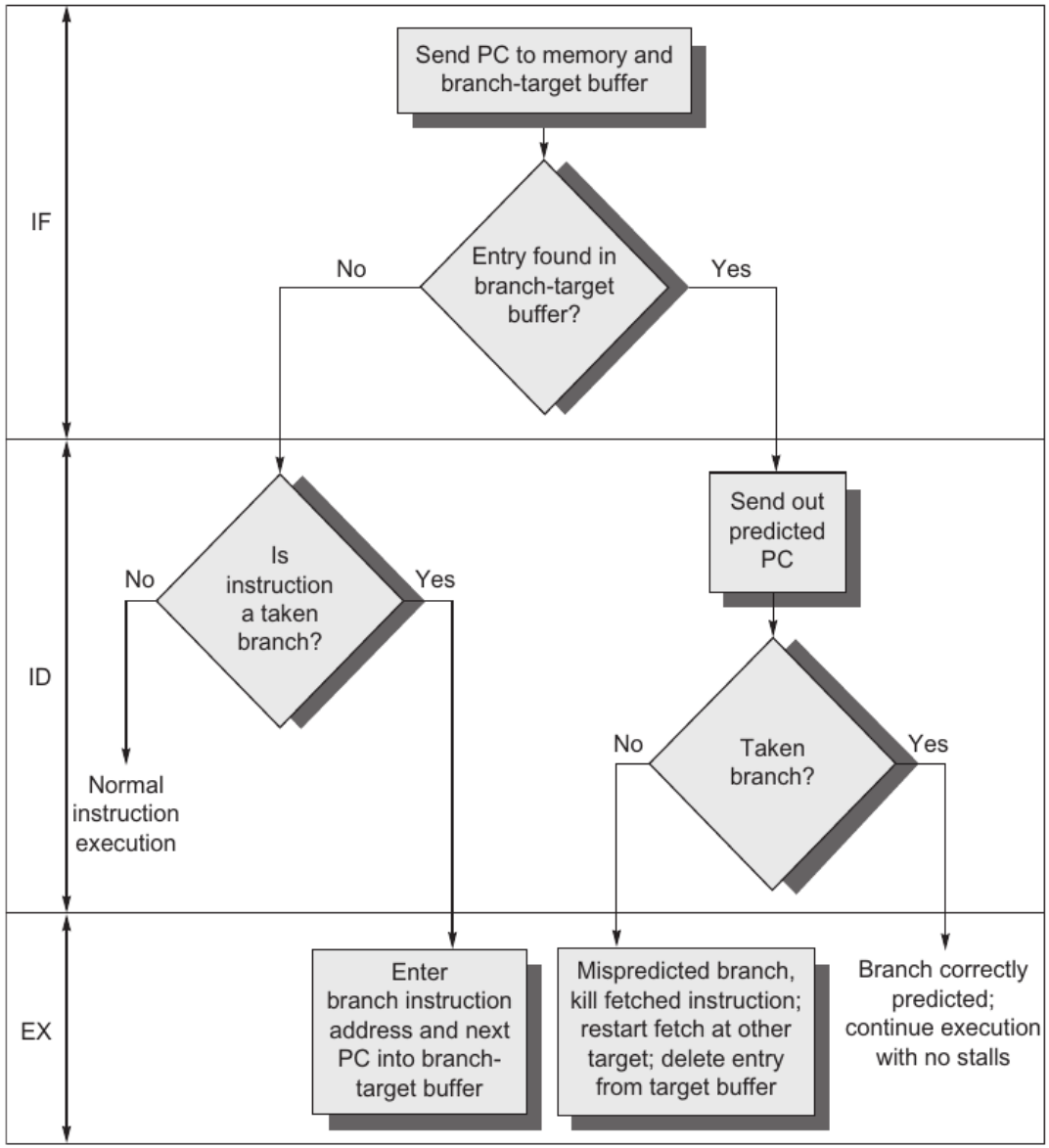
Branch-Target Buffer

Minden branch utasításnál tudni kell, hogy az adott utasítás elágazás-e és hogy mi a cél PC értéke (számolni kell).

Ennek megkönnyitésére azoknak az utasításoknak a PC-évl indexelve eltárolhatjuk a cél PC értékét, melyek a jóslás szerint taken elágazások. $\Rightarrow$ *Branch-Target Buffer*



Nincs stall, ha jó a prediction (5 lépcsős pipeline)



Ha nem jó a jóslás vagy nincs benne a BTB-ben:

Instruction in buffer	Prediction	Actual branch	Penalty cycles
Yes	Taken	Taken	0
Yes	Taken	Not taken	2
No		Taken	2
No		Not taken	0

Lehetséges a BTB-ben eltárolni a cél PC-e mellett néhány prefetch-elt utasítást, így

1. a BTB lassabb elérése is megfelelőbb $\Rightarrow$ nagyobb BTB
2. nem kerül órajelbe a (feltétel nélküli) elágazás. (branch folding)

Return address predictors

Motiváció: indirekt jump jóslása.

Az ilyen jump-ok leginkább eljárásokból való visszatéréseknél (15%) fordulnak elő (de select, case, stb. is lehet) és ha nem sok különböző helyről kerül meghívásra az adott eljárás, akkor BTB megoldhatja a problémát.

Megoldás: egy stack-ben tárolni a megfelelő eljárások hívásának helyét, hogy azt a programhívás után tudjuk használni. $\Rightarrow$ Return Address Buffer

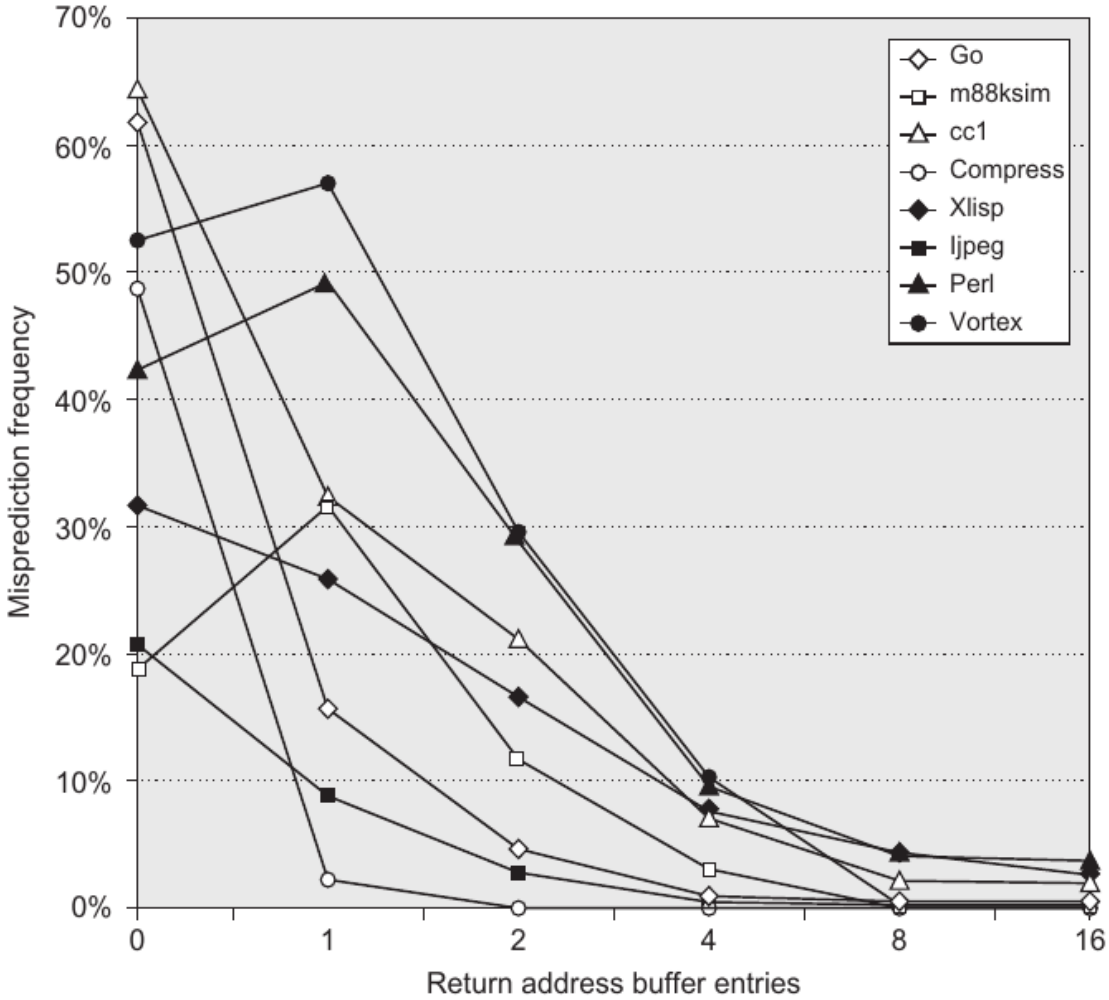


Figure 3.28 Prediction accuracy for a return address buffer operated as a stack on a number of SPEC CPU95 benchmarks. The accuracy is the fraction of return addresses predicted correctly. A buffer of 0 entries implies that the standard branch prediction is used. Because call depths are typically not large, with some exceptions, a modest buffer works well. These data come from Skadron et al. (1999) and use a fix-up mechanism to prevent corruption of the cached return addresses.

Integrated Instruction Fetch Unit

Többszörös kibocsájtású processzorok esetében – ahogy láttuk – nehéz az instruction fetch-et egyetlen pipline állomásként megoldani, így sokszor különálló egységgel szokták ezt a feladatot megoldani, mely a következő feladatokat végzi el:

- integrated branch prediction
- instruction prefetch
- instruction memory access and buffering

Megjegyzések – Spekuláció

- A spekuláció elrejti a késleltetéseket. Ez azonban túl költséges műveleteknél több problémát hordoz magában, így spekulatív módban az ilyen kivételek nem engedélyezettek.
- A spekuláció történhet egyszerre több branch esetén is, – akár egy órajelen beül – ami tovább növelheti a hatékonyságot, de nagyban bonyolíthatja a processzort (főként ha egy órajelen belül tesszük ezt).
- *Value prediction* Bizonyos esetekben az utasítások eredményét is lehet jósolni, pl. load. Gyakorlatban nem használt (jelenleg).
- *Address aliasing prediction* Ha két store vagy egy load és egy store nem ugyanazt a memóriaterületet használja akkor azok felcserélhetőek. Ehhez elég csak az egyenlőség teljesülését megjósolni, ami jóval könnyebb mint a value prediction. Használt technika.

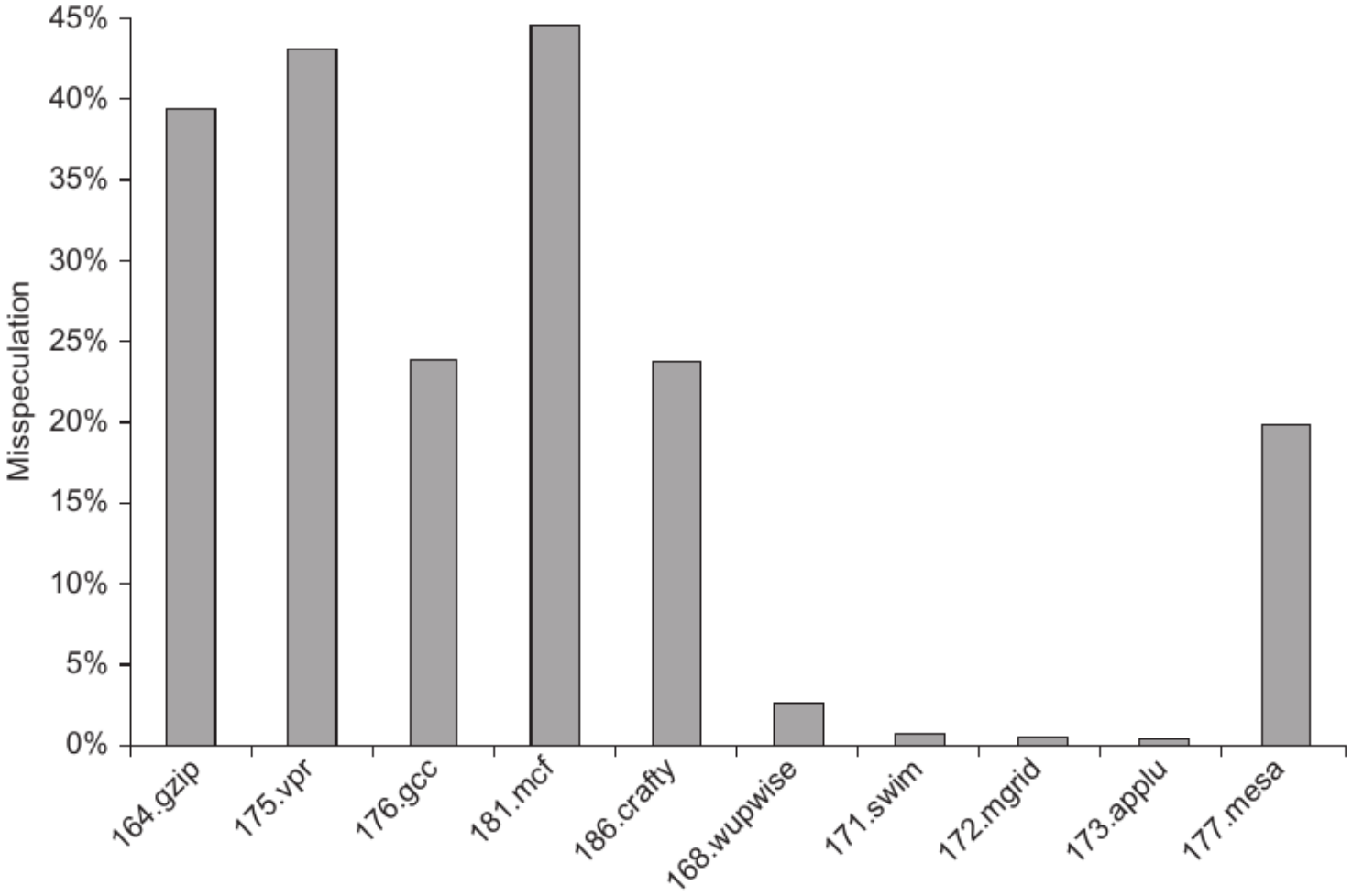


Figure 3.30 The fraction of instructions that are executed as a result of misspeculation is typically much higher for integer programs (the first five) versus FP programs (the last five).

Instr. #	Instruction	Physical register assigned or destination	Instruction with physical register numbers	Rename map changes
1	`add x1,x2,x3`	p32	`add p32,p2,p3`	**x1-> p32**
2	`sub x1,x1,x2`	p33	`sub p33,p32,p2`	**x1->p33**
3	`add x2,x1,x2`	p34	`add p34,p33,x2`	**x2->p34**
4	`sub x1,x3,x2`	p35	`sub p35,p3,p34`	**x1->p35**
5	`add x1,x1,x2`	p36	`add p36,p35,p34`	**x1->p36**
6	`sub x1,x3,x1`	p37	`sub p37,p3,p36`	**x1->p37**

Figure 3.29 An example of six instructions to be issued in the same clock cycle and what has to happen. The instructions are shown in program order: 1–6; they are, however, issued in 1 clock cycle! The notation `pi` is used to refer to a physical register; the contents of that register at any point is determined by the renaming map. For simplicity, we assume that the physical registers holding the architectural registers `x1`, `x2`, and `x3` are initially `p1`, `p2`, and `p3` (they could be any physical register). The instructions are issued with physical register numbers, as shown in column four. The rename map, which appears in the last column, shows how the map would change if the instructions were issued sequentially. The difficulty is that all this renaming and replacement of architectural registers by physical renaming registers happens effectively in 1 cycle, not sequentially. The issue logic must find all the dependences and "rewrite" the instruction in parallel.

Nagy hatékonyságú számítások . . .
8. előadás – ILP összefoglalás; DLP

Nagy Ádám
nagyadam888@gmail.com

Eötvös Loránd Tudományegyetem
Informatikai Kar

2022. november 1.

ILP – Határok

Mennyi párhuzamosság használható ki az előzőekben ismertetett technikákkal (elmélteben)?

Ideális processzor (csak data flow jelent megkötést)

- végtelen számú regiszter (átnevezéshez)
- tökéletes branch prediction
- tökéletes jump prediction (tökéletes spekuláció)
- tökéletes memory address alias analysis
- tökéletes cache-elés (minden töltés 1 órajel)
- unlimited-issue
- funkcionális egységek késleltetése 1 órajel

Tökéletes processzor

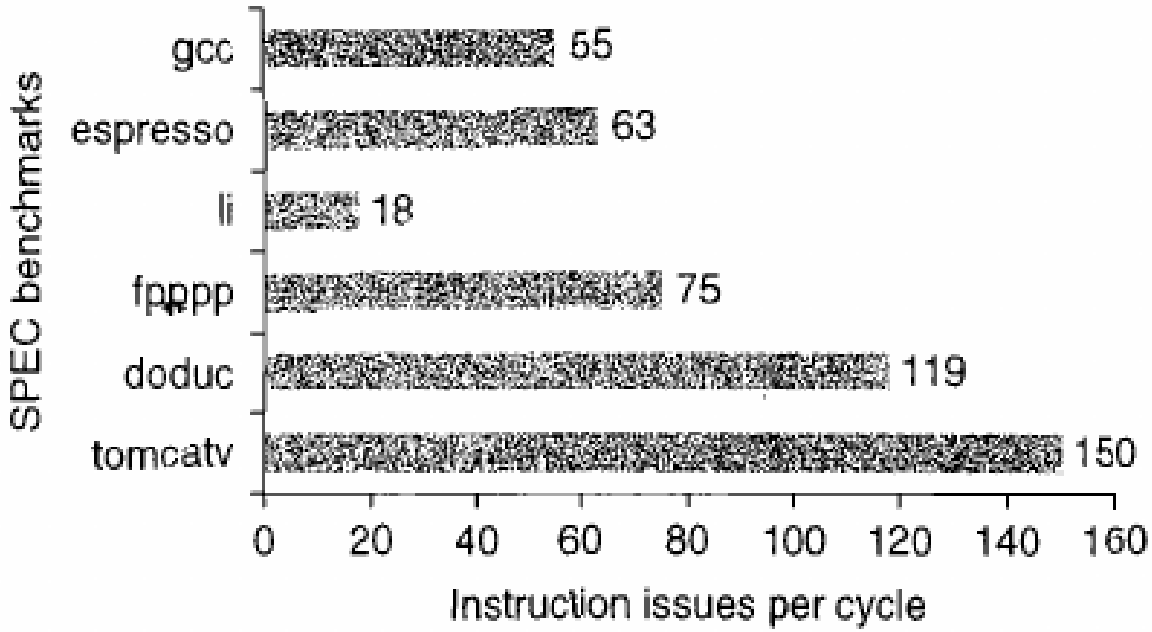


Figure 3.26 **ILP available in a perfect processor for six of the SPEC92 benchmarks.** The first three programs are integer programs, and the last three are floating-point programs. The floating-point programs are loop intensive and have large amounts of loop-level parallelism.

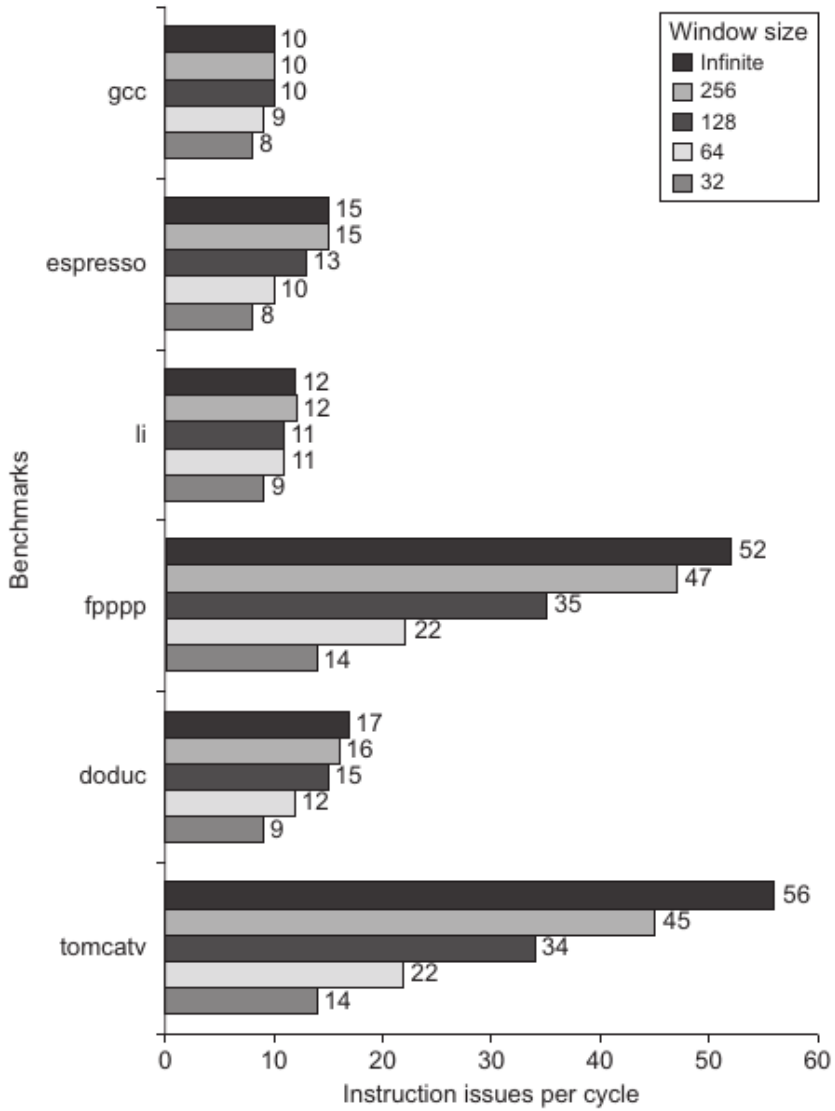


Figure 3.45 The amount of parallelism available versus the window size for a variety of integer and floating-point programs with up to 64 arbitrary instruction issues per clock. Although there are fewer renaming registers than the window size, the fact that all operations have 1-cycle latency and that the number of renaming registers equals the issue width allows the processor to exploit parallelism within the entire window.

ILP gyakorlatban

Gyakorlatban a tökéletes processzorral szemben

- WAW és RAW memóriában nem követhető jól,
- unnecessary dependecies (pl. ciklusváltozó, stackpointer),
- data flow limit.

Spekuláció több ágon lehetséges, de manapság inkább a jóslást próbálják javítani.

Konklúzió: több ILP kihasználása nem igazán kifizetődő.

Dinamikus vs. Statikus

- Memóriaelérések out-of-order ütemezése dinamikus esetben lehetséges, statikus esetben azoban általában nem, vagy korlátozottan kivitelezhető.
- Ha az elágazás nem jósolható jól (statikusan), akkor a dinamikus ütemezésnél használt jóslás sokkal jobban viselkedik. (integer programok)
- Dinamikus esetben nincs szükség további ellenőrző vagy javító kódra.
- Statikus ütemezésnél sokkal nagyobb kódrészletet (esetleg az egészet) vizsgálhatjuk egyszerre.
- Hordozhatóságra nincs hatással a dinamikus ütemezés.
- Dinamikus harwerrel és költséggel jár.

Statikus és dinamikus ütemezés együtt használat néha nem várt eredményt okozhat! (conditional move)

Multithreading a késleltetések elfedésére

ILP kihasználása sok esetben igen nehéz vagy egyáltalán nem lehetséges (pl. memória elérés késleltetése, hosszú ideig tartó utasítások, stb.).

A többszörös kibocsájtás tényleges kihasználására és a nagyobb késleltetések idejének kihasználására jól alkalmazható a multithreading (TLP) egyetlen processzoron.

Egy több szál használatára képes processzor a különböző szálak számára privát területet biztosít (reg-k, stb.), így azok a memóriát megosztva használhatják. Aszerint, hogy a processzor milyen gyakran tud váltani a szálak között megkülönböztetünk

- coarse (csak nagyobb késleltetések esetén)
- fine (órajelenként)
- simultaneous (egyszerre több szál is az egy órajel alatt kibocsájtható utasításoknak megfelelően)

szálkezelést.

MT típusok

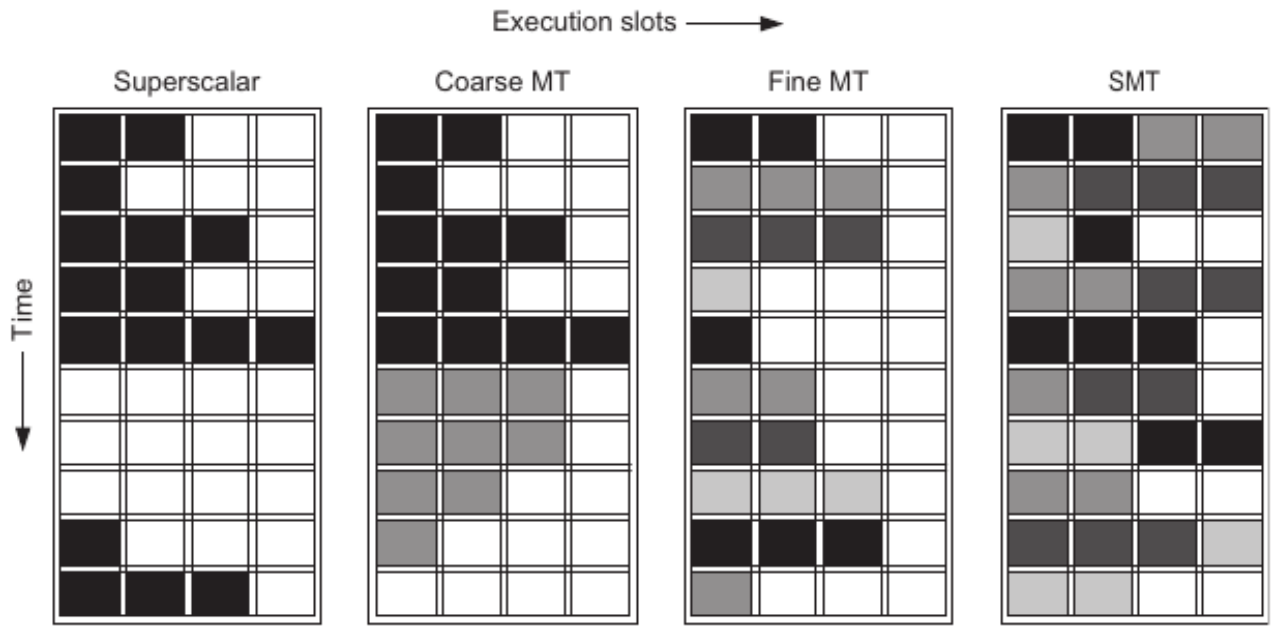


Figure 3.31 How four different approaches use the functional unit execution slots of a superscalar processor. The horizontal dimension represents the instruction execution capability in each clock cycle. The vertical dimension represents a sequence of clock cycles. An empty (white) box indicates that the corresponding execution slot is unused in that clock cycle. The shades of gray and black correspond to four different threads in the multithreading processors. Black is also used to indicate the occupied issue slots in the case of the superscalar without multithreading support. The Sun T1 and T2 (aka Niagara) processors are fine-grained, multithreaded processors, while the Intel Core i7 and IBM Power7 processors use SMT. The T2 has 8 threads, the Power7 has 4, and the Intel i7 has 2. In all existing SMTs, instructions issue from only one thread at a time. The difference in SMT is that the subsequent decision to execute an instruction is decoupled and could execute the operations coming from several different instructions in the same clock cycle.

SMT on i7

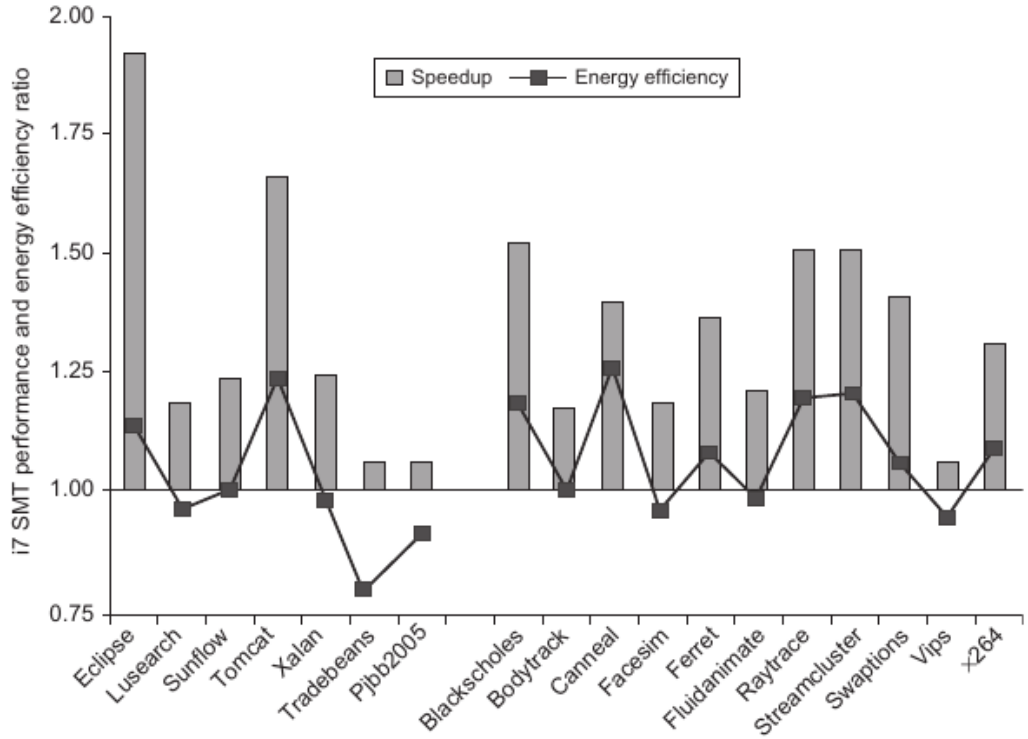


Figure 3.33 The speedup from using multithreading on one core on an i7 processor averages 1.28 for the Java benchmarks and 1.31 for the PARSEC benchmarks (using an unweighted harmonic mean, which implies a workload where the total time spent executing each benchmark in the single-threaded base set was the same). The energy efficiency averages 0.99 and 1.07, respectively (using the harmonic mean). Recall that anything above 1.0 for energy efficiency indicates that the feature reduces execution time by more than it increases average power. Two of the Java benchmarks experience little speedup and have significant negative energy efficiency because of this issue. Turbo Boost is off in all cases. These data were collected and analyzed by Esmaeilzadeh et al. (2011) using the Oracle (Sun) HotSpot build 16.3-b01 Java 1.6.0 Virtual Machine and the gcc v4.4.1 native compiler.

Példa 1. – ARM Cortex A53 – Pipeline

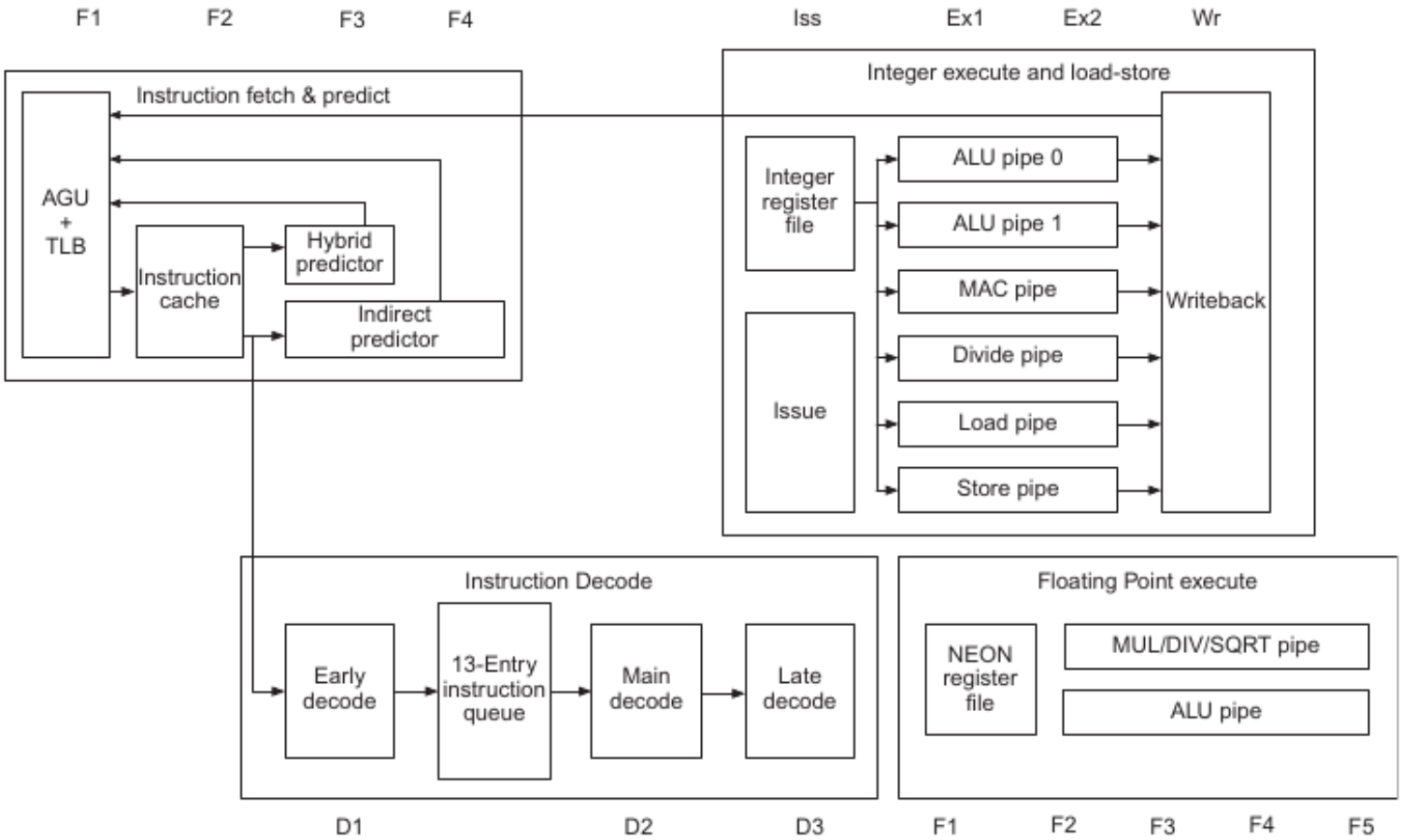


Figure 3.34 The basic structure of the A53 integer pipeline is 8 stages: F1 and F2 fetch the instruction, D1 and D2 do the basic decoding, and D3 decodes some more complex instructions and is overlapped with the first stage of the execution pipeline (ISS). After ISS, the Ex1, EX2, and WB stages complete the integer pipeline. Branches use four different predictors, depending on the type. The floating-point execution pipeline is 5 cycles deep, in addition to the 5 cycles needed for fetch and decode, yielding 10 stages in total.

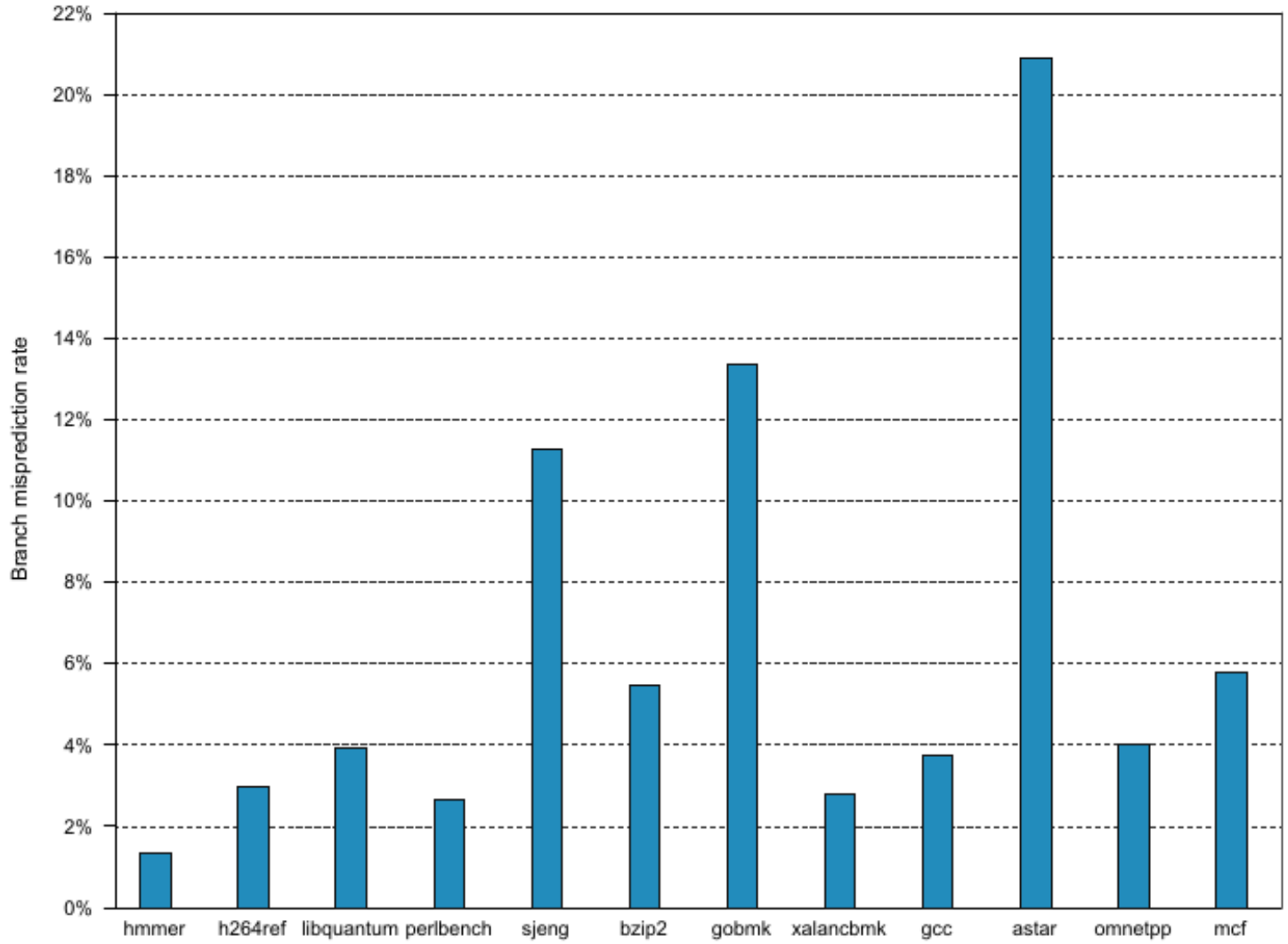


Figure 3.35 Misprediction rate of the A53 branch predictor for SPECint2006.

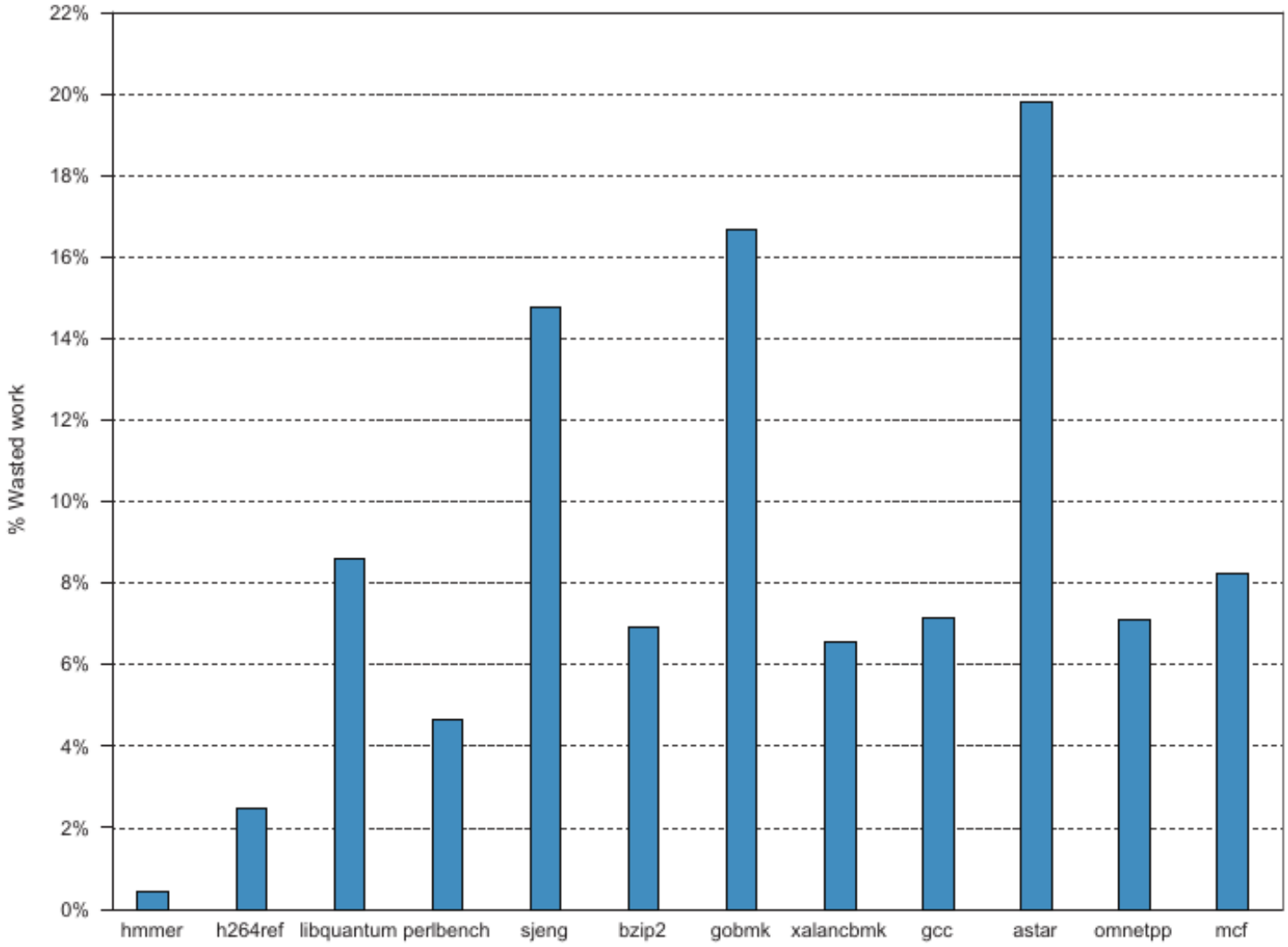


Figure 3.36 Wasted work due to branch misprediction on the A53. Because the A53 is an in-order machine, the amount of wasted work depends on a variety of factors, including data dependences and cache misses, both of which will cause a stall.

Példa 1. – Stalls

Pipeline stalls:

- A két végrehatjtásra kerülő utasítás ugyanazt a funkcionális egységet használná.
- Adatfüggőség. (Ha az első megáll, akkor a második is.)
- Rossz jóslás.

A8-ig a processzor statikusan ütemezett, így a compiler feleadata, hogy a fenti esetek a lehető legkevesebbszer forduljanak elő.

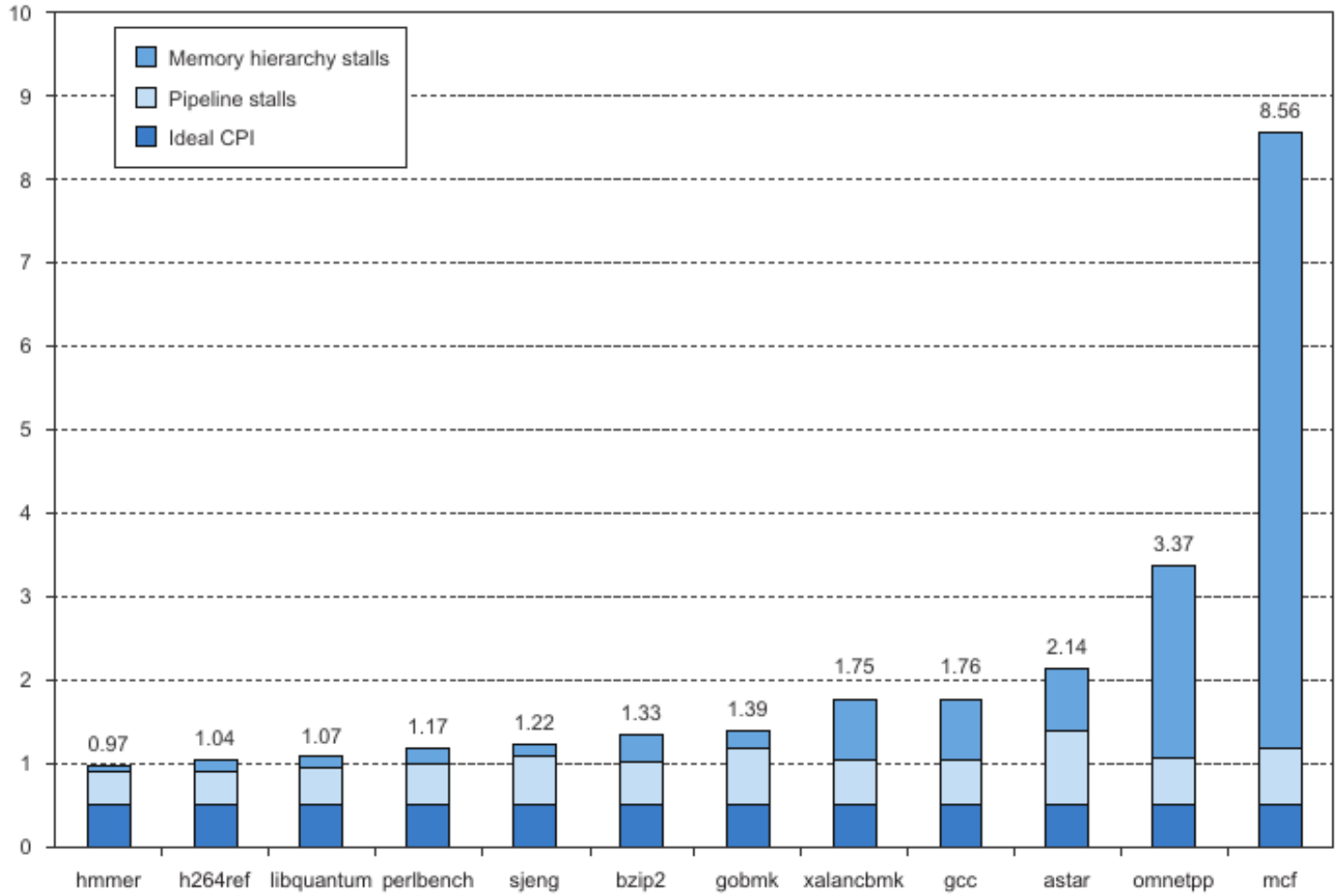


Figure 3.37 The estimated composition of the CPI on the ARM A53 shows that pipeline stalls are significant but are outweighed by cache misses in the poorest performing programs. This estimate is obtained by using the L1 and L2 miss rates and penalties to compute the L1 and L2 generated stalls per instruction. These are subtracted from the CPI measured by a detailed simulator to obtain the pipeline stalls. Pipeline stalls include all three hazards.

Példa 1. – ARM Cortex A8 vs A9 (dynamic scheduling)

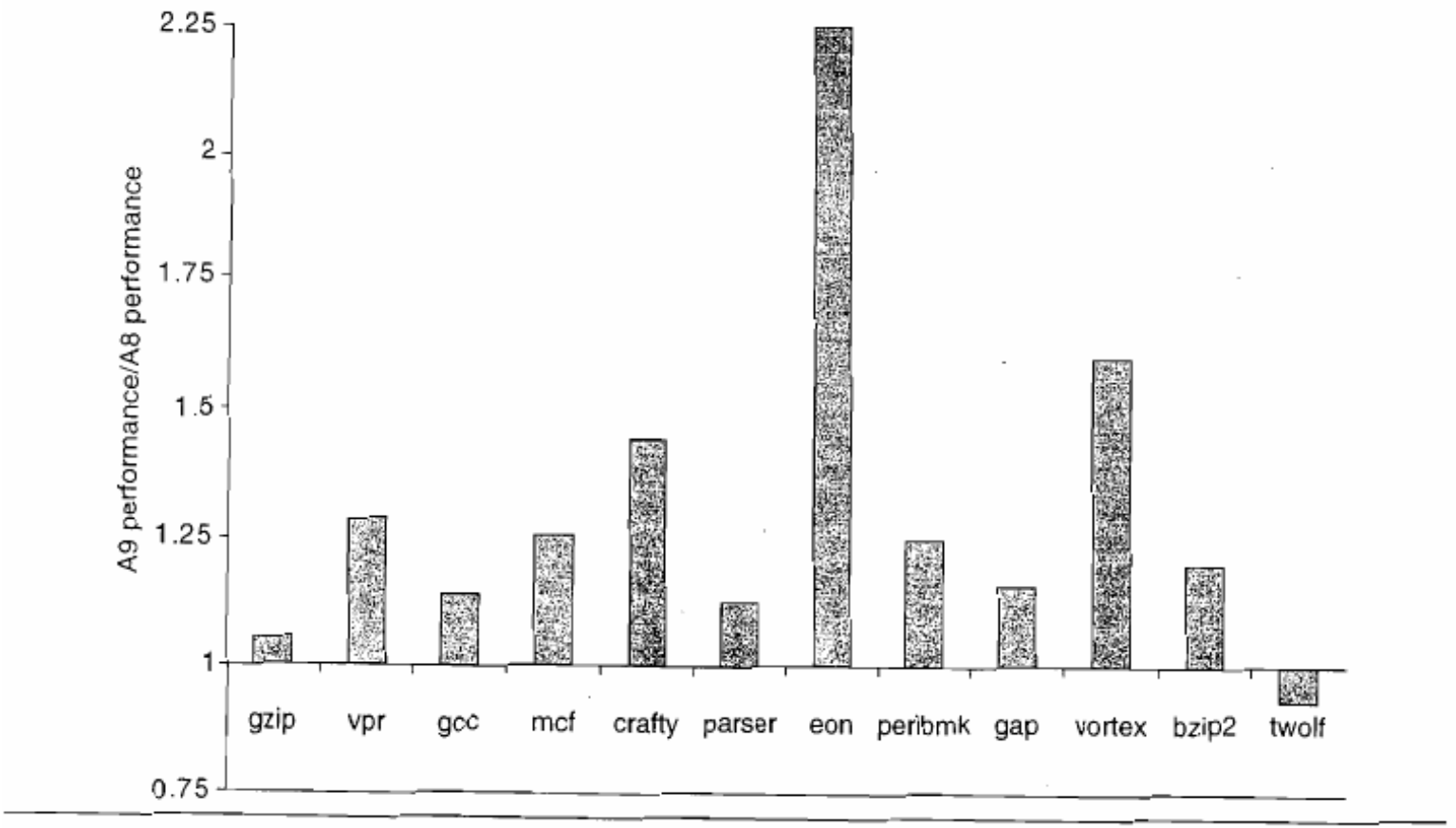


Figure 3.40 The performance ratio for the A9 compared to the A8, both using a 1 GHz clock and the same size caches for L1 and L2, shows that the A9 is about 1.28 times faster. Both runs use a 32 KB primary cache and a 1 MB secondary cache, which is 8-way set associative for the A8 and 16-way for the A9. The block sizes in the caches are 64 bytes for the A8 and 32 bytes for the A9. As mentioned in the caption of Figure 3.39, eon makes intensive use of integer multiply, and the combination of dynamic scheduling and a faster multiply pipeline significantly improves performance on the A9. twolf experiences a small slowdown, likely due to the fact that its cache behavior is worse with the smaller L1 block size of the A9.

Példa 2. – Intel Core i7

Area	Specific characteristic	Intel i7 920 Four cores, each with FP	ARM A8 One core, no FP	Intel Atom 230 One core, with FP
Physical chip properties	Clock rate	2.66 GHz	1 GHz	1.66 GHz
	Thermal design power	130 W	2 W	4 W
	Package	1366-pin BGA	522-pin BGA	437-pin BGA
	TLB	Two-level All four-way set associative 128 I/64 D 512 L2	One-level fully associative 32 I/32 D	Two-level All four-way set associative 16 I/16 D 64 L2
	Caches	Three-level 32 KiB/32 KiB 256 KiB 2–8 MiB	Two-level 16/16 or 32/32 KiB 128 KiB–1 MiB	Two-level 32/24 KiB 512 KiB
Memory system	Peak memory BW	17 GB/s	12 GB/sec	8 GB/s
	Peak issue rate	4 ops/clock with fusion	2 ops/clock	2 ops/clock
	Pipe line scheduling	Speculating out of order	In-order dynamic issue	In-order dynamic issue
Pipeline structure	Branch prediction	Two-level	Two-level 512-entry BTB 4 K global history 8-entry return stack	Two-level

Figure 3.42 An overview of the four-core Intel i7 920, an example of a typical ARM A8 processor chip (with a 256 MiB L2, 32 KiB L1s, and no floating point), and the Intel ARM 230, clearly showing the difference in design philosophy between a processor intended for the PMD (in the case of ARM) or netbook space (in the case of Atom) and a processor for use in servers and high-end desktops. Remember, the i7 includes four cores, each of which is higher in performance than the one-core A8 or Atom. All these processors are implemented in a comparable 45 nm technology.

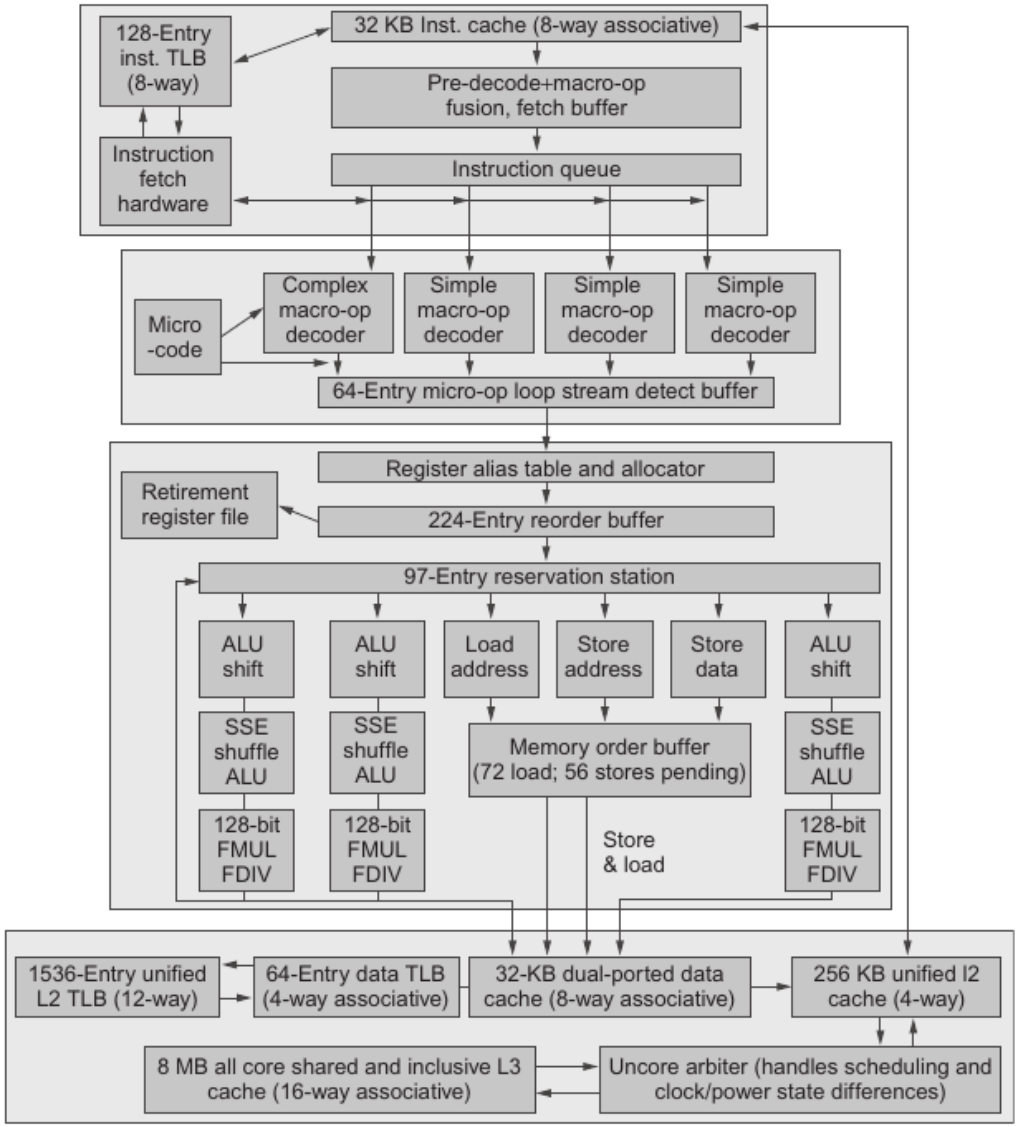


Figure 3.38 The Intel Core i7 pipeline structure shown with the memory system components. The total pipeline depth is 14 stages, with branch mispredictions typically costing 17 cycles, with the extra few cycles likely due to the time to reset the branch predictor. The six independent functional units can each begin execution of a ready micro-op in the same cycle. Up to four micro-ops can be processed in the register renaming table.

Példa 2. – Intel Core i7 – Through the pipeline

1. *Instruction fetch* 16 byte a IC-ből, többszintű BTB és RS segítségével. (Misprediction $\Rightarrow$ 15c stall)
2. 16 btye elhelyezése az instruction buffer-be. Itt összegyűjtésre kerül egy Micro-op utasításnak megfelelő bytesorozat, mely lehet több x86 utasítás is (x86 utasítások 1-17 byte hosszúak). Legvégül az így kapott bytesorozat bekerül a 18 helyes instruction queue-be.
3. Micro-op utasítások (ezek jól pipeline-olhatók) elkészítése a x86 utasításokból (egy utasításból lehet több is). Egy órajel alatt legfeljebb 4 Micro-op utasítás kerül a 28 helyes Micro-op buffer-be.
4. *Loop stream detection, microfusion* Ha egy ciklus elfér a buffer-be ($\leq$ 28 utasítás vagy $\leq$ 256b), akkor az a buffer segítségével az utasítások újraolvasása nélkül kezelhető.

Példa 2. – Intel Core i7 – Through the pipeline

5. Issue: regiszter keresése regiszter táblában és átnevezés, ROB bejegyzés, operandusok töltése, elhelyezés reservation station-be
6. Egy közös reservation station-ből (36 hely) a 6 független (max ennyi Micro-op egy órajel alatt) funkcionális egység egyikébe kerül az utasítás.
7. Az eredmény a funkcinális egységből a ROB-ba, RS-be ill. a register retirement unit-ba.
8. Ha az ROB bejegyzés nem spekulatív, akkor onnan kikerül és a RRU véglegesíti a regiszter értékét.

Resource	i7 920 (Nehalem)	i7 6700 (Skylake)
Micro-op queue (per thread)	28	64
Reservation stations	36	97
Integer registers	NA	180
FP registers	NA	168
Outstanding load buffer	48	72
Outstanding store buffer	32	56
Reorder buffer	128	256

Figure 3.39 The buffers and queues in the first generation i7 and the latest generation i7. Nehalem used a reservation station plus reorder buffer organization. In later microarchitectures, the reservation stations serve as scheduling resources, and register renaming is used rather than the reorder buffer; the reorder buffer in the Skylake microarchitecture serves only to buffer control information. The choices of the size of various buffers and renaming registers, while appearing sometimes arbitrary, are likely based on extensive simulation.

Példa 2. – Intel Core i7 – CPI (egy szál, SMT nincs)

A pipeline nagy részben cache miss és misprediction miatt áll.

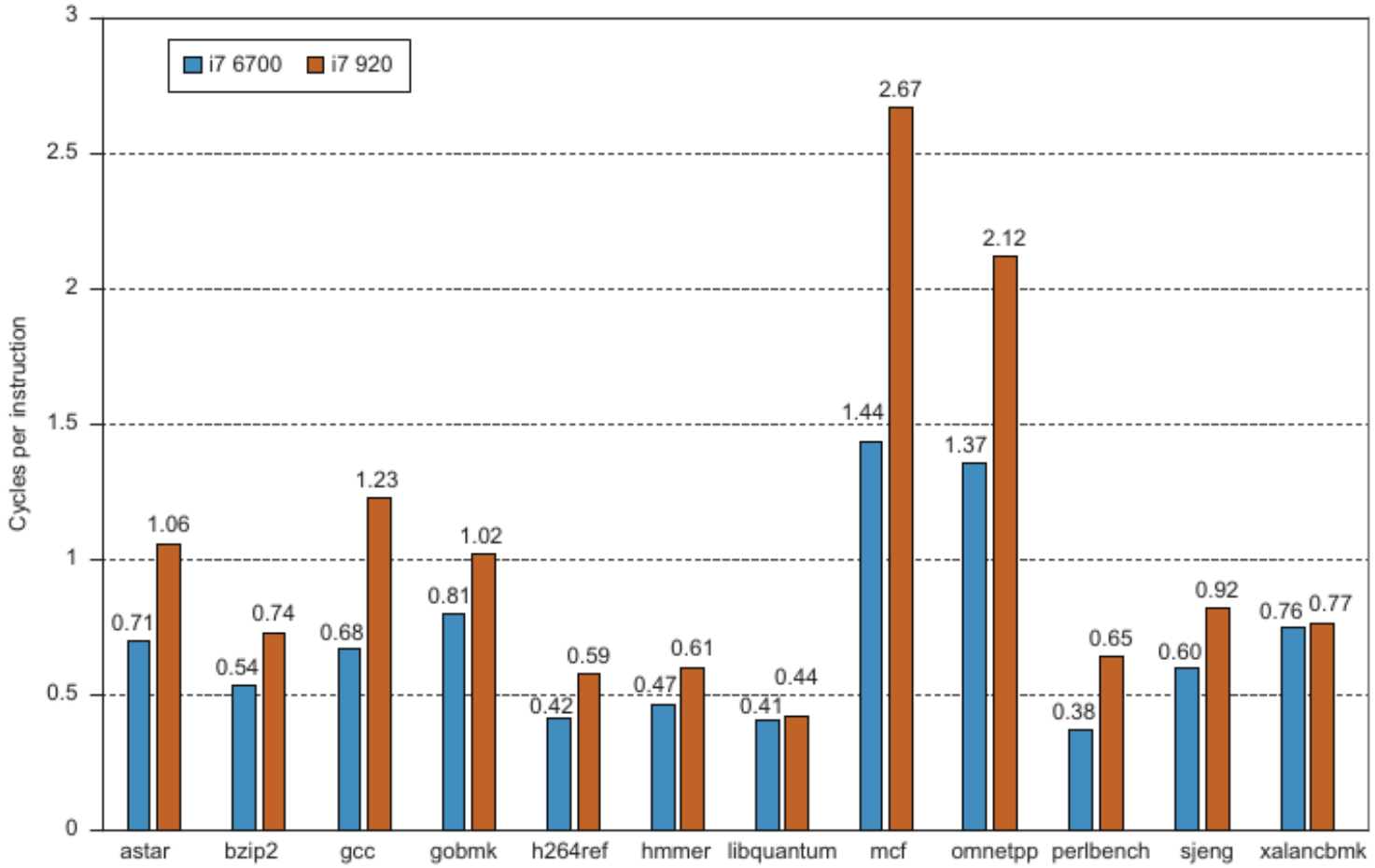


Figure 3.40 The CPI for the SPECCPUint2006 benchmarks on the i7 6700 and the i7 920. The data in this section were collected by Professor Lu Peng and PhD student Qun Liu, both of Louisiana State University.

Benchmark	CPI ratio (920/6700)	Branch mispredict ratio (920/6700)	L1 demand miss ratio (920/6700)
ASTAR	1.51	1.53	2.14
GCC	1.82	2.54	1.82
MCF	1.85	1.27	1.71
OMNETPP	1.55	1.48	1.96
PERLBENCH	1.70	2.11	1.78

Figure 3.41 An analysis of the five integer benchmarks with the largest performance gap between the i7 6700 and 920. These five benchmarks show an improvement in the branch prediction rate and a reduction in the L1 demand miss rate.

Példa 2. – Intel Core i7 vs Atom 230

Egy mag (többi power save), turbo boost

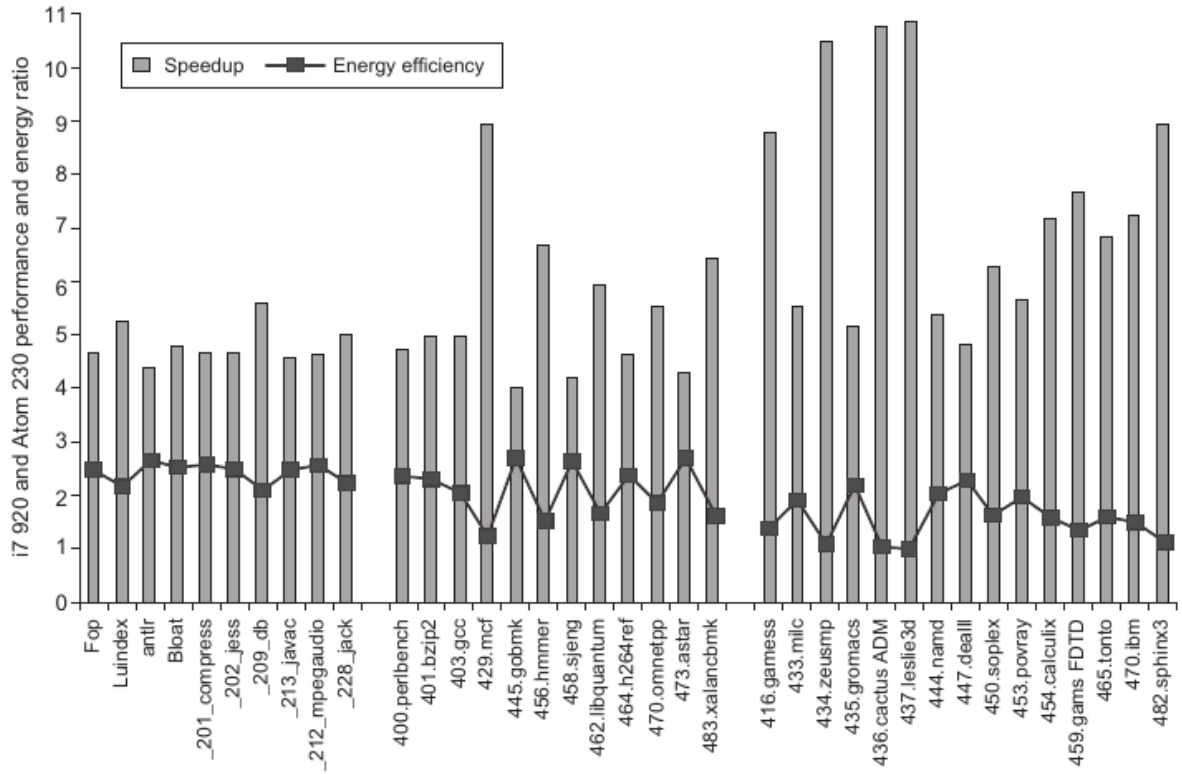


Figure 3.43 The relative performance and energy efficiency for a set of single-threaded benchmarks shows the i7 920 is 4 to over 10 times faster than the Atom 230 but that it is about 2 times *less* power-efficient on average! Performance is shown in the columns as i7 relative to Atom, which is execution time (i7)/execution time (Atom). Energy is shown with the line as Energy (Atom)/Energy (i7). The i7 never beats the Atom in energy efficiency, although it is essentially as good on four benchmarks, three of which are floating point. The data shown here were collected by Esmaeilzadeh et al. (2011). The SPEC benchmarks were compiled with optimization using the standard Intel compiler, while the Java benchmarks use the Sun (Oracle) Hotspot Java VM. Only one core is active on the i7, and the rest are in deep power saving mode. Turbo Boost is used on the i7, which increases its performance advantage but slightly decreases its relative energy efficiency.

ILP – Tévhitek és csapdák

- F A teljesítmény és energiafelhasználás könnyen megbecsülhető két azonos technológiával készült, azonos utasításkészletet használó különböző processzor esetén.
- F Alacsonyabb CPI/nagyobb órajel ⇒ gyorsabb processzor.

Processor	Implementation technology	Clock rate	Power	SPECCInt2006 base	SPECCFP2006 baseline
Intel Pentium 4 670	90 nm	3.8 GHz	115 W	11.5	12.2
Intel Itanium 2	90 nm	1.66 GHz	104 W approx. 70 W one core	14.5	17.3
Intel i7 920	45 nm	3.3 GHz	130 W total approx. 80 W one core	35.5	38.4

Figure 3.44 Three different Intel processors vary widely. Although the Itanium processor has two cores and the i7 four, only one core is used in the benchmarks; the Power column is the thermal design power with estimates for only one core active in the multicore cases.

  - Pentium 4: 20+ hosszú pipline, nagy energiafelvétel
  - Core i7: nagyobb cache,
  - Itanium: VLIW típusú architektúra

- P Sometimes bigger and dumber is better. (Pentium 4)
- P Sometimes smarter is better than bigger and dumber.
- P Túlbecsülni a rendelkezésre álló ILP-t.

ILP – Merre tovább?

ILP kihasználása egyre inkább nehézkes (nem éri meg) $\Rightarrow$ TLP, DLP

TLP: processzorról programozóra, fordítóra helyezi a hangsúlyt

DLP: bizonyos felhasználási területeken jó (media, mátrix, tudományos), de ...

Konklúzió:

- kiegyensúlyozott ILP (out-of-order, spekuláció, nagyobb cache, lassan növekvő kibocsájtásszám);
- több szál, SMT, több mag;
- speciális célra SIMD

	Power4	Power5	Power6	Power7	Power8
Introduced	2001	2004	2007	2010	2014
Initial clock rate (GHz)	1.3	1.9	4.7	3.6	3.3 GHz
Transistor count (M)	174	276	790	1200	4200
Issues per clock	5	5	7	6	8
Functional units per core	8	8	9	12	16
SMT threads per core	0	2	2	4	8
Cores/chip	2	2	2	8	12
SMT threads per core	0	2	2	4	8
Total on-chip cache (MiB)	1.5	2	4.1	32.3	103.0

Figure 3.46 Characteristics of five generations of IBM Power processors. All except the Power6, which is static and in-order, were dynamically scheduled; all the processors support two load/store pipelines. The Power6 has the same functional units as the Power5 except for a decimal unit. Power7 and Power8 use embedded DRAM for the L3 cache. Power9 has been described briefly; it further expands the caches and supports off-chip HBM.

DLP – Bevezető

We call these algorithms *data parallel* algorithms because their parallelism comes from simultaneous operations across large sets of data rather than from multiple threads of control.

W. Daniel Hillis and Guy L. Steele,
"Data parallel algorithms," *Commun. ACM* (1986)

If you were plowing a field, which would you rather use: two strong oxen or 1024 chickens?

Seymour Cray, Father of the Supercomputer
(arguing for two powerful vector processors versus many simple processors)

DLP – Bevezető

SIMD (vs MIMD):

- energiafelhasználás
- szekvenciális gondolkodás és párhuzamosság
- nagy memória sávszélesség

SIMD architektúrák:

- vektorprocesszor
- SIMD egységek (x86 – MMX, SSE, AVX)
- GPU

DLP – SIMD és MIMD speedup

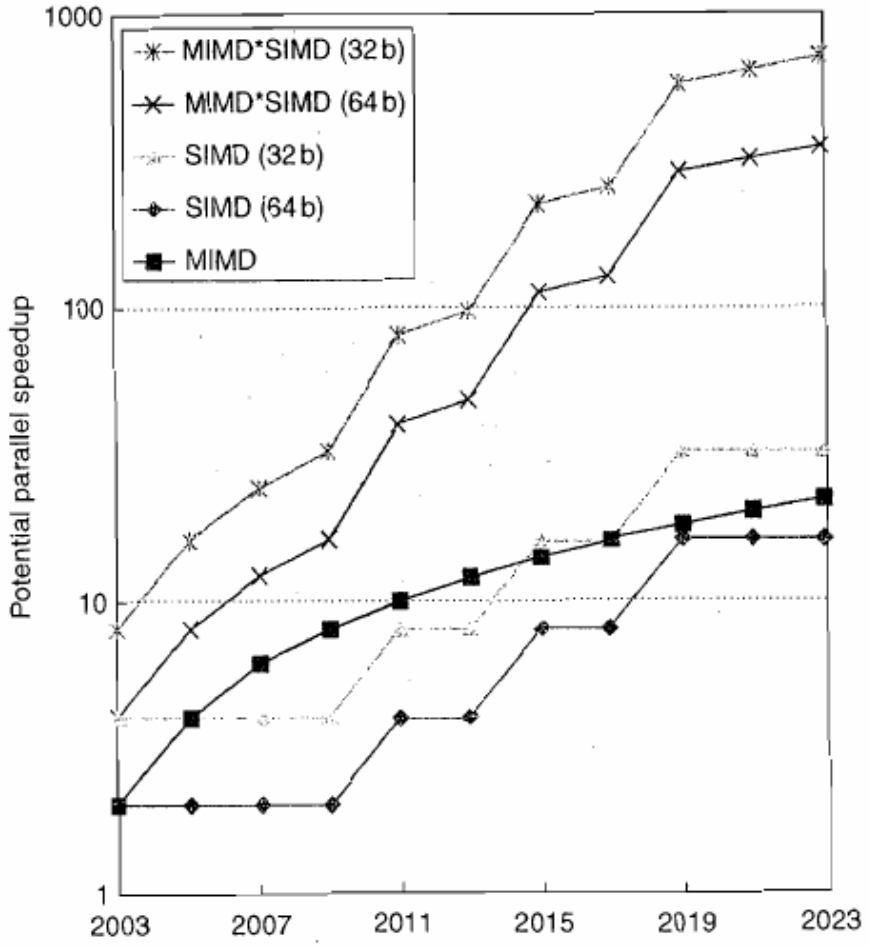


Figure 4.1 Potential speedup via parallelism from MIMD, SIMD, and both MIMD and SIMD over time for x86 computers. This figure assumes that two cores per chip for MIMD will be added every two years and the number of operations for SIMD will double every four years.

DLP – Vektorarchitektúra

Vektorprocesszor betölt egy, a memóriában szétszortan elhelyezkedő adatot egy szekvenciális regiszterfájlba (gather); elvégzi rajtuk a megfelelő (azonos) műveleteket; majd az eredményeket visszaírja (scatter).

Vektorprocesszor részei (itt: VMIPS based on Cray-I)

- *Vekotrregiszterek*: minden regiszter egy fix szélességű és hosszúságú adatvektor megfelelő számú port-tal a funkcionális egységek és memória (R/W) felé.
- *Vektor funkcionális egységek*: Pipeline-olt funkcionális egységek és control unit a data/control hazard-ok felderítésére.
- *Vector load/store unit*: Pipeline-olt load/store egység, mely teljes szavak olvasására és írására képes (néhány órajel után).
- *Skalárregiszterek*: Skaláris értékek tárolására megfelelő számú port-tal (a funkcionális egységek felé).
- *Speciális regiszterek*: vector-length, vector-mask

Vektorprocesszor szerkezete

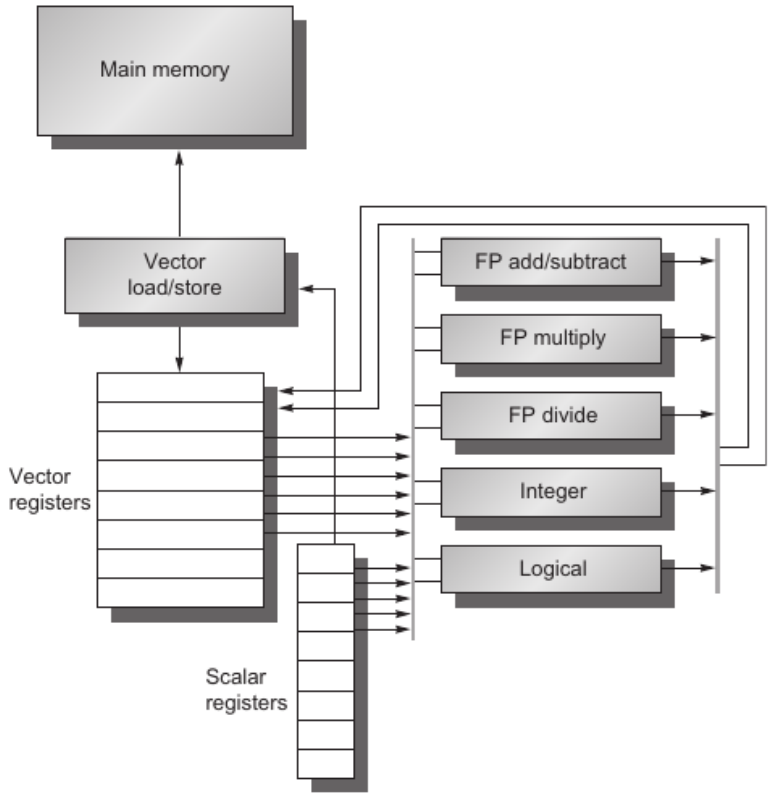


Figure 4.1 The basic structure of a vector architecture, RV64V, which includes a RISC-V scalar architecture. There are also 32 vector registers, and all the functional units are vector functional units. The vector and scalar registers have a significant number of read and write ports to allow multiple simultaneous vector operations. A set of crossbar switches *(thick gray lines)* connects these ports to the inputs and outputs of the vector functional units.

Vektorprocesszorok – Hogyan működik

Megj.: A vektor mérete határozza meg, hogy maximálisan hány elem esetén lehet elkezdeni a végrehajtást. pl.: 64 db 64 bit = 128 db 32 bit = ...

64 elemet tartalmazó adatvektorokkal a DAXPY (vectorizable – no loop-carried dependencies) MIPS utasításkészlettel közel 600 utasítás:

Here is the RISC-V code:

```
        fld     f0,a            # Load scalar a
        addi    x28,x5,#256     # Last address to load
Loop:   fld     f1,0(x5)        # Load X[i]
        fmul.d  f1,f1,f0        # a × X[i]
        fld     f2,0(x6)        # Load Y[i]
        fadd.d  f2,f2,f1        # a × X[i] + Y[i]
        fsd     f2,0(x6)        # Store into Y[i]
        addi    x5,x5,#8        # Increment index to X
        addi    x6,x6,#8        # Increment index to Y
        bne     x28,x5,Loop     # Check if done
```

Vektorprocesszorok – Hogyan működik

VMIPS esetén elég 6 utasítás és a MUL-ADD-STORE késleltetés is csak egyszer fordul elő (64x kevesebbszer).

Here is the RV64V code for DAXPY:

```
vsetdcfg  4*FP64        # Enable 4 DP FP vregs
fld       f0,a          # Load scalar a
vld       v0,x5         # Load vector X
vmul      v1,v0,f0      # Vector-scalar mult
vld       v2,x6         # Load vector Y
vadd      v3,v1,v2      # Vector-vector add
vst       v3,x6         # Store the sum
vdisable                # Disable vector regs
```

Vector execution time, hullám (convoy)

A futási idő 3 körülményen alapszik:

1. A bemeneti vektorok hossza.
2. Structural hazards among operations
3. Adatfüggőség

Miden órajel esetén a lehetséges végrehajtásra kerülő utasítások száma a vektorregiszter szélessége, amelyet lehet hullámnak (itt convoy) nevezni.

Ha a hullám tartalmaz valamilyen függőséged vagy hazard-ot, akkor a hullámot át kell alakítani (veszteség).

(Flexible) Chaining, Chime

Chaining: Adatfüggőség esetén a megfelelő vektorelem továbbítható a funkcionális egységeknek közvetlenül (forwarding; vektor regiszter írása és olvasása egy időben). RAW lehet egy hullámban lévő utasítások között.

Flexible chaining: Chaining bármely vektorművelet között, ha ez nem okoz hibát.

Chime: Egy hullám végrahajtásához szükséges idő. (Az utasítások kibocsájtására felhasznált időt nem tekintjük itt.)

```
vld   v0,x5        # Load vector X
vmul  v1,v0,f0     # Vector-scalar multiply
vld   v2,x6        # Load vector Y
vadd  v3,v1,v2     # Vector-vector add
vst   v3,x6        # Store the sum
```

1. `vld` `vmul`
2. `vld` `vadd`
3. `vst`

Nagy hatékonyságú számítások . . .
9. előadás – DLP – Vektorprocesszor

Nagy Ádám
nagyadam888@gmail.com

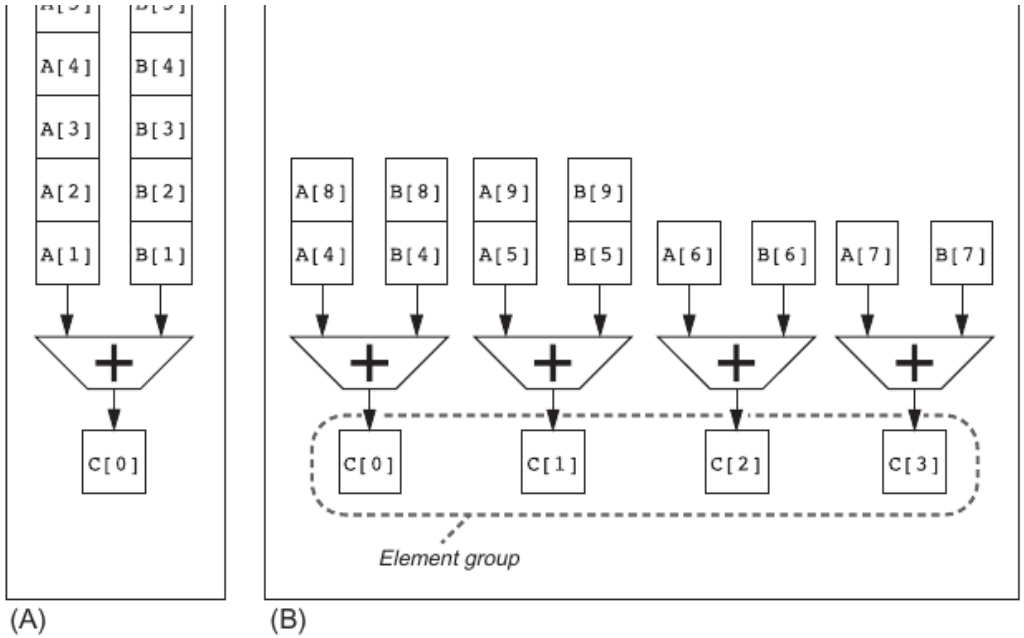
Eötvös Loránd Tudományegyetem
Informatikai Kar

2022. november 8.

Vektorprocesszorok – Sáv

Eddig valójában: egy funkcionális egység végezte a számításokat egy pipeline-on $\Rightarrow$ chime kb. az elemek számának megfelelő órajel.

Sávok (lanes): több pipeline (egy számítási egységen belül) használata a párhuzamosítható számításokhoz.



Akár 64 utasítás órajelenként így: 16 sáv, 2 ALU, 2 l/s

Vektorprocesszorok – Sáv

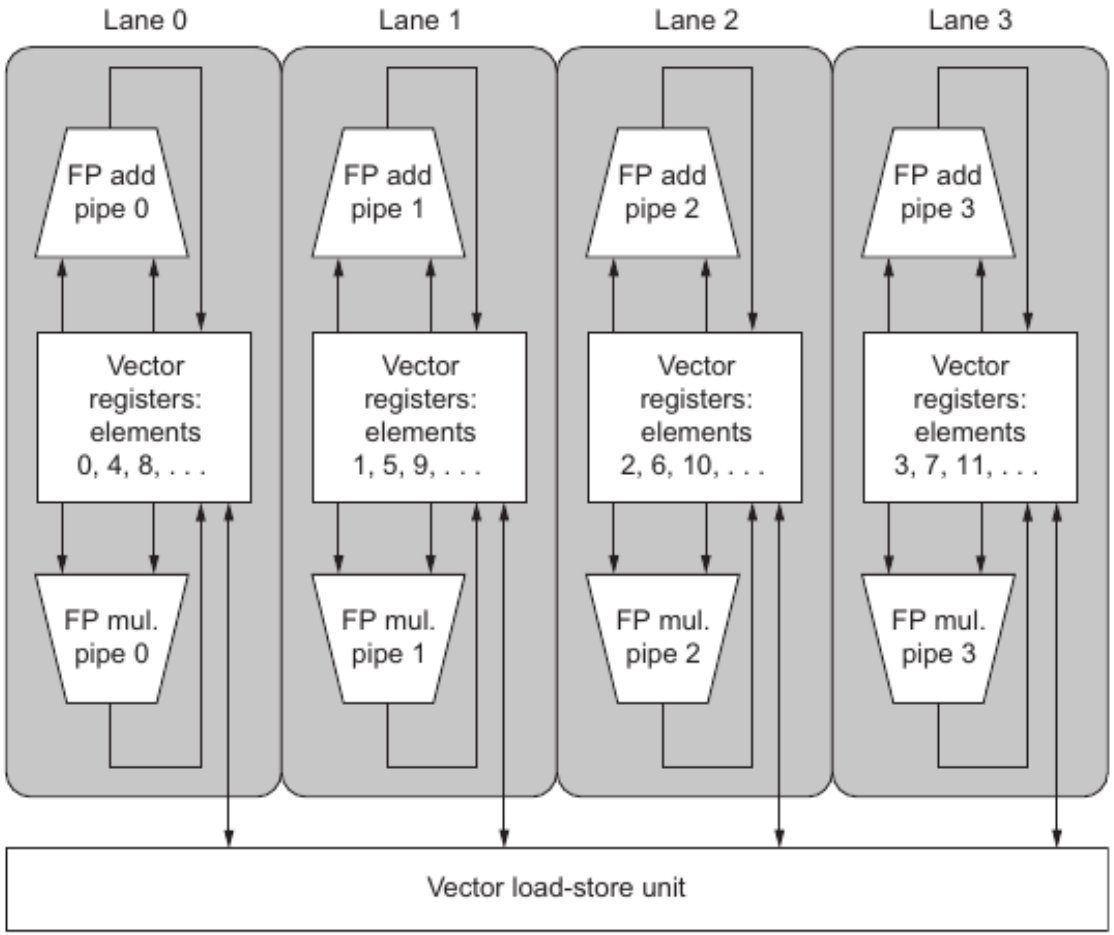


Figure 4.5 Structure of a vector unit containing four lanes. The vector register mem-

Vektorprocesszorok – Vektorok hossza

Valós esetben a vetorok hossza csak ritkán egyezik meg (és nem is osztható) az architektúra regisztervektorának hosszával és sokszor nem is ismert fordítási időben.

Segítség:

- vector-length-register (VLR)
- maximum vector length (MVL)

MVL használatával az architktúra változtatása esetén nem kell a teljes utasításkészletet újradefiniálni (SIMD-nél nincs).

Vektorprocesszorok – Vektorok hossza

Minden utasítás a VLR-nek megfelő számú elemre kerül végrehajtásra és tetszőleges n hosszúságú vektor felírható $n = m * MVL + \varepsilon$ alakban, így egy $m + 1$ hosszú ciklust kapunk mely utolsó iterációjában a VLR értéke ε lesz.

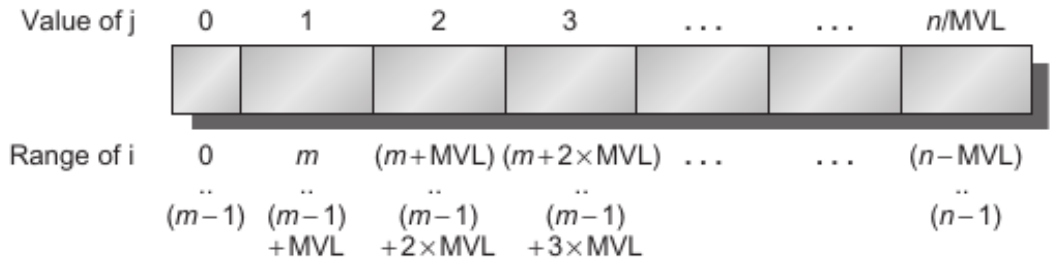


Figure 4.6 A vector of arbitrary length processed with strip mining. All blocks but the first are of length MVL, utilizing the full power of the vector processor. In this figure, we use the variable m for the expression (`n % MVL`). (The C operator `%` is modulo.)

Vektorprocesszorok – Elágazás

Elágazásokat nem tudjuk végrehajtani egyszerre egy vektorra, ha a feltétel a vektor egyes elemeinek értékétől függ.

```
for (i = 0; i < 64;  i=i+1)
  if (X[i] != 0)
    X[i] = X[i] - Y[i];
```

Vector-mask-register (VMR) használata: A vektor egyes elemeire kiszámoljuk az igazságértéket és ezeket egy bináris regiszter bitjeiben tároljuk. A ,,számolás" ezután feltételesen történik, annak megfelelően hogy a VMR megelelő bitje 0 vagy 1.

```
vsetdcfg    2*FP64      # Enable 2 64b FP vector regs
vsetpcfgi   1           # Enable 1 predicate register
vld         v0,x5       # Load vector X into v0
vld         v1,x6       # Load vector Y into v1
fmv.d.x     f0,x0       # Put (FP) zero into f0
```

Így jobb eredményt kapunk, mintha skalárszámítással hajtottuk volna végre a ciklust (akkor is ha sok 0 van a VMR-ben).

Vektorprocesszorok – Memória sávszélesség

A vektor load/store egységeknek képesnek kell lenniük teljes szavak olvasására órajelenként egy adott felfutási idő után.

A legtöbb vektorprocesszor memóriabankokat használ, ui.:

- lehetővé teszi az akár egymástól független elérések szimultán végrehajtását,
- a memóriamodul órajele kisebb mint a processzoré,
- a memóriaelérés lehet nem szekvenciális is (gather/scatter),
- a vektorprocesszorokat használó architektúrák általában több processzort tartalmaznak, melyek ugyanazt a memóriát használják.

Vektorprocesszorok – Többdimenziós tömbök kezelése

Példa: Mátrixszorzás

```
for (i = 0; i < 100;  i=i+1)
  for (j = 0; j < 100;  j=j+1) {
    A[i][j] = 0.0;
    for (k = 0; k < 100; k=k+1)
      A[i][j] = A[i][j] + B[i][k] * D[k][j];
    }
```

Valamelyik dimenzió szerint az elemek elérése nem lesz szekvenciális (nem unit-stride), az elemek távolsága viszont azonos (stride).

Vektorprocesszorok load/store egységei általában képesek kezelni (külön utasítás) az ilyen (non-unit-stride) adatszerkezeteket és tömörített formában tárolni a vektorregiszterekben.

Memóriabankok és a stride kapcsolata:

$$\frac{\text{Bankok száma}}{\text{gcd(stride, bankok száma)}} < \text{Bank késleltetés}$$

Vektorprocesszorok – Ritka mátrixok, Gather-Scatter

Ritka mátrixok elérése indirekt címzéssel történik

```
for (i = 0; i < n;  i=i+1)
  A[K[i]] = A[K[i]] + C[M[i]];
```

A (modernebb) vektorprocesszorok load/store egységei képesek hatékony indirekt memóriakezelésre úgy, hogy a megfelelő címeket képesek egy vektorregiszterből olvasni.

```
vsetdcfg  4*FP64       # 4  64b  FP vector registers
vld       v0, x7       # Load K[]
vldx      v1, x5, v0)  # Load A[K[]]
vld       v2, x28      # Load M[]
vldi      v3, x6, v2)  # Load C[M[]]
vadd      v1, v1, v3   # Add them
vstx      v1, x5, v0)  # Store A[K[]]
vdisable               # Disable vector registers
```

Az itt alkalmazott ún. gather-scatter memóriakezelés azonban messze elmarad attól, ha az adatok távolsága egyenlő.

Vektorporcesszorok – Programozás

A vektorprocesszorok alkalmazása akkor hatékony, ha a probléma megoldása során a processzor vektor módban számol, ahol lehet.

A vektor módban való számítás lehetőségét felfedheti

- fordítóprogram,
- fordítóprogram programozói hintek segítségével,
- programozó (alacsony szint).

Nagy hangsúly van azon, hogy a program milyen struktúrával és technikával készült el, mennyire lehet az alkalmazott adat- és programozási struktúrák (pl. ciklus) esetén vektor számítási módot.

Vektorprocesszorok – Vektorizáció

Benchmark name	Operations executed in vector mode, compiler-optimized	Operations executed in vector mode, with programmer aid	Speedup from hint optimization
BDNA	96.1%	97.2%	1.52
MG3D	95.1%	94.5%	1.00
FLO52	91.5%	88.7%	N/A
ARC3D	91.1%	92.0%	1.01
SPEC77	90.3%	90.4%	1.07
MDG	87.7%	94.2%	1.49
TRFD	69.8%	73.7%	1.67
DYFESM	68.8%	65.6%	N/A
ADM	42.9%	59.6%	3.60
OCEAN	42.8%	91.2%	3.92
TRACK	14.4%	54.6%	2.52
SPICE	11.5%	79.9%	4.06
QCD	4.2%	75.1%	2.15

Figure 4.7 Level of vectorization among the Perfect Club benchmarks when executed on the Cray Y-MP (Vajapeyam, 1991). The first column shows the vectorization level obtained with the compiler without hints, and the second column shows the results after the codes have been improved with hints from a team of Cray Research programmers.

DLP – SIMD egységek

Sok, leginkább multimédia alkalmazás használ kisebb méretű adattípusokat (8, 16, 32 bit) és leginkább csak egyszerű aritmetikai műveleteket úgy, hogy az adatok között nem nagy mértékű a függőség.

Nem nagy költséggel jár egy kevés utasítással rendelkező 256 bit hosszú adatvektorokkal dolgozó SIMD egység alkalmazása.

Instruction category	Operands
Unsigned add/subtract	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Maximum/minimum	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Average	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Shift right/left	Thirty-two 8-bit, sixteen 16-bit, eight 32-bit, or four 64-bit
Floating point	Sixteen 16-bit, eight 32-bit, four 64-bit, or two 128-bit

Figure 4.8 Summary of typical SIMD multimedia support for 256-bit-wide operations. Note that the IEEE 754-2008 floating-point standard added half-precision (16-bit) and quad-precision (128-bit) floating-point operations.

SIMD egységek vs. Vektorprocesszorok

A SIMD egységek a vektorprocesszorokkal szemben

- kevés SIMD regiszterrel rendelkeznek (ált. 8);
- SIMD esetén az operandusok hosszát az utasítás határozza meg, így kevés utasítás esetén is több száz valódi utasítás lesz (vektorarchitektúrák esetén ezt egy regiszter értéke határozza meg);
- nincs vektor load/store egység, így nincs stride és gather-scatter R/W
- általában nincsennek maszk és hossz regiszterek.

Ezek a tulajdonságok azt eredményezik, hogy a fodítóprogramok sokkal kevésbé tudják használni az ilyen egységet és programozásuk is nehézkes (a speciális alkalmazási területeken kívül). Az adatokat megfelelően kell align-olni.

SIMD x86

- *MMX*: fp regiszterek használata 8 és 16 bites műveletekre
- *SSEx*: 128 bit SIMD regiszterek, 8, 16, 32 és később ($x > 2$) 64 bites számításokhoz
- *AVX(-512)*: 256 (512) bit SIMD regiszterek 8,15,32 és 64 bites számításokhoz.
- *FMA*: utasításkészlet összetett utasításokhoz SIMD egységek esetén

SIMD egységek vs. Vektorprocesszorok (round 2)

A SIMD egységek a vektorprocesszorokkal szemben

- olcsó kiegészítés szuperskalár processzorok mellé;
- könnyű szálkezelés (váltás a szálak között);
- nem igényel akkora sávszélességet;
- aligned memóriaelérés (nincs laphiba);
- kisebb vektorméret miatt több komplex utasítás lehetséges.

SIMD – Néhány ADX utasítás

AVX instruction	Description
VADDPD	Add four packed double-precision operands
VSUBPD	Subtract four packed double-precision operands
VMULPD	Multiply four packed double-precision operands
VDIVPD	Divide four packed double-precision operands
VFMADDPD	Multiply and add four packed double-precision operands
VFMSUBPD	Multiply and subtract four packed double-precision operands
VCMPxx	Compare four packed double-precision operands for EQ, NEQ, LT, LE, GT, GE, ...
VMOVAPD	Move aligned four packed double-precision operands
VBROADCASTSD	Broadcast one double-precision operand to four locations in a 256-bit register

Figure 4.9 AVX instructions for x86 architecture useful in double-precision floating-point programs. Packed-double for 256-bit AVX means four 64-bit operands executed in SIMD mode. As the width increases with AVX, it is increasingly important to add data permutation instructions that allow combinations of narrow operands from different parts of the wide registers. AVX includes instructions that shuffle 32-bit, 64-bit, or 128-bit operands within a 256-bit register. For example, BROADCAST replicates a 64-bit operand four times in an AVX register. AVX also includes a large variety of fused multiply-add/subtract instructions; we show just two here.

SIMD – Példa MIPS kiegészítésével DAXPY

Here is the RISC-V SIMD code:

```
        fld       f0,a          #Load scalar a
        splat.4D  f0,f0         #Make 4 copies of a
        addi      x28,x5,#256   #Last address to load
Loop:   fld.4D    f1,0(x5)      #Load X[i] ... X[i+3]
        fmul.4D   f1,f1,f0      #a×X[i] ... a×X[i+3]
        fld.4D    f2,0(x6)      #Load Y[i] ... Y[i+3]
        fadd.4D   f2,f2,f1      # a×X[i]+Y[i]...
                                # a×X[i+3]+Y[i+3]
        fsd.4D    f2,0(x6)      #Store Y[i]... Y[i+3]
        addi      x5,x5,#32     #Increment index to X
        addi      x6,x6,#32     #Increment index to Y
        bne       x28,x5,Loop   #Check if done
```

Arithmetic intensity

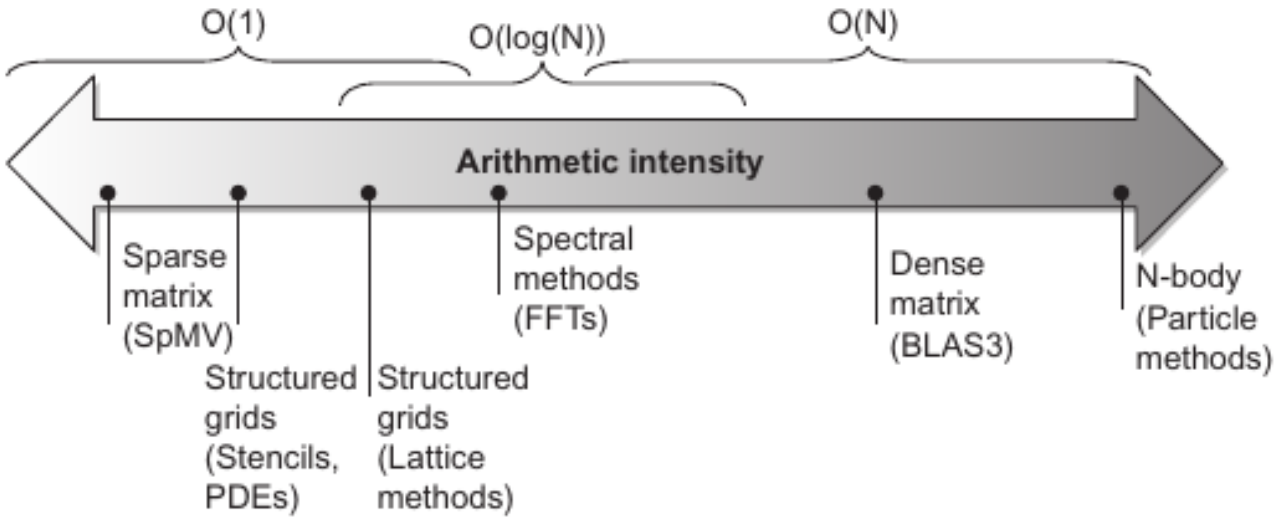


Figure 4.10 Arithmetic intensity, specified as the number of floating-point operations to run the program divided by the number of bytes accessed in main memory (Williams et al., 2009). Some kernels have an arithmetic intensity that scales with problem size, such as a dense matrix, but there are many kernels with arithmetic intensities independent of problem size.

A xFLOP/s megtalálható a hardver leírásában, a memóriacsúcs teljesítménye viszont csak benchmark segítségével készíthető el megbízhatóan (Stream), a memória heterogén szerkezete miatt.

Roofline model

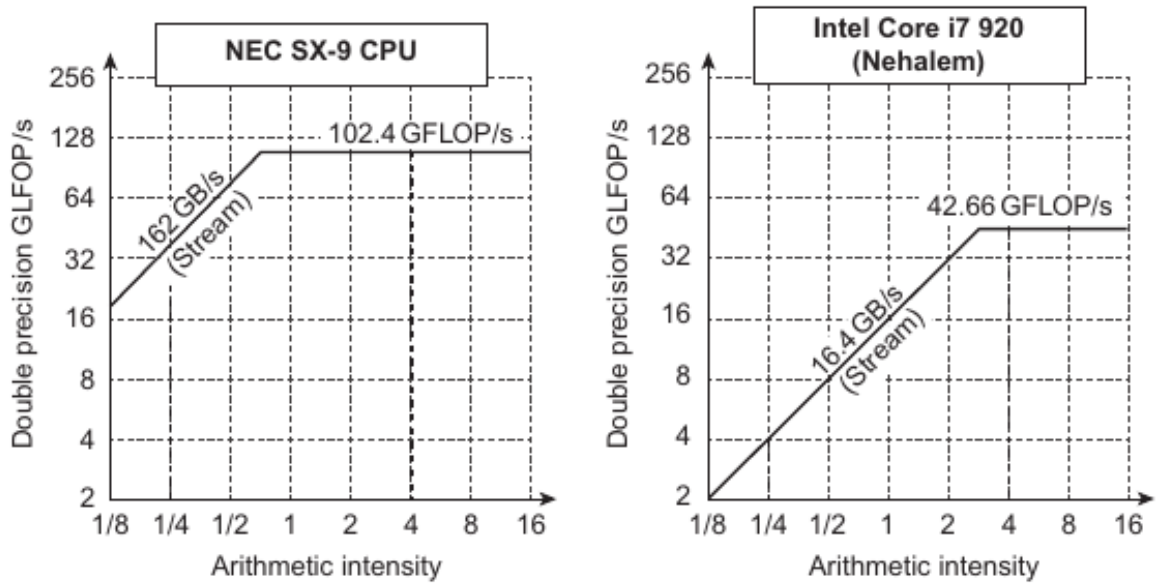


Figure 4.11 Roofline model for one NEC SX-9 vector processor on the left and the Intel Core i7 920 multicore computer with SIMD extensions on the right (Williams et al., 2009). This Roofline is for unit-stride memory accesses and double-precision floating-point performance. NEC SX-9 is a vector supercomputer announced in 2008 that cost millions of dollars. It has a peak DP FP performance of 102.4 GFLOP/s and a peak memory bandwidth of 162 GB/s from the Stream benchmark. The Core i7 920 has a peak DP FP performance of 42.66 GFLOP/s and a peak memory bandwidth of 16.4 GB/s. The dashed vertical lines at an arithmetic intensity of 4 FLOP/byte show that both processors operate at peak performance. In this case, the SX-9 at 102.4 FLOP/s is 2.4× faster than the Core i7 at 42.66 GFLOP/s. At an arithmetic intensity of 0.25 FLOP/byte, the SX-9 is 10× faster at 40.5 GFLOP/s versus 4.1 GFLOP/s for the Core i7.

DLP – GPU

> People can buy a GPU chip with thousands of parallel floating-point units for a few hundred dollars and plug it into their desk side PC. Such affordability and convenience makes high performance computing available to many. The interest in GPU computing blossomed when this potential was combined with a programming language that made GPUs easier to program. Therefore many programmers of scientific and multimedia applications today are pondering whether to use GPUs or CPUs. For programmers interested in machine learning, which is the subject of Chapter 7, GPUs are currently the preferred platform.

GPU programozás során a konkrét programon kívül a fejlesztőnek figyelnie kell/ ki kell használnia a CPU-t és a CPU-memória, GPU-memória közötti adatátvitelt.

Programozás: leginkább C/C++ alapú nyelveken/eszközökkel – OpenCL, Vulcan, SPIR-V, CUDA; és korlátozottan ASM nyelven is

Kérdés: productivity <> performance

GPU – elnevezések 1.

Type	Descriptive name	Closest old term outside of GPUs	Official CUDA/NVIDIA GPU term	Short explanation
Program abstractions	Vectorizable Loop	Vectorizable Loop	Grid	A vectorizable loop, executed on the GPU, made up of one or more Thread Blocks (bodies of vectorized loop) that can execute in parallel
	Body of Vectorized Loop	Body of a (Strip-Mined) Vectorized Loop	Thread Block	A vectorized loop executed on a multithreaded SIMD Processor, made up of one or more threads of SIMD instructions. They can communicate via local memory
	Sequence of SIMD Lane Operations	One iteration of a Scalar Loop	CUDA Thread	A vertical cut of a thread of SIMD instructions corresponding to one element executed by one SIMD Lane. Result is stored depending on mask and predicate register
Machine object	A Thread of SIMD Instructions	Thread of Vector Instructions	Warp	A traditional thread, but it only contains SIMD instructions that are executed on a multithreaded SIMD Processor. Results stored depending on a per-element mask
	SIMD Instruction	Vector Instruction	PTX Instruction	A single SIMD instruction executed across SIMD Lanes

GPU – elnevezések 2.

Processing hardware	Multithreaded SIMD Processor	(Multithreaded) Vector Processor	Streaming Multiprocessor	A multithreaded SIMD Processor executes threads of SIMD instructions, independent of other SIMD Processors
	Thread Block Scheduler	Scalar Processor	Giga Thread Engine	Assigns multiple Thread Blocks (bodies of vectorized loop) to multithreaded SIMD Processors
	SIMD Thread Scheduler	Thread Scheduler in a Multithreaded CPU	Warp Scheduler	Hardware unit that schedules and issues threads of SIMD instructions when they are ready to execute; includes a scoreboard to track SIMD Thread execution
	SIMD Lane	Vector Lane	Thread Processor	A SIMD Lane executes the operations in a thread of SIMD instructions on a single element. Results stored depending on mask

GPU – elnevezések 3.

Memory hardware	GPU Memory	Main Memory	Global Memory	DRAM memory accessible by all multithreaded SIMD Processors in a GPU
	Private Memory	Stack or Thread Local Storage (OS)	Local Memory	Portion of DRAM memory private to each SIMD Lane
	Local Memory	Local Memory	Shared Memory	Fast local SRAM for one multithreaded SIMD Processor, unavailable to other SIMD Processors
	SIMD Lane Registers	Vector Lane Registers	Thread Processor Registers	Registers in a single SIMD Lane allocated across a full Thread Block (body of vectorized loop)

Nagy hatékonyságú számítások . . .
10. előadás – DLP – GPU

Nagy Ádám
nagyadam888@gmail.com

Eötvös Loránd Tudományegyetem
Informatikai Kar

2021. november 26.

GPU

Vektorprocesszoroknál megismert technikák közül:

- gather–scatter (minden load/store ilyen, de spec. hw segít)
- mask registerek (elágazás kicsit máshogy)
- több regiszter (nagyságrendileg) vektorprocesszorokhoz képest
- nincs vagy csak korlátozottan használható skalárprocesszor
- több szálat is képes egyszerre kezelni (memóriakésleltetések elfedése), melyet az ütemező thread blokkokként kezel, a programozó pedig SIMD utasítások csoportjaként ad meg

Könyv: NVidia Tesla (Pascal)

Példa: 2 db 8192 elemet tartalmazó vektor szorzása elemenként

GPU – Grid (vectorizable loop)

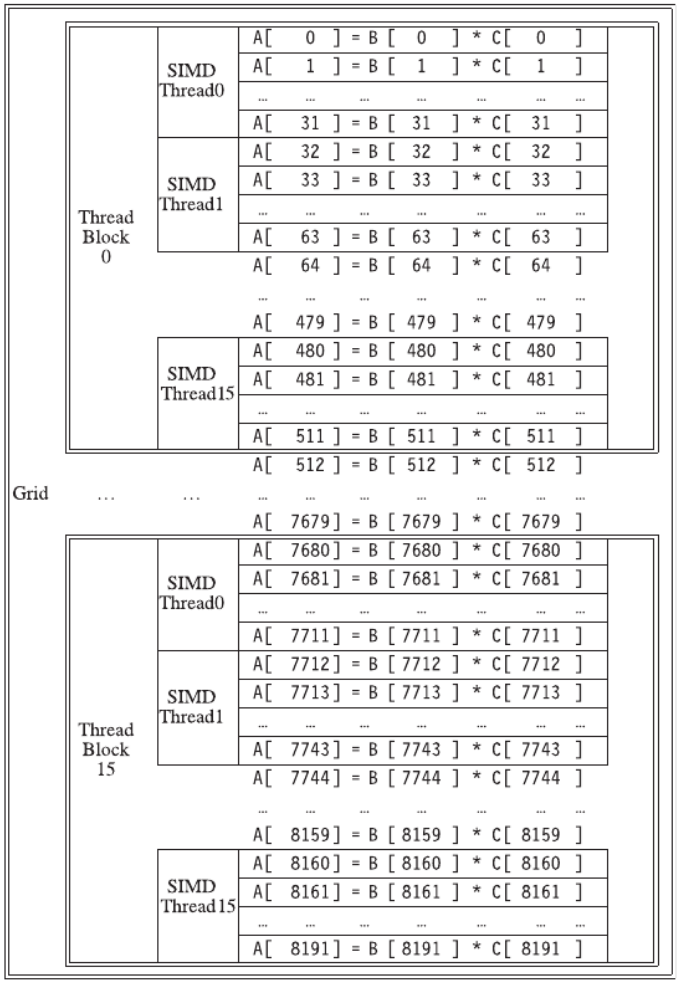


Figure 4.13 The mapping of a Grid (vectorizable loop), Thread Blocks (SIMD basic blocks), and threads of SIMD instructions to a vector-vector multiply, with each vector being 8192 elements long. Each thread of SIMD instructions calculates 32 elements per instruction, and in this example, each Thread Block contains 16 threads of SIMD instructions and the Grid contains 16 Thread Blocks. The hardware Thread Block Scheduler assigns Thread Blocks to multithreaded SIMD Processors, and the hardware Thread Scheduler picks which thread of SIMD instructions to run each clock cycle within a SIMD Processor. Only SIMD Threads in the same Thread Block can communicate via local memory. (The maximum number of SIMD Threads that can execute simultaneously per Thread Block is 32 for Pascal GPUs.)

GPU – SIMD processzor (Thread block scheduler)

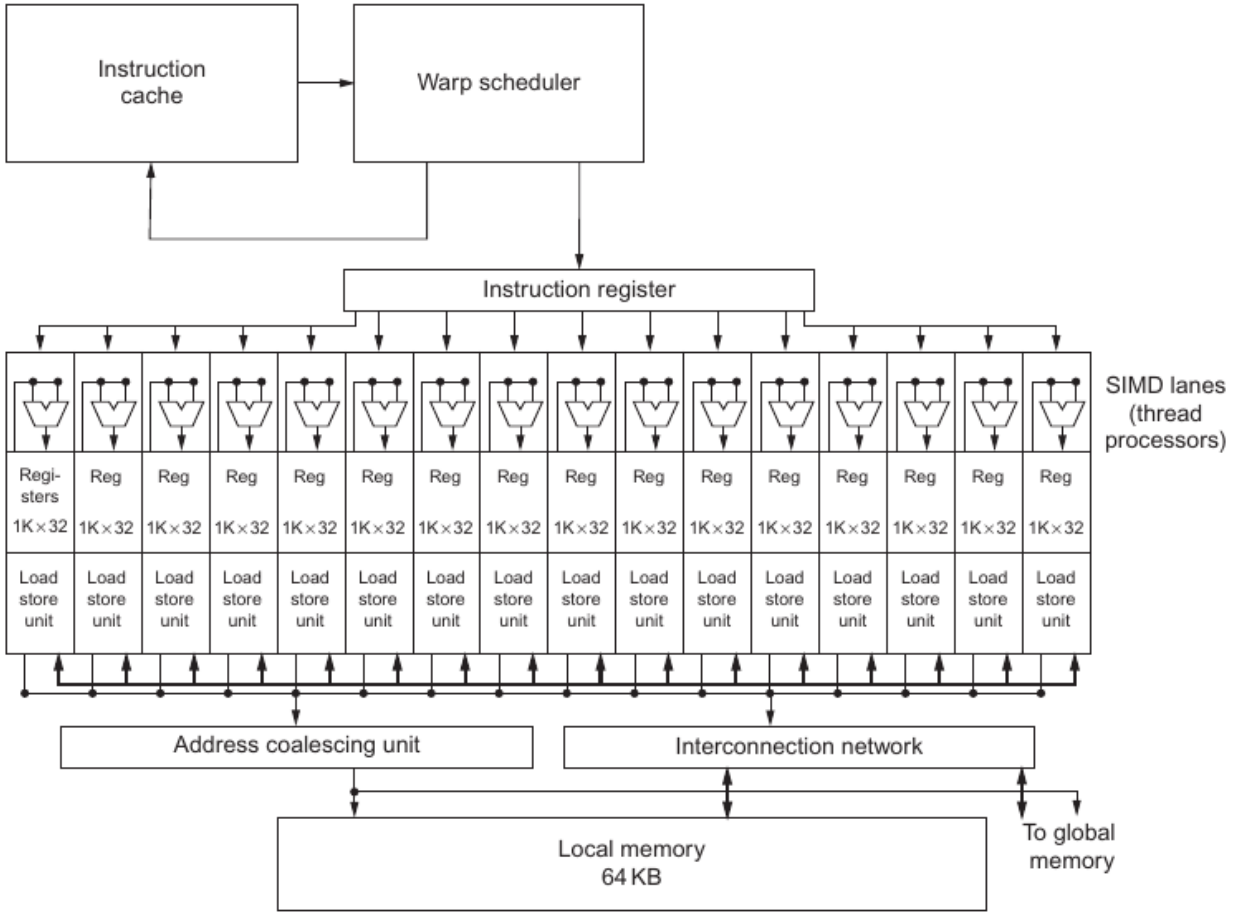


Figure 4.14 Simplified block diagram of a multithreaded SIMD Processor. It has 16 SIMD Lanes. The SIMD Thread Scheduler has, say, 64 independent threads of SIMD instructions that it schedules with a table of 64 program counters (PCs). Note that each lane has 1024 32-bit registers.

GPU – Ütemezés

GPU architektúrák két szintű ütemezést használnak:

1. Thread blokkok ütemezése SIMD processzorokhoz
2. SIMD processzorokon belül az SIMD utsaításokból álló szálak ütemezése

SIMD processzorokban több sáv (SIMD lane) áll rendelkezésre a vektorprocesszorokhoz hasonlóan, habár itt a sokkal több van (chime rövidebb).

Minden sávhoz több ezer regiszter tartozik/tartozhat.

GPU – Ütemezés

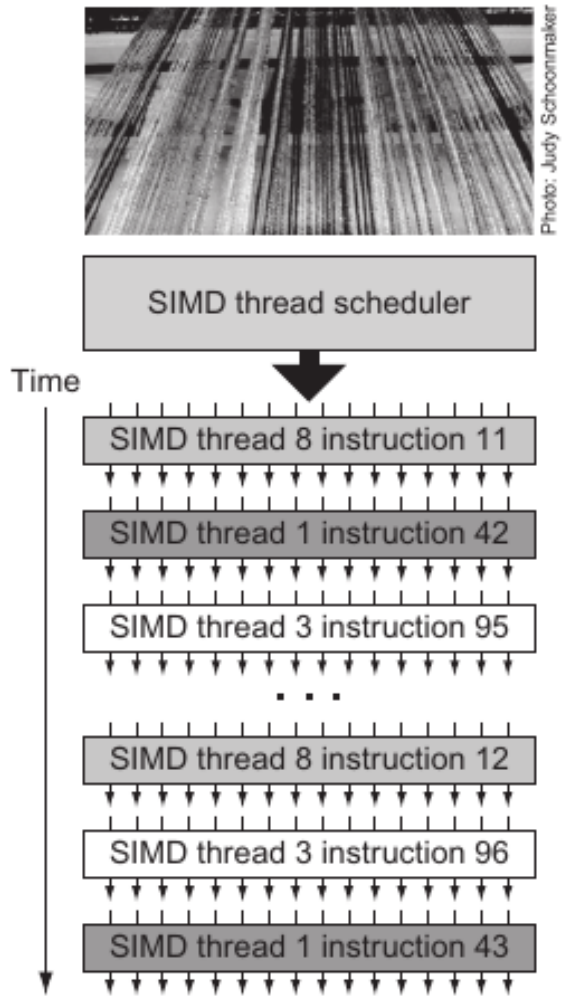


Figure 4.16 Scheduling of threads of SIMD instructions. The scheduler selects a ready thread of SIMD instructions and issues an instruction synchronously to all the SIMD Lanes executing the SIMD Thread. Because threads of SIMD instructions are independent, the scheduler may select a different SIMD Thread each time.

GPU – Utasítások

A GPU gépi utasításai hasonlóak a x86 utasításokhoz abban az értelemben, hogy végrehajtás előtt mindekettő transzformálásra kerül (x86 *to* MicroOp). A különbség az hogy a GPU utasítások (pl. PTX) load time kerülnek fordításra.

Az utasításokhoz tartozik egy a típust jelölő "rész" is

Type	.type specifier
Untyped bits 8, 16, 32, and 64 bits	`.b8, .b16, .b32, .b64`
Unsigned integer 8, 16, 32, and 64 bits	`.u8, .u16, .u32, .u64`
Signed integer 8, 16, 32, and 64 bits	`.s8, .s16, .s32, .s64`
Floating Point 16, 32, and 64 bits	`.f16, .f32, .f64`

GPU – PTX utasítások

Group	Instruction	Example	Meaning	Comments
Arithmetic	arithmetic .type = .s32, .u32, .f32, .s64, .u64, .f64			
	add.type	add.f32 d, a, b	d = a + b;	
	sub.type	sub.f32 d, a, b	d = a - b;	
	mul.type	mul.f32 d, a, b	d = a * b;	
	mad.type	mad.f32 d, a, b, c	d = a * b + c;	multiply-add
	div.type	div.f32 d, a, b	d = a / b;	multiple microinstructions
	rem.type	rem.u32 d, a, b	d = a % b;	integer remainder
	abs.type	abs.f32 d, a	d = |a|;	
	neg.type	neg.f32 d, a	d = 0 - a;	
	min.type	min.f32 d, a, b	d = (a < b)? a:b;	floating selects non-NaN
	max.type	max.f32 d, a, b	d = (a > b)? a:b;	floating selects non-NaN
	setp.cmp.type	setp.lt.f32 p, a, b	p = (a < b);	compare and set predicate
	numeric .cmp = eq, ne, lt, le, gt, ge; unordered cmp = equ, neu, ltu, leu, gtu, geu, num, nan			
	mov.type	mov.b32 d, a	d = a;	move
	selp.type	selp.f32 d, a, b, p	d = p? a: b;	select with predicate
	cvt.dtype.atype	cvt.f32.s32 d, a	d = convert(a);	convert atype to dtype
Special function	special .type = .f32 (some .f64)			
	rcp.type	rcp.f32 d, a	d = 1/a;	reciprocal
	sqrt.type	sqrt.f32 d, a	d = sqrt(a);	square root
	rsqrt.type	rsqrt.f32 d, a	d = 1/sqrt(a);	reciprocal square root
	sin.type	sin.f32 d, a	d = sin(a);	sine
	cos.type	cos.f32 d, a	d = cos(a);	cosine
	lg2.type	lg2.f32 d, a	d = log(a)/log(2)	binary logarithm
	ex2.type	ex2.f32 d, a	d = 2 ** a;	binary exponential
Logical	logic.type = .pred,.b32, .b64			
	and.type	and.b32 d, a, b	d = a & b;	
	or.type	or.b32 d, a, b	d = a | b;	
	xor.type	xor.b32 d, a, b	d = a ^b;	
	not.type	not.b32 d, a, b	d = ~a;	one's complement
	cnot.type	cnot.b32 d, a, b	d = (a==0)? 1:0;	C logical not
	shl.type	shl.b32 d, a, b	d = a << b;	shift left
	shr.type	shr.s32 d, a, b	d = a >> b;	shift right
Memory access	memory.space = .global, .shared, .local, .const; .type = .b8, .u8, .s8, .b16, .b32, .b64			
	ld.space.type	ld.global.b32 d, [a+off]	d = *(a+off);	load from memory space
	st.space.type	st.shared.b32 [d+off], a	*(d+off) = a;	store to memory space
	tex.nd.dtyp.btype	tex.2d.v4.f32.f32 d, a, b	d = tex2d(a, b);	texture lookup
	atom.spc.op.type	atom.global.add.u32 d,[a], b atom.global.cas.b32 d,[a], b, c	atomic { d = *a; *a = op(*a, b); }	atomic read-modify-write operation
	atom.op = and, or, xor, add, min, max, exch, cas; .spc = .global; .type = .b32			
Control flow	branch	@p bra target	if (p) goto target;	conditional branch
	call	call (ret), func, (params)	ret = func(params);	call function
	ret	ret	return;	return from function call
	bar.sync	bar.sync d	wait for threads	barrier synchronization
	exit	exit	exit;	terminate thread execution

Figure 4.17 Basic PTX GPU thread instructions.

GPU – CUDA DAXPY

```
shl.u32 R8, blockIdx, 8   ; Thread Block ID * Block size
                          ;(256 or 2^8)
add.u32 R8, R8, threadIdx ; R8 = i = my CUDA Thread ID
shl.u32 R8, R8, 3         ; byte offset
ld.global.f64 RD0, [X+R8]; RD0 = X[i]
ld.global.f64 RD2, [Y+R8]; RD2 = Y[i]
mul.f64 RD0, RD0, RD4     ; Product in RD0 = RD0 * RD4
                          ; (scalar a)
add.f64 RD0, RD0, RD2     ; Sum in RD0 = RD0 + RD2 (Y[i])
st.global.f64 [Y+R8], RD0; Y[i] = sum (X[i]*a + Y[i])
```

GPU – Elágazás (Conditional execution)

Vektorprocesszorhoz hasonlóan maszkok segítségével, de itt nem a szoftver hanem a hardver végzi el az átalakítást.

A maszk regiszterek továbbra is használhatóak.

Branch synchronization stack és instruction marker az ágak szétválasztásához és egyesítéséhez (szinkronizáció).

Minden sáv használhat független jóslást (néhány bit), amelyet az assembler figyelembe vesz (lehet, hogy nem is lesz elágazás).

Az assembler végzi el az elágazások optimalizációját és a ciklusok kiválasztását és egy stack segítségével a végrehajtásukat több sávon.

Fontos: egymásba ágyazás növeli a idle állapot arányát.

GPU – Memória (Tesla)

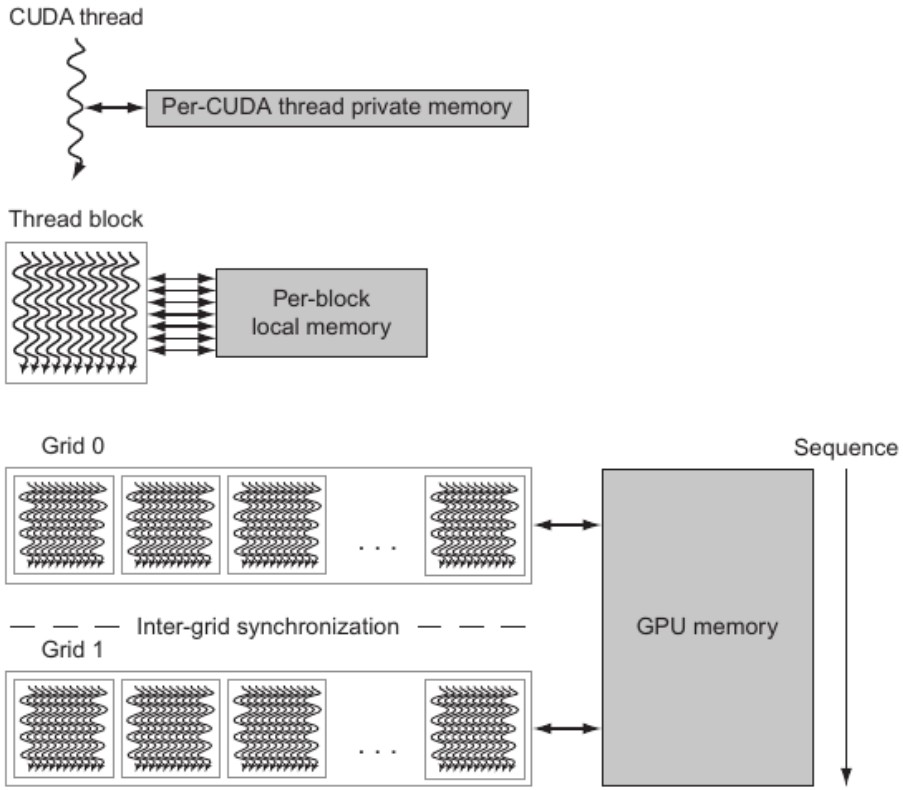


Figure 4.18 GPU memory structures. GPU memory is shared by all Grids (vectorized loops), local memory is shared by all threads of SIMD instructions within a Thread Block (body of a vectorized loop), and private memory is private to a single CUDA Thread. Pascal allows preemption of a Grid, which requires that all local and private memory be able to be saved in and restored from global memory. For completeness sake, the GPU can also access CPU memory via the PCIe bus. This path is commonly used for a final result when its address is in host memory. This option eliminates a final copy from the GPU memory to the host memory.

GPU – Pascal

Figure 4.20 Block diagram of the multithreaded SIMD Processor of a Pascal GPU. Each of the 64 SIMD Lanes (cores) has a pipelined floating-point unit, a pipelined integer unit, some logic for dispatching instructions and operands to these units, and a queue for holding results. The 64 SIMD Lanes interact with 32 double-precision ALUs (DP units) that perform 64-bit floating-point arithmetic, 16 load-store units (LD/STs), and 16 special function units (SFUs) that calculate functions such as square roots, reciprocals, sines, and cosines.

GPU vs vektorprocesszor

- GPU MIMD architektúra, ami több szál kezelését teszi lehetővé
- GPU-nál nincs vektorregiszter, a vektoroknak megfelelő adatok az egyes regiszterfájlokban vannak
- több sáv, rövidebb chime
- memóriakésleltetés elrejtése: VP: pipeline, GPU: multithreading
- multithreading elrejtése több szál kezelésével
- elágazások kezelését hw végzi
- nincs kontrolprocesszor ...
- nincs skalárprocesszor ...

GPU vs vektorprocesszor

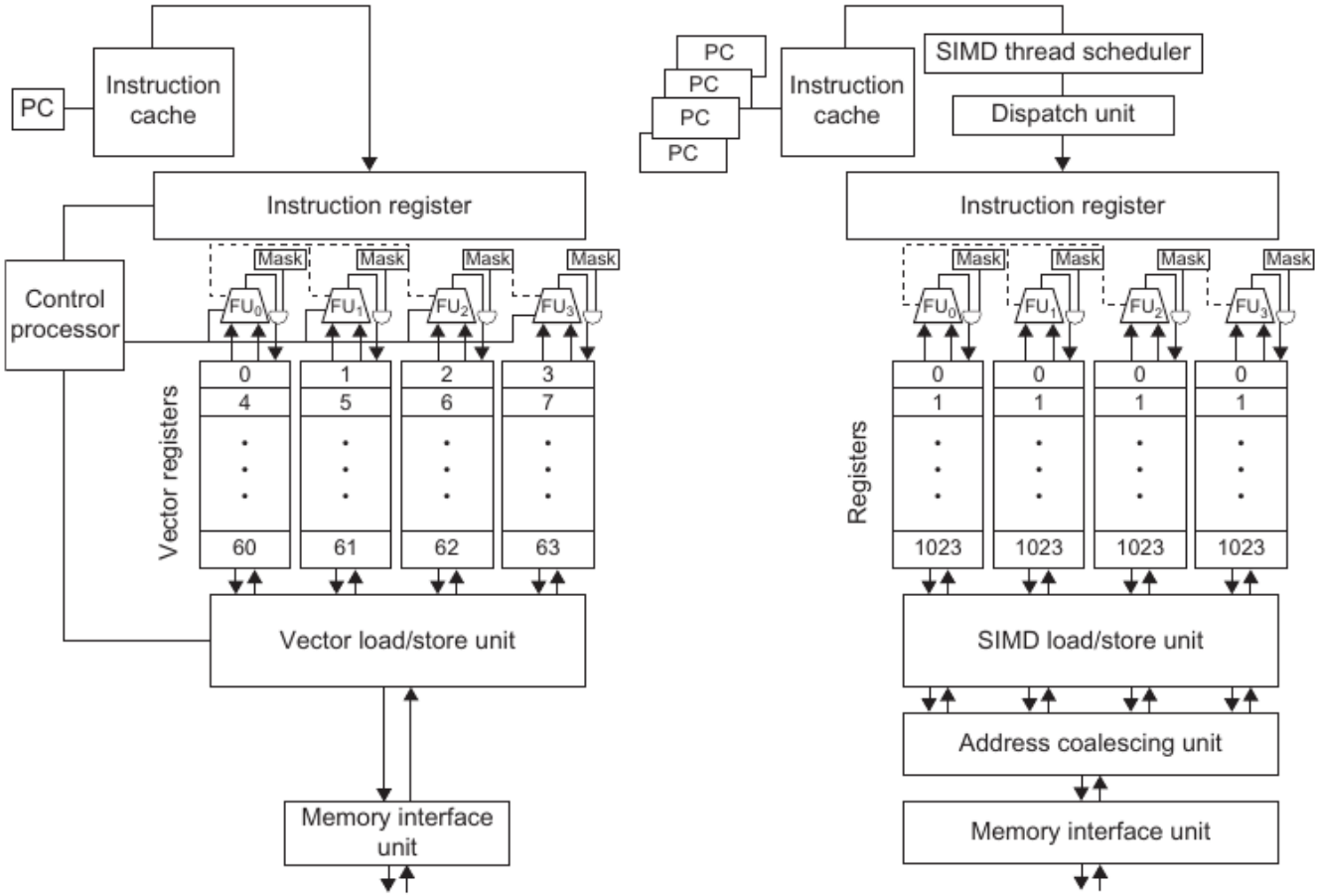


Figure 4.22 A vector processor with four lanes on the left and a multithreaded SIMD Processor of a GPU with four SIMD Lanes on the right. (GPUs typically have 16 or 32 SIMD Lanes.) The Control Processor supplies scalar operands for scalar-vector operations, increments addressing for unit and nonunit stride accesses to memory, and performs other accounting-type operations. Peak memory performance occurs only in a GPU when the Address Coalescing Unit can discover localized addressing. Similarly, peak computational performance occurs when all internal mask bits are set identically. Note that the SIMD Processor has one PC per SIMD Thread to help with multithreading.

GPU vs SIMD egységek

Feature	Multicore with SIMD	GPU
SIMD Processors	4–8	8–32
SIMD Lanes/Processor	2–4	up to 64
Multithreading hardware support for SIMD Threads	2–4	up to 64
Typical ratio of single-precision to double-precision performance	2:1	2:1
Largest cache size	40 MB	4 MB
Size of memory address	64-bit	64-bit
Size of main memory	up to 1024 GB	up to 24 GB
Memory protection at level of page	Yes	Yes
Demand paging	Yes	Yes
Integrated scalar processor/SIMD Processor	Yes	No
Cache coherent	Yes	Yes on some systems

Figure 4.23 Similarities and differences between multicore with multimedia SIMD extensions and recent GPUs.

GPU elnevezések

Type	More descriptive name used in this book	Official CUDA/ NVIDIA term	Short explanation and AMD and OpenCL terms	Official CUDA/NVIDIA definition
Program abstractions	Vectorizable loop	Grid	A vectorizable loop, executed on the GPU, made up of one or more "Thread Blocks" (or bodies of vectorized loop) that can execute in parallel. OpenCL name is "index range." AMD name is "NDRange"	A Grid is an array of Thread Blocks that can execute concurrently, sequentially, or a mixture
	Body of Vectorized loop	Thread Block	A vectorized loop executed on a multithreaded SIMD Processor, made up of one or more threads of SIMD instructions. These SIMD Threads can communicate via local memory. AMD and OpenCL name is "work group"	A Thread Block is an array of CUDA Threads that execute concurrently and can cooperate and communicate via shared memory and barrier synchronization. A Thread Block has a Thread Block ID within its Grid
	Sequence of SIMD Lane operations	CUDA Thread	A vertical cut of a thread of SIMD instructions corresponding to one element executed by one SIMD Lane. Result is stored depending on mask. AMD and OpenCL call a CUDA Thread a "work item"	A CUDA Thread is a lightweight thread that executes a sequential program and that can cooperate with other CUDA Threads executing in the same Thread Block. A CUDA Thread has a thread ID within its Thread Block
Machine object	A thread of SIMD instructions	Warp	A traditional thread, but it contains just SIMD instructions that are executed on a multithreaded SIMD Processor. Results are stored depending on a per-element mask. AMD name is "wavefront"	A warp is a set of parallel CUDA Threads (e.g., 32) that execute the same instruction together in a multithreaded SIMT/SIMD Processor
	SIMD instruction	PTX instruction	A single SIMD instruction executed across the SIMD Lanes. AMD name is "AMDIL" or "FSAIL" instruction	A PTX instruction specifies an instruction executed by a CUDA Thread

Figure 4.24 Conversion from terms used in this chapter to official NVIDIA/CUDA and AMD jargon. OpenCL names are given in the book's definitions.

GPU elnevezések

Type	More descriptive name used in this book	Official CUDA/ NVIDIA term	Short explanation and AMD and OpenCL terms	Official CUDA/NVIDIA definition
Processing hardware	Multithreaded SIMD processor	Streaming multiprocessor	Multithreaded SIMD Processor that executes thread of SIMD instructions, independent of other SIMD Processors. Both AMD and OpenCL call it a "compute unit." However, the CUDA programmer writes program for one lane rather than for a "vector" of multiple SIMD Lanes	A streaming multiprocessor (SM) is a multithreaded SIMT/SIMD Processor that executes warps of CUDA Threads. A SIMT program specifies the execution of one CUDA Thread, rather than a vector of multiple SIMD Lanes
	Thread Block Scheduler	Giga Thread Engine	Assigns multiple bodies of vectorized loop to multithreaded SIMD Processors. AMD name is "Ultra-Threaded Dispatch Engine"	Distributes and schedules Thread Blocks of a grid to streaming multiprocessors as resources become available
	SIMD Thread scheduler	Warp scheduler	Hardware unit that schedules and issues threads of SIMD instructions when they are ready to execute; includes a scoreboard to track SIMD Thread execution. AMD name is "Work Group Scheduler"	A warp scheduler in a streaming multiprocessor schedules warps for execution when their next instruction is ready to execute
	SIMD Lane	Thread processor	Hardware SIMD Lane that executes the operations in a thread of SIMD instructions on a single element. Results are stored depending on mask. OpenCL calls it a "processing element." AMD name is also "SIMD Lane"	A thread processor is a datapath and register file portion of a streaming multiprocessor that executes operations for one or more lanes of a warp

GPU elnevezések

Memory hardware				
	GPU Memory	Global memory	DRAM memory accessible by all multithreaded SIMD Processors in a GPU. OpenCL calls it "global memory"	Global memory is accessible by all CUDA Threads in any Thread Block in any grid; implemented as a region of DRAM, and may be cached
	Private memory	Local memory	Portion of DRAM memory private to each SIMD Lane. Both AMD and OpenCL call it "private memory"	Private "thread-local" memory for a CUDA Thread; implemented as a cached region of DRAM
	Local memory	Shared memory	Fast local SRAM for one multithreaded SIMD Processor, unavailable to other SIMD Processors. OpenCL calls it "local memory." AMD calls it "group memory"	Fast SRAM memory shared by the CUDA Threads composing a Thread Block, and private to that Thread Block. Used for communication among CUDA Threads in a Thread Block at barrier synchronization points
	SIMD Lane registers	Registers	Registers in a single SIMD Lane allocated across body of vectorized loop. AMD also calls them "registers"	Private registers for a CUDA Thread; implemented as multithreaded register file for certain lanes of several warps for each thread processor

Figure 4.25 Conversion from terms used in this chapter to official NVIDIA/CUDA and AMD jargon. Note that our descriptive terms "local memory" and "private memory" use the OpenCL terminology. NVIDIA uses SIMT (single-instruction multiple-thread) rather than SIMD to describe a streaming multiprocessor. SIMT is preferred over SIMD because the per-thread branching and control flow are unlike any SIMD machine.

GPU Memory vs Main Memory

- Mindkettő magas sávszélességet követel meg
- nincs DIMM, közvetlenül forrasztva van
- nagyobb órajel
- kisebb méret
- bonyolultabb hw az eléréshez
- nagy szélességű adatsinek használata
- TLB bonyolultabb

GPU vs Mobile GPU

	NVIDIA Tegra 2	NVIDIA Tesla P100
Market	Automotive, Embedded, Console, Tablet	Desktop, server
System processor	Six-Core ARM (2 Denver2 +4 A57)	Not applicable
System interface	Not applicable	PCI Express × 16 Gen 3
System interface bandwidth	Not applicable	16 GB/s (each direction), 32 GB/s (total)
Clock rate	1.5 GHz	1.4 GHz
SIMD multiprocessors	2	56
SIMD Lanes/SIMD multiprocessor	128	64
Memory interface	128-bit LP-DDR4	4096-bit HBM2
Memory bandwidth	50 GB/s	732 GB/s
Memory capacity	up to 16 GB	up to 16 GB
Transistors	7 billion	15.3 billion
Process	TSMC 16 nm FinFET	TSMC 16 nm FinFET
Die area	147 mm^2	645 mm^2
Power	20 W	300 W

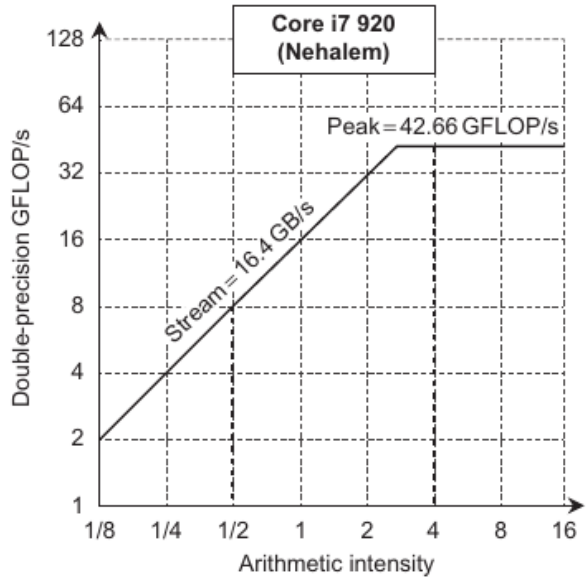
Figure 4.26 Key features of the GPUs for embedded clients and servers.

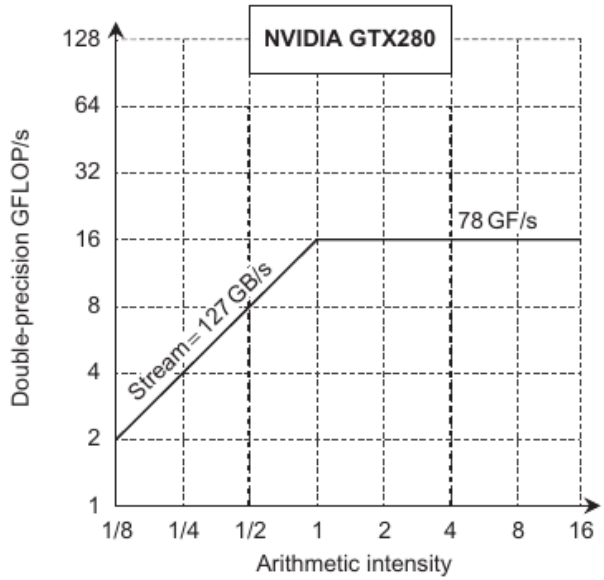
GPU vs CPU

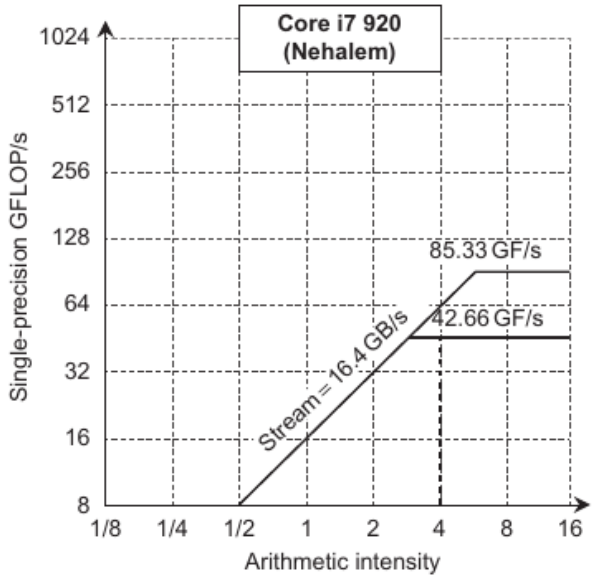
	Core i7-960	GTX 280	Ratio 280/i7
Number of processing elements (cores or SMs)	4	30	7.5
Clock frequency (GHz)	3.2	1.3	0.41
Die size	263	576	2.2
Technology	Intel 45 nm	TSMC 65 nm	1.6
Power (chip, not module)	130	130	1.0
Transistors	700 M	1400 M	2.0
Memory bandwidth (GB/s)	32	141	4.4
Single-precision SIMD width	4	8	2.0
Double-precision SIMD width	2	1	0.5
Peak single-precision scalar FLOPS (GFLOP/S)	26	117	4.6
Peak single-precision SIMD FLOPS (GFLOP/S)	102	311–933	3.0–9.1
(SP 1 add or multiply)	N.A.	(311)	(3.0)
(SP 1 instruction fused multiply-adds)	N.A.	(622)	(6.1)
(Rare SP dual issue fused multiply-add and multiply)	N.A.	(933)	(9.1)
Peak double-precision SIMD FLOPS (GFLOP/S)	51	78	1.5

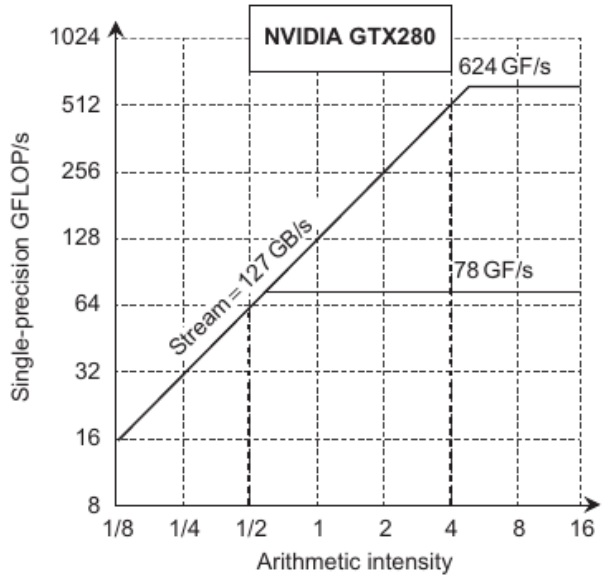
Figure 4.27 Intel Core i7-960 and NVIDIA GTX 280. The rightmost column shows the ratios of GTX 280 to Core i7. For single-precision SIMD FLOPS on the GTX 280, the higher speed (933) comes from a very rare case of dual issuing of fused multiply-add and multiply. More reasonable is 622 for single fused multiply-adds. Note that these memory bandwidths are higher than in Figure 4.28 because these are DRAM pin bandwidths and those in Figure 4.28 are at the processors as measured by a benchmark program. From Table 2 in Lee, W.V., et al., 2010. Debunking the 100 × GPU vs. CPU myth: an evaluation of throughput computing on CPU and GPU. In: Proc. 37th Annual Int'l. Symposium on Computer Architecture (ISCA), June 19–23, 2010, Saint-Malo, France.

GPU vs CPU


Core i7 920
(Nehalem)
Peak = 42.66 GFLOP/s
Stream = 16.4 GB/s
Double-precision GFLOP/s
128
64
32
16
8
4
2
1
1/8
1/4
1/2
1
2
4
8
16
Arithmetic intensity


NVIDIA GTX280
78 GF/s
Stream = 127 GB/s
Double-precision GFLOP/s
128
64
32
16
8
4
2
1
1/8
1/4
1/2
1
2
4
8
16
Arithmetic intensity


Core i7 920
(Nehalem)
85.33 GF/s
42.66 GF/s
Stream = 16.4 GB/s
Single-precision GFLOP/s
1024
512
256
128
64
32
16
8
1/8
1/4
1/2
1
2
4
8
16
Arithmetic intensity


NVIDIA GTX280
624 GF/s
78 GF/s
Stream = 127 GB/s
Single-precision GFLOP/s
1024
512
256
128
64
32
16
8
1/8
1/4
1/2
1
2
4
8
16
Arithmetic intensity

GPU vs CPU

Kernel	Application	SIMD	TLP	Characteristics
SGEMM (**SGEMM**)	Linear algebra	Regular	Across 2D tiles	Compute bound after tiling
Monte Carlo (**MC**)	Computational finance	Regular	Across paths	Compute bound
Convolution (**Conv**)	Image analysis	Regular	Across pixels	Compute bound; BW bound for small filters
FFT (**FFT**)	Signal processing	Regular	Across smaller FFTs	Compute bound or BW bound depending on size
SAXPY (**SAXPY**)	Dot product	Regular	Across vector	BW bound for large vectors
LBM (**LBM**)	Time migration	Regular	Across cells	BW bound
Constraint solver (**Solv**)	Rigid body physics	Gather/ Scatter	Across constraints	Synchronization bound
SpMV (**SpMV**)	Sparse solver	Gather	Across nonzero	BW bound for typical large matrices
GJK (**GJK**)	Collision detection	Gather/ Scatter	Across objects	Compute bound
Sort (**Sort**)	Database	Gather/ Scatter	Across elements	Compute bound
Ray casting (**RC**)	Volume rendering	Gather	Across rays	4–8 MB first level working set; over 500 MB last level working set
Search (**Search**)	Database	Gather/ Scatter	Across queries	Compute bound for small tree, BW bound at bottom of tree for large tree
Histogram (**Hist**)	Image analysis	Requires conflict detection	Across pixels	Reduction/synchronization bound
Bilateral (**Bilat**)	Image analysis	Regular	Across pixels	Compute bound

Figure 4.29 Throughput computing kernel characteristics. The name in parentheses identifies the benchmark name in this section. The authors suggest that code for both machines had equal optimization effort. From

GPU vs CPU

Kernel	Units	Core i7-960	GTX 280	GTX 280/ i7-960
SGEMM	GFLOP/s	94	364	3.9
MC	Billion paths/s	0.8	1.4	1.8
Conv	Million pixels/s	1250	3500	2.8
FFT	GFLOP/s	71.4	213	3.0
SAXPY	GB/s	16.8	88.8	5.3
LBM	Million lookups/s	85	426	5.0
Solv	Frames/s	103	52	0.5
SpMV	GFLOP/s	4.9	9.1	1.9
GJK	Frames/s	67	1020	15.2
Sort	Million elements/s	250	198	0.8
RC	Frames/s	5	8.1	1.6
Search	Million queries/s	50	90	1.8
Hist	Million pixels/s	1517	2583	1.7
Bilat	Million pixels/s	83	475	5.7

Figure 4.30 Raw and relative performance measured for the two platforms. In this study, SAXPY is used only as a measure of memory bandwidth, so the right unit is GB/s and not GFLOP/s. Based on Table 3 in Lee, W.V., et al., 2010. Debunking the

GPU vs CPU, okok

- *Memória sávszélesség* GPU 4.4x sávszélessége olyan problémáknál, ami nem fér el a CPU cache-ben (SAXPY, LBM)
- *Számítási sávszélesség* Sp jobb GPU-n (főleg a MULADD utasítást használó programoknál), de DP támogatás gyenge.
- *Cache fontossága* Ahol a program elfér a cache-ben a GPU-val nem volt számottevő a nyereség.
- *Gather-scatter* a SIMD nem tud és ráadásul csak align-olt adatot tud olvasni, így nem sokat segít a CPU-n, ha a program sok gather-scatter memóriaelérést végez (GJK)
- *Szinkronizáció* Atomic utasítások és a hozzájuk tartozó szinkronizáció a GPU-k esetén jelentős sebességnövekedést jelenthet (Hist).

GPU csapdák

Sok szál segít: A nem elegendő memória (sávszélesség) problémáját általában nem tudjuk megoldani még több szál indításával. Ennek oka az, hogy az ütemező véletlenszerűen választ a szálak közül, így a lokalitás elve (és vele a hatékony memóriaelérés) annál valószínűbben sérül minél több várakozó szál van.

GPU általános számításokra készült koprocesszor

GPU vs CPU

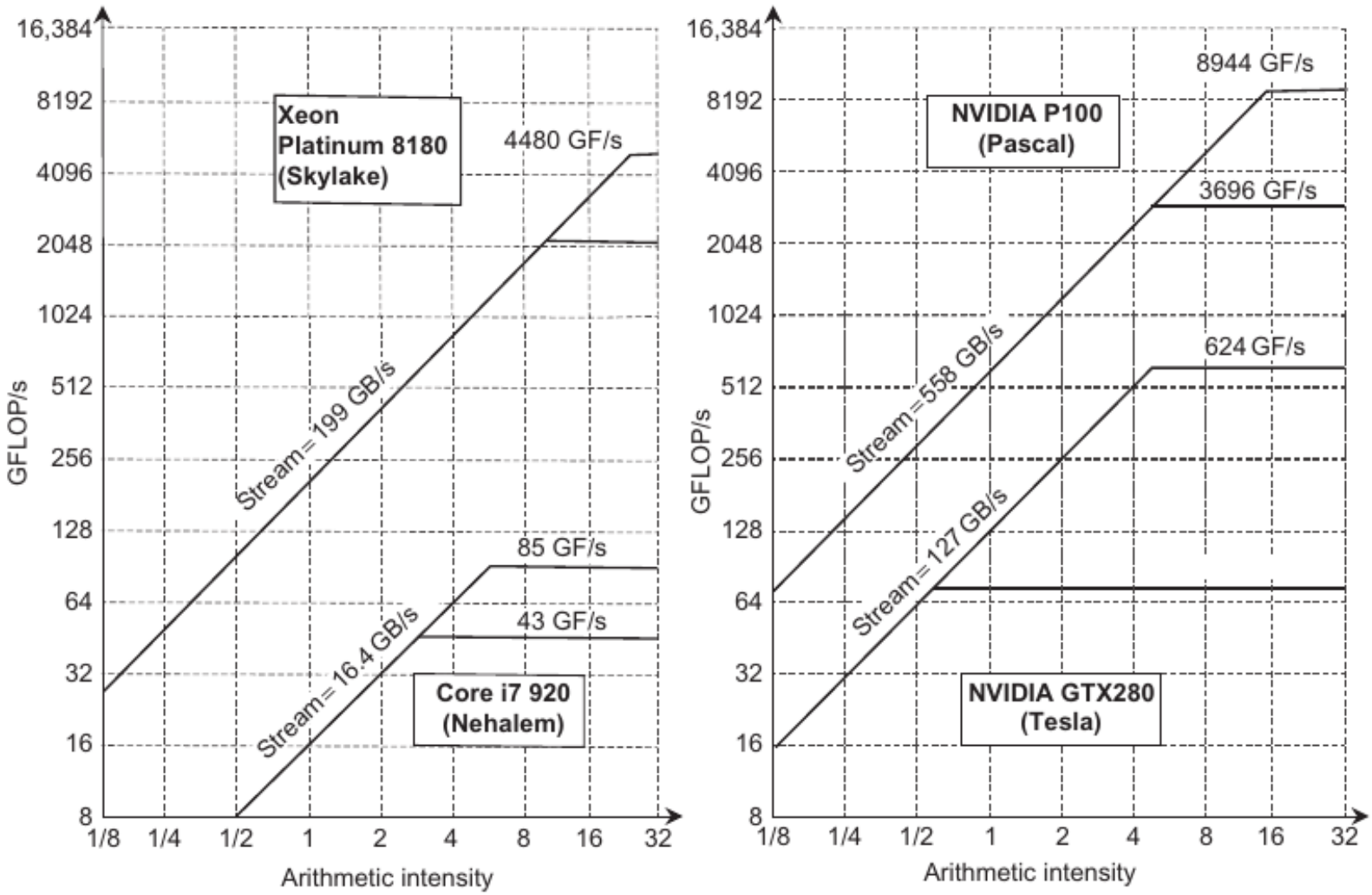


Figure 4.33 Roofline models of older and newer CPUs versus older and newer GPUs. The higher roofline for each computer is single-precision floating-point performance, and the lower one is double-precision performance.

GPGPU

- Intel Xeon Phi (koprocesszor) -2020
- AMD Instinct: GCN, CDNA
- NVIDIA Tesla (A100)

Könyv után

- HBM2
- jobb kommunikáció PCIe-n
- load balance
- ...

Nagy hatékonyságú számítások . . .
11. előadás – Thread Level Parallelism

Nagy Ádám
nagyadam888@gmail.com

Eötvös Loránd Tudományegyetem
Informatikai Kar

2022. november 22.

Thread Level Parallelism

Miért fontos (manapság egyre jobban):

- ILP korlátozott és jelenleg (manapság használt technológiával) nem éri meg többet használni, így perspektíva váltás kell.
- Több high-end szerver és cloud-computing.
- Adatintenzív alkalmazások az Interneten található adat feldolgozására.
- Egy külön (asztali) számítógép sebességének növelése nem szempont (kivéve média alkalmazások) manapság.
- Olcsóbb többet használni, mint jobbat tervezni.

Most: multiprocesszor (2-32) közös, megosztott memória címtérrel.

1. parallel processing (több szál egy munkán).
2. request-level parallelism (viszonylag elkülönülő munkák).

Szálak esetén fontos a *Grain size*: egy szál által végzett számítás.

Symmetric (shared-memory) multiprocessors (SMPs)

A memória és sok esetben egy réteg cache közös. A különböző processzorok memóriaelérésének késleltetse is gyakran azonos (UMA).

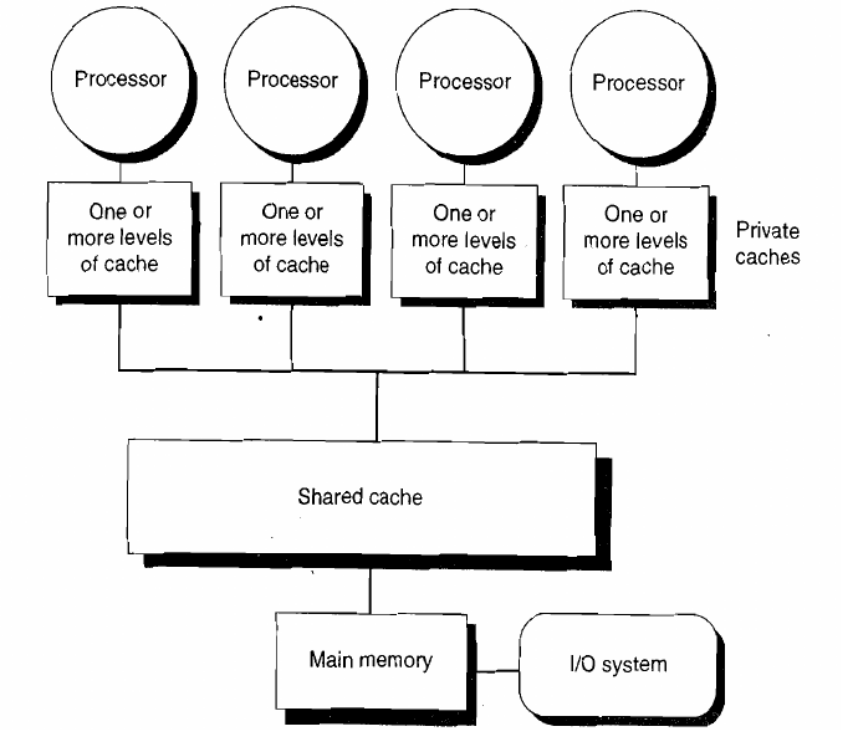


Figure 5.1 Basic structure of a centralized shared-memory multiprocessor based on a multicore chip. Multiple processor–cache subsystems share the same physical memory, typically with one level of shared cache, and one or more levels of private per-core cache. The key architectural property is the uniform access time to all of the memory from all of the processors. In a multichip version the shared cache would be omitted and the bus or interconnection network connecting the processors to memory would run between chips as opposed to within a single chip.

Distributed Shared Memory (DSM)

Minden processzor rendelkezik saját memóriaegységgel, így a sávszélesség egy processzorra nézve és a lokális késleltetés kedvezőbb; viszont itt más problémák merülnek fel.

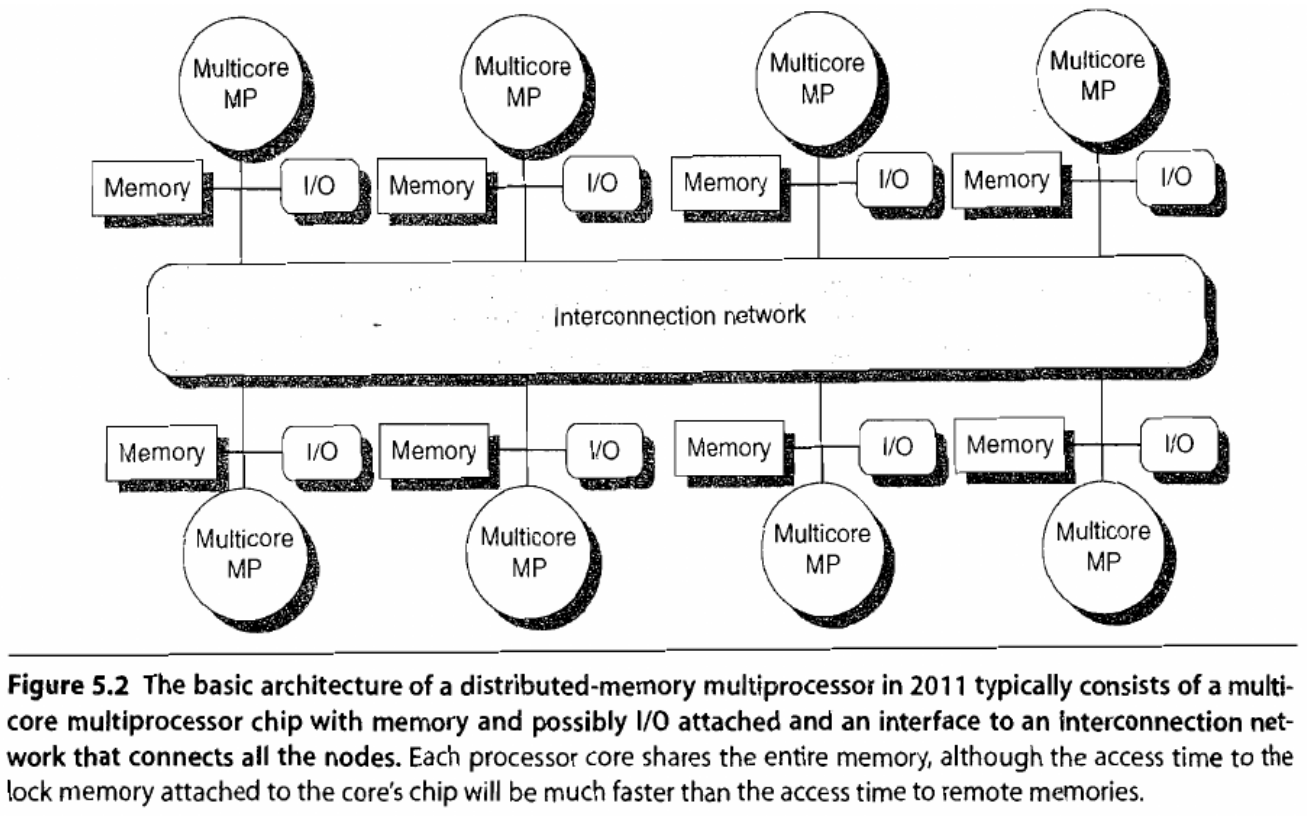


Figure 5.2 The basic architecture of a distributed-memory multiprocessor in 2011 typically consists of a multicore multiprocessor chip with memory and possibly I/O attached and an interface to an interconnection network that connects all the nodes. Each processor core shares the entire memory, although the access time to the lock memory attached to the core's chip will be much faster than the access time to remote memories.

Problémák a párhuzamos számításokkal

Leginkább kétféle probléma szokott nehézséget okozni:

1. Korlátolt párhuzamosság a programban, melyere az algoritmus módoítása nyújthat megoldást:
   - hatékonyabb párhuzamos futás
   - processzorok kihasználásának növelése
2. Kommunikáció magas költsége, mely csökkentető az architektúra vagy a program módosításával:
   - caching (prefetching) shared data
   - adatszerkezetek átalakítása úgy, hogy a memóriaelérések lokálisak legyenek

Cache coherence problem

SMP esetén is előfordulhat, hogy a memóriában tárolt globális állapot különbözik valamelyik vagy esetleg minkettő cache-ben tárolt lokális állapottól.

Time	Event	Cache contents for processor A	Cache contents for processor B	Memory contents for location X
0				1
1	Processor A reads X	1		1
2	Processor B reads X	1	1	1
3	Processor A stores 0 into X	0	1	0

Figure 5.3 The cache coherence problem for a single memory location (X), read and written by two processors (A and B). We initially assume that neither cache contains the variable and that X has the value 1. We also assume a write-through cache; a write-back cache adds some additional but similar complications. After the value of X has been written by A, A's cache and the memory both contain the new value, but B's cache does not, and if B reads the value of X it will receive 1!

Koherens memória

A koherens memóriához koherencia (mit add vissza egy olvasás) és konzisztenica (egy kiírt érték mikor olvasható) is szükséges.

Koherenciához:

1. Ha egy processzor olvas, majd ír egy X értéket, akkor a P ismételt olvasására az írt értéket kell visszakapnia, feltéve hogy más processzor nem írta X értékét.
2. Ha egy processzor olvassa X értékét, egy másik processzor pedig írja, akkor az olvasónak az írt értékkel kell visszatérnie, ha az olvasás és az írás között elég idő eltelt és közben újabb írás nem történt. (Nem ,,ragadhat be" egy régi érték folyamatos olvasásnál.)

Koherens memória

3. Az írások sorrendben történnek, azaz ha egy értéket két processzor egymás után ír, akkor egyik processzor sem olvashatja az utóbbi írás eredményét egy olyan olvasás előtt, amely az első írás értékét adja vissza.

Konzisztenciához: később, most feltesszük, hogy egy írás akkor fejeződik be, ha minden processzor látja az eredményt és az írások sorrendje nem változik különböző helyek írása esetén sem.

Koherencia megőrzése

Migration: egy megosztott adat transparent mozgatása lokális helyre. (Az érintett elem elérési idejének javítása, közös memória sávszélesség terhelésének csökkentése.)

Replication: szimultán olvasott értékek másolása lokális helyre. (Késleltetés csökkentése.)

Ezen elveket és a koherenciát megvalósító protokollok lehetnek

- *Directory based* Az egyes közös memóriablokkok állapota egy helyen (SMP esetén egyszerű).
- *Snooping* A lokális adatot tároló egység figyeli (pl. adatsín) a tárolt blokk állapotát.

Snooping coherence protocols

- *write update/broadcast* Az írás után mindenhova eljut az új érték (és közben megeszi a sávszélességet).
- *write invalidate* Az írás után invaliddá teszi a többi példányt, de írás nem történik a következő olvasásig.

Processor activity	Bus activity	Contents of processor A's cache	Contents of processor B's cache	Contents of memory location X
				0
Processor A reads X	Cache miss for X	0		0
Processor B reads X	Cache miss for X	0	0	0
Processor A writes a 1 to X	Invalidation for X	1		0
Processor B reads X	Cache miss for X	1	1	1

Figure 5.4 An example of an invalidation protocol working on a snooping bus for a single cache block (X) with write-back caches. We assume that neither cache initially holds X and that the value of X in memory is 0. The processor and memory contents show the value after the processor and bus activity have both completed. A blank indicates no activity or no copy cached. When the second miss by B occurs, processor A responds with the value canceling the response from memory. In addition, both the contents of B's cache and the memory contents of X are updated. This update of memory, which occurs when a block becomes shared, simplifies the protocol, but it is possible to track the ownership and force the write-back only if the block is replaced. This requires the introduction of an additional state called "owner," which indicates that a block may be shared, but the owning processor is responsible for updating any other processors and memory when it changes the block or replaces it. If a multicore uses a shared cache (e.g., L3), then all memory is seen through the shared cache; L3 acts like the memory in this example, and coherency must be handled for the private L1 and L2 for each core. It is this observation that led some designers to opt for a directory protocol within the multicore. To make this work the L3 cache must be inclusive (see page 397).

Snooping – Write Invalidate

A médium, amelyen keresztül az invalidáció továbbításra kerül az adatsín a megfelelő memóriaegységek között. Az invalidációhoz egy adatsín elérés kell, melyet az összes többi processzor figyel és ha az egy ottani lokális adathoz tartozik, akkor az invalid lesz.

A médium (adatsín) elérése szekvenciálisnak kell lenni a koherencia szerint (az elérési sorrend eldöntése nem feladat egyelőre).

Cache miss esetén az adat elérése (és annak nehézsége) függ az írás módjától:

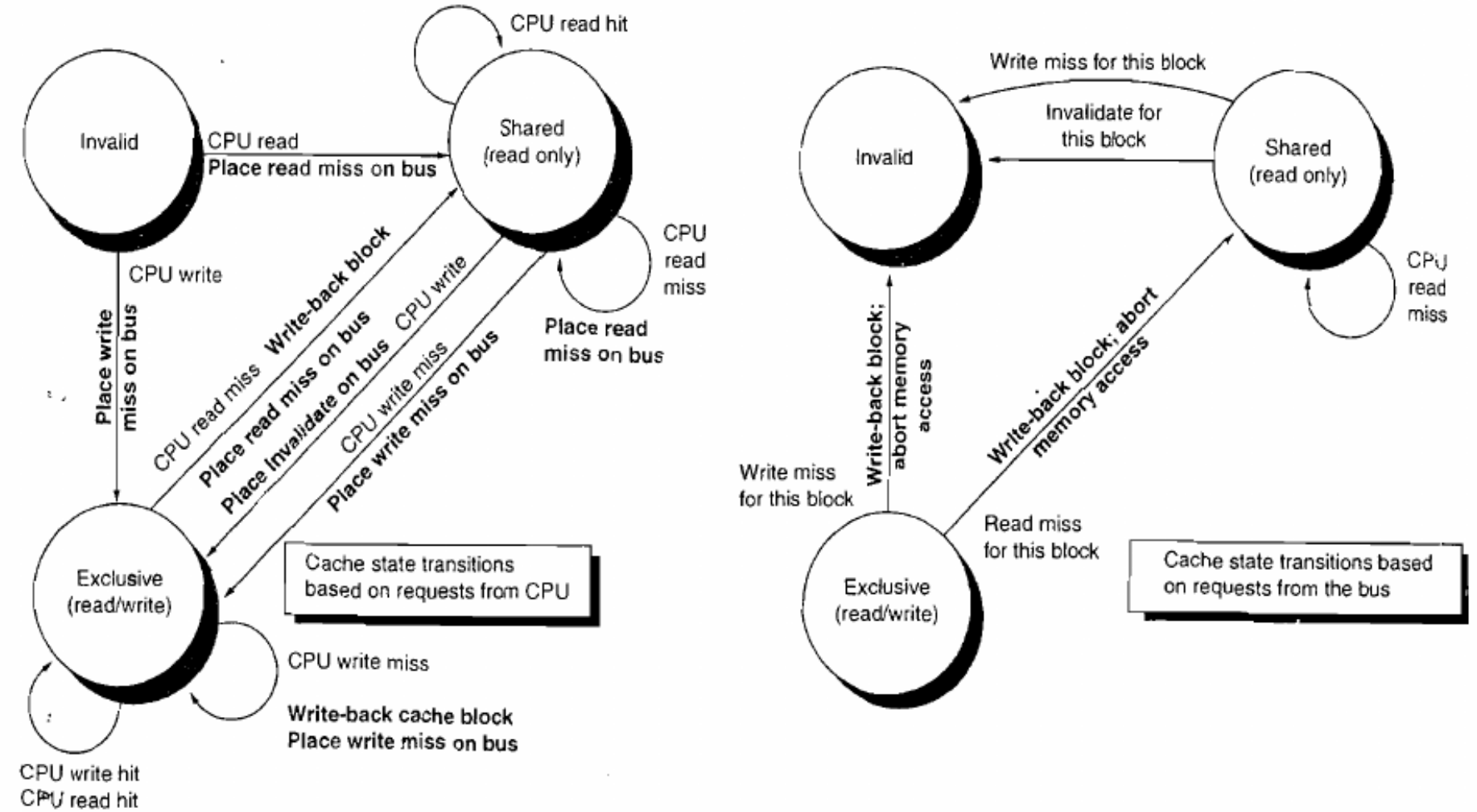
- write-through: memóriában
- write-through with write buffers: memóriában lesz vagy a buffer elérhető
- write-back: az adat egy másik processzor privát területén is lehet (shared bit alkalmazása a valid bit mellett shared/modified/invalid)

Példa: Snooping + write-back, write invalidate

Request	Source	State of addressed cache block	Type of cache action	Function and explanation
Read hit	Processor	Shared or modified	Normal hit	Read data in local cache.
Read miss	Processor	Invalid	Normal miss	Place read miss on bus.
Read miss	Processor	Shared	Replacement	Address conflict miss: place read miss on bus.
Read miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place read miss on bus.
Write hit	Processor	Modified	Normal hit	Write data in local cache.
Write hit	Processor	Shared	Coherence	Place invalidate on bus. These operations are often called upgrade or *ownership* misses, since they do not fetch the data but only change the state.
Write miss	Processor	Invalid	Normal miss	Place write miss on bus.
Write miss	Processor	Shared	Replacement	Address conflict miss: place write miss on bus.
Write miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place write miss on bus.
Read miss	Bus	Shared	No action	Allow shared cache or memory to service read miss.
Read miss	Bus	Modified	Coherence	Attempt to share data: place cache block on bus and change state to shared.
Invalidate	Bus	Shared	Coherence	Attempt to write shared block; invalidate the block.
Write miss	Bus	Shared	Coherence	Attempt to write shared block; invalidate the cache block.
Write miss	Bus	Modified	Coherence	Attempt to write block that is exclusive elsewhere; write-back the cache block and make its state invalid in the local cache.

Figure 5.5 The cache coherence mechanism receives requests from both the core's processor and the shared bus and responds to these based on the type of request, whether it hits or misses in the local cache, and the state of the local cache block specified in the request. The fourth column describes the type of cache action as normal hit or miss (the same as a uniprocessor cache would see), replacement (a uniprocessor cache replacement miss), or coherence (required to maintain cache coherence); a normal or replacement action may cause a coherence action depending on the state of the block in other caches. For read, misses, write misses, or invalidates snooped from the bus, an action is required *only* if the read or write addresses match a block in the local cache and the block is valid.

Példa: Snooping + write-back, write invalidate



Probléma: az átmeneteknek atominak kellene lennie (deadlock veszély)

MSI (Modify, Share, Invalid) protokoll kiterjesztése + problémák

- *MESI* Exclusive állapot hozzáadása (segíti az írást, ha csak egy szál dolgozik az adott elemen) Intel i7
- *MESFI* (Intel i7) Forward állapot hozzáadása MESI-hez
- *MOESI* (AMD Opteron) Owned állapot hozzáadása MESI-hez (az író egy állapota, jelzi saját magának, hogy a memóriában lévő példány nem érvényes)

Egyre több core egyre nagyobb memória sávszélesség igénnyel nagy terhet jelent a memória elosztott részére és annak interfészére.

Megoldások: snooping és directory együttes használata (i7, Xeon); több adatsín alkalmazása; interconnection networks cachek között – bankok; közvetlen point-to-point a közös memória és cache között, explicit acknowledgements (AMD Opteron)

Interconnection network használata

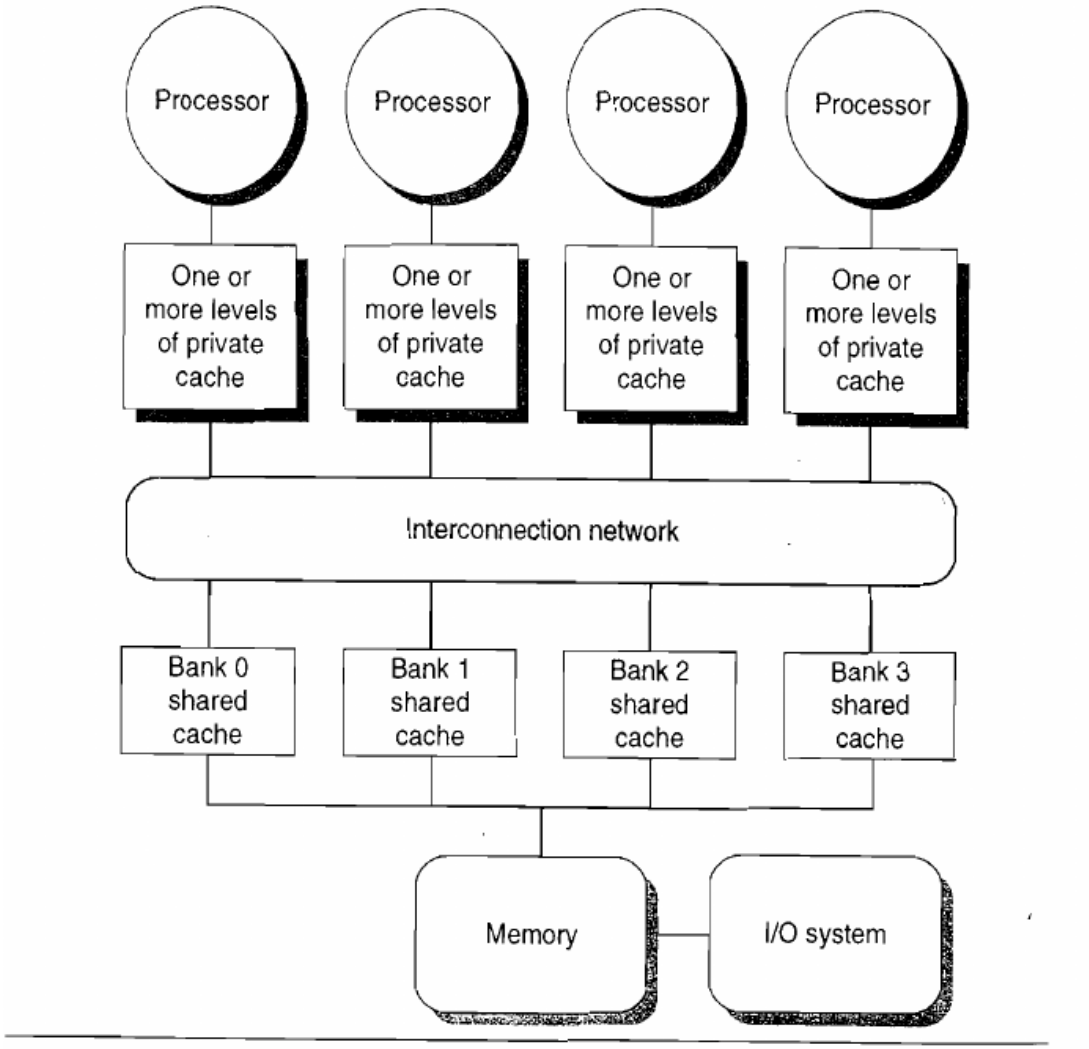


Figure 5.8 A multicore single-chip multiprocessor with uniform memory access through a banked shared cache and using an interconnection network rather than a bus.

Snooping performance (Commercial workload)

Coherence misses:

- *true sharing misses*: ownership megszerzése, read modified
- *false sharing*: a blokkon belül egy másik szó miatti miss

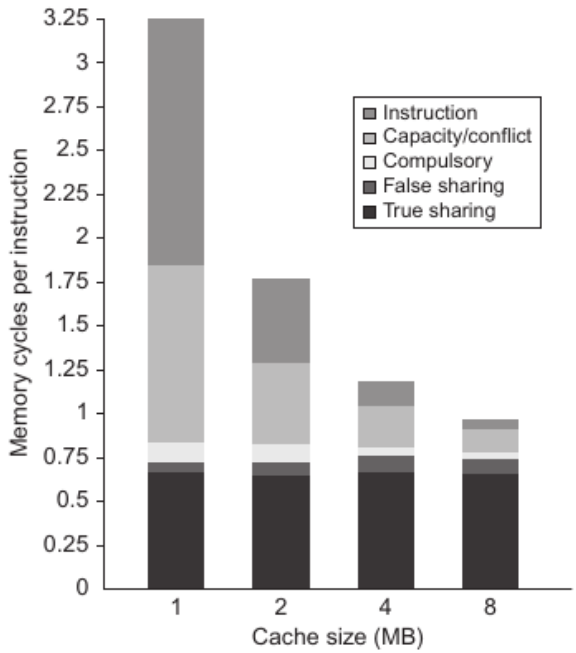


Figure 5.11 The contributing causes of memory access cycle shift as the cache size is increased. The L3 cache is simulated as two-way set associative.

Snooping performance (Commercial workload)

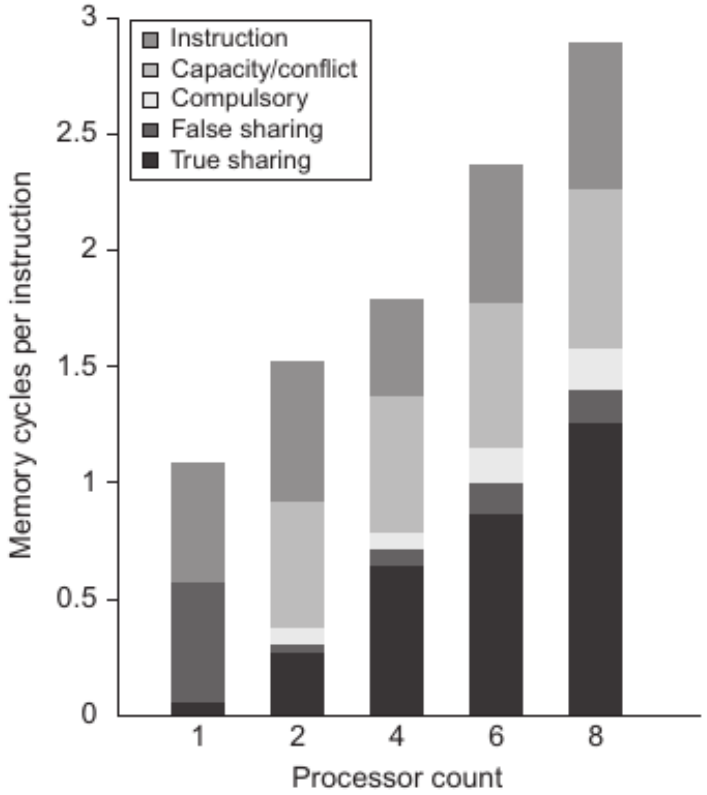


Figure 5.12 The contribution to memory access cycles increases as processor count increases primarily because of increased true sharing. The compulsory misses slightly increase because each processor must now handle more compulsory misses.

Snooping performance (Commercial workload)

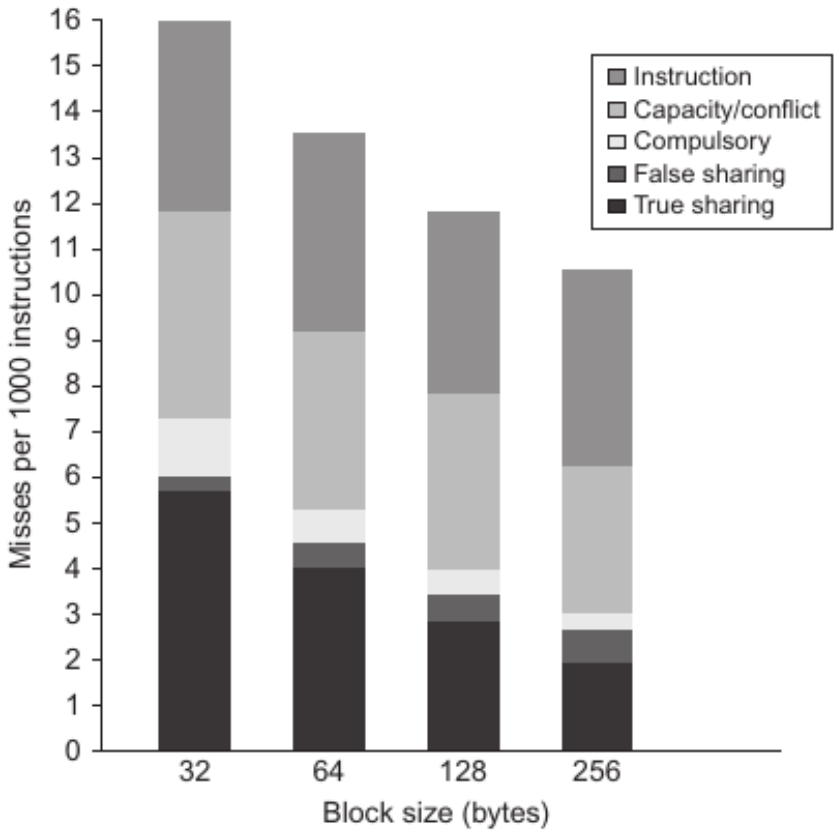


Figure 5.13 The number of misses per 1000 instructions drops steadily as the block size of the L3 cache is increased, making a good case for an L3 block size of at least 128 bytes. The L3 cache is 2 MiB, two-way set associative.

True sharing feleződik (lokalitás), False sharing duplázódik

Directory protokoll

Tábla tartalmazza minden blokkra, hogy az hol található (esetleg több helyen is) és milyen állapotban van. (i7)

A tábla maga megosztott kell legyen és midig tudni kell az érvényes bejegyzést (az egyes blokkokra) helyét.

Minden blokk esetén egy a magok számával megegyező bitvektor tárolása, így néhány száz processzor jól kezelhető (ésszerű blokkmérettel).

Directory protokoll

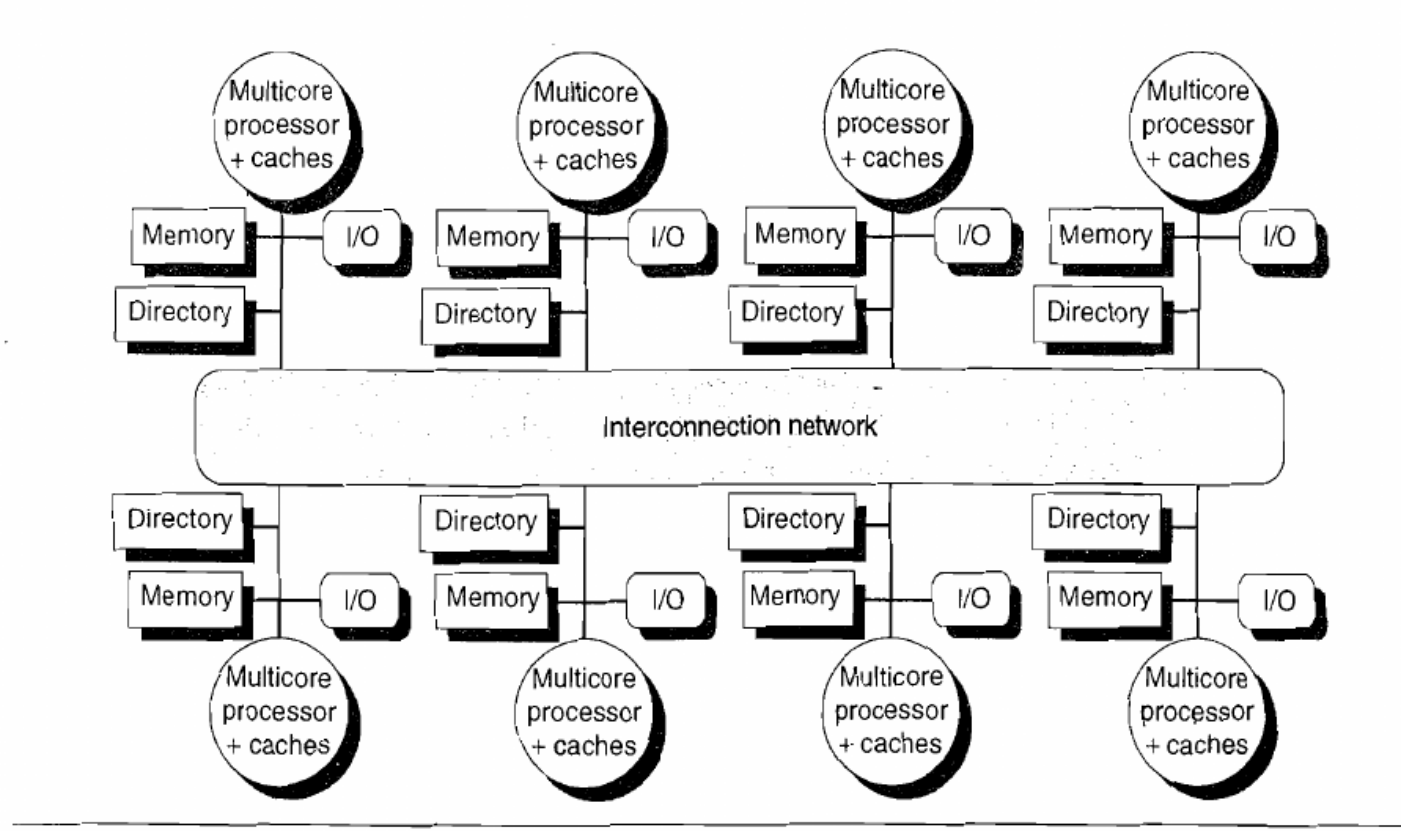


Figure 5.20 A directory is added to each node to implement cache coherence in a distributed-memory multiprocessor. In this case, a node is shown as a single multicore chip, and the directory information for the associated memory may reside either on or off the multicore. Each directory is responsible for tracking the caches that share the memory addresses of the portion of memory in the node. The coherence mechanism would handle both the maintenance of the directory information and any coherence actions needed within the multicore node.

Directory protokoll

Message type	Source	Destination	Message contents	Function of this message
Read miss	Local cache	Home directory	P, A	Node P has a read miss at address A; request data and make P a read sharer.
Write miss	Local cache	Home directory	P, A	Node P has a write miss at address A; request data and make P the exclusive owner.
Invalidate	Local cache	Home directory	A	Request to send invalidates to all remote caches that are caching the block at address A.
Invalidate	Home directory	Remote cache	A	Invalidate a shared copy of data at address A.
Fetch	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; change the state of A in the remote cache to shared.
Fetch/invalidate	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; invalidate the block in the cache.
Data value reply	Home directory	Local cache	D	Return a data value from the home memory.
Data write-back	Remote cache	Home directory	A, D	Write-back a data value for address A.

Figure 5.21 The possible messages sent among nodes to maintain coherence, along with the source and destination node, the contents (where P = requesting node number, A = requested address, and D = data contents), and the function of the message. The first three messages are requests sent by the local node to the home. The fourth through sixth messages are messages sent to a remote node by the home when the home needs the data to satisfy a read or write miss request. Data value replies are used to send a value from the home node back to the requesting node. Data value write-backs occur for two reasons: when a block is replaced in a cache and must be written back to its home memory, and also in reply to fetch or fetch/invalidate messages from the home. Writing back the data value whenever the block becomes shared simplifies the number of states in the protocol, since any dirty block must be exclusive and any shared block is always available in the home memory.

Directory protokoll

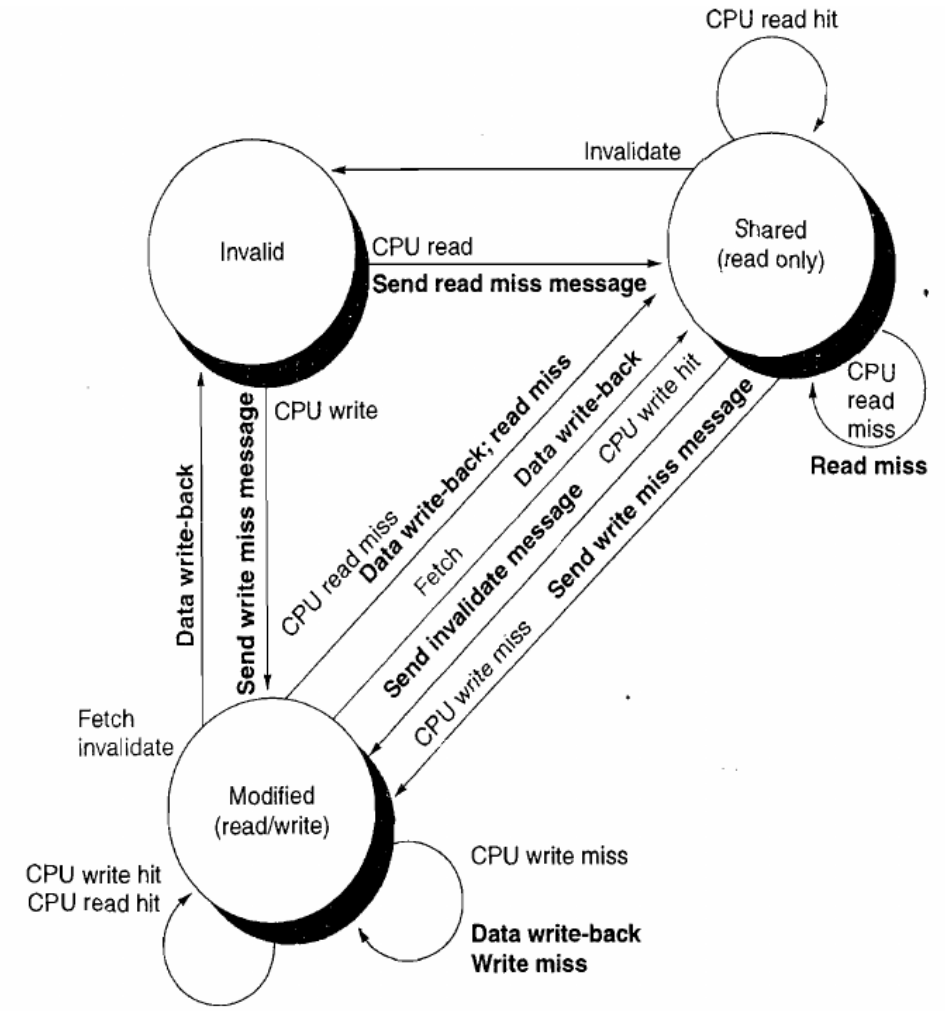


Figure 5.22 State transition diagram for an individual cache block in a directory-based system. Requests by the local processor are shown in black, and those from the home directory are shown in gray. The states are identical to those in the snooping case, and the transactions are very similar, with explicit invalidate and write-back

Nagy hatékonyságú számítások . . .
12. előadás – Thread Level Parallelism

Nagy Ádám
nagyadam888@gmail.com

Eötvös Loránd Tudományegyetem
Informatikai Kar

2022. december 12.

Szinkronizáció – Alapok

Általában szofter (system tools) valósítja meg egyszerű hardveres (atomi) utasítások alapján:

- atomic exchange (reg $\leftrightarrow$ mem, lock: 0 $\leftrightarrow$ 1 , unlock: 1 $\leftrightarrow$ 0)
- test-and-set (lock: `a = (a == 0) ? 1`, unlock: `a = 0`)
- fetch-and-increment (lock: `a == 0`, unlock: `a = 0`)
- fence
- load reserved (load locked `M[rs1]` $\rightarrow$ `rd, R[rs1] = 1`),
  store conditional
  (`rd = (rs2` $\rightarrow$ `M[rs1]` $\wedge$ `R[rs1] = 1) ? 1 : 0`)
  minden írás `M[rs1]`-en `R[rs1] = 0`

Atomic xchg (`x1` $\leftrightarrow$ `x4`):

```
try:    mov   x3,x4      ;mov exchange value
        lr    x2,x1      ;load reserved from
        sc    x3,0(x1)   ;store conditional
        bnez  x3,try     ;branch store fails
        mov   x4,x2      ;put load value in x4
```

Atomic fetch-and-increment:

```
try:    lr    x2,x1      ;load reserved 0(x1)
        addi  x3,x2,1    ;increment
        sc    x3,0(x1)   ;store conditional
        bnez  x3,try     ;branch store fails
```

Megjegyzések:

- most nem cél nagy mértekű konkurenica kezelése
- alacsony szintű utasítások általában nem (átlagos) programozók számára vannak
- bonyolult megoldások $\Rightarrow$ drága hw

Spin locks

```
        addi  x2,R0,#1
lockit: EXCH  x2,0(x1)     ;atomic exchange
        bnez  x2,lockit    ;already locked?
```

"spinning" lehet cached lokális, ha a koherencia kezelve van

```
lockit: ld    x2,0(x1)   ;load of lock
        bnez  x2,lockit  ;not available-spin
        addi  x2,R0,#1   ;load locked value
        EXCH  x2,0(x1)   ;swap
        bnez  x2,lockit  ;branch if lock wasn't 0
```

`lr` nem okoz forgalmat a közös bus-on

```
lockit: lr    x2,0(x1)   ;load reserved
        bnez  x2,lockit  ;not available-spin
        addi  x2,R0,#1   ;locked value
        sc    x2,0(x1)   ;store
        bnez  x2,lockit  ;branch if store fails
```

Step	P0	P1	P2	Coherence state of lock at end of step	Bus/directory activity
1	Has lock	Begins spin, testing if lock = 0	Begins spin, testing if lock = 0	Shared	Cache misses for P1 and P2 satisfied in either order. Lock state becomes shared.
2	Set lock to 0	(Invalidate received)	(Invalidate received)	Exclusive (P0)	Write invalidate of lock variable from P0.
3		Cache miss	Cache miss	Shared	Bus/directory services P2 cache miss; write-back from P0; state shared.
4		(Waits while bus/ directory busy)	Lock = 0 test succeeds	Shared	Cache miss for P2 satisfied.
5		Lock = 0	Executes swap, gets cache miss	Shared	Cache miss for P1 satisfied.
6		Executes swap, gets cache miss	Completes swap: returns 0 and sets lock = 1	Exclusive (P2)	Bus/directory services P2 cache miss; generates invalidate; lock is exclusive.
7		Swap completes and returns 1, and sets lock = 1	Enter critical section	Exclusive (P1)	Bus/directory services P1 cache miss; sends invalidate and generates write-back from P2.
8		Spins, testing if lock = 0			None

Figure 5.22 Cache coherence steps and bus traffic for three processors, P0, P1, and P2. This figure assumes write invalidate coherence. P0 starts with the lock (step 1), and the value of the lock is 1 (i.e., locked); it is initially exclusive and owned by P0 before step 1 begins. P0 exits and unlocks the lock (step 2). P1 and P2 race to see which reads the unlocked value during the swap (steps 3–5). P2 wins and enters the critical section (steps 6 and 7), while P1's attempt fails, so it starts spin waiting (steps 7 and 8). In a real system, these events will take many more than 8 clock ticks because acquiring the bus and replying to misses take much longer. Once step 8 is reached, the process can repeat with P2, eventually getting exclusive access and setting the lock to 0.

Konzisztencia

Eddig feltettük a teljes konzisztenciát és a koherenciát vizsgaltuk, de valójában soha nincs teljes konzisztenciánk.

Ha nincs teljes konzisztencia, akkor mi történik?

```
P1:     A = 0;          P2:     B = 0;
        .....                   .....
        A = 1;                  B = 1;
L1:     if (B == 0)...  L2:     if (A == 0)...
```

Szekvenciális konzisztencia: mintha egy processzoron lennének az utasítások végrehajtva tetszoőleges sorrendben.

Relaxált konzisztencia

Szekvenciális konzisztencia problémás lehet, ha sok processzor van, vagy ha a performancia nagyban függ a cacheléstől (write buffer).

- sequential consistency + latency-hiding
- relaxing cosistency

Cél: minél több data-race-free

Valamelyik $R \to W$, $R \to R$, $W \to W$ sorrendjét "elengedjük".

Megj.: a magasabb szintű párhuzamosságot támogató lib-ek elrejtik.

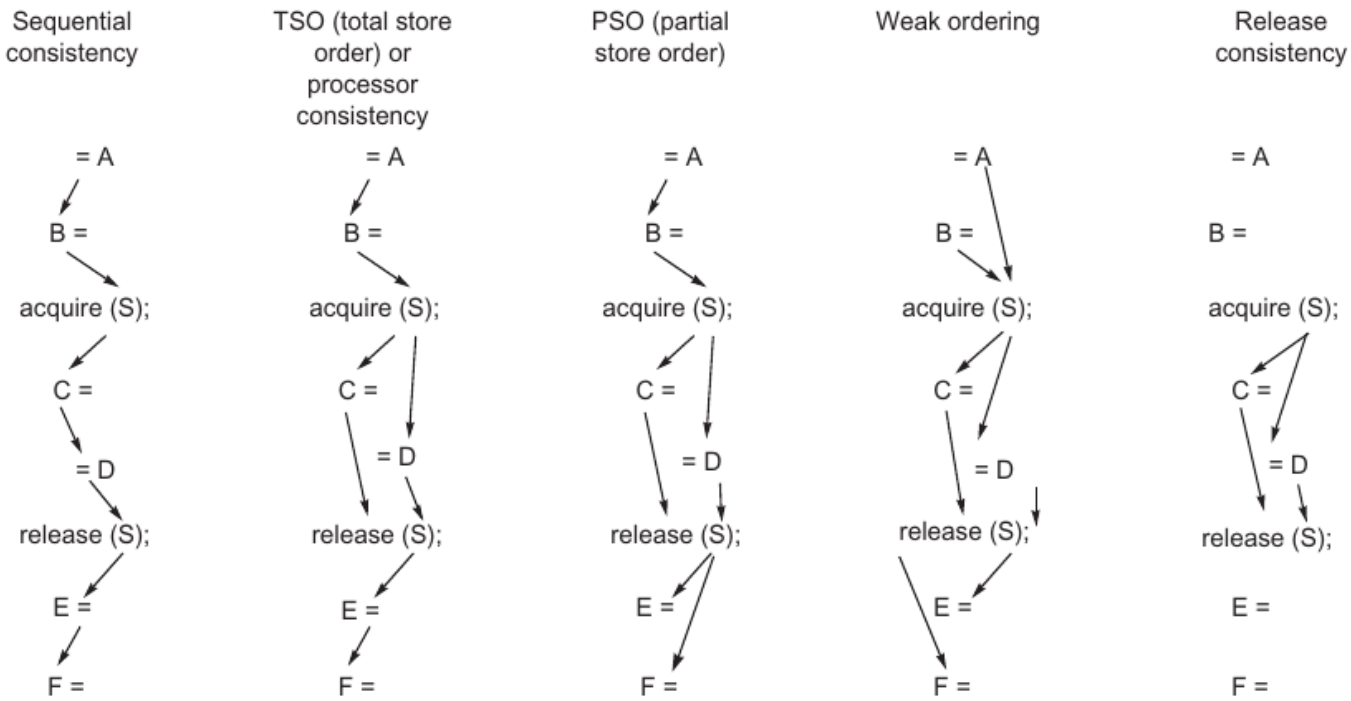


Figure 5.24 These examples of the five consistency models discussed in this section show the reduction in the number of orders imposed as the models become more relaxed. Only the minimum orders are shown with arrows. Orders implied by transitivity, such as the write of C before the release of S in the sequential consistency model or the acquire before the release in weak ordering or release consistency, are not shown.

Model	Used in	Ordinary orderings	Synchronization orderings
Sequential consistency	Most machines as an optional mode	$R \rightarrow R$, $R \rightarrow W$, $W \rightarrow R$, $W \rightarrow W$	$S \rightarrow W$, $S \rightarrow R$, $R \rightarrow S$, $W \rightarrow S$, $S \rightarrow S$
Total store order or processor consistency	IBMS/370, DEC VAX, SPARC	$R \rightarrow R$, $R \rightarrow W$, $W \rightarrow W$	$S \rightarrow W$, $S \rightarrow R$, $R \rightarrow S$, $W \rightarrow S$, $S \rightarrow S$
Partial store order	SPARC	$R \rightarrow R$, $R \rightarrow W$	$S \rightarrow W$, $S \rightarrow R$, $R \rightarrow S$, $W \rightarrow S$, $S \rightarrow S$
Weak ordering	PowerPC		$S \rightarrow W$, $S \rightarrow R$, $R \rightarrow S$, $W \rightarrow S$, $S \rightarrow S$
Release consistency	MIPS, RISC V, Armv8, C, and C++ specifications		$S_A \rightarrow W$, $S_A \rightarrow R$, $R \rightarrow S_R$, $W \rightarrow S_R$, $S_A \rightarrow S_A$, $S_A \rightarrow S_R$, $S_R \rightarrow S_A$, $S_R \rightarrow S_R$

Figure 5.23 The orderings imposed by various consistency models are shown for both ordinary accesses and synchronization accesses. The models grow from most restrictive (sequential consistency) to least restrictive (release consistency), allowing increased flexibility in the implementation. The weaker models rely on fences created by synchronization operations, as opposed to an implicit fence at every memory operation. S_A and S_R stand for acquire and release operations, respectively, and are needed to define release consistency. If we were to use the notation S_A and S_R for each S consistently, each ordering with one S would become two orderings (e.g., $S \rightarrow W$ becomes $S_A \rightarrow W$, $S_R \rightarrow W$), and each $S \rightarrow S$ would become the four orderings shown in the last line of the bottom-right table entry.

MegjegyzéseK

- Compiler problémák
- Strict consistency + spekulatív végrehajtás
- Inclusion (susbset property), different block sizes

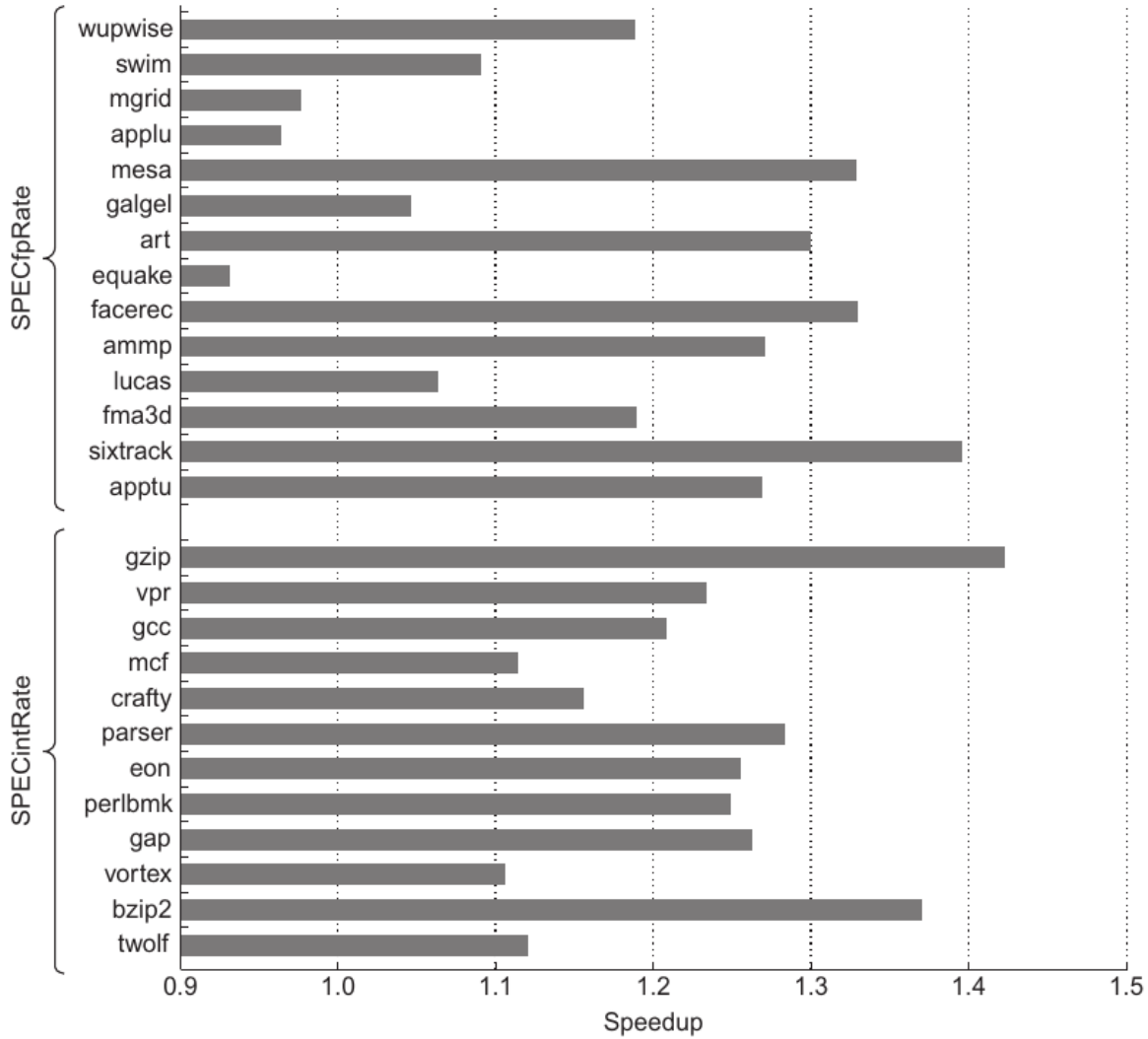
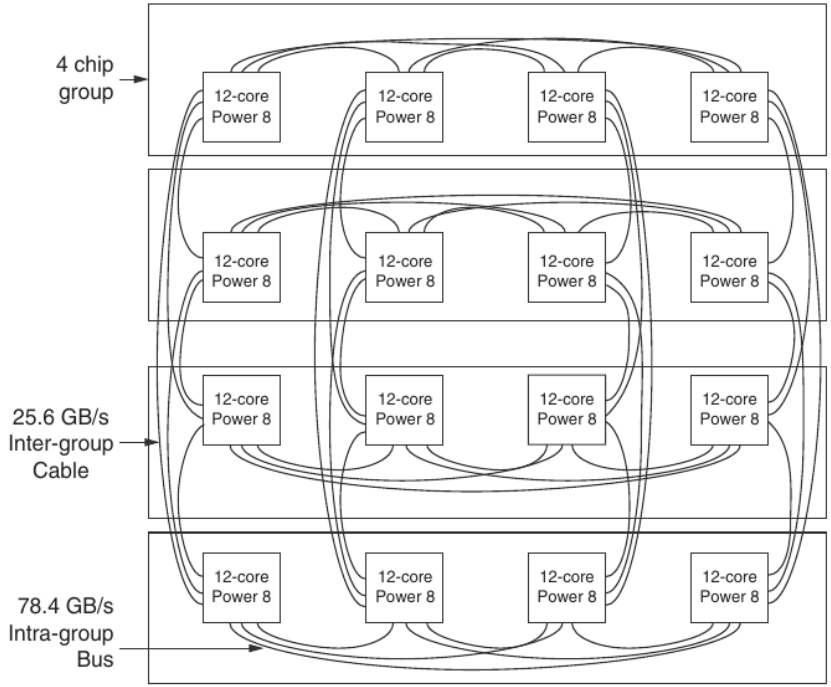


Figure 5.25 A comparison of SMT and single-thread (ST) performance on the 8-processor IBM eServer p5 575 using SPECfpRate (top half) and SPECintRate (bottom half) as benchmarks. Note that the *x*-axis starts at a speedup of 0.9, a performance loss. Only one processor in each Power5 core is active, which should slightly improve the results from SMT by decreasing destructive interference in the memory system. The SMT results are obtained by creating 16 user threads, whereas the ST results use only eight threads; with only one thread per processor, the Power5 is switched to single-threaded mode by the OS. These results were collected by John McCalpin at IBM. As we can see from the data, the standard deviation of the results for the SPECfpRate is higher than for SPECintRate (0.13 versus 0.07), indicating that the SMT improvement for FP programs is likely to vary widely.

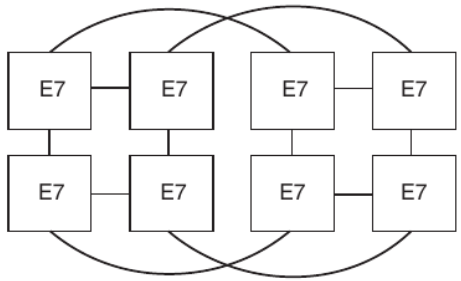
Multicore

Feature	IBM Power8	Intel Xeon E7	Fujitsu SPARC64 X+
Cores/chip	4, 6, 8, 10, 12	4, 8, 10, 12, 22, 24	16
Multithreading	SMT	SMT	SMT
Threads/core	8	2	2
Clock rate	3.1–3.8 GHz	2.1–3.2 GHz	3.5 GHz
L1 I cache	32 KB per core	32 KB per core	64 KB per core
L1 D cache	64 KB per core	32 KB per core	64 KB per core
L2 cache	512 KB per core	256 KB per core	24 MiB shared
L3 cache	L3: 32–96 MiB: 8 MiB per core (using eDRAM); shared with nonuniform access time	10–60 MiB @ 2.5 MiB per core; shared, with larger core counts	None
Inclusion	Yes, L3 superset	Yes, L3 superset	Yes
Multicore coherence protocol	Extended MESI with behavioral and locality hints (13-states)	MESIF: an extended form of MESI allowing direct transfers of clean blocks	MOESI
Multichip coherence implementation	Hybrid strategy with snooping and directory	Hybrid strategy with snooping and directory	Hybrid strategy with snooping and directory
Multiprocessor interconnect support	Can connect up to 16 processor chips with 1 or 2 hops to reach any processor	Up to 8 processor chips directly via Quickpath; larger system and directory support with additional logic	Crossbar interconnect chip, supports up to 64 processors; includes directory support
Processor chip range	1–16	2–32	1–64
Core count range	4–192	12–576	8–1024

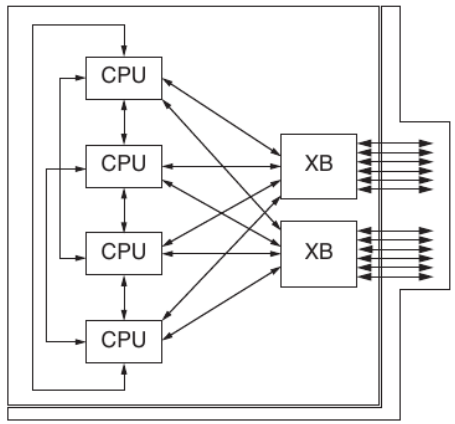
Figure 5.26 Summary of the characteristics of three recent high-end multicore processors (2015–2017 releases) designed for servers. The table shows the range of processor counts, clock rates, and cache sizes within each pro-



(A) Power8 system with up to 16 chips.



(B) Xeon E7 system showing with up to 8 chips.



(C) SPARC64 X+ with the 4-chip building block.

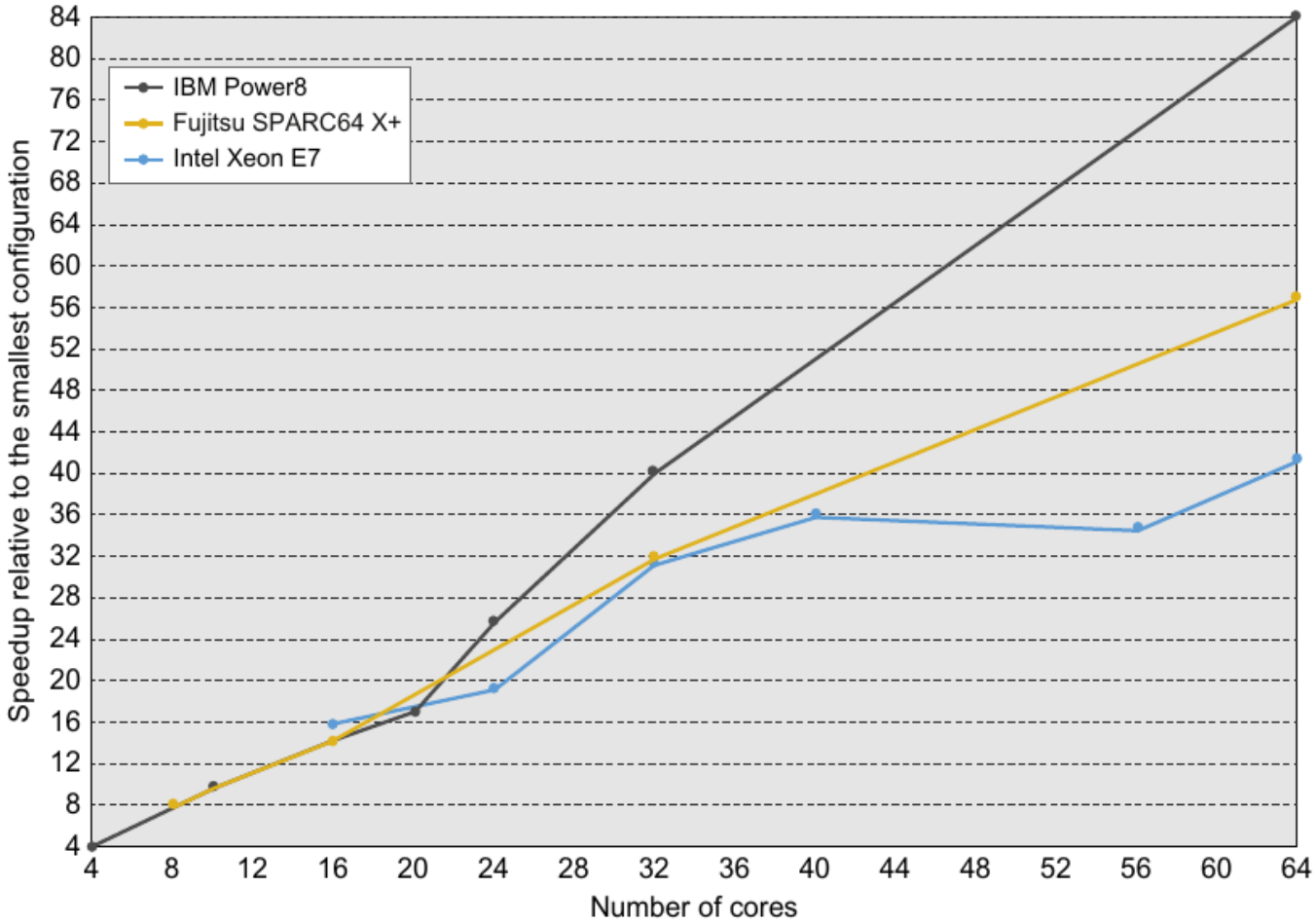


Figure 5.29 The performance scaling on the SPECintRate benchmarks for four multicore processors as the number of cores is increased to 64. Performance for each processor is plotted relative to the smallest configuration and assuming that configuration had perfect speedup. Although this chart shows how a given multiprocessor scales with additional cores, it does not supply any data about performance among processors. There are differences in the clock rates, even within a given processor family. These are generally swamped by the core scaling effects, except for the Power8 that shows a clock range spread of 1.5× from the smallest configuration to the 64 core configuration.

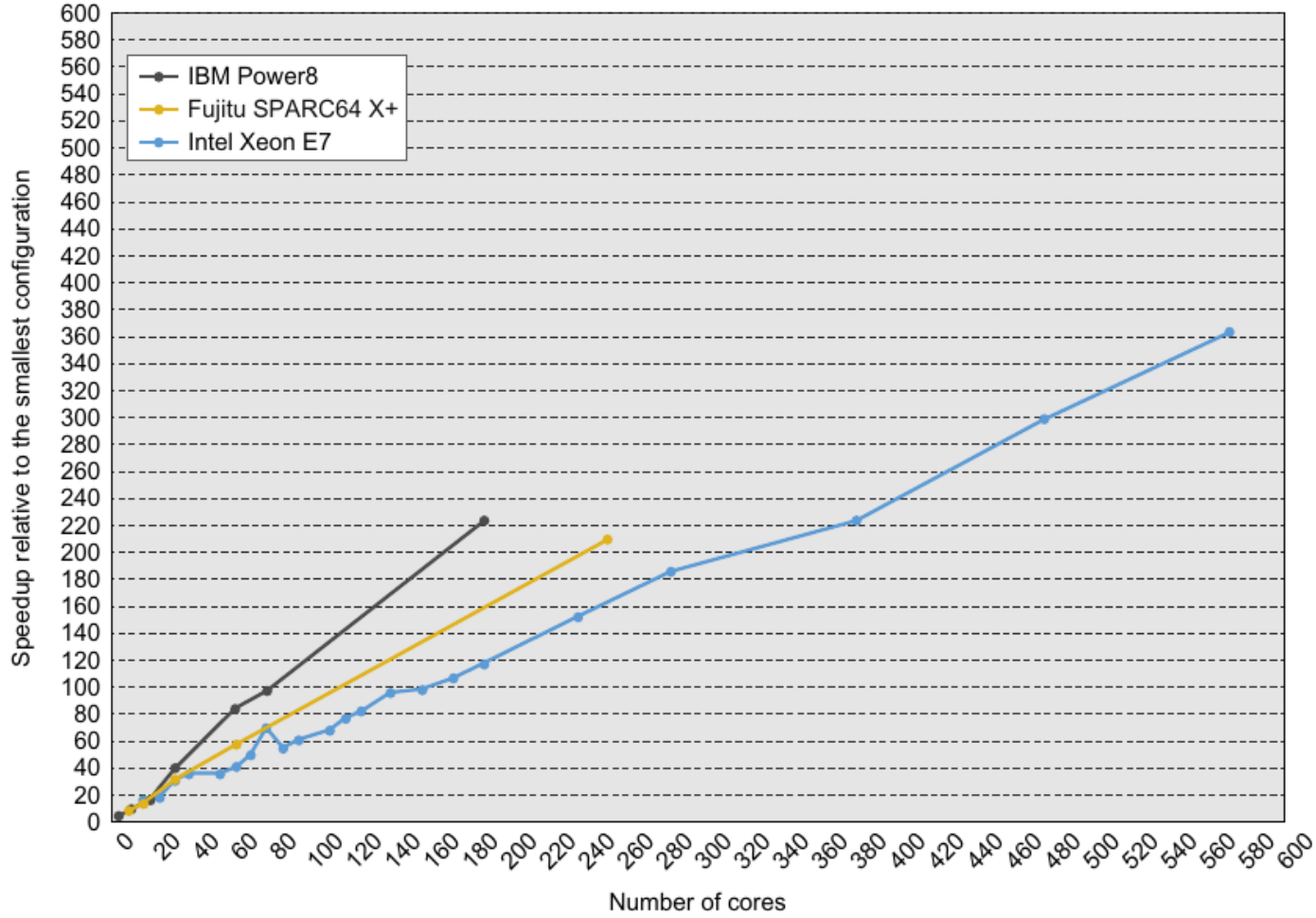


Figure 5.30 The scaling of relative performance for multiprocessor multicore. As before, performance is shown relative to the smallest available system. The Xeon result at 80 cores is the same L3 effect that showed up for smaller configurations. All systems larger than 80 cores have between 2.5 and 3.8 MiB of L3 per core, and the 80-core, or smaller, systems have 6 MiB per core.

Workload type and scaling (Java, VM, Sci)

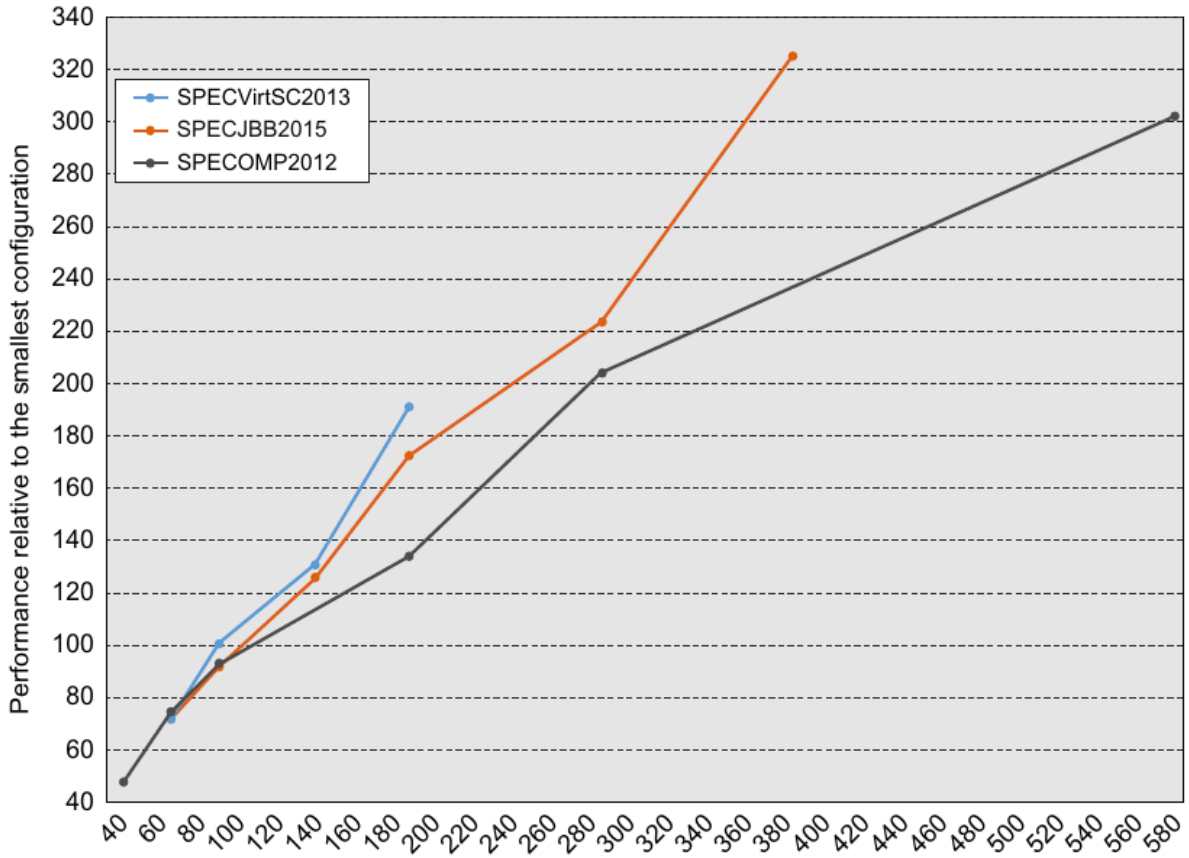


Figure 5.31 Scaling of performance on a range of Xeon E7 systems showing performance relative to the smallest benchmark configuration, and assuming that configuration gets perfect speedup (e.g., the smallest SPEWCOMP configuration is 30 cores and we assume a performance of 30 for that system). Only relative performance can be assessed from this data, and comparisons across the benchmarks have no relevance. Note the difference in the scale of the vertical and horizontal axes.

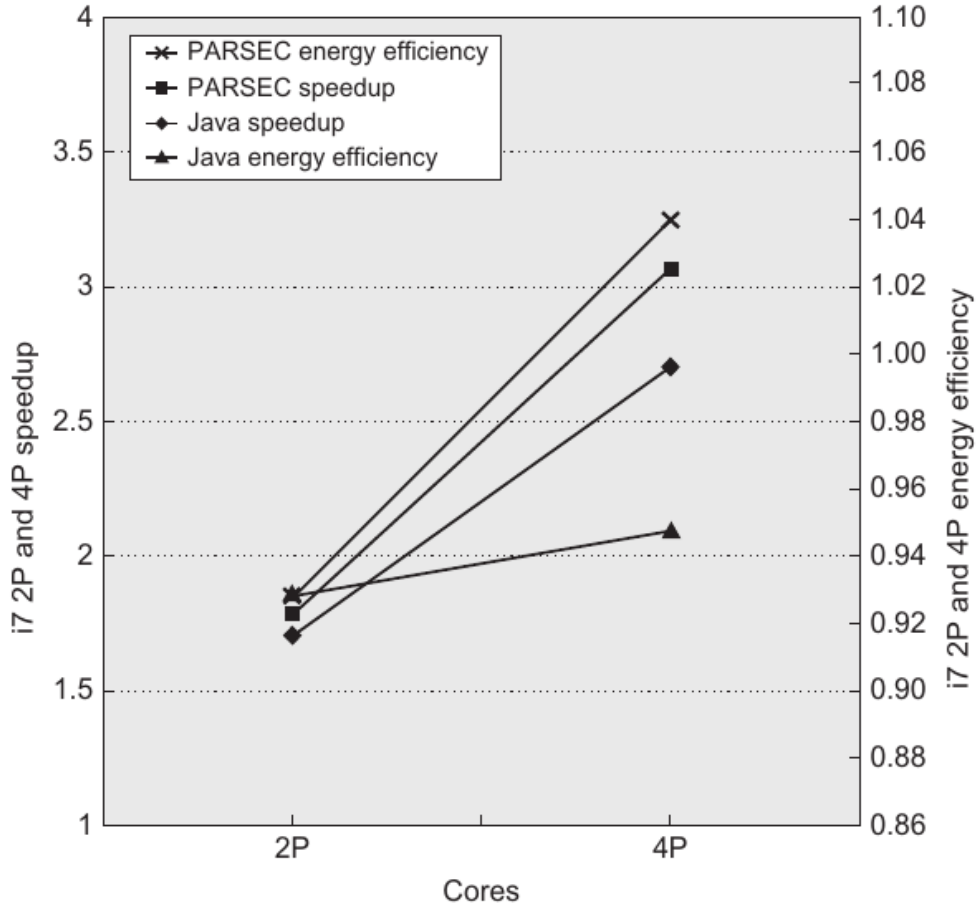


Figure 5.32 This chart shows the speedup and energy efficiency for two- and four-core executions of the parallel Java and PARSEC workloads without SMT. These data were collected by Esmaeilzadeh et al. (2011) using the same setup as described in Chapter 3. Turbo Boost is turned off. The speedup and energy efficiency are summarized using harmonic mean, implying a workload where the total time spent running each benchmark on 2 cores is equivalent.

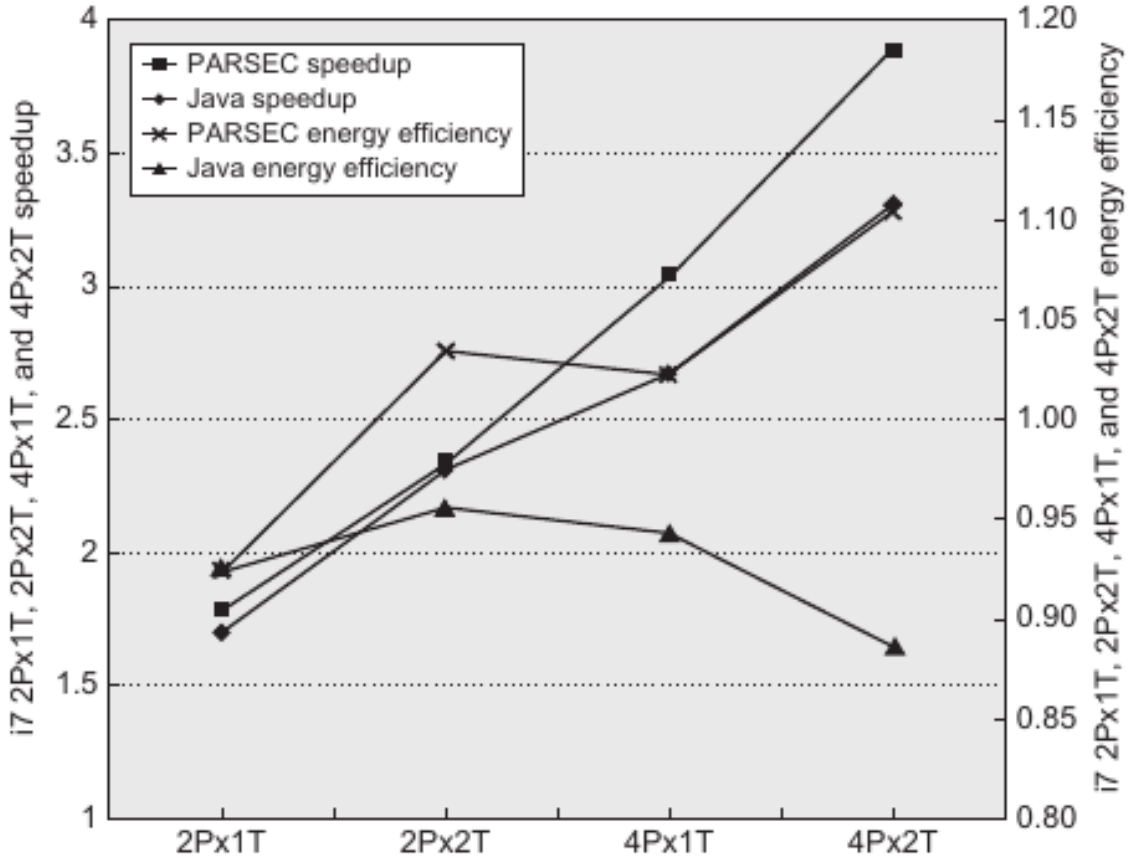


Figure 5.33 This chart shows the speedup for two- and four-core executions of the parallel Java and PARSEC workloads both with and without SMT. Remember that the preceding results vary in the number of threads from two to eight and reflect both architectural effects and application characteristics. Harmonic mean is used to summarize results, as discussed in the Figure 5.32 caption.

Csapdák és tévhivtek

- Performacia lineáris skálázást feltételezve
- Amdahl törvénye érvényét veszti párhuzamosság esetén
- Optimalizáció egy számítási egységre
- Lineáris gyorsulás $\Rightarrow$ költséghatékony

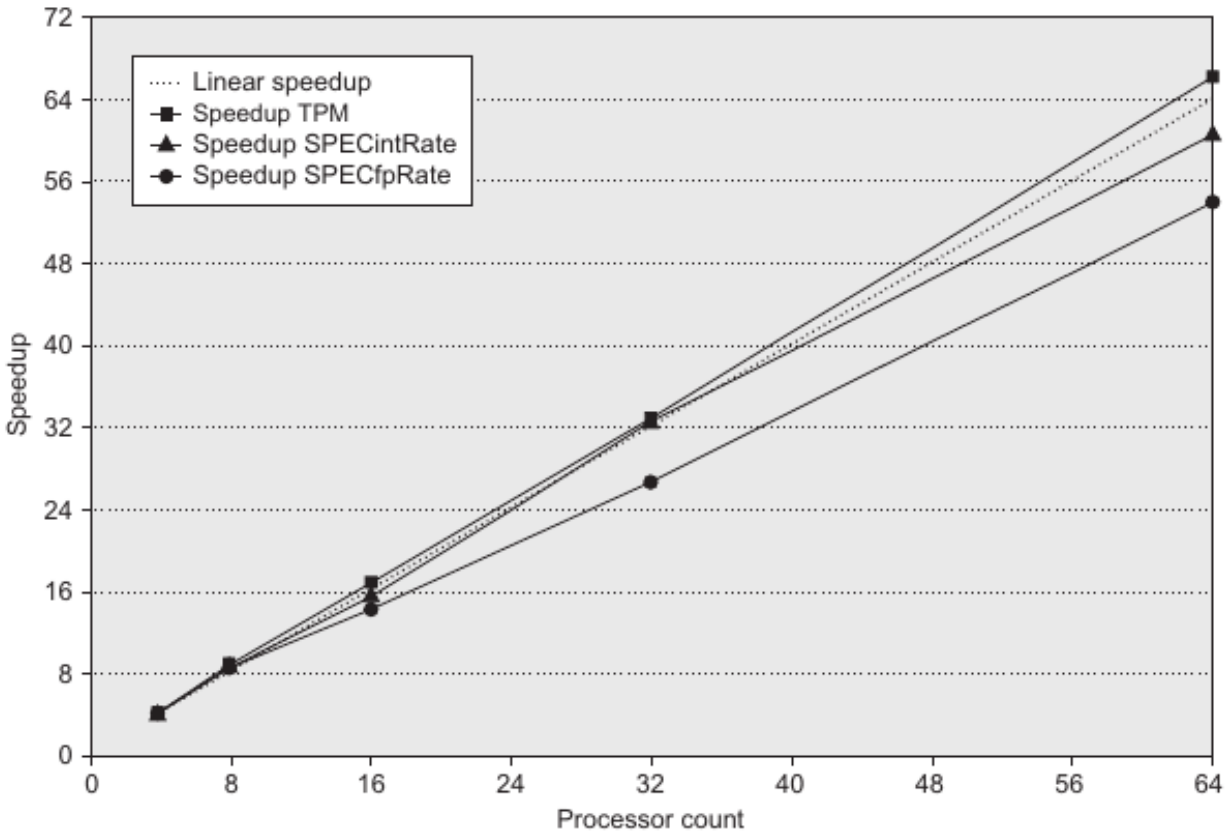


Figure 5.34 Speedup for three benchmarks on an IBM eServer p5 multiprocessor when configured with 4, 8, 16, 32, and 64 processors. The *dashed line* shows linear speedup.

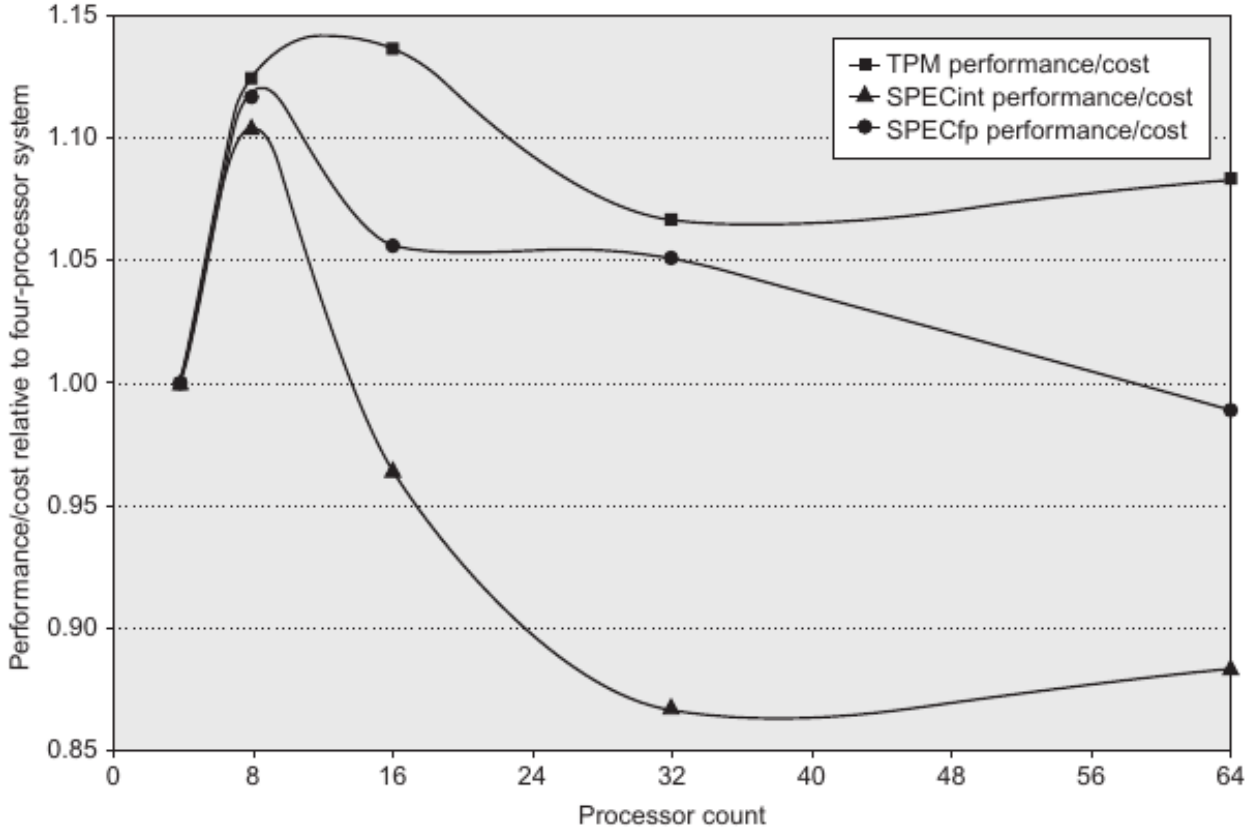


Figure 5.35 The performance/cost for IBM eServer p5 multiprocessors with 4–64 processors is shown relative to the 4-processor configuration. Any measurement above 1.0 indicates that the configuration is more cost-effective than the 4-processor system. The 8-processor configurations show an advantage for all three benchmarks, whereas two of the three benchmarks show a cost-performance advantage in the 16- and 32-processor configurations. For TPC-C, the configurations are those used in the official runs, which means that disk and memory scale nearly linearly with processor count, and a 64-processor machine is approximately twice as expensive as a 32-processor version. In contrast, the disk and memory are scaled more slowly (although still faster than necessary to achieve the best SPECRate at 64 processors). In particular, the disk configurations go from one drive for the 4-processor version to four drives (140 GB) for the 64-processor version. Memory is scaled from 8 GiB for the 4-processor system to 20 GiB for the 64-processor system.

Nagy hatékonyságú számítások . . .
13. előadás – Massively Parallel Processors

Nagy Ádám
nagyadam888@gmail.com

Eötvös Loránd Tudományegyetem
Informatikai Kar

2022. december 7.

MPP – Alapok

Problem: parallel commputational units require high amount of collaboration (true parallel programming)

Intense intertask communication

1. Communication
2. Specialization (specific application)
3. Synchronization (scale-it-up)
4. Monitoring performance
5. Interconnection network

Interprocessor communication

- More computational unit $\Rightarrow$ higher cost
- Complex architecture $\Leftarrow$ application related
- Metrics:
  1. Bandwith (+ architecture, occupancy)
  2. Latency
     - Sender/receiver overhead (communication, implementation)
     - Time of flight (fixed)
     - Transmission time (interconnection network))
  3. Commnuncation latency hiding (how to do, how to measure)
     Usually not trivial to overlap communication with calculation/communication

Shared memory vs Message passing

- Compatibility (standardized programming interface)
- Ease of programming (simplify compiler)
- Can focus on the important (performance) issues.
- Lower overhead for small items (memory mapping)
- Hardware controlled caching
  (reduceses latency and access time of shared data)

Shared memory vs Message passing

- Less complex hardware (no caching, coherence, etc.)
- Communication is explicit (can be seen when it happens)
- Less incorrect synchronization
- Sender initiated (better performance)
- Simpler fault isolation (when topology not dense)
- Only this possible for large enough systems

Applications – FFT

For $n \to \sqrt{n}$ lenght that minimize communication.

The roots of unity in a table precalcualted (distributed).

Steps:

1. Transform data matrix
2. 1D FFT on rows
3. Mul with root of unity
4. Transpose matrix
5. 1D FFT on rows
6. Transpose matrix

Communication (large amount) at transpose phase only.

Application – LU

Blocking algorithm with

- 2D tiling (grid of blocks)
- block size small enough to keep the cache miss rate low
- block size big enough to reduce time in less parallel parts
- multiplication performed by the owner of the destinition block
- 4D indices (global + local)

Application – Barnes, Ocean

Barnes-Hut

- n-body for galaxy evolution (points + gravity represented as a tree)
- octree (cube decomposition of space)
- tree depth calculation
- requires efficiently implemented shared-memory

Ocean

- simulates large-sclae flow in ocean
- red-black Gauss-Seidel multigrid, elliptical PDEs
- grid allocation dynamic according to error
- processors own part of the grid and share nearest-neighbour at time iterations

Exceptions

The ration of copmutation and communication very important as problem scales

Application	Scaling of computation	Scaling of communication	Scaling of computation-to-communication
FFT	$\frac{n \log n}{p}$	$\frac{n}{p}$	$\log n$
LU	$\frac{n}{p}$	$\frac{\sqrt{n}}{\sqrt{p}}$	$\frac{\sqrt{n}}{\sqrt{p}}$
Barnes	$\frac{n \log n}{p}$	approximately $\frac{\sqrt{n}(\log n)}{\sqrt{p}}$	approximately $\frac{\sqrt{n}}{\sqrt{p}}$
Ocean	$\frac{n}{p}$	$\frac{\sqrt{n}}{\sqrt{p}}$	$\frac{\sqrt{n}}{\sqrt{p}}$

Figure I.1 Scaling of computation, of communication, and of the ratio are critical factors in determining performance on parallel multiprocessors. In this table, p is the increased processor count and n is the increased dataset size. Scaling is on a per-processor basis. The computation scales up with n at the rate given by $O(\,)$ analysis and scales down linearly as p is increased. Communication scaling is more complex. In FFT, all data points must interact, so communication increases with n and decreases with p. In LU and Ocean, communication is proportional to the boundary of a block, so it scales with dataset size at a rate proportional to the side of a square with n points, namely, $\sqrt{n}$; for the same reason communication in these two applications scales inversely to $\sqrt{p}$. Barnes has the most complex scaling properties. Because of the fall-off of interaction between bodies, the basic number of interactions among bodies that require communication scales as $\sqrt{n}$. An additional factor of $\log n$ is needed to maintain the relationships among the bodies. As processor count is increased, communication scales inversely to $\sqrt{p}$.

Sychronization – Avoid contention

- Spin lock:

```
            DADDUI   R3,R0,#1    ;R3 = initial delay
lockit:     LL       R2,0(R1)    ;load linked
            BNEZ     R2,lockit   ;not available-spin
            DADDUI   R2,R2,#1    ;get locked value
            SC       R2,0(R1)    ;store conditional
            BNEZ     R2,gotit    ;branch if store succeeds
            DSLL     R3,R3,#1    ;increase delay by factor of 2
            PAUSE    R3          ;delays by value in R3
            J        lockit
gotit:      use data protected by lock
```

Figure I.5 A spin lock with exponential back-off. When the store conditional fails, the process delays itself by the value in R3. The delay can be implemented by decrementing a copy of the value in R3 until it reaches 0. The exact timing of the delay is multiprocessor dependent, although it should start with a value that is approximately the time to perform the critical section and release the lock. The statement `pause R3` should cause a delay of R3 of these time units. The value in R3 is increased by a factor of 2 every time

- Queuing lock
- Barrier + combining tree

Performance on shared memory

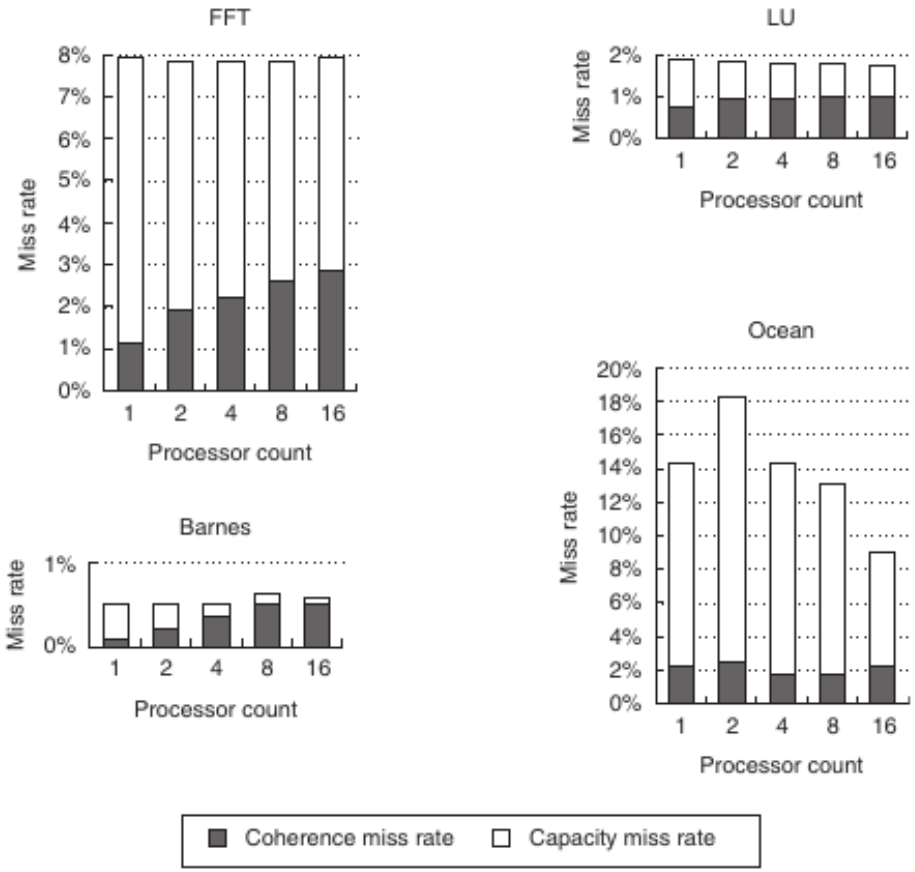


Figure I.8 Data miss rates can vary in nonobvious ways as the processor count is increased from 1 to 16. The miss rates include both coherence and capacity miss rates. The compulsory misses in these benchmarks are all very small and are included in the capacity misses. Most of the misses in these applications are generated by accesses to data that are potentially shared, although in the applications with larger miss rates (FFT and Ocean), it is the capacity misses rather than the coherence misses that comprise the majority of the miss rate. Data are potentially shared if they are allocated in a portion of

Performance on shared memory

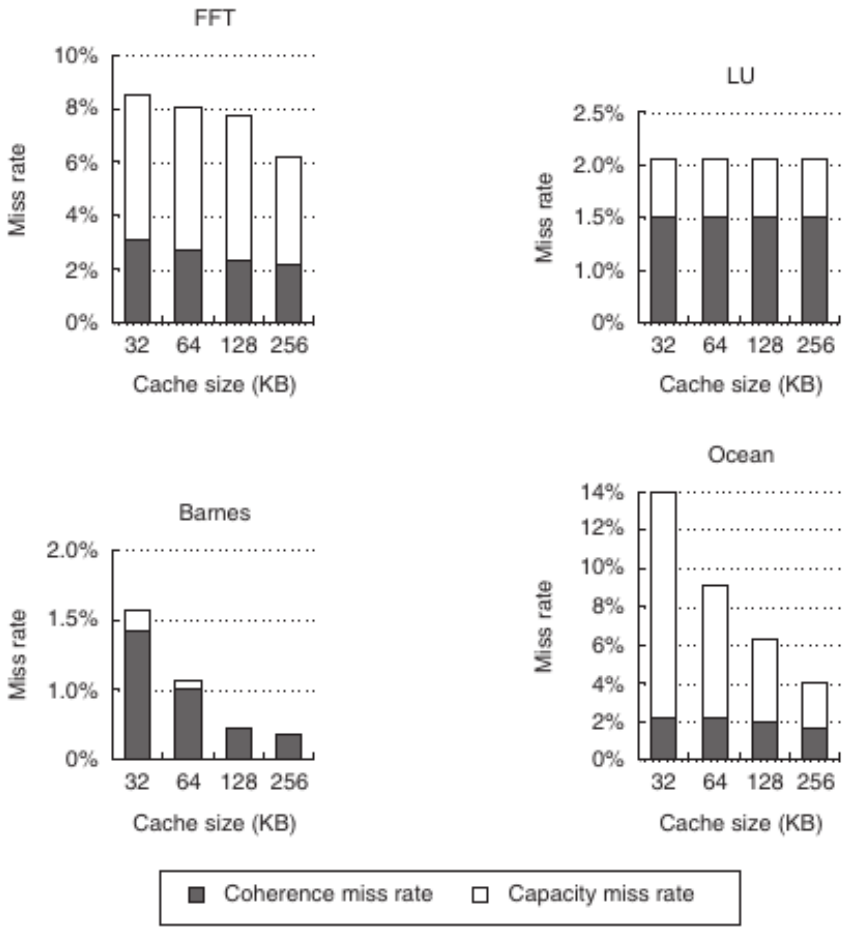


Figure I.9 The miss rate usually drops as the cache size is increased, although coherence misses dampen the effect. The block size is 32 bytes and the cache is two-way set associative. The processor count is fixed at 16 processors. Observe that the scale for each graph is different.

Performance on shared memory

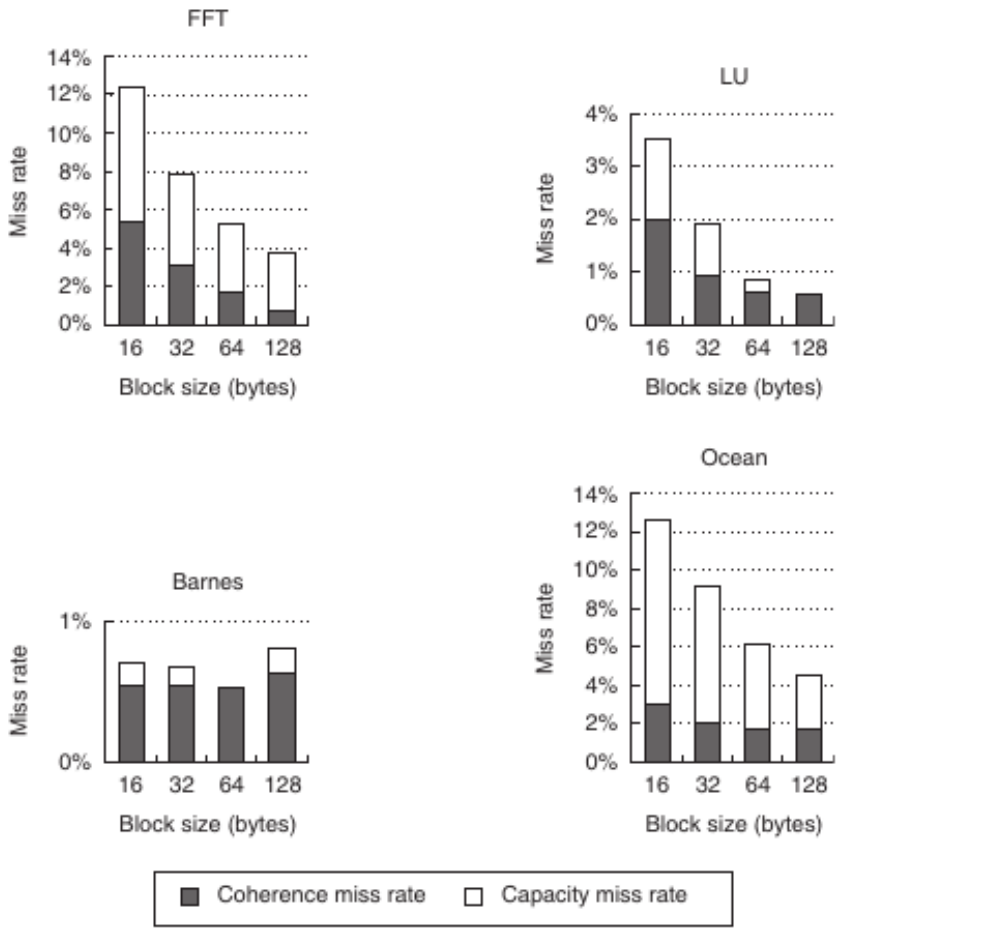


Figure I.10 The data miss rate drops as the cache block size is increased. All these results are for a 16-processor run with a 64 KB cache and two-way set associativity. Once again we use different scales for each benchmark.

Performance on shared memory

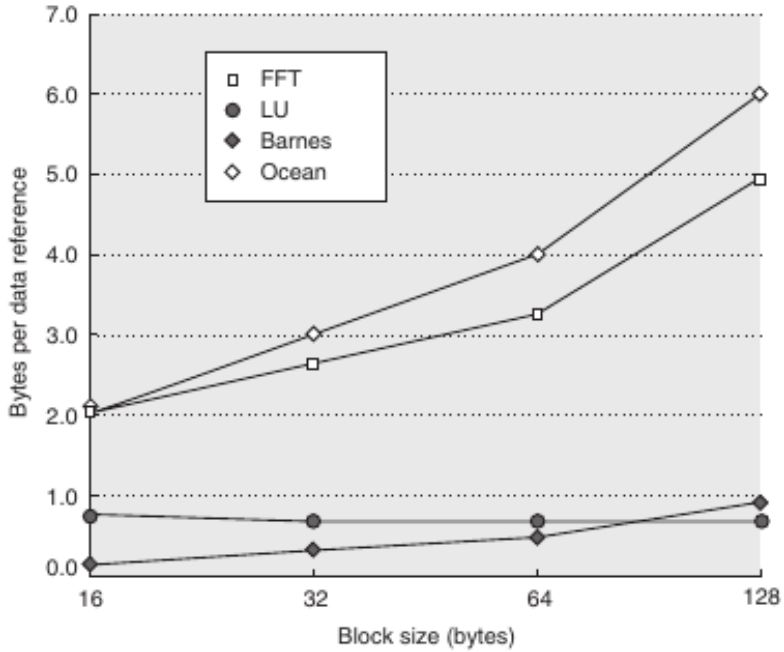


Figure I.11 Bus traffic for data misses climbs steadily as the block size in the data cache is increased. The factor of 3 increase in traffic for Ocean is the best argument against larger block sizes. Remember that our protocol treats ownership or upgrade misses the same as other misses, slightly increasing the penalty for large cache blocks; in both Ocean and FFT, this simplification accounts for less than 10% of the traffic.

Performance on distributed memory

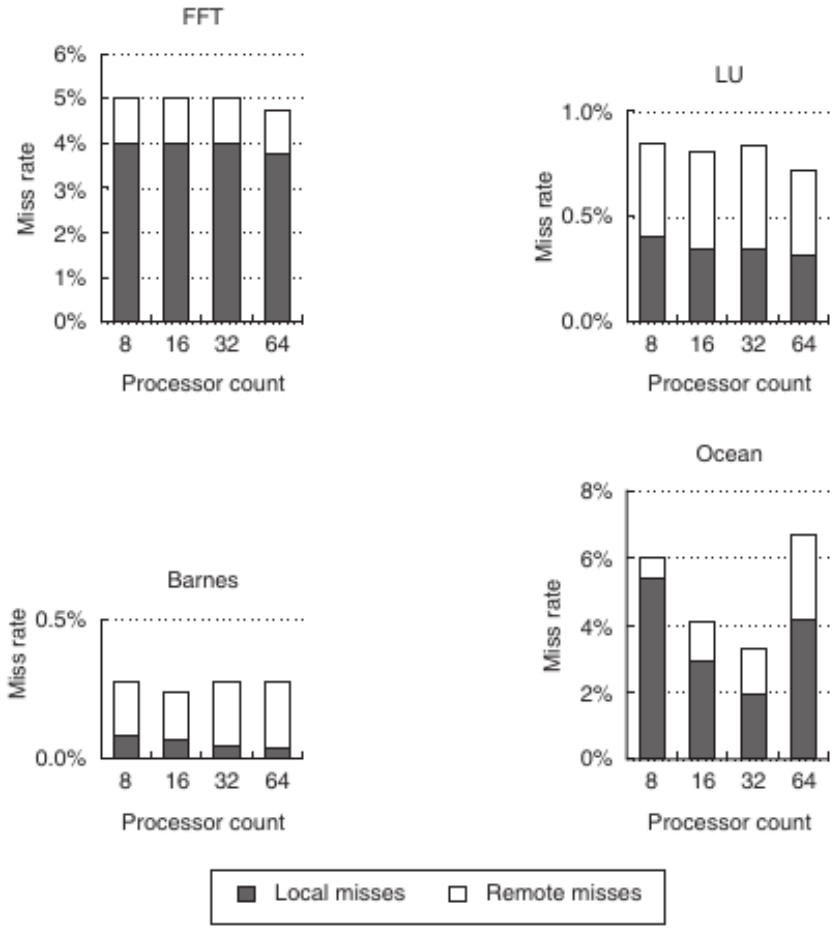


Figure I.12 The data miss rate is often steady as processors are added for these benchmarks. Because of its grid structure, Ocean has an initially decreasing miss rate, which rises when there are 64 processors. For Ocean, the local miss rate drops from 5% at 8 processors to 2% at 32, before rising to 4% at 64. The remote miss rate in Ocean, driven primarily by communication, rises monotonically from 1% to 2.5%. Note that, to show the detailed behavior of each benchmark, different scales are used on the *y*-axis. The cache for all these runs is 128 KB, two-way set associative, with 64-byte blocks.

Performance on distributed memory

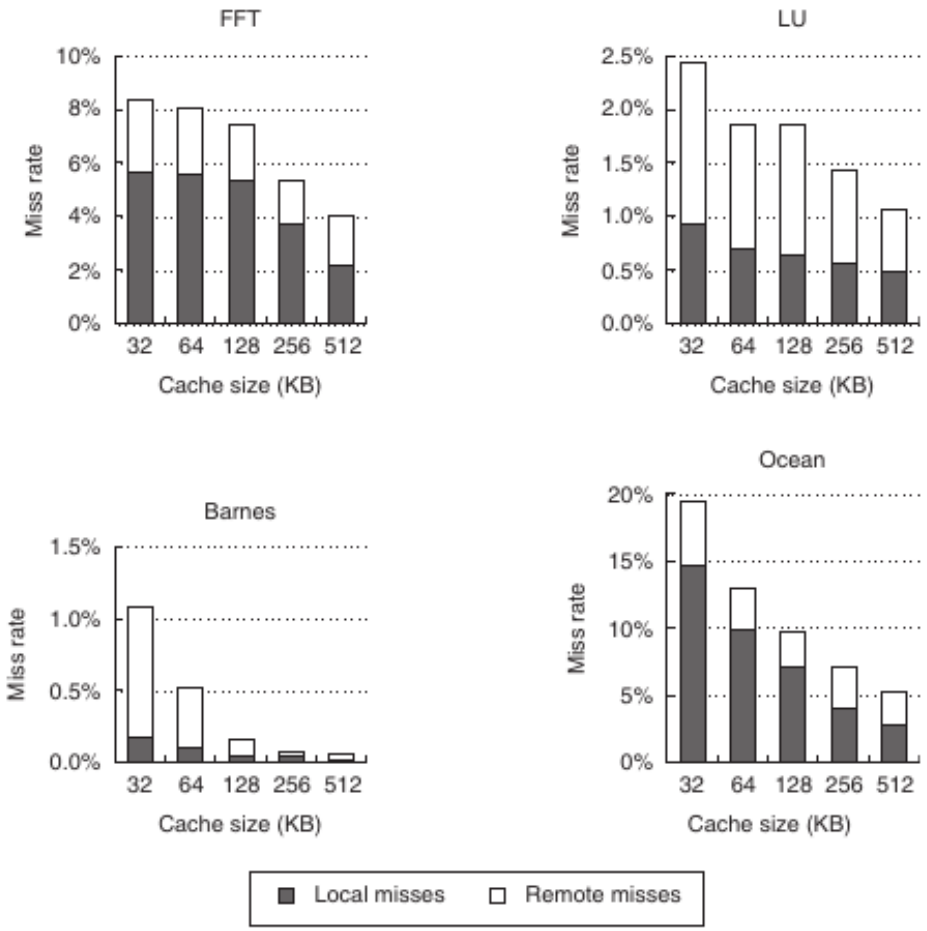


Figure I.13 Miss rates decrease as cache sizes grow. Steady decreases are seen in the local miss rate, while the remote miss rate declines to varying degrees, depending on whether the remote miss rate had a large capacity component or was driven primarily by communication misses. In all cases, the decrease in the local miss rate is larger than the decrease in the remote miss rate. The plateau in the miss rate of FFT, which we mentioned in the last section, ends once the cache exceeds 128 KB. These runs were done with 64 processors and 64-byte cache blocks.

Performance on distributed memory

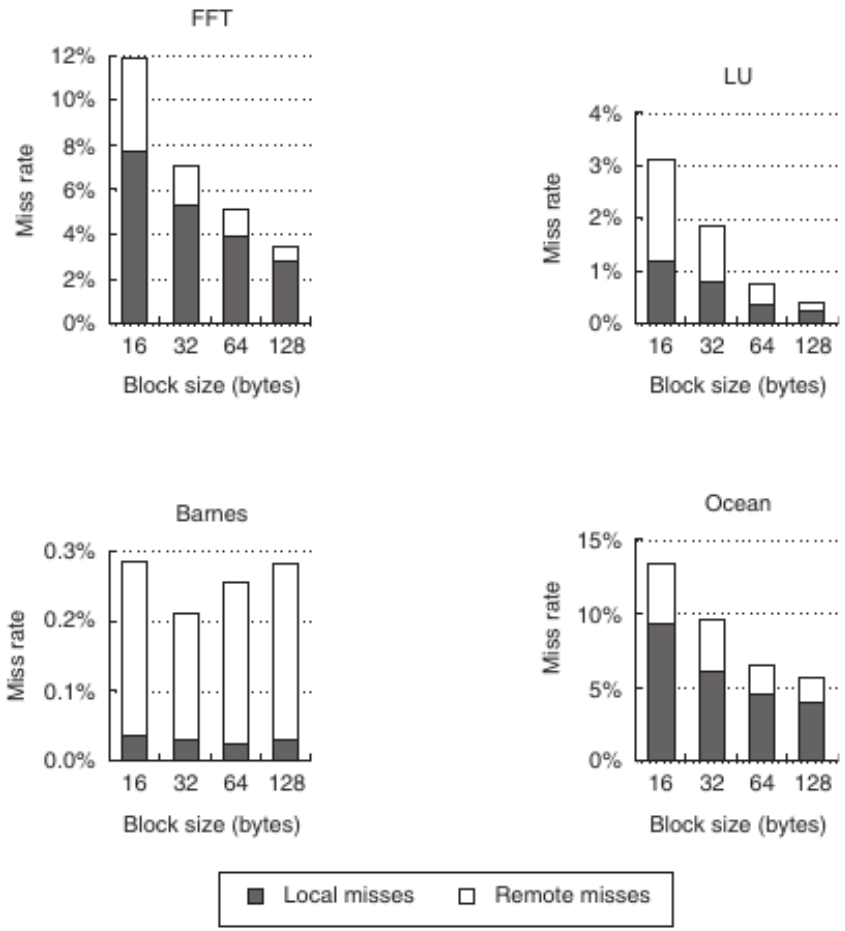


Figure I.14 Data miss rate versus block size assuming a 128 KB cache and 64 processors in total. Although difficult to see, the coherence miss rate in Barnes actually rises for the largest block size, just as in the last section.

Performance on distributed memory

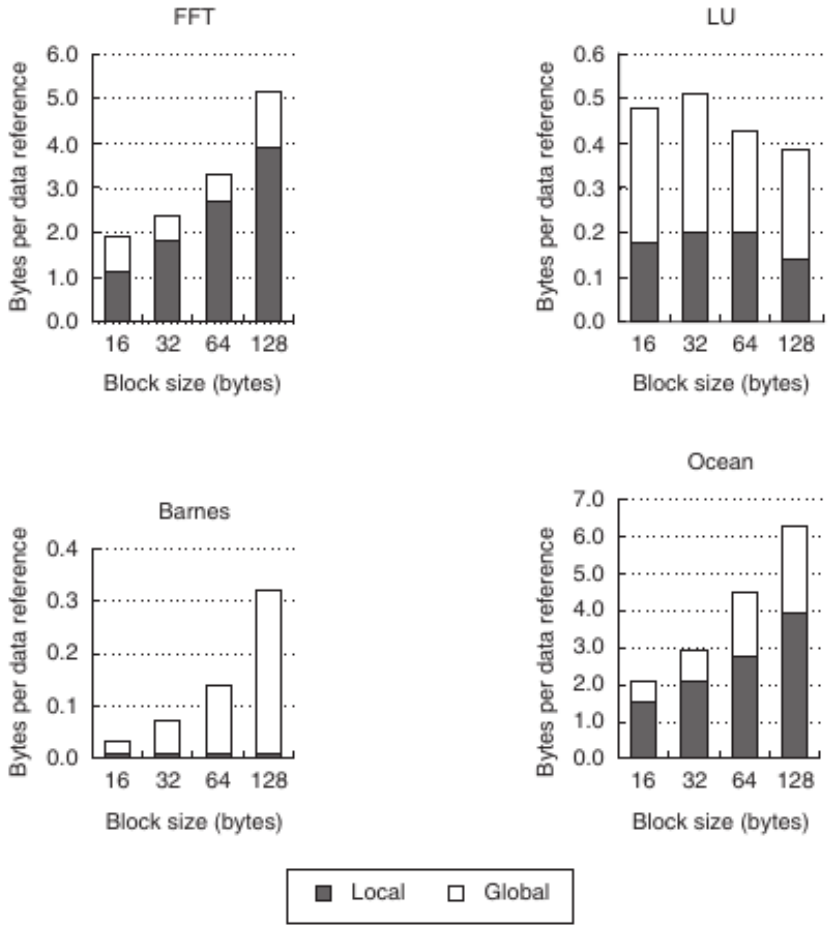


Figure I.15 The number of bytes per data reference climbs steadily as block size is increased. These data can be used to determine the bandwidth required per node both internally and globally. The data assume a 128 KB cache for each of 64 processors.

Performance on distributed memory

Characteristic	Processor clock cycles ≤16 processors	Processor clock cycles 17–64 processors
Cache hit	1	1
Cache miss to local memory	85	85
Cache miss to remote home directory	125	150
Cache miss to remotely cached data (three-hop miss)	140	170

Figure I.16 Characteristics of the example directory-based multiprocessor. Misses can be serviced locally (including from the local directory), at a remote home node, or using the services of both the home node and another remote node that is caching an exclusive copy. This last case is called a three-hop miss and has a higher cost because it requires interrogating both the home directory and a remote cache. Note that this simple model does not account for invalidation time but does include some factor for increasing interconnect time. These remote access latencies are based on those in an SGI Origin 3000, the fastest scalable interconnect system in 2001, and assume a 500 MHz processor.

Performance on distributed memory

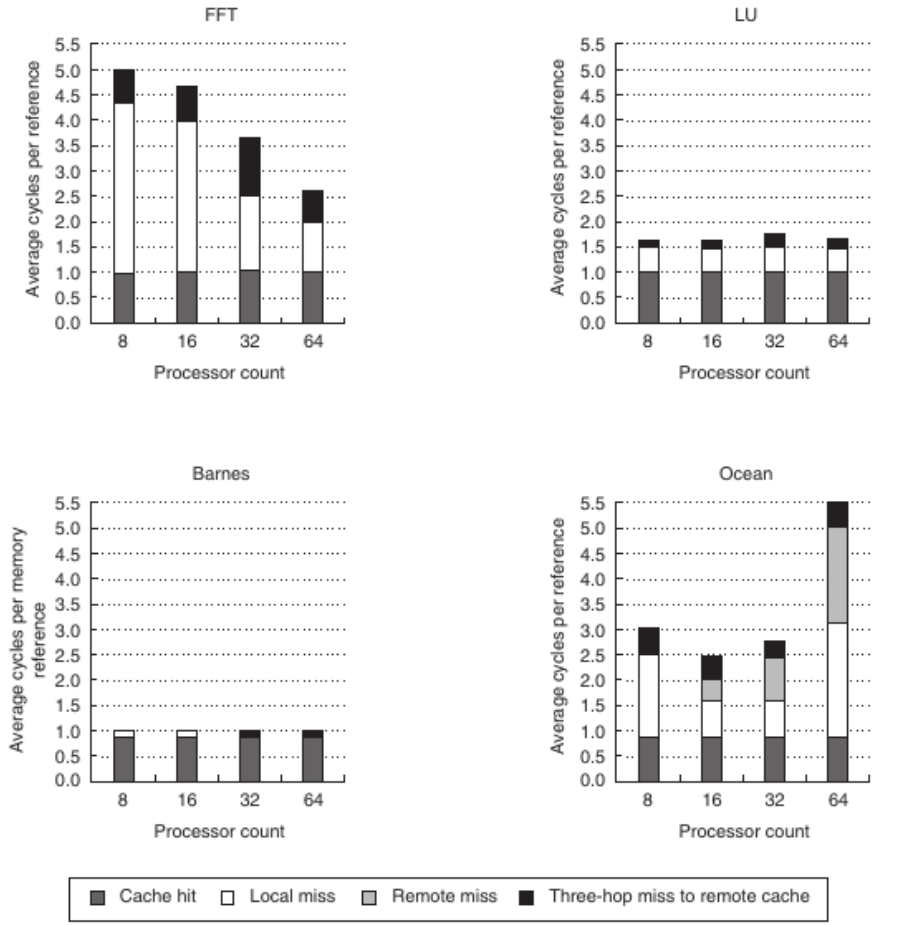


Figure I.17 The effective latency of memory references in a DSM multiprocessor depends both on the relative frequency of cache misses and on the location of the memory where the accesses are served. These plots show the memory access cost (a metric called average memory access time in Chapter 2) for each of the benchmarks for 8, 16, 32, and 64 processors, assuming a 512 KB data cache that is two-way set associative with 64-byte blocks. The average memory access cost is composed of four different types of accesses, with the cost of each type given in Figure I.16. For the Barnes

Measure performace and scale

- Benchmark
- CPU-time
- Wall-time
- Scale up
  - memory-constrained scaling (constant memory/processor)
  - time-constrained scaling (total execution time, perfect speedup)
- how the algorithm scale (convergence)

Interconnection network

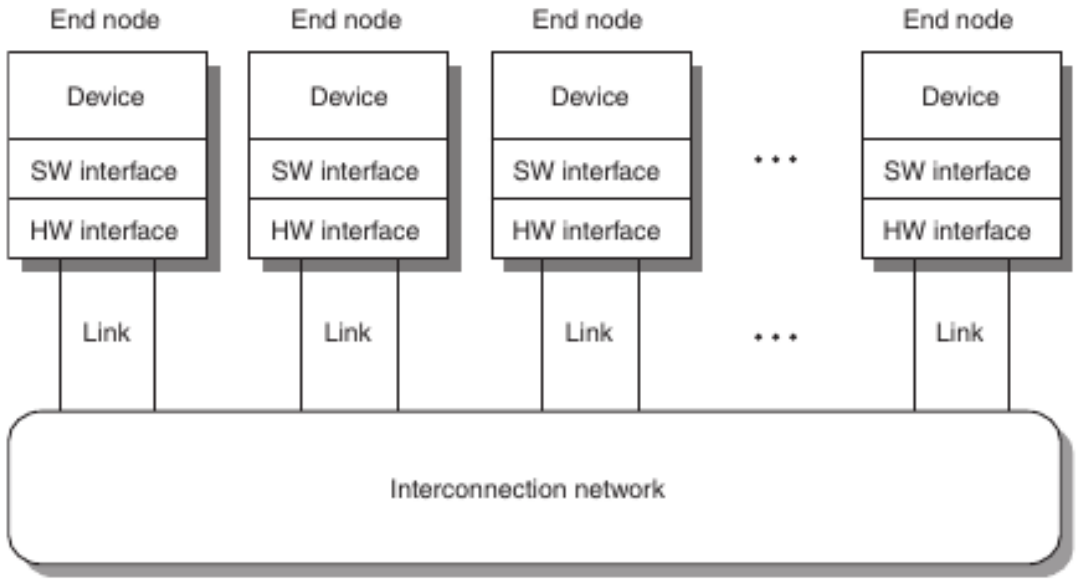


Figure F.1 A conceptual illustration of an interconnected community of devices.

Interconnection domains

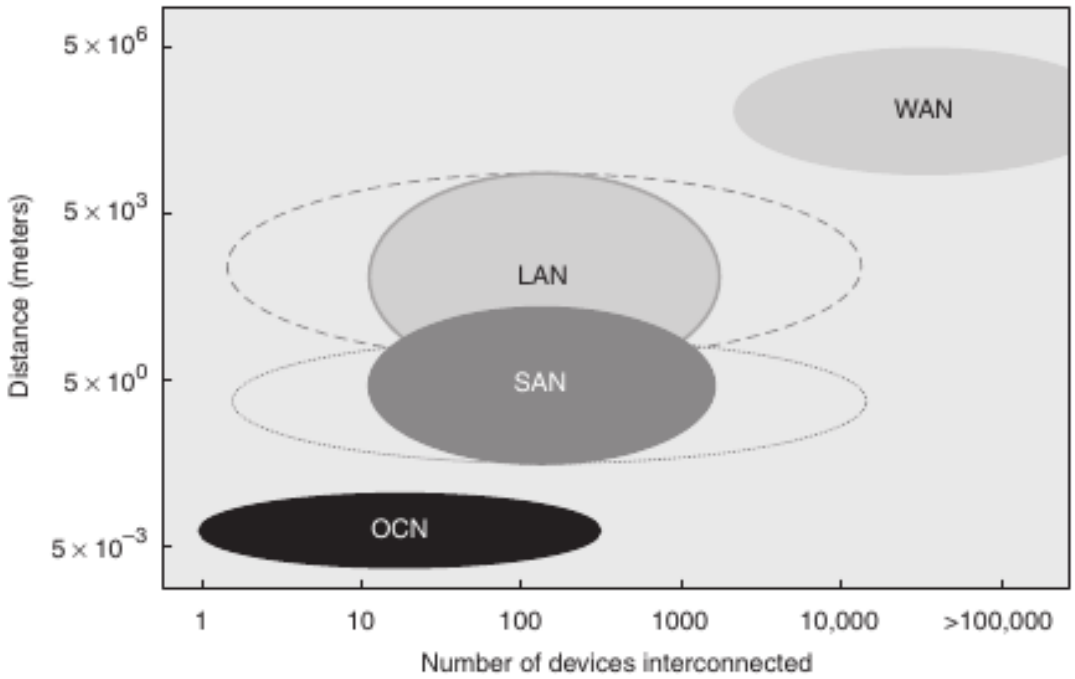


Figure F.2 Relationship of the four interconnection network domains in terms of number of devices connected and their distance scales: on-chip network (OCN), system/storage area network (SAN), local area network (LAN), and wide area network (WAN). Note that there are overlapping ranges where some of these networks compete. Some supercomputer systems use proprietary custom networks to interconnect several thousands of computers, while other systems, such as multicomputer clusters, use standard commercial networks.

Packets costs

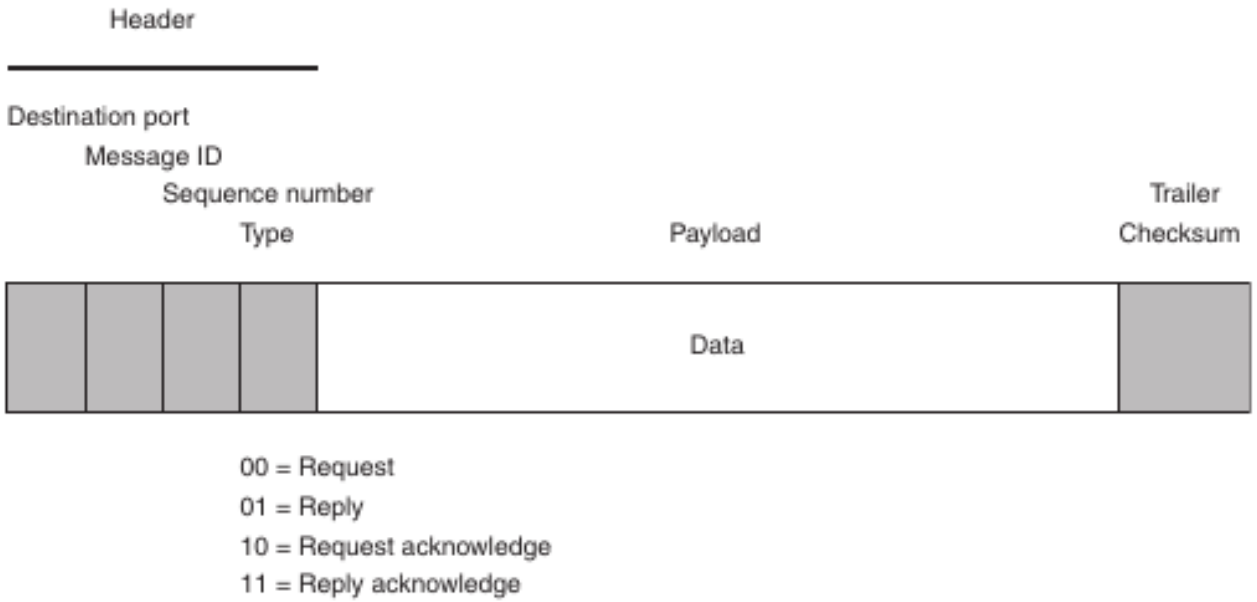


Figure F.4 An example packet format with header, payload, and checksum in the trailer.

Transport

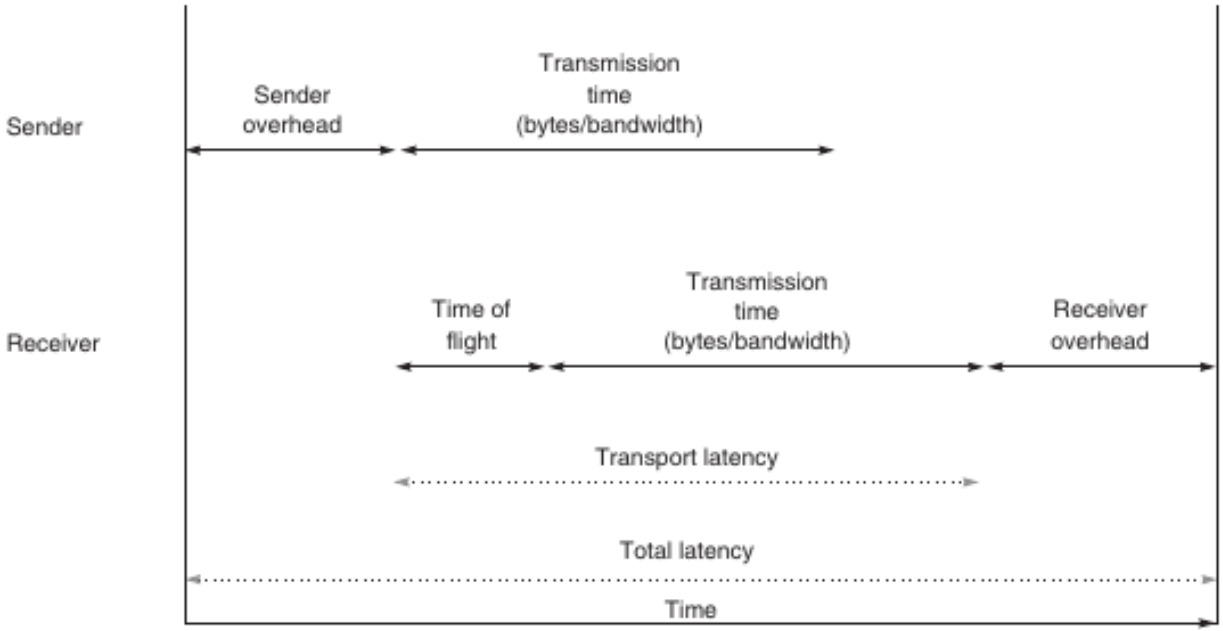


Figure F.5 Components of packet latency. Depending on whether it is an OCN, SAN, LAN, or WAN, the relative amounts of sending and receiving overhead, time of flight, and transmission time are usually quite different from those illustrated here.

Bandwidth

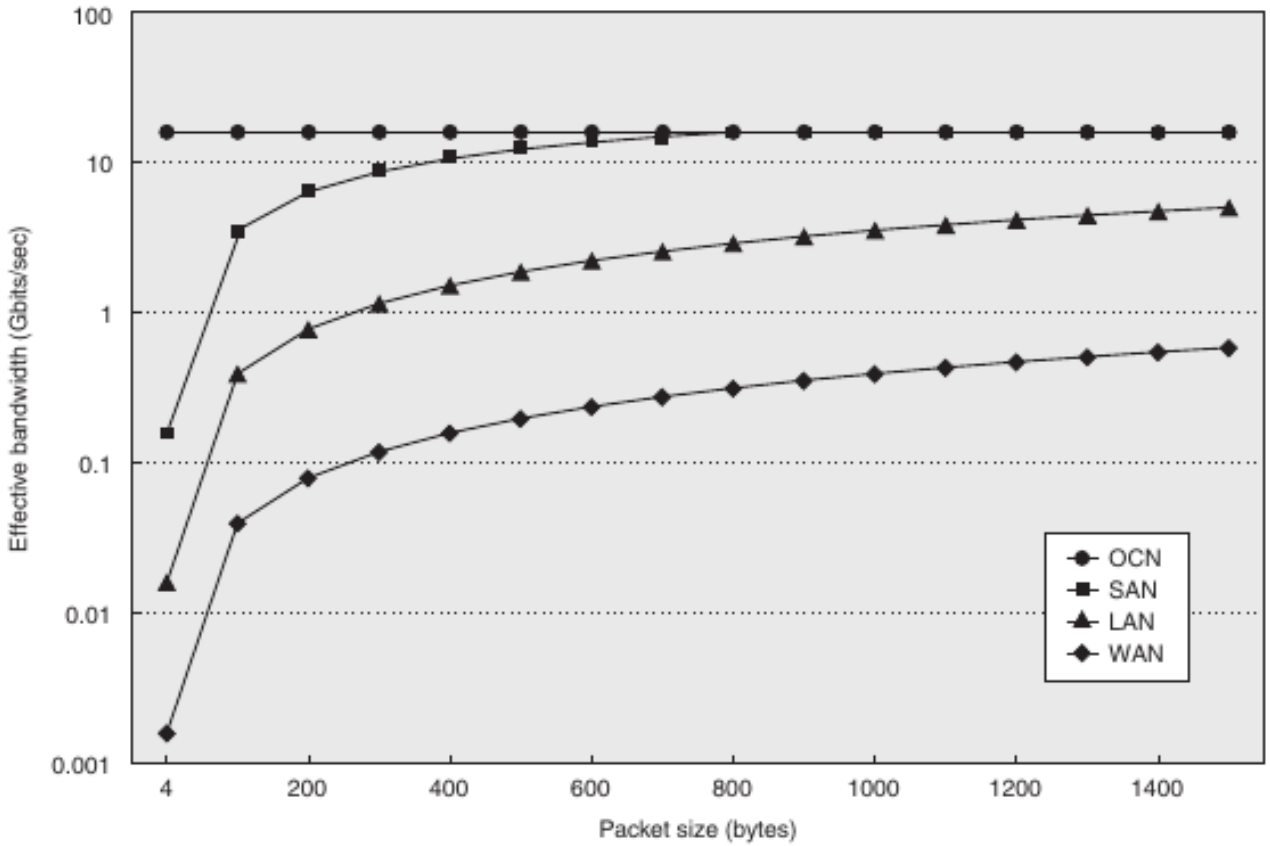


Figure F.6 Effective bandwidth versus packet size plotted in semi-log form for the four network domains. Overhead can be amortized by increasing the packet size, but for too large of an overhead (e.g., for WANs and some LANs) scaling the packet size is of little help. Other considerations come into play that limit the maximum packet size.

Shared vs Switched

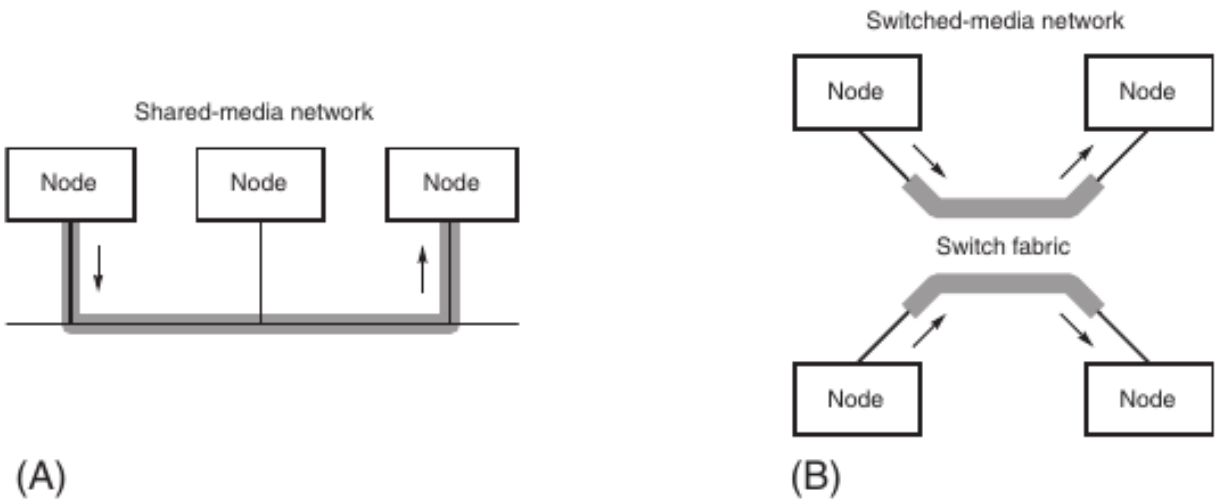


Figure F.8 (a) A shared-media network versus (b) a switched-media network. Ethernet was originally a shared media network, but switched Ethernet is now available. All nodes on the shared-media networks must dynamically share the raw bandwidth of one link, but switched-media networks can support multiple links, providing higher raw aggregate bandwidth.

Shared vs Switched (latency)

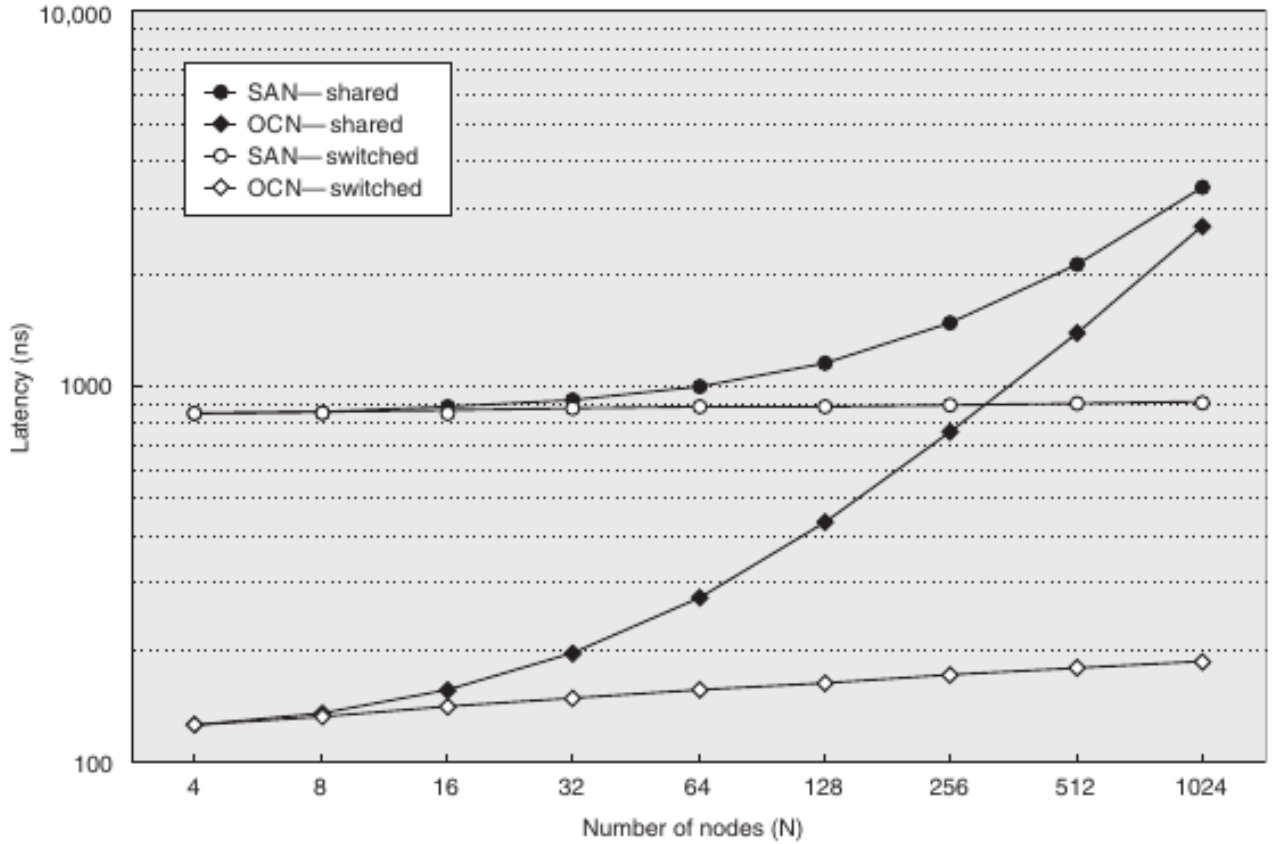


Figure F.9 Latency versus number of interconnected nodes plotted in semi-log form for OCNs and SANs. Routing, arbitration, and switching have more of an impact on latency for networks in these two domains, particularly for networks with a large number of nodes, given the low sending and receiving overheads and low propagation delay.

Shared vs Switched (bandwith)

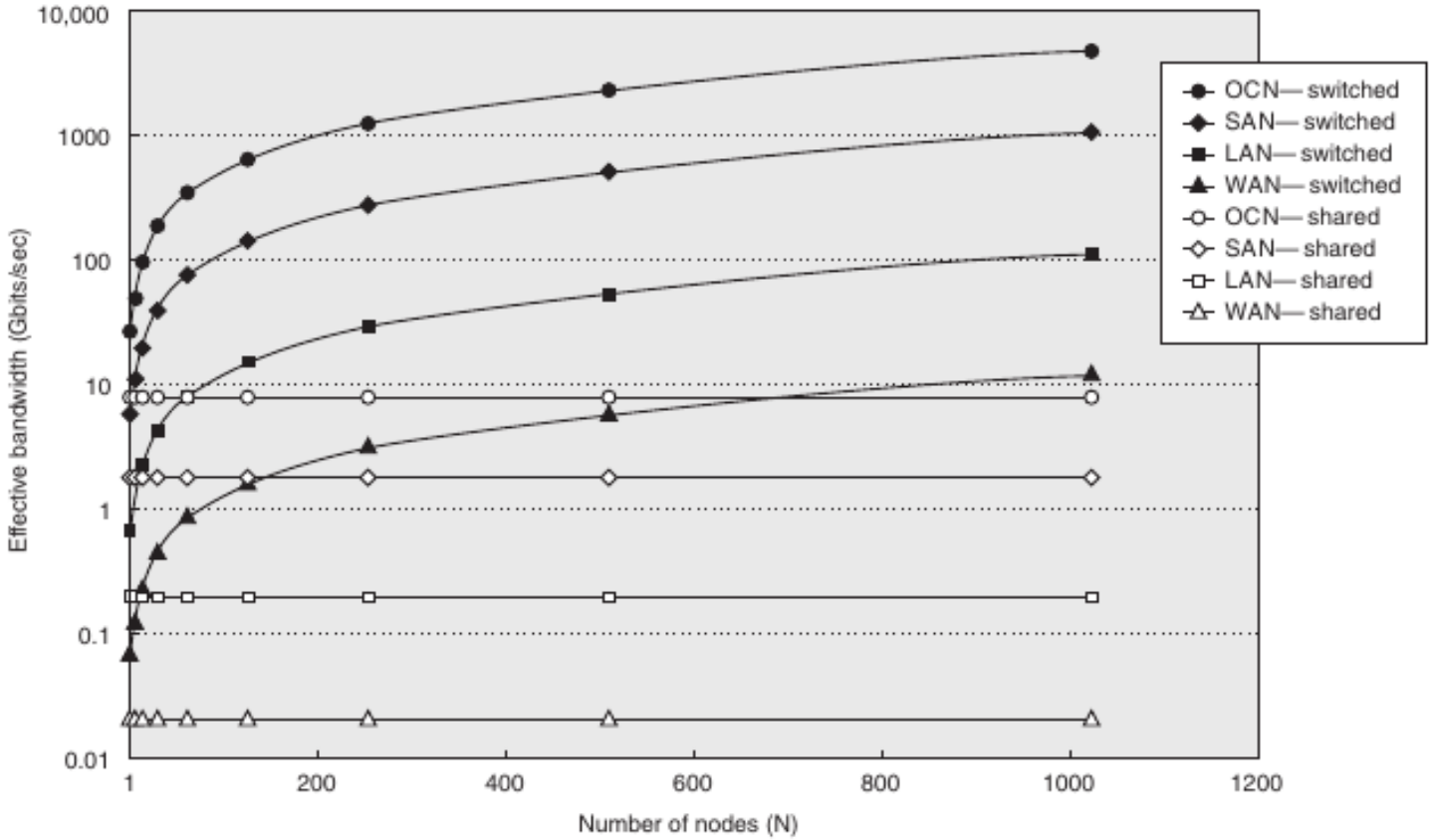


Figure F.10 Effective bandwidth versus number of interconnected nodes plotted in semi-log form for the four network domains. The disparity in effective bandwidth between shared- and switched-media networks for all interconnect domains widens significantly as the number of nodes in the network increases. Only the switched on-chip network is able to achieve an effective bandwidth equal to the aggregate bandwidth for the parameters given in this example.

Centralized switch topology

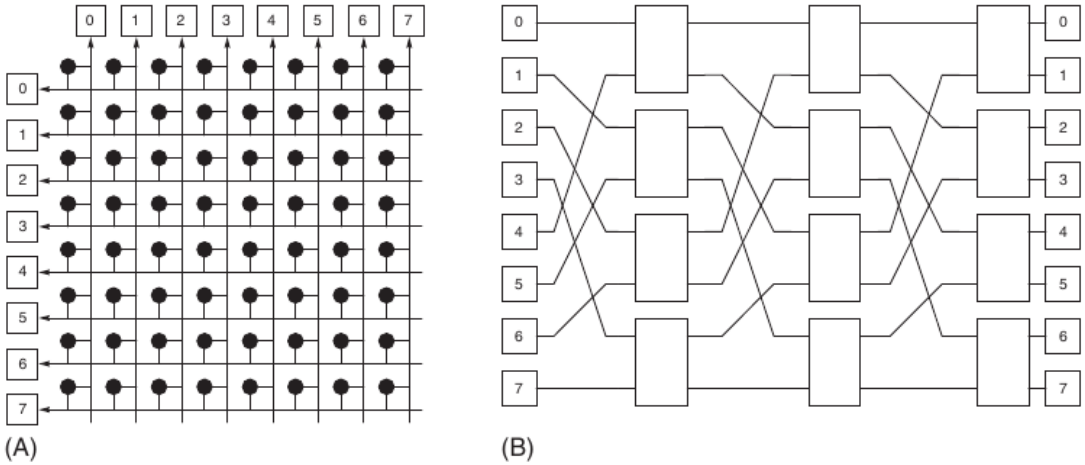


Figure F.11 Popular centralized switched networks: (a) the crossbar network requires N^2 crosspoint switches, shown as black dots; (b) the Omega, a MIN, requires $N/2 \log_2 N$ switches, shown as vertical rectangles. End node devices are shown as numbered squares (total of eight). Links are unidirectional—data enter at the left and exit out the top or right.

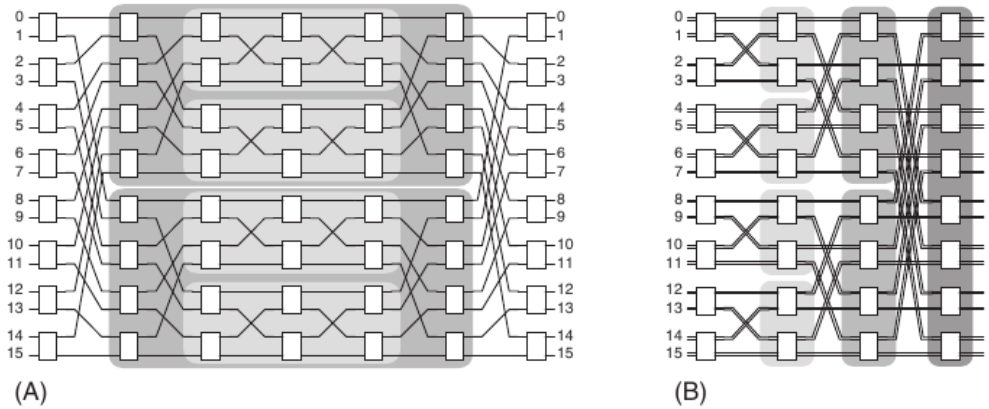


Figure F.12 Two Beneš networks. (a) A 16-port Clos topology, where the middle-stage switches shown in the darker shading are implemented with another Clos network whose middle-stage switches shown in the lighter shading are implemented with yet another Clos network, and so on, until a Beneš network is produced that uses only 2×2 switches everywhere. (b) A folded Beneš network (bidirectional) in which 4×4 switches are used; end nodes attach to the innermost set of the Beneš network (unidirectional) switches. This topology is equivalent to a fat tree, where tree vertices are shown in shades.

Distributed switch topology

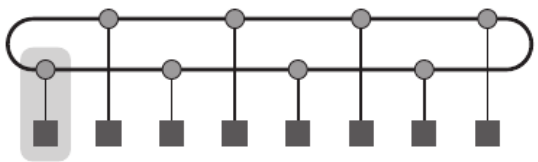


Figure F.13 A ring network topology, folded to reduce the length of the longest link. Shaded circles represent switches, and black squares represent end node devices. The gray rectangle signifies a network node consisting of a switch, a device, and its connecting link.

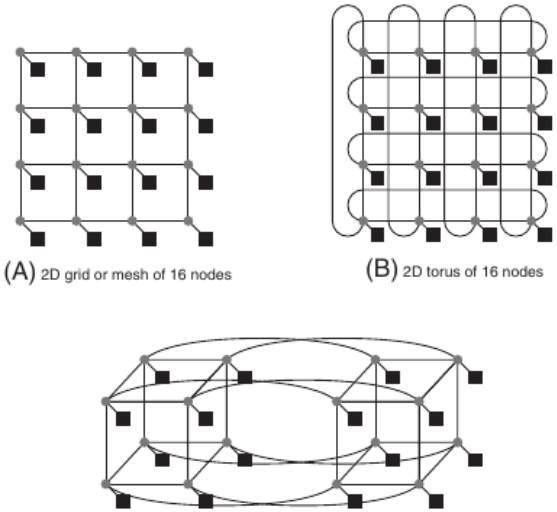


Figure F.14 Direct network topologies that have appeared in commercial systems, mostly supercomputers.
The shaded circles represent switches, and the black squares represent end node devices. Switches have many bidirectional network links, but at least one link goes to the end node device. These basic topologies can be supplemented with extra links to improve performance and reliability. For example, connecting the switches on the periphery of the 2D mesh, shown in (a), using the unused ports on each switch forms a 2D torus, shown in (b). The hypercube topology, shown in (c) is an n-dimensional interconnect for 2^n nodes, requiring $n+1$ ports per switch: one for the n nearest neighbor nodes and one for the end node device.

Cost and performance

Evaluation category	Bus	Ring	2D mesh	2D torus	Hypercube	Fat tree	Fully connected
Performance							
$BW_{Bisection}$ in # links	1	2	8	16	32	32	1024
Max (ave.) hop count	1 (1)	32 (16)	14 (7)	8 (4)	6 (3)	11 (9)	1 (1)
Cost							
I/O ports per switch	NA	3	5	5	7	4	64
Number of switches	NA	64	64	64	64	192	64
Number of net. links	1	64	112	128	192	320	2016
Total number of links	1	128	176	192	256	384	2080

Figure F.15 Performance and cost of several network topologies for 64 nodes. The bus is the standard reference at unit network link cost and bisection bandwidth. Values are given in terms of bidirectional links and ports. Hop count includes a switch and its output link, but not the injection link at end nodes. Except for the bus, values are given for the number of network links and total number of links, including injection/reception links between end node devices and the network.

Routing

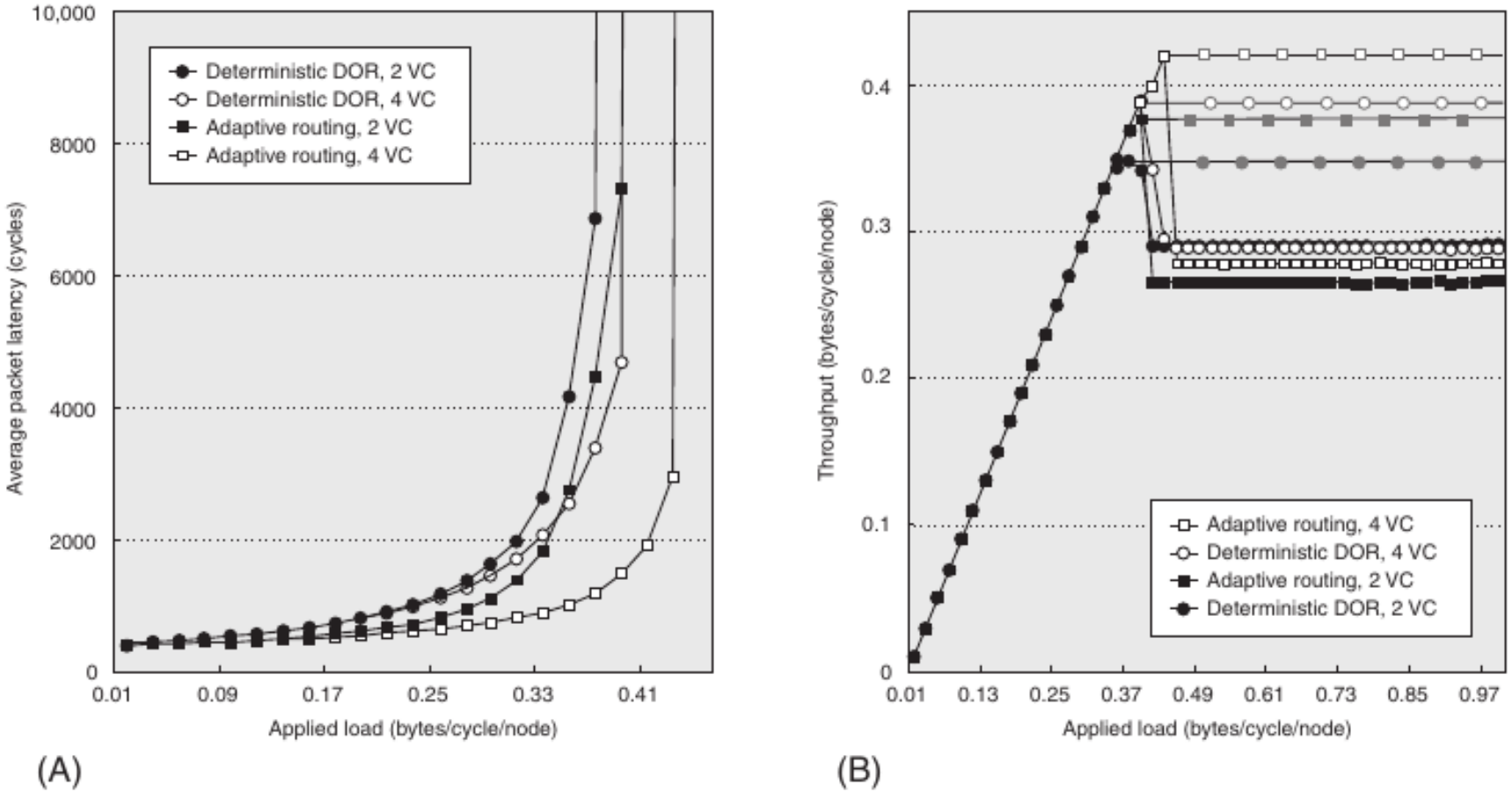


Figure F.19 Deterministic routing is compared against adaptive routing, both with either two or four virtual channels, assuming uniformly distributed traffic on a 4 K node 3D torus network with virtual cut-through switching and bubble flow control to avoid deadlock. (a) Average latency is plotted versus applied load, and (b) throughput is plotted versus applied load (the upper grayish plots show peak throughput, and the lower black plots show sustained throughput). Simulation data were collected by P. Gilabert and J. Flich at the Universidad Politècnica de València, Spain (2006).

Landscape

Terminology	Characteristics	Examples
MPP	Originally referred to a class of architectures characterized by large numbers of small, typically custom processors and usually using an SIMD style architecture.	Connection Machines CM-2
SMP (symmetric multiprocessor)	Shared-memory multiprocessors with a symmetric relationship to memory; also called UMA (uniform memory access). Scalable versions of these architectures used multistage interconnection networks, typically configured with at most 64 to 128 processors.	SUN Sunfire, NEC Earth Simulator
DSM (distributed shared memory)	A class of architectures that support scalable shared memory in a distributed fashion. These architectures are available both with and without cache coherence and typically can support hundreds to thousands of processors.	SGI Origin and Altix, Cray T3E, Cray X1, IBM p5 590/5
Cluster	A class of multiprocessors using message passing. The individual nodes are either commodities or customized, likewise the interconnect.	See commodity and custom clusters
Commodity cluster	A class of clusters where the nodes are truly commodities, typically headless workstations, motherboards, or blade servers, connected with a SAN or LAN usually accessible via an I/O bus.	"Beowulf" and other "homemade" clusters
Custom cluster	A cluster architecture where the nodes and the interconnect are customized and more tightly integrated than in a commodity cluster. Also called distributed memory or message passing multiprocessors.	IBM Blue Gene, Cray XT3
Constellation	Large-scale multiprocessors that use clustering of smaller-scale multiprocessors, typically with a DSM or SMP architecture and 32 or more processors.	Larger SGI Origin/Altix, ASC Purple

Figure I.20 A classification of large-scale multiprocessors. The term *MPP*, which had the original meaning described above, has been used more recently, and less precisely, to refer to all large-scale multiprocessors. None of the commercial shipping multiprocessors is a true MPP in the original sense of the word, but such an approach may make sense in the future. Both the SMP and DSM class includes multiprocessors with vector support. The term *constellation* has been used in different ways; the above usage seems both intuitive and precise [Dongarra et al. 2005].

Relation of classes

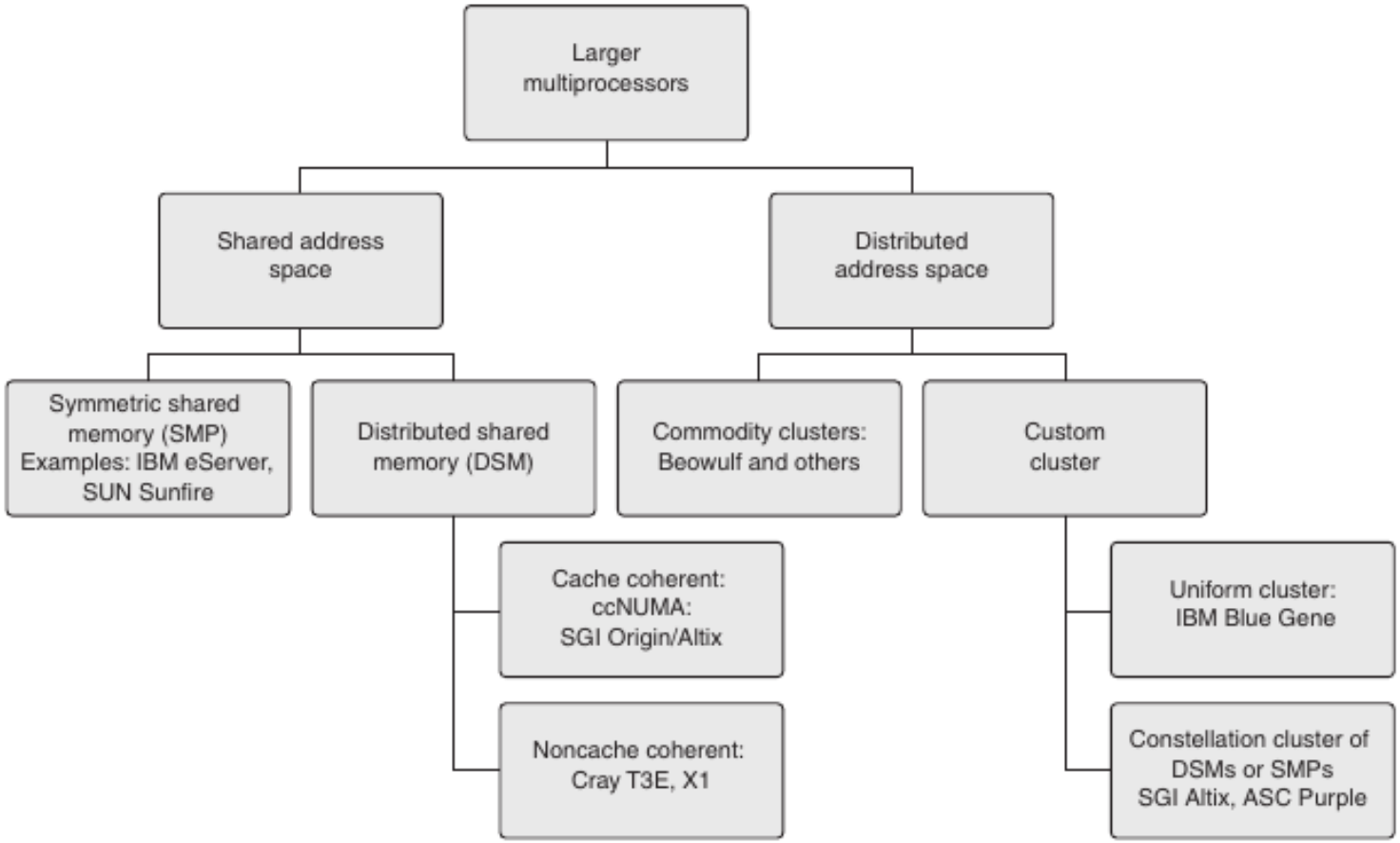


Figure I.21 The space of large-scale multiprocessors and the relation of different classes.